Ansible Automation - Use Cases

A reference of the use cases by which Ansible Automation Platform is commonly implemented, by domain. Each use case is described in the same format (Description, Collection, Key content, Notes) and mapped to Red Hat Certified and Validated content.

16 solution use cases · 384 product use cases · generated 2026-10-21 · community reference, not an official Red Hat publication

Contents

1. Network
    - Solution use cases (16)
        - Back up device configuration
        - Detect configuration drift
        - Restore configuration
        - Gather network facts and inventory
        - Enforce configuration compliance
        - Upgrade network OS and firmware
        - Validate operational state and health checks
        - Generate network reports
        - Manage scoped configuration with a source of truth
        - Test links and circuits with approvals
        - Provision network devices with zero-touch onboarding
        - Manage DNS, DHCP and IPAM
        - Manage load balancers and application delivery
        - Automate data center fabrics
        - Build and validate VPN tunnels
        - Stream telemetry into Event-Driven Ansible
    - Product use cases
        - Network automation (common) (4)
        - SNMP (1)
        - Ansible utilities (6)
        - YANG and NETCONF (1)
        - Arista AVD (EOS) (5)
        - Arista CloudVision Portal (2)
        - Arista EOS (6)
        - Aruba AOS-CX (5)
        - Cisco ACI (24)
        - Cisco Catalyst Center (35)
        - Cisco Nexus Dashboard Fabric Controller (NDFC/DCNM) (6)
        - Cisco Catalyst Center (DNA Center) (33)
        - Cisco IOS and IOS XE (7)
        - Cisco IOS-XR (5)
        - Cisco Meraki (24)
        - Cisco Nexus Dashboard Orchestrator (ACI Multi-Site) (10)
        - Cisco Nexus Dashboard (9)
        - Cisco NX-OS (10)
        - Dell Enterprise SONiC (10)
        - Dell EMC SmartFabric OS10 (7)
        - F5 BIG-IP and BIG-IQ (7)
        - F5 BIG-IP (1)
        - F5 BIG-IP and BIG-IQ (14)
        - F5 BIG-IP (3)
        - F5OS (rSeries and VELOS) (7)
        - F5 BIG-IP Next (3)
        - FRRouting (1)
        - Gluware Control (3)
        - H3C Comware 7 (9)
        - Infoblox NIOS (6)
        - Infoblox Universal DDI (6)
        - Juniper Apstra (5)
        - Juniper Junos (7)
        - Juniper Junos (5)
        - Kentik (1)
        - Lantronix LMOS (1)
        - Lantronix Percepxion and SLC9000 (6)
        - NetBox (9)
        - Citrix NetScaler ADC (33)
        - Network access control lists (1)
        - Network device backup (1)
        - Network device management (2)
        - BGP (1)
        - Network device health checks (3)
        - Network interface management (2)
        - Network device configuration and reporting (2)
        - Red Hat OpenShift (SDN to OVN-Kubernetes migration) (4)
        - OSPF (1)
        - Network telemetry (2)
        - VPN (1)
        - Nautobot (Network to Code) (10)
        - Nokia SR Linux (1)
        - Open vSwitch (1)
        - Radware Alteon (9)
        - Seiko SmartCS console server (1)
        - VyOS (5)

Network

Back up, configure, validate, upgrade and report on multi-vendor networks.

Solution use cases

Use Case: Back up device configuration

Description	Use Ansible to collect the running configuration of every router, switch and firewall and store it as a versioned file, in a Git repository or an S3-compatible bucket. Backups run on a schedule from Automation Platform, so there is always a known-good copy to compare against or restore from.
Collection	`network.backup` (Validated, v5.1.0, updated 2026-01-16), `ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22), `cisco.ios` (Certified, v11.6.0, updated 2026-09-17), `arista.eos` (Certified, v12.3.0, updated 2026-09-17), `junipernetworks.junos` (Certified, v11.1.1, updated 2026-03-09)
Key content	• `network.backup.backup` — Create network backup and write to datastore. • `ansible.netcommon.cli_backup` — Back up device configuration from network devices over network_cli • `cisco.ios.ios_config` — Module to manage configuration sections. • `arista.eos.eos_config` — Manage Arista EOS configuration sections • `junipernetworks.junos.junos_config` — Manage configuration on devices running Juniper JUNOS
Notes	• Pair with Detect configuration drift and Restore configuration to cover the full lifecycle. • Where SSH is not available, SNMP-based collection is an alternative for basic facts.

Use Case: Detect configuration drift

Description	Find out, on a schedule or after an incident, which devices no longer match their approved configuration. The result is a per-device list of what changed (interfaces, BGP, OSPF, ACLs and other resources), so engineers can decide whether to accept the change or correct it.
Collection	`network.base` (Validated, v8.0.0, updated 2025-09-03), `network.interfaces` (Validated, v7.0.0, updated 2025-09-03), `network.bgp` (Validated, v8.0.0, updated 2025-09-03), `network.ospf` (Validated, v7.0.0, updated 2025-09-03), `network.acls` (Validated, v4.0.0, updated 2025-01-17), `ansible.utils` (Certified, v6.1.1, updated 2026-09-17)
Key content	• `network.base.persist` — The `persist` role enables users to fetch facts for specified network resources and store them in a YAML-formatted structure. These host variables can be saved either locally or in a remote data store, acting as a single source of truth (SOT) for network configurations. • `network.base.detect` — The `detect` role enables users to identify configuration drifts between the current running configurations and the desired state provided in a local or remote inventory. This role ensures that network devices remain compliant with their intended configurations. • `network.interfaces.detect` — Detect role will read the facts from the local provided/default inventory and detect if any configuration diff exists w.r.t running-config. • `network.bgp.detect` — The `bgp.detect` role enables users to identify bgp configuration drifts between the current running configurations and the desired state provided in a local or remote inventory. This role ensures that network devices remain compliant with their intended configurations. • `network.ospf.detect` — Detect role will read the facts from the local provided/default inventory and detect if any configuration diff exists w.r.t running-config. • `network.acls.detect` — The `detect` role will read the ACL facts from the local provided/default inventory and detect if any configuration drift exists w.r.t running-config. This role enables users to identify differences between the current running ACL configurations and the desired state, ensuring access control policie • `ansible.utils.fact_diff` — Find the difference between currently set facts
Notes	• Report only by default. Correct drift with Restore configuration or the `remediate` roles, ideally after an approval step. • Detection against a source of truth is described in Manage scoped configuration with a source of truth. • Pair with Event-Driven Ansible to start a detect run when a device sends a configuration-change syslog message, instead of waiting for the schedule. • Fabric controllers have their own drift views, for example `cisco.nd.nd_compliance_analysis` for Nexus Dashboard Insights.

Use Case: Restore configuration

Description	Return a router, switch or firewall to a previous good configuration after a failed change, a hardware replacement or detected drift. Restores run from Automation Platform with the same credentials and logging as every other network job.
Collection	`network.backup` (Validated, v5.1.0, updated 2026-01-16), `network.base` (Validated, v8.0.0, updated 2025-09-03), `ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22), `cisco.ios` (Certified, v11.6.0, updated 2026-09-17), `arista.eos` (Certified, v12.3.0, updated 2026-09-17), `junipernetworks.junos` (Certified, v11.1.1, updated 2026-03-09)
Key content	• `network.backup.restore` — A role to provide a platform-agnostic way of restoring previously backed-up network configurations across supported platforms such as IOS, IOS-XR, NX-OS, EOS, and Junos. • `ansible.netcommon.cli_restore` — Restore device configuration to network devices over network_cli • `network.base.remediate` — The `remediate` role enables users to fix configuration drifts by overriding the running configuration with the desired state provided in the inventory host variables. This role ensures that network devices are restored to their intended configurations. • `cisco.ios.ios_config` — Module to manage configuration sections. • `arista.eos.eos_config` — Manage Arista EOS configuration sections • `junipernetworks.junos.junos_config` — Manage configuration on devices running Juniper JUNOS
Notes	• Restore can drop sessions. Use a workflow with an approval node before the restore job, and run it with limited forks. • Use `check_mode` first where the module supports it to preview the difference. • Pair with Detect configuration drift to find the devices that need a restore.

Use Case: Gather network facts and inventory

Description	Build an accurate, current picture of what is in the network: models, software versions, serial numbers, interfaces, VLANs and routing configuration. The data is collected in the same structure for every vendor, so it can feed reports, audits, upgrade planning and a source of truth.
Collection	`network.base` (Validated, v8.0.0, updated 2025-09-03), `network.inventory_reports` (Validated, v2.0.0, updated 2025-08-26), `ansible.utils` (Certified, v6.1.1, updated 2026-09-17), `ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22), `cisco.ios` (Certified, v11.6.0, updated 2026-09-17), `arista.eos` (Certified, v12.3.0, updated 2026-09-17), `junipernetworks.junos` (Certified, v11.1.1, updated 2026-03-09)
Key content	• `network.base.gather` — The `gather` role enables users to collect and display structured facts for provided network resources. This role helps administrators gain insights into the current state of network configurations in a standardized format. • `network.inventory_reports.gather` — The `gather` role collects comprehensive network device facts, including hardware details using native parsers. • `ansible.utils.cli_parse` — Parse cli output or text using a variety of parsers • `cisco.ios.ios_facts` — Module to collect facts from remote devices. • `arista.eos.eos_facts` — Collect facts from remote devices running Arista EOS • `junipernetworks.junos.junos_facts` — Collect facts from remote devices running Juniper Junos • `ansible.netcommon.cli_command` — Run a cli command on cli-based network devices
Notes	• Turn the data into HTML or CSV with Generate network reports. • Use the persisted data as the baseline for Detect configuration drift. • Controllers such as Cisco Catalyst Center also expose inventory information, for example through `cisco.catalystcenter.network_devices_info_workflow_manager`.

Use Case: Enforce configuration compliance

Description	Define the settings every device must have, for example approved NTP servers, SNMP communities or users, syslog targets and access lists, then verify them across all vendors. Violations are listed per device and can be corrected with the same automation.
Collection	`ansible.utils` (Certified, v6.1.1, updated 2026-09-17), `ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22), `network.base` (Validated, v8.0.0, updated 2025-09-03), `network.acls` (Validated, v4.0.0, updated 2025-01-17), `cisco.ios` (Certified, v11.6.0, updated 2026-09-17), `arista.eos` (Certified, v12.3.0, updated 2026-09-17), `junipernetworks.junos` (Certified, v11.1.1, updated 2026-03-09)
Key content	• `ansible.utils.validate` — Validate data with provided criteria • `ansible.utils.cli_parse` — Parse cli output or text using a variety of parsers • `network.base.remediate` — The `remediate` role enables users to fix configuration drifts by overriding the running configuration with the desired state provided in the inventory host variables. This role ensures that network devices are restored to their intended configurations. • `network.acls.detect` — The `detect` role will read the ACL facts from the local provided/default inventory and detect if any configuration drift exists w.r.t running-config. This role enables users to identify differences between the current running ACL configurations and the desired state, ensuring access control policie • `cisco.ios.ios_ntp_global` — Resource module to configure NTP. • `cisco.ios.ios_snmp_server` — Resource module to configure snmp server. • `arista.eos.eos_ntp_global` — Manages ntp resource module • `junipernetworks.junos.junos_ntp_global` — Manage NTP configuration on Junos devices.
Notes	• Start in report-only mode and enable remediation per rule after owners agree. • Related: Detect configuration drift compares with a recorded state, while this use case compares with a policy. • Report results with Generate network reports.

Use Case: Upgrade network OS and firmware

Description	Move a fleet of network devices to an approved software release without manual device-by-device work. Automation backs up the device, checks space, stages and installs the image, reboots when needed and proves that the device is healthy afterwards.
Collection	`ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22), `network.backup` (Validated, v5.1.0, updated 2026-01-16), `network.healthchecks` (Validated, v2.0.0, updated 2025-08-26), `cisco.nxos` (Certified, v13.0.0, updated 2026-09-17), `juniper.device` (Certified, v2.0.1, updated 2025-11-19), `dellemc.enterprise_sonic` (Certified, v4.1.0, updated 2026-01-27), `arubanetworks.aoscx` (Certified, v4.4.0, updated 2024-04-30), `cisco.catalystcenter` (Certified, v2.10.2, updated 2026-08-11)
Key content	• `network.backup.backup` — Create network backup and write to datastore. • `network.healthchecks.filesystem` — The `network.healthchecks.filesystem` role allows monitoring of filesystem usage on network devices. This helps detect insufficient disk space, ensuring device stability and performance. The role provides a comprehensive health check view that shows the status of filesystem utilization and overall s • `ansible.netcommon.net_put` — Copy a file from Ansible Controller to a network device • `cisco.nxos.nxos_install_os` — Set boot options like boot, kickstart image and issu. • `juniper.device.software` — Install software on a Junos device • `dellemc.enterprise_sonic.sonic_image_management` — Manage installation of Enterprise SONiC image, software patch and firmware updater • `arubanetworks.aoscx.aoscx_upload_firmware` — Uploads a firmware image onto the AOS-CX switch. • `cisco.catalystcenter.swim_workflow_manager` — Configure device credentials on Cisco Catalyst Center
Notes	• Use a workflow job template with an approval node between waves. • Other options: `arista.cvp.cv_image_v3` for EOS images managed by CloudVision, `cisco.dcnm.dcnm_image_upgrade` for Nexus switches managed by NDFC, `juniper.apstra.os_upgrade` for Apstra blueprints and `f5networks.f5_bigip.bigip_software_install` for BIG-IP. • Use Restore configuration if the new release needs a configuration rollback.

Use Case: Validate operational state and health checks

Description	Confirm that the network is working, not only that it is configured: BGP sessions are established, OSPF neighbors are up, interfaces are in the expected state and devices have headroom. The same checks run before and after a change, and on a schedule, with a clear pass or fail result.
Collection	`network.healthchecks` (Validated, v2.0.0, updated 2025-08-26), `network.bgp` (Validated, v8.0.0, updated 2025-09-03), `network.ospf` (Validated, v7.0.0, updated 2025-09-03), `network.interfaces` (Validated, v7.0.0, updated 2025-09-03), `ansible.utils` (Certified, v6.1.1, updated 2026-09-17), `ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22)
Key content	• `network.healthchecks.bgp` — The `network.healthchecks.bgp` role helps in monitoring BGP (Border Gateway Protocol) neighbor status and session health on network devices. This role provides a comprehensive health check view that shows the status of BGP sessions and overall routing health. • `network.healthchecks.ospf` — The role enables user to manage the OSPF resources independent of platforms and perform OSPF health checks. • `network.healthchecks.interfaces` — The role enables users to perform interface health checks on network devices. It provides comprehensive monitoring of interface states and operational status. • `network.healthchecks.ip_reachability` — The `network.healthchecks.ip_reachability` role validates network connectivity by pinging specified IP targets from a network device. It is designed to verify reachability to critical endpoints and provide a detailed status report for observability and automated diagnostics. This role is useful for • `network.healthchecks.cpu` — The `network.healthchecks.cpu` role allows monitoring of CPU usage on network devices. This helps detect excessive CPU consumption, ensuring device stability and performance. The role provides a comprehensive health check view that shows the status of CPU utilization and overall system health. • `network.healthchecks.memory` — The `network.healthchecks.memory` role allows monitoring of memory usage on network devices. This helps detect excessive memory consumption, ensuring device stability and performance. The role provides a comprehensive health check view that shows the status of memory utilization and overall system h • `network.bgp.health_checks` — network.bgp.health_checks • `ansible.utils.cli_parse` — Parse cli output or text using a variety of parsers
Notes	• Combine with Upgrade network OS and firmware and Restore configuration as the verification step. • Feed the results to Generate network reports for management summaries. • Pair with Event-Driven Ansible: a monitoring alert for a BGP session can start the same health check job and attach the output to the ticket.

Use Case: Generate network reports

Description	Give managers, auditors and other teams a readable view of the network without sending them to the CLI. Automation collects device data and turns it into a report that can be opened in a browser or loaded into a spreadsheet.
Collection	`network.inventory_reports` (Validated, v2.0.0, updated 2025-08-26), `network.healthchecks` (Validated, v2.0.0, updated 2025-08-26), `ansible.utils` (Certified, v6.1.1, updated 2026-09-17)
Key content	• `network.inventory_reports.gather` — The `gather` role collects comprehensive network device facts, including hardware details using native parsers. • `network.inventory_reports.generate_report` — The `generate_report` role allows users to generate HTML reports based on collected network device facts. This role parses the gathered facts and presents them in a visually structured HTML format. • `network.inventory_reports.flatten_dict` — Prepares structured data for CSV export. • `network.healthchecks.health_check_view` — Generate the filtered health check dict based on the provided target. • `ansible.utils.cli_parse` — Parse cli output or text using a variety of parsers
Notes	• Combine with Enforce configuration compliance to report pass or fail by rule. • Controllers have their own reporting: `cisco.catalystcenter.reports_workflow_manager` for Catalyst Center and `juniper.apstra.blueprint_report` for Apstra blueprints. • Automation Platform job and host metrics for the automation itself are covered in the reporting domain.

Use Case: Manage scoped configuration with a source of truth

Description	Describe the intended configuration of selected resources (interfaces, VLANs, BGP, OSPF, ACLs) as structured data and let automation apply it. Changes are scoped to the resources named in the change, so a VLAN update does not touch routing, and every change starts as a reviewable data change.
Collection	`network.base` (Validated, v8.0.0, updated 2025-09-03), `network.interfaces` (Validated, v7.0.0, updated 2025-09-03), `network.bgp` (Validated, v8.0.0, updated 2025-09-03), `network.ospf` (Validated, v7.0.0, updated 2025-09-03), `network.acls` (Validated, v4.0.0, updated 2025-01-17), `netbox.netbox` (Certified, v3.21.0, updated 2025-03-12), `networktocode.nautobot` (Certified, v6.3.0, updated 2026-09-17), `ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22), `cisco.ios` (Certified, v11.6.0, updated 2026-09-17)
Key content	• `network.base.persist` — The `persist` role enables users to fetch facts for specified network resources and store them in a YAML-formatted structure. These host variables can be saved either locally or in a remote data store, acting as a single source of truth (SOT) for network configurations. • `network.base.deploy` — The `deploy` role enables users to read the config as `host_vars` from a local or remote data store and deploy network configurations if any changes are found. This role is designed to ensure that network configurations remain consistent and compliant across devices. • `network.interfaces.deploy` — The `deploy` role will read the interfaces facts from the provided/default or remote inventory and deploy the changes onto the appliances. It enables users to read the config as `host_vars` from a local or remote data store and deploy network configurations if any changes are found. This role is des • `network.acls.deploy` — The `deploy` role will read the ACL facts from the provided/default or remote inventory and deploy the changes onto the appliances. It enables users to read the config as host_vars from a local or remote data store and deploy ACL configurations if any changes are found. This role is designed to ensu • `netbox.netbox.nb_inventory` — NetBox inventory source • `networktocode.nautobot.inventory` — Nautobot inventory source • `ansible.netcommon.network_resource` — Manage resource modules • `cisco.ios.ios_vlans` — Resource module to configure VLANs.

Notes	• Resource modules use `state` values such as `merged`, `replaced`, `overridden` and `deleted`. Agree on which are allowed, because `overridden` removes anything not in the data. • Related: Detect configuration drift and Manage DNS, DHCP and IPAM. • Pair with Event-Driven Ansible to start a deploy when a record in the source of truth changes.

Use Case: Test links and circuits with approvals

Description	Prove that a new or changed link works before traffic is moved to it. A workflow in Automation Platform runs the tests from both ends of the circuit, stops for approval at defined points and records the outcome in the ticket and inventory records.
Collection	`ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22), `network.healthchecks` (Validated, v2.0.0, updated 2025-08-26), `cisco.ios` (Certified, v11.6.0, updated 2026-09-17), `cisco.nxos` (Certified, v13.0.0, updated 2026-09-17), `juniper.device` (Certified, v2.0.1, updated 2025-11-19), `netbox.netbox` (Certified, v3.21.0, updated 2025-03-12), `servicenow.itsm` (Certified, v2.16.0, updated 2026-08-24)
Key content	• `ansible.netcommon.net_ping` — Tests reachability using ping from a network device • `network.healthchecks.ip_reachability` — The `network.healthchecks.ip_reachability` role validates network connectivity by pinging specified IP targets from a network device. It is designed to verify reachability to critical endpoints and provide a detailed status report for observability and automated diagnostics. This role is useful for • `cisco.ios.ios_ping` — Tests reachability using ping from IOS switch. • `cisco.nxos.nxos_ping` — Tests reachability using ping from Nexus switch. • `juniper.device.ping` — Execute ping from a Junos device • `juniper.device.pmtud` — Perform path MTU discovery from a Junos device to a destination • `netbox.netbox.netbox_circuit` — Create, update or delete circuits within NetBox • `servicenow.itsm.change_request` — Manage ServiceNow change requests
Notes	• Use the workflow approval node for the human decision, and keep test and change jobs separate so the tests can be rerun on demand. • Related: Validate operational state and health checks. • `networktocode.nautobot.circuit` is the equivalent for Nautobot.

Use Case: Provision network devices with zero-touch onboarding

Description	Ship a device to a site, power it on and have it receive its software and configuration without an engineer on site. Automation prepares the records and templates that the onboarding service needs, then verifies that the device registered and is configured.
Collection	`cisco.catalystcenter` (Certified, v2.10.2, updated 2026-08-11), `juniper.apstra` (Certified, v1.1.0, updated 2026-07-31), `arista.cvp` (Certified, v3.12.0, updated 2024-11-19), `arista.avd` (Certified, v6.4.0, updated 2026-09-18), `network.base` (Validated, v8.0.0, updated 2025-09-03), `netbox.netbox` (Certified, v3.21.0, updated 2025-03-12)
Key content	• `cisco.catalystcenter.pnp_workflow_manager` — Manage operations - Add, claim, and delete devices of Onboarding Configuration (PnP) • `cisco.catalystcenter.provision_workflow_manager` — Configure device credentials on Cisco Catalyst Center • `cisco.catalystcenter.template_workflow_manager` — Template workflow manager • `juniper.apstra.ztp_config` — Manage ZTP VM configuration (DHCP, firmware, passwords) • `juniper.apstra.ztp_device` — Manage ZTP (Zero Touch Provisioning) devices in Apstra • `arista.cvp.dhcp_configuration` — Role to configure DHCP service to act as ZTP server • `arista.avd.cv_deploy` — Deploy device configurations and tags to CloudVision • `network.base.deploy` — The `deploy` role enables users to read the config as `host_vars` from a local or remote data store and deploy network configurations if any changes are found. This role is designed to ensure that network configurations remain consistent and compliant across devices.
Notes	• Keep device credentials and image repositories in the platform credentials and controller, not in templates. • Related: Upgrade network OS and firmware and Manage scoped configuration with a source of truth. • Pair with Event-Driven Ansible: a controller or DHCP event for a new device can start the onboarding workflow.

Use Case: Manage DNS, DHCP and IPAM

Description	Take the manual tickets out of address management. Automation reserves the next free address, creates the DNS record and DHCP reservation and records the result in the IPAM, so servers, devices and applications get consistent names and addresses the moment they are requested.
Collection	`infoblox.nios_modules` (Certified, v1.8.0, updated 2025-01-29), `infoblox.universal_ddi` (Certified, v1.1.0, updated 2025-12-03), `netbox.netbox` (Certified, v3.21.0, updated 2025-03-12), `networktocode.nautobot` (Certified, v6.3.0, updated 2026-09-17)
Key content	• `infoblox.nios_modules.nios_next_ip` — Return the next available IP address for a network • `infoblox.nios_modules.nios_host_record` — Configure Infoblox NIOS host records • `infoblox.nios_modules.nios_network` — Configure Infoblox NIOS network object • `infoblox.universal_ddi.ipam_subnet` — Manages a Subnet. • `infoblox.universal_ddi.dns_record` — Manages a DNS resource record in an authoritative zone. • `infoblox.universal_ddi.dhcp_fixed_address` — Manage a Fixed Address • `netbox.netbox.netbox_prefix` — Creates or removes prefixes from NetBox • `netbox.netbox.netbox_ip_address` — Creates or removes IP addresses from NetBox
Notes	• Expose the job as a survey or service catalog item so other automation can call it, for example during Provision network devices with zero-touch onboarding. • Both Infoblox collections can also act as inventory sources, for example `infoblox.nios_modules.nios_inventory`. • Related: Manage scoped configuration with a source of truth.

Use Case: Manage load balancers and application delivery

Description	Publish an application behind a load balancer from a request, rather than a manual change on the appliance. Automation creates the pools, members, health monitors and virtual servers consistently, and takes servers in and out of rotation during maintenance.
Collection	`f5networks.f5_modules` (Certified, v1.44.0, updated 2026-09-08), `f5networks.f5_bigip` (Certified, v3.14.0, updated 2025-07-03), `f5networks.f5_eda_security` (Validated, v1.0.1, updated 2024-12-05), `netscaler.adc` (Certified, v2.17.0, updated 2026-07-15), `vmware.alb` (Certified, v32.1.4, updated 2026-10-14)
Key content	• `f5networks.f5_modules.bigip_virtual_server` — Manage LTM virtual servers on a BIG-IP • `f5networks.f5_modules.bigip_pool` — Manages F5 BIG-IP LTM pools • `f5networks.f5_modules.bigip_pool_member` — Manages F5 BIG-IP LTM pool members • `f5networks.f5_bigip.bigip_as3_deploy` — Manages AS3 declarations sent to BIG-IP • `netscaler.adc.lbvserver` — Configuration for Load Balancing Virtual Server resource. • `netscaler.adc.servicegroup` — Configuration for service group resource. • `vmware.alb.avi_virtualservice` — Module for setup of VirtualService Avi RESTful Object
Notes	• Create the DNS name and address first with Manage DNS, DHCP and IPAM. • `f5networks.f5_eda_security` provides Event-Driven Ansible playbooks that react to security events on BIG-IP, for example blocking IP addresses in an ASM policy. • BIG-IP software and UCS archives are handled with `f5networks.f5_bigip.bigip_software_install` and `f5networks.f5_modules.bigip_ucs`.

Use Case: Automate data center fabrics

Description	Manage a data center fabric through its controller or design model, not box by box. Teams describe tenants, VRFs, networks and policies as data, review the change, and automation pushes it to the fabric in a repeatable way.
Collection	`cisco.aci` (Certified, v2.13.0, updated 2025-12-09), `cisco.dcnm` (Certified, v3.11.0, updated 2026-04-13), `juniper.apstra` (Certified, v1.1.0, updated 2026-07-31), `arista.avd` (Certified, v6.4.0, updated 2026-09-18), `arista.cvp` (Certified, v3.12.0, updated 2024-11-19)
Key content	• `cisco.aci.aci_tenant` — Manage tenants (fv:Tenant) • `cisco.aci.aci_vrf` — Manage contexts or VRFs (fv:Ctx) • `cisco.dcnm.dcnm_vrf` — Add and remove VRFs from a ND managed VXLAN fabric. • `cisco.dcnm.dcnm_network` — Add and remove Networks from a ND managed VXLAN fabric. • `juniper.apstra.blueprint` — Manage Apstra blueprints • `juniper.apstra.virtual_network` — Manage virtual networks in Apstra • `arista.avd.eos_designs` — eos_designs is a role that provides an abstracted data model to deploy various network designs and use cases. • `arista.avd.cv_deploy` — Deploy device configurations and tags to CloudVision
Notes	• Multi-site ACI uses `cisco.mso` and Nexus Dashboard modules (`cisco.nd`), which have their own object models. • Campus fabrics follow the same pattern with Catalyst Center (`cisco.catalystcenter`) and Meraki (`cisco.meraki`) collections. • Related: Detect configuration drift and Upgrade network OS and firmware.

Use Case: Build and validate VPN tunnels

Description	Connect a data center or branch to a cloud network with an IPsec VPN, and prove that it works. Both ends of the tunnel are configured from the same data, so peer addresses, pre-shared keys and routes cannot drift apart.
Collection	`network.vpn` (Validated, v6.0.0, updated 2025-09-03), `ansible.netcommon` (Certified, v8.7.1, updated 2026-09-22)
Key content	• `network.vpn.deploy` — The `deploy` role reads VPN configuration details from the provided/default or remote inventory and deploys them to network appliances. It ensures that VPN settings remain consistent and compliant across devices. Users can source configurations from local files or remote repositories and deploy only • `network.vpn.validate` — This role serves as the validation point for the `network.vpn` collection. Based on the provider and validation parameters specified, the correct validation tasks are executed. • `network.vpn.azure_rm_localnetworkgateway` — Manage Azure local network gateways • `network.vpn.azure_rm_virtualnetworkgatewayconnection` — Manage Azure Virtual Network Gateway Connections • `ansible.netcommon.net_ping` — Tests reachability using ping from a network device
Notes	• Check the collection documentation for the providers and appliances supported by each role. • Related: Validate operational state and health checks. • Pair with Event-Driven Ansible to re-run validation, or open a ticket, when a tunnel-down alert arrives from monitoring.

Use Case: Stream telemetry into Event-Driven Ansible

Description	React to what the network reports instead of waiting for a schedule. Devices stream telemetry to a collector, and the events reach Event-Driven Ansible, which starts a diagnostic or remediation job in Automation Platform when a condition is met.
Collection	`network.telemetry` (Validated, v3.0.0, updated 2024-06-13), `network.healthchecks` (Validated, v2.0.0, updated 2025-08-26), `kentik.ansible_eda` (Certified, v1.0.7, updated 2024-03-25)

Key content	• `network.telemetry.deploy` — The `network.telemetry.deploy` role reads telemetry configuration facts from a provided/default or remote inventory and deploys the changes onto network appliances. It enables users to read the config as host_vars from a local or remote data store and deploy telemetry configurations if any changes a • `network.telemetry.deploy_collector` — The `deploy_collector` role automates the deployment of a telemetry collection infrastructure using Docker and Telegraf. This role is designed to set up a containerized environment for collecting network telemetry data, particularly from Cisco devices using Model-Driven Telemetry (MDT). • `network.telemetry.health_checks` — The `deploy_health_check` role automates monitoring of telemetry configuration health on Cisco NXOS devices. This role verifies telemetry configuration state and transport connectivity. • `network.telemetry.detect` — The `network.telemetry.detect` role identifies configuration drift in telemetry settings by comparing current device configurations against a previously persisted state. This role helps maintain configuration consistency by detecting unauthorized or unexpected changes in telemetry configurations acr • `network.telemetry.remediate` — The `network.telemetry.remediate` role automatically corrects configuration drift in telemetry settings by comparing current device configurations against a trusted source of truth and applying necessary changes to restore the intended state. • `kentik.ansible_eda.kentik_webhook` —
Notes	• Start with rules that collect data and notify, and add automatic remediation after the results have been reviewed. • Related: Validate operational state and health checks. • Other event sources for network use cases include syslog and monitoring webhooks.

Product use cases

Grouped from the modules of each certified or validated collection. Key content lists the components each use case supports, with the modules that manage them (create, update, delete) and query them (_info).

Network automation (common) `ansible.netcommon`

Certified Red Hat · version 8.7.1 · updated 2026-09-22 · 16 modules → 4 use cases

Use Case: Connect to network devices

Description	Connect over network_cli, NETCONF, httpapi, gRPC, libssh and persistent sockets.
Collection	`ansible.netcommon` (Certified, Red Hat, v8.7.1, updated 2026-09-22)
Key content	**CLI over SSH** connection `network_cli` **NETCONF** connection `netconf` **HTTP API** connection `httpapi` **gRPC** connection `grpc` **libssh** connection `libssh` **Persistent socket** connection `persistent` **Telnet commands** `telnet`

Use Case: Run commands and push configuration

Description	Run CLI commands, push, back up and restore text configuration, and manage resource modules.
Collection	`ansible.netcommon` (Certified, Red Hat, v8.7.1, updated 2026-09-22)
Key content	**CLI commands** `cli_command` **CLI configuration** `cli_config` **Configuration backup** `cli_backup` **Configuration restore** `cli_restore` **Resource modules** `network_resource` **Reachability tests** `net_ping` **Copy files to devices** `net_put` **Copy files from devices** `net_get`

Use Case: Manage devices over NETCONF, RESTCONF and gRPC

Description	Fetch and change configuration and state data and run operations on devices that expose model-driven interfaces.
Collection	`ansible.netcommon` (Certified, Red Hat, v8.7.1, updated 2026-09-22)
Key content	**NETCONF data** `netconf_get` **NETCONF configuration** `netconf_config` **NETCONF operations** `netconf_rpc` **RESTCONF data** `restconf_get` **RESTCONF configuration** `restconf_config` **gRPC data** `grpc_get` **gRPC configuration** `grpc_config`

Use Case: Parse device output and prepare data

Description	Parse CLI and XML output, work with VLAN lists, hash passwords and edit ACL entries in playbooks.

Collection	`ansible.netcommon` (Certified, Red Hat, v8.7.1, updated 2026-09-22)
Key content	**CLI parsing** filter `parse_cli` **TextFSM parsing** filter `parse_cli_textfsm` **XML parsing** filter `parse_xml` **VLAN expansion** filter `vlan_expander` **VLAN parsing** filter `vlan_parser` **ACL entry removal** filter `pop_ace` **Type 5 password comparison** filter `comp_type5` **Type 5 password hash** filter `type5_pw` **Hash salt** filter `hash_salt`

SNMP `ansible.snmp`

Certified Red Hat · version 3.1.0 · updated 2026-03-31 · 3 modules → 1 use cases

Use Case: Query and set values on devices over SNMP

Description	Run SNMP get, set and walk operations on OIDs using SNMP v1, v2c or v3 USM connections.
Collection	`ansible.snmp` (Certified, Red Hat, v3.1.0, updated 2026-03-31)
Key content	**SNMP get** `get` **SNMP set** `set` **SNMP walk** `walk` **SNMP v1 connection** connection `v1` **SNMP v2c connection** connection `v2c` **SNMP v3 USM connection** connection `v3_usm` **Interface loop test playbook** playbook `int_looper` **Site test playbook** playbook `site`

Ansible utilities `ansible.utils`

Certified Red Hat · version 6.1.1 · updated 2026-09-17 · 4 modules → 6 use cases

Use Case: Parse, compare and validate data

Description	Parse CLI output into structured data, validate data against criteria, and compare or update facts.
Collection	`ansible.utils` (Certified, Red Hat, v6.1.1, updated 2026-09-17)
Key content	**CLI parsing** `cli_parse` **Data validation** `validate` **Fact comparison** `fact_diff` **Fact updates** `update_fact` **Validate filter** filter `validate` **Validate lookup** lookup `validate` **Validate test** test `validate` **Fact diff filter** filter `fact_diff`

Use Case: Navigate and reshape data structures

Description	Find values and indices in nested data, flatten it to paths, consolidate facts, add, keep, remove or rename keys, and convert between XML and JSON.
Collection	`ansible.utils` (Certified, Red Hat, v6.1.1, updated 2026-09-17)
Key content	**Get path filter** filter `get_path` **Get path lookup** lookup `get_path` **Index-of filter** filter `index_of` **Index-of lookup** lookup `index_of` **To-paths filter** filter `to_paths` **To-paths lookup** lookup `to_paths` **Consolidate facts** filter `consolidate` **Keep keys** filter `keep_keys` **Remove keys** filter `remove_keys` **Replace keys** filter `replace_keys` **Parameter list comparison** filter `param_list_compare` **XML to dictionary** filter `from_xml` **JSON to XML** filter `to_xml`

Use Case: Manipulate IP addresses and subnets

Description	Query, filter and convert IP addresses, subnets and ranges, merge CIDRs, and do host and subnet arithmetic.
Collection	`ansible.utils` (Certified, Red Hat, v6.1.1, updated 2026-09-17)
Key content	**IP address query** filter `ipaddr` **IPv4 filter** filter `ipv4` **IPv6 filter** filter `ipv6` **IPv6 formatting** filter `ipv6form` **IPv6 bracket wrapping** filter `ipwrap` **IPv4 to hex** filter `ip4_hex` **IP bit cutting** filter `ipcut` **IP arithmetic** filter `ipmath` **Subnet manipulation** filter `ipsubnet` **CIDR merge** filter `cidr_merge` **Nth host** filter `nthhost` **Next usable address** filter `next_nth_usable` **Previous usable address** filter `previous_nth_usable` **Usable range** filter `usable_range` **Network in network** filter `network_in_network` **Address usable in network** filter `network_in_usable` **Reduce on network** filter `reduce_on_network` **SLAAC address** filter `slaac`

Use Case: Handle MAC addresses

Description	Filter and format hardware (MAC) addresses.
Collection	`ansible.utils` (Certified, Red Hat, v6.1.1, updated 2026-09-17)
Key content	**Hardware address filter** filter `hwaddr` **MAC address filter** filter `macaddr` **MAC address test** test `mac`

Use Case: Test network membership and address type

Description	Test whether an IP or network is a subnet, supernet or member of one or more networks, and whether it is private, public, loopback, multicast or reserved.
Collection	`ansible.utils` (Certified, Red Hat, v6.1.1, updated 2026-09-17)
Key content	**In network** test `in_network` **In any network** test `in_any_network` **In exactly one network** test `in_one_network` **Subnet of** test `subnet_of` **Supernet of** test `supernet_of` **IP or network** test `ip` **IP address** test `ip_address` **Private address** test `private` **Public address** test `public` **Loopback address** test `loopback` **Multicast address** test `multicast` **Reserved address** test `reserved` **Unspecified address** test `unspecified` **Resolvable name or address** test `resolvable`

Use Case: Test IPv4 and IPv6 address formats

Description	Test whether a value is a valid IPv4 or IPv6 address, netmask, hostmask or special IPv6 form such as 6to4, Teredo or IPv4-mapped.
Collection	`ansible.utils` (Certified, Red Hat, v6.1.1, updated 2026-09-17)
Key content	**IPv4** test `ipv4` **IPv4 address** test `ipv4_address` **IPv4 hostmask** test `ipv4_hostmask` **IPv4 netmask** test `ipv4_netmask` **IPv6** test `ipv6` **IPv6 address** test `ipv6_address` **IPv4-mapped IPv6** test `ipv6_ipv4_mapped` **6to4 address** test `ipv6_sixtofour` **Teredo address** test `ipv6_teredo`

YANG and NETCONF `ansible.yang`

Certified Red Hat · version 3.1.0 · updated 2026-03-31 · 4 modules → 1 use cases

Use Case: Work with YANG models and NETCONF configuration

Description	Fetch YANG models, generate schemas and skeletons, read device configuration as JSON, push JSON configuration over NETCONF, and convert between JSON and XML.
Collection	`ansible.yang` (Certified, Red Hat, v3.1.0, updated 2026-03-31)

Key content	**Fetch YANG models** `fetch` **Generate model specifications** `generate_spec` **Get device configuration** `get` **Push configuration over NETCONF** `configure` **Model specification lookup** lookup `spec` **JSON to XML lookup** lookup `json2xml` **XML to JSON lookup** lookup `xml2json`

Arista AVD (EOS) `arista.avd`

Certified Arista · version 6.4.0 · updated 2026-09-18 · 9 modules → 5 use cases

Use Case: Design the network fabric

Description	Turn an abstracted data model into structured configuration and fabric documentation, and validate inputs against AVD schemas.
Collection	`arista.avd` (Certified, Arista, v6.4.0, updated 2026-09-18)
Key content	**Network designs** role `eos_designs` **eos_designs facts** `eos_designs_facts` **Structured configuration** `eos_designs_structured_config` **Fabric documentation** `eos_designs_documentation` **Input validation** `validate_inputs` **Variables as facts** `set_vars` **Python requirements check** `verify_requirements` **Output folders** role `build_output_folders`

Use Case: Generate EOS configuration

Description	Generate EOS CLI configuration and device documentation from the intended data model.
Collection	`arista.avd` (Certified, Arista, v6.4.0, updated 2026-09-18)
Key content	**EOS CLI configuration** role `eos_cli_config_gen` **EOS configuration generation** `eos_cli_config_gen`

Use Case: Deploy configuration

Description	Deploy intended configuration to devices over eAPI or to CloudVision, with tags and workflows.
Collection	`arista.avd` (Certified, Arista, v6.4.0, updated 2026-09-18)
Key content	**Deploy over eAPI** role `eos_config_deploy_eapi` **Deploy to CloudVision** role `cv_deploy` **CloudVision workflow** `cv_workflow` **CloudVision configlet upload (removed)** role `cvp_configlet_upload` **CloudVision deployment (removed)** role `eos_config_deploy_cvp` **DHCP provisioner (removed)** role `dhcp_provisioner`

Use Case: Validate and snapshot network state

Description	Run ANTA tests against EOS devices and collect show command snapshots and reports.
Collection	`arista.avd` (Certified, Arista, v6.4.0, updated 2026-09-18)
Key content	**ANTA runner** role `anta_runner` **ANTA workflow** `anta_workflow` **Show command snapshots** role `eos_snapshot` **State validation (removed)** role `eos_validate_state`

Use Case: Use AVD filters, tests and examples

Description	Helper filters and tests for sorting, hashing, encrypting passwords, expanding ranges and rendering Markdown, plus example installation.
Collection	`arista.avd` (Certified, Arista, v6.4.0, updated 2026-09-18)

Key content	**Markdown table of contents** filter `add_md_toc` **Decrypt passwords** filter `decrypt` **Encrypt passwords** filter `encrypt` **Default value** filter `default` **Hide passwords** filter `hide_passwords` **IP address sort** filter `ip_address_sort` **Hostname filter match** filter `is_in_filter` **List compression** filter `list_compress` **Natural sort** filter `natural_sort` **Range expansion** filter `range_expand` **Secure hash** filter `secure_hash` **SNMP passphrase hash** filter `snmp_hash` **Status rendering** filter `status_render` **Contains test** test `contains` **Defined test** test `defined` **Install examples** playbook `install_examples`

Arista CloudVision Portal `arista.cvp`

Certified Arista · version 3.12.0 · updated 2024-11-19 · 9 modules → 2 use cases

Use Case: Provision devices and configuration

Description	Manage configlets, the provisioning topology, containers and devices, images, tags and ZTP DHCP service.
Collection	`arista.cvp` (Certified, Arista, v3.12.0, updated 2024-11-19)
Key content	**Configlets** `cv_configlet_v3` **Containers** `cv_container_v3` **Devices** `cv_device_v3` **EOS images** `cv_image_v3` **Tags** `cv_tag_v3` **Configlet synchronization** role `configlets_sync` **ZTP DHCP service** role `dhcp_configuration`

Use Case: Control changes and validate

Description	Manage change controls, execute or cancel tasks, validate configlets and collect facts.
Collection	`arista.cvp` (Certified, Arista, v3.12.0, updated 2024-11-19)
Key content	**Change controls** `cv_change_control_v3` **Tasks** `cv_task_v3` **Configlet validation** `cv_validate_v3` **CloudVision facts** `cv_facts_v3`

Arista EOS `arista.eos`

Certified Arista · version 12.3.0 · updated 2026-09-17 · 33 modules → 6 use cases

Use Case: Configure Layer 2 interfaces and VLANs

Description	Manage interfaces, VLANs, L2 and L3 interface settings, LAG and LACP.
Collection	`arista.eos` (Certified, Arista, v12.3.0, updated 2026-09-17)
Key content	**Interfaces** `eos_interfaces` **Layer 2 interfaces** `eos_l2_interfaces` **Layer 3 interfaces** `eos_l3_interfaces` **VLANs** `eos_vlans` **LAG interfaces** `eos_lag_interfaces` **LACP** `eos_lacp` **LACP interfaces** `eos_lacp_interfaces`
Notes	• Module names have the eos_ prefix omitted.

Use Case: Configure routing: static routes, BGP, OSPF and VRFs

Description	Manage static routes, BGP, OSPFv2 and OSPFv3, prefix lists, route maps and VRFs.
Collection	`arista.eos` (Certified, Arista, v12.3.0, updated 2026-09-17)

Key content	**Static routes** `eos_static_routes` **BGP global** `eos_bgp_global` **BGP address families** `eos_bgp_address_family` **OSPFv2** `eos_ospfv2` **OSPFv3** `eos_ospfv3` **OSPF interfaces** `eos_ospf_interfaces` **Prefix lists** `eos_prefix_lists` **Route maps** `eos_route_maps` **VRFs** `eos_vrf` **VRF definitions** `eos_vrf_global`
Notes	• Module names have the eos_ prefix omitted.

Use Case: Apply access control lists

Description	Define ACLs and bind them to interfaces.
Collection	`arista.eos` (Certified, Arista, v12.3.0, updated 2026-09-17)
Key content	**ACLs** `eos_acls` **ACL interface bindings** `eos_acl_interfaces`
Notes	• Module names have the eos_ prefix omitted.

Use Case: Configure discovery, logging, time and monitoring

Description	Manage LLDP, logging, NTP and SNMP.
Collection	`arista.eos` (Certified, Arista, v12.3.0, updated 2026-09-17)
Key content	**LLDP** `eos_lldp` **LLDP global** `eos_lldp_global` **LLDP interfaces** `eos_lldp_interfaces` **Logging** `eos_logging_global` **NTP** `eos_ntp_global` **SNMP server** `eos_snmp_server`
Notes	• Module names have the eos_ prefix omitted.

Use Case: Administer devices: users, banners and management access

Description	Set hostname and system attributes, local users, banners and eAPI.
Collection	`arista.eos` (Certified, Arista, v12.3.0, updated 2026-09-17)
Key content	**Hostname** `eos_hostname` **System attributes** `eos_system` **Local users** `eos_user` **Banners** `eos_banner` **eAPI** `eos_eapi`
Notes	• Module names have the eos_ prefix omitted.

Use Case: Run commands, push configuration and gather facts

Description	Run arbitrary commands, manage configuration sections and collect device facts.
Collection	`arista.eos` (Certified, Arista, v12.3.0, updated 2026-09-17)
Key content	**Commands** `eos_command` **Configuration sections** `eos_config` **Facts** `eos_facts`
Notes	• Module names have the eos_ prefix omitted.

Aruba AOS-CX `arubanetworks.aoscx`

Certified Aruba · version 4.4.0 · updated 2024-04-30 · 40 modules → 5 use cases

Use Case: Configure interfaces, VLANs and Layer 2

Description	Configure physical, Layer 2, Layer 3, LAG and VLAN interfaces, VLANs, static MACs, PoE and VSX.
Collection	`arubanetworks.aoscx` (Certified, Aruba, v4.4.0, updated 2024-04-30)

Key content	**Interfaces** `aoscx_interface` **Layer 2 interfaces** `aoscx_l2_interface` **Layer 3 interfaces** `aoscx_l3_interface` **LAG interfaces** `aoscx_lag_interface` **VLANs** `aoscx_vlan` **VLAN interfaces** `aoscx_vlan_interface` **Static MACs** `aoscx_static_mac` **MAC table** `aoscx_mac` **Power over Ethernet** `aoscx_poe` **VSX** `aoscx_vsx`

Use Case: Configure routing

Description	Configure VRFs, static routes, and OSPFv2 and OSPFv3 areas, routers, interfaces and virtual links.
Collection	`arubanetworks.aoscx` (Certified, Aruba, v4.4.0, updated 2024-04-30)
Key content	**VRFs** `aoscx_vrf` **Static routes** `aoscx_static_route` **OSPF routers** `aoscx_ospf_router` **OSPF areas** `aoscx_ospf_area` **OSPF interfaces** `aoscx_ospf_interface` **OSPF virtual links** `aoscx_ospf_vlink` **OSPFv3 routers** `aoscx_ospfv3_router` **OSPFv3 areas** `aoscx_ospfv3_area` **OSPFv3 virtual links** `aoscx_ospfv3_vlink`

Use Case: Manage access control lists and object groups

Description	Create ACLs and object groups and apply ACLs to interfaces and VLANs.
Collection	`arubanetworks.aoscx` (Certified, Aruba, v4.4.0, updated 2024-04-30)
Key content	**ACLs** `aoscx_acl` **ACLs on interfaces** `aoscx_acl_interface` **ACLs on VLANs** `aoscx_acl_vlan` **Object groups** `aoscx_object_group`

Use Case: Configure quality of service

Description	Configure QoS schedule profiles, COS and DSCP trust modes, queues and queue profiles.
Collection	`arubanetworks.aoscx` (Certified, Aruba, v4.4.0, updated 2024-04-30)
Key content	**QoS schedule profiles** `aoscx_qos` **COS trust modes** `aoscx_qos_cos` **DSCP trust modes** `aoscx_qos_dscp` **Queues** `aoscx_queue` **Queue profiles** `aoscx_queue_profile` **Queue profile entries** `aoscx_queue_profile_entry`

Use Case: Administer the switch, firmware and configuration

Description	Set global configuration, banners and DNS, run commands, back up, upload and checkpoint configuration, upload and boot firmware, and gather facts.
Collection	`arubanetworks.aoscx` (Certified, Aruba, v4.4.0, updated 2024-04-30)
Key content	**Global system settings** `aoscx_system` **Banners** `aoscx_banner` **DNS client** `aoscx_dns` **CLI commands** `aoscx_command` **Configuration commands** `aoscx_config` **Device facts** `aoscx_facts` **Configuration backup** `aoscx_backup_config` **Configuration upload** `aoscx_upload_config` **Configuration checkpoints** `aoscx_checkpoint` **Firmware upload** `aoscx_upload_firmware` **Firmware boot** `aoscx_boot_firmware` **AOS-CX connection** connection `aoscx`

Cisco ACI `cisco.aci`

Certified Cisco · version 2.13.0 · updated 2025-12-09 · 287 modules → 24 use cases

Use Case: Manage tenants, application profiles and EPGs

Description	Create tenants, application profiles and endpoint groups, bind EPGs to domains and static paths, and tag endpoints.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Application profiles** `aci_ap` **Bulk static path bindings to EPGs** `aci_bulk_static_binding_to_epg` **Endpoint IP tags** `aci_endpoint_tag_ip` **Endpoint MAC tags** `aci_endpoint_tag_mac` **Endpoint groups (EPGs)** `aci_epg` **EPG monitoring policies** `aci_epg_monitoring_policy` **EPG Subnets** `aci_epg_subnet` **EPG to contract bindings** `aci_epg_to_contract` **EPG to consumed contract interface bindings** `aci_epg_to_contract_interface` **EPG contract masters** `aci_epg_to_contract_master` **EPG to domain bindings** `aci_epg_to_domain` **uSeg attribute block statements** `aci_epg_useg_attribute_block_statement` **uSeg attribute simple statements** `aci_epg_useg_attribute_simple_statement` **Static path bindings to EPGs** `aci_static_binding_to_epg` **Object tags** `aci_tag` **Tenants** `aci_tenant` **Endpoint retention policies** `aci_tenant_ep_retention_policy`

Use Case: Manage endpoint security groups

Description	Create ESGs, define their EPG, IP subnet and tag selectors, and attach contracts.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Endpoint security groups (ESGs)** `aci_esg` **ESG contract masters** `aci_esg_contract_master` **ESG EPG selectors** `aci_esg_epg_selector` **ESG IP subnet selectors** `aci_esg_ip_subnet_selector` **ESG tag selectors** `aci_esg_tag_selector` **ESG to contract bindings** `aci_esg_to_contract`

Use Case: Manage contracts, filters and policy enforcement

Description	Define contracts, subjects and filters, and bind them to EPGs, L2/L3 external EPGs, management EPGs and vzAny.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Contract resources** `aci_contract` **Contract interfaces (exports)** `aci_contract_export` **Contract subjects** `aci_contract_subject` **Subject to filter bindings** `aci_contract_subject_to_filter` **Subject to service graph bindings** `aci_contract_subject_to_service_graph` **Filters** `aci_filter` **Filter entries** `aci_filter_entry` **L2Out external EPG contract bindings** `aci_l2out_extepg_to_contract` **L3Out external EPG contract bindings** `aci_l3out_extepg_to_contract` **Management network profile contract bindings** `aci_management_network_instance_profile_to_contract` **Node management EPG contract bindings** `aci_node_mgmt_epg_to_contract` **Out-of-band contracts** `aci_oob_contract` **Subject Labels** `aci_subject_label` **Taboo contracts** `aci_taboo_contract` **vzAny contract bindings** `aci_vzany_to_contract`

Use Case: Manage VRFs, bridge domains and DHCP relay

Description	Create VRFs and bridge domains with their subnets, L3Out and NetFlow bindings, and configure DHCP relay and option policies.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Bridge domains** `aci_bd` **Bridge domain DHCP labels** `aci_bd_dhcp_label` **Rogue exception MACs** `aci_bd_rogue_exception_mac` **Bridge domain subnets** `aci_bd_subnet` **Bridge domain to L3Out bindings** `aci_bd_to_l3out` **Bridge domain NetFlow monitor bindings** `aci_bd_to_netflow_monitor_policy` **DHCP options** `aci_dhcp_option` **DHCP option policies** `aci_dhcp_option_policy` **DHCP relay policies** `aci_dhcp_relay` **DHCP relay providers** `aci_dhcp_relay_provider` **VRFs** `aci_vrf` **VRF fallback route groups** `aci_vrf_fallback_route_group` **VRF leaked subnets** `aci_vrf_leak_internal_subnet`

Use Case: Connect external networks with L3Outs

Description	Create L3Outs with their node and interface profiles, external EPGs and subnets, floating SVIs and static routes.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Fabric external connection profiles** `aci_fabric_external_connection_profile` **Fabric external routing profiles** `aci_fabric_external_routing_profile` **L3Outs** `aci_l3out` **L3Out DHCP relay labels** `aci_l3out_dhcp_relay_label` **L3Out external EPGs** `aci_l3out_extepg` **External subnets** `aci_l3out_extsubnet` **Floating SVIs** `aci_l3out_floating_svi` **Floating SVI paths** `aci_l3out_floating_svi_path` **Floating SVI path secondary IPs** `aci_l3out_floating_svi_path_secondary_ip` **Floating SVI secondary IPs** `aci_l3out_floating_svi_secondary_ip` **L3Out interfaces** `aci_l3out_interface` **L3Out interface secondary IPs** `aci_l3out_interface_secondary_ip` **Logical interface profiles** `aci_l3out_logical_interface_profile` **vPC member nodes** `aci_l3out_logical_interface_vpc_member` **Logical nodes** `aci_l3out_logical_node` **Logical node profiles** `aci_l3out_logical_node_profile` **Route tag policies** `aci_l3out_route_tag_policy` **Static routes** `aci_l3out_static_routes` **Static route next hops** `aci_l3out_static_routes_nexthop` **SR-MPLS infra L3Out bindings** `aci_l3out_to_sr_mpls_infra_l3out`

Use Case: Configure L3Out routing protocols (BGP, OSPF, EIGRP, HSRP, BFD)

Description	Configure BGP peers, OSPF and EIGRP interface profiles, HSRP groups and BFD profiles for L3Outs, plus the matching interface policies.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**BFD multihop node policies** `aci_bfd_multihop_node_policy` **BFD interface policies** `aci_interface_policy_bfd` **BFD multihop interface policies** `aci_interface_policy_bfd_multihop` **EIGRP interface policies** `aci_interface_policy_eigrp` **HSRP interface policies** `aci_interface_policy_hsrp` **OSPF interface policies** `aci_interface_policy_ospf` **L3Out BFD interface profiles** `aci_l3out_bfd_interface_profile` **L3Out BFD multihop interface profiles** `aci_l3out_bfd_multihop_interface_profile` **L3Out BGP peers** `aci_l3out_bgp_peer` **L3Out BGP protocol profiles** `aci_l3out_bgp_protocol_profile` **L3Out EIGRP interface profiles** `aci_l3out_eigrp_interface_profile` **HSRP groups** `aci_l3out_hsrp_group` **L3Out HSRP interface profiles** `aci_l3out_hsrp_interface_profile` **HSRP secondary virtual IPs** `aci_l3out_hsrp_secondary_vip` **L3Out OSPF interface profiles** `aci_l3out_logical_interface_profile_ospf_policy`

Use Case: Control routing with BGP policies and route maps

Description	Define BGP policies and route reflectors, route control profiles, match rules and action rules, and routing authentication keys.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Additional community action rules** `aci_action_rule_additional_communities` **AS path action rules** `aci_action_rule_set_as_path` **AS path ASNs** `aci_action_rule_set_as_path_asn` **BGP address family context policies** `aci_bgp_address_family_context_policy` **BGP best path policies** `aci_bgp_best_path_policy` **BGP peer prefix policies** `aci_bgp_peer_prefix_policy` **BGP route summarization policies** `aci_bgp_route_summarization_policy` **BGP route reflector ASN** `aci_bgp_rr_asn` **BGP route reflector nodes** `aci_bgp_rr_node` **BGP timers policies** `aci_bgp_timers_policy` **Key policies** `aci_key_policy` **Keychain policies** `aci_keychain_policy` **Match AS path regex terms** `aci_match_as_path_regex_term` **Match community factors** `aci_match_community_factor` **Match community regex terms** `aci_match_community_regex_term` **Match community terms** `aci_match_community_term` **Match route destinations** `aci_match_route_destination` **Match rules** `aci_match_rule` **Route control contexts** `aci_route_control_context` **Route control profiles** `aci_route_control_profile` **Action rule profiles** `aci_tenant_action_rule_profile`

Use Case: Configure multicast (PIM and IGMP)

Description	Configure PIM interface and route map policies, IGMP interface policies and VRF multicast.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**IGMP interface policies** `aci_igmp_interface_policy` **PIM interface policies** `aci_interface_policy_pim` **PIM route map entries** `aci_pim_route_map_entry` **PIM route map policies** `aci_pim_route_map_policy` **VRF multicast** `aci_vrf_multicast`

Use Case: Connect Layer 2 external networks with L2Outs

Description	Create L2Outs with their node, interface and path profiles and external EPGs.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)

Key content	**L2Outs** `aci_l2out` **L2Out external EPGs** `aci_l2out_extepg` **L2Out logical interface paths** `aci_l2out_logical_interface_path` **L2Out logical interface profiles** `aci_l2out_logical_interface_profile` **L2Out logical node profiles** `aci_l2out_logical_node_profile`

Use Case: Insert L4-L7 services and service graphs

Description	Register L4-L7 devices and interfaces, build service graph templates, and configure device selection and policy-based redirect.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Concrete devices** `aci_l4l7_concrete_device` **Concrete interfaces** `aci_l4l7_concrete_interface` **Concrete interface attachments** `aci_l4l7_concrete_interface_attachment` **L4-L7 devices** `aci_l4l7_device` **Device selection interface contexts** `aci_l4l7_device_selection_interface_context` **Device selection policies** `aci_l4l7_device_selection_policy` **Logical interfaces** `aci_l4l7_logical_interface` **Policy-based redirect policies** `aci_l4l7_policy_based_redirect` **Policy-based redirect destinations** `aci_l4l7_policy_based_redirect_destination` **Redirect health groups** `aci_l4l7_redirect_health_group` **Service graph templates** `aci_l4l7_service_graph_template` **Service graph connections** `aci_l4l7_service_graph_template_connection` **Service graph connector connections** `aci_l4l7_service_graph_template_connection_to_connector` **Service graph functional connections** `aci_l4l7_service_graph_template_functional_connection` **Service graph nodes** `aci_l4l7_service_graph_template_node` **Service graph terminal nodes** `aci_l4l7_service_graph_template_term_node`

Use Case: Manage domains, VLAN pools and AEPs

Description	Create physical, virtual, bridged and routed domains, VLAN/encap pools and ranges, and attachable access entity profiles with their bindings.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Attachable access entity profiles (AEPs)** `aci_aep` **AEP to domain bindings** `aci_aep_to_domain` **AEP to EPG bindings** `aci_aep_to_epg` **Physical, virtual, bridged, routed and FC domains** `aci_domain` **Domain to encap pool bindings** `aci_domain_to_encap_pool` **Domain to VLAN pool bindings** `aci_domain_to_vlan_pool` **Encap pools** `aci_encap_pool` **Encap pool ranges** `aci_encap_pool_range` **VLAN pools** `aci_vlan_pool` **VLAN pool encap blocks** `aci_vlan_pool_encap_block`

Use Case: Define interface policies and policy groups

Description	Configure CDP, LLDP, LACP, link-level, spanning-tree, storm-control and other interface policies and combine them into leaf and spine policy groups.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Access switch policy groups** `aci_access_switch_policy_group` **Fabric interface policy groups** `aci_fabric_interface_policy_group` **CDP interface policies** `aci_interface_policy_cdp` **Fibre Channel interface policies** `aci_interface_policy_fc` **Layer 2 interface policies** `aci_interface_policy_l2` **Leaf breakout port groups** `aci_interface_policy_leaf_breakout_port_group` **Leaf Fibre Channel policy groups** `aci_interface_policy_leaf_fc_policy_group` **Leaf interface policy groups** `aci_interface_policy_leaf_policy_group` **Leaf interface profiles** `aci_interface_policy_leaf_profile` **FEX policy groups** `aci_interface_policy_leaf_profile_fex_policy_group` **Link-level interface policies** `aci_interface_policy_link_level` **LLDP interface policies** `aci_interface_policy_lldp` **MCP interface policies** `aci_interface_policy_mcp` **Port channel policies** `aci_interface_policy_port_channel` **Port channel member policies** `aci_interface_policy_port_channel_member` **Port security policies** `aci_interface_policy_port_security` **Spanning tree policies** `aci_interface_policy_spanning_tree` **Spine interface policy groups** `aci_interface_policy_spine_policy_group` **Storm control policies** `aci_interface_policy_storm_control`

Use Case: Configure access ports, switch profiles and selectors

Description	Build leaf and spine interface and switch profiles, port selectors and blocks, vPC protection groups, and set port-level configuration and descriptions.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)

Key content	**Port blocks of access port selectors** `aci_access_port_block_to_access_port` **Leaf interface profile port selectors** `aci_access_port_to_interface_policy_leaf_profile` **Spine interface profiles** `aci_access_spine_interface_profile` **Spine interface to switch profile bindings** `aci_access_spine_interface_profile_to_spine_switch_profile` **Spine interface selectors** `aci_access_spine_interface_selector` **Spine switch profiles** `aci_access_spine_switch_profile` **Spine switch selectors** `aci_access_spine_switch_selector` **Sub port blocks of access port selectors** `aci_access_sub_port_block_to_access_port` **Interface enable and disable** `aci_interface_blacklist` **Access and fabric port configuration (ACI 5.2+)** `aci_interface_config` **Interface descriptions** `aci_interface_description` **Interface selector to switch profile bindings** `aci_interface_selector_to_switch_policy_leaf_profile` **Node blocks** `aci_node_block` **Switch access policy configuration** `aci_switch_access_config` **Leaf selectors** `aci_switch_leaf_selector` **Leaf switch profiles** `aci_switch_policy_leaf_profile` **vPC protection groups** `aci_switch_policy_vpc_protection_group`

Use Case: Build the fabric: nodes, pods and fabric policies

Description	Register and decommission fabric nodes, define pod, leaf and spine profiles and policy groups, and set fabric-wide settings.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Fabric leaf profiles** `aci_fabric_leaf_profile` **Leaf switch associations** `aci_fabric_leaf_switch_assoc` **Fabric node members** `aci_fabric_node` **Fabric node controls** `aci_fabric_node_control` **Node commissioning and decommissioning** `aci_fabric_node_decommission` **Pod setup policies** `aci_fabric_pod` **Pod connectivity profiles** `aci_fabric_pod_connectivity_profile` **Pod external TEPs** `aci_fabric_pod_external_tep` **Pod policy groups** `aci_fabric_pod_policy_group` **Pod profiles** `aci_fabric_pod_profile` **Pod remote pools** `aci_fabric_pod_remote_pool` **Pod selectors** `aci_fabric_pod_selector` **Fabric spine profiles** `aci_fabric_spine_profile` **Spine switch associations** `aci_fabric_spine_switch_assoc` **Switch blocks** `aci_fabric_switch_block` **Fabric switch policy groups** `aci_fabric_switch_policy_group` **Fabric-wide settings** `aci_fabric_wide_settings` **Switch fabric policy configuration** `aci_switch_fabric_config`

Use Case: Upgrade firmware and schedule maintenance

Description	Manage firmware sources, policies and groups, maintenance policies and groups, and the schedulers that trigger them.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Schedulers** `aci_fabric_scheduler` **Firmware groups** `aci_firmware_group` **Firmware group nodes** `aci_firmware_group_node` **Firmware policies** `aci_firmware_policy` **Firmware image sources** `aci_firmware_source` **Maintenance groups** `aci_maintenance_group` **Maintenance group nodes** `aci_maintenance_group_node` **Maintenance policies** `aci_maintenance_policy`

Use Case: Mirror and monitor traffic (SPAN, NetFlow, IP SLA)

Description	Configure access, fabric and tenant SPAN sessions, NetFlow exporters, monitors and records, and IP SLA monitoring policies.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Access SPAN destination groups** `aci_access_span_dst_group` **Access SPAN filter groups** `aci_access_span_filter_group` **Access SPAN filter group entries** `aci_access_span_filter_group_entry` **Access SPAN source groups** `aci_access_span_src_group` **Access SPAN sources** `aci_access_span_src_group_src` **Access SPAN source paths** `aci_access_span_src_group_src_path` **Fabric SPAN destination groups** `aci_fabric_span_dst_group` **Fabric SPAN source groups** `aci_fabric_span_src_group` **Fabric SPAN sources** `aci_fabric_span_src_group_src` **Fabric SPAN source nodes** `aci_fabric_span_src_group_src_node` **Fabric SPAN source paths** `aci_fabric_span_src_group_src_path` **IP SLA monitoring policies** `aci_ip_sla_monitoring_policy` **NetFlow exporter policies** `aci_netflow_exporter_policy` **NetFlow monitor policies** `aci_netflow_monitor_policy` **NetFlow monitor to exporter bindings** `aci_netflow_monitor_to_exporter` **NetFlow record policies** `aci_netflow_record_policy` **Tenant SPAN destination groups** `aci_tenant_span_dst_group` **Tenant SPAN source groups** `aci_tenant_span_src_group` **Tenant SPAN sources** `aci_tenant_span_src_group_src` **SPAN source to destination bindings** `aci_tenant_span_src_group_to_dst_group`

Use Case: Send SNMP and syslog telemetry

Description	Configure SNMP policies, clients and users, and syslog groups, sources and remote destinations.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**SNMP clients** `aci_snmp_client` **SNMP client groups** `aci_snmp_client_group` **SNMP community policies** `aci_snmp_community_policy` **SNMP policies** `aci_snmp_policy` **SNMP v3 users** `aci_snmp_user` **Syslog groups** `aci_syslog_group` **Syslog remote destinations** `aci_syslog_remote_dest` **Syslog sources** `aci_syslog_source`

Use Case: Administer users, roles and management security

Description	Manage AAA users, roles, domains, privileges, certificates and key rings, and secure the fabric management access and HTTPS ciphers.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Certificate authorities** `aci_aaa_certificate_authority` **RBAC custom privileges** `aci_aaa_custom_privilege` **Security domains** `aci_aaa_domain` **Key rings** `aci_aaa_key_ring` **Roles** `aci_aaa_role` **Security default settings** `aci_aaa_security_default_settings` **SSH authentication** `aci_aaa_ssh_auth` **Users** `aci_aaa_user` **User certificates** `aci_aaa_user_certificate` **User security domains** `aci_aaa_user_domain` **User roles** `aci_aaa_user_role` **Fabric management access** `aci_fabric_management_access` **HTTPS cipher configuration** `aci_fabric_management_access_https_cipher` **Global AES passphrase encryption** `aci_system_global_aes_passphrase_encryption`

Use Case: Configure management network, NTP, DNS and system settings

Description	Configure in-band and out-of-band management EPGs and addresses, NTP, DNS, banners and system-wide endpoint controls.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**DNS domains** `aci_dns_domain` **DNS profiles** `aci_dns_profile` **DNS providers** `aci_dns_provider` **External management network profiles** `aci_management_network_instance_profile` **In-band and out-of-band management EPGs** `aci_node_mgmt_epg` **NTP policies** `aci_ntp_policy` **NTP servers** `aci_ntp_server` **Static node management addresses** `aci_static_node_mgmt_address` **System information** `aci_system` **Alias and banners** `aci_system_banner` **APIC connectivity preferences** `aci_system_connectivity_preference` **Endpoint controls (IP aging, loop protection)** `aci_system_endpoint_controls`

Use Case: Apply quality of service policies

Description	Define custom QoS policies and map Dot1P and DSCP classes.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Custom QoS policies** `aci_qos_custom_policy` **Dot1P classes** `aci_qos_dot1p_class` **DSCP classes** `aci_qos_dscp_class`

Use Case: Back up, snapshot and roll back configuration

Description	Export configuration, take snapshots, roll back and preview changes, and define remote file paths.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Configuration export policies** `aci_config_export_policy` **Configuration rollback and preview** `aci_config_rollback` **Configuration snapshots** `aci_config_snapshot` **Remote file paths** `aci_file_remote_path`

Use Case: Extend ACI to public clouds with Cloud APIC

Description	Configure cloud context profiles, CIDRs, subnets, cloud EPGs and selectors, and cloud provider, region and VPN gateway objects.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)

Key content	**Cloud application profiles** `aci_cloud_ap` **AWS providers** `aci_cloud_aws_provider` **Cloud BGP ASN profiles** `aci_cloud_bgp_asn` **Cloud CIDRs** `aci_cloud_cidr` **Cloud context profiles** `aci_cloud_ctx_profile` **Cloud EPGs** `aci_cloud_epg` **Cloud endpoint selectors** `aci_cloud_epg_selector` **Cloud external EPGs** `aci_cloud_external_epg` **Cloud external endpoint selectors** `aci_cloud_external_epg_selector` **Cloud providers (query)** `aci_cloud_provider` **Cloud regions** `aci_cloud_region` **Cloud subnets** `aci_cloud_subnet` **Cloud VPN gateways (routers)** `aci_cloud_vpn_gateway` **Cloud availability zones** `aci_cloud_zone`

Use Case: Integrate with virtual machine managers (VMM)

Description	Configure VMM controllers, credentials, uplinks, vSwitch policies and enhanced LAG policies.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**VMM controllers** `aci_vmm_controller` **VMM credentials** `aci_vmm_credential` **Enhanced LAG policies** `aci_vmm_enhanced_lag_policy` **VMM uplinks** `aci_vmm_uplink` **VMM uplink containers** `aci_vmm_uplink_container` **vSwitch policies** `aci_vmm_vswitch_policy`

Use Case: Call the APIC API, inventory and helpers

Description	Send raw REST calls to the APIC, build Ansible inventory from ACI, expand interface ranges and flatten ACI data.
Collection	`cisco.aci` (Certified, Cisco, v2.13.0, updated 2025-12-09)
Key content	**Direct REST API access** `aci_rest` **listify filter (flatten ACI model data)** filter `listify` **ACI inventory plugin** inventory `aci_inventory_system` **Interface range lookup** lookup `interface_range`

Cisco Catalyst Center `cisco.catalystcenter`

Certified Cisco · version 2.10.2 · updated 2026-08-11 · 1551 modules → 35 use cases

Use Case: Generate playbook inputs from an existing Catalyst Center

Description	Export the current Catalyst Center configuration (sites, devices, credentials, policies, fabric, wireless and more) as playbook variable files to start automating a brownfield environment.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Accesspoint location playbook** `accesspoint_location_playbook_config_generator` **Accesspoint playbook** `accesspoint_playbook_config_generator` **Application policy playbook** `application_policy_playbook_config_generator` **Assurance device health score settings playbook** `assurance_device_health_score_settings_playbook_config_generator` **Assurance issue playbook** `assurance_issue_playbook_config_generator` **Backup and restore playbook** `backup_and_restore_playbook_config_generator` **Device credential playbook** `device_credential_playbook_config_generator` **Discovery playbook** `discovery_playbook_config_generator` **Events and notifications playbook** `events_and_notifications_playbook_config_generator` **Inventory playbook** `inventory_playbook_config_generator` **ISE radius integration playbook** `ise_radius_integration_playbook_config_generator` **Network profile switching playbook** `network_profile_switching_playbook_config_generator` **Network profile wireless playbook** `network_profile_wireless_playbook_config_generator` **Network settings playbook** `network_settings_playbook_config_generator` **Accesspoint location playbook config generator (playbook)** playbook `accesspoint_location_playbook_config_generator` **Accesspoint playbook config generator (playbook)** playbook `accesspoint_playbook_config_generator` **Application policy playbook config generator (playbook)** playbook `application_policy_playbook_config_generator` **Assurance device health score settings playbook config generator (playbook)** playbook `assurance_device_health_score_settings_playbook_config_generator` **Assurance issue playbook config generator (playbook)** playbook `assurance_issue_playbook_config_generator` **Backup and restore playbook config generator (playbook)** playbook `backup_and_restore_playbook_config_generator` **Device credential playbook config generator (playbook)** playbook `device_credential_playbook_config_generator` **Discovery playbook config generator (playbook)** playbook `discovery_playbook_config_generator` **Events and notifications playbook config generator (playbook)** playbook `events_and_notifications_playbook_config_generator` **Inventory playbook config generator (playbook)** playbook `inventory_playbook_config_generator` **ISE radius integration playbook config generator (playbook)** playbook `ise_radius_integration_playbook_config_generator` **Network profile switching playbook config generator (playbook)** playbook `network_profile_switching_playbook_config_generator` **Network profile wireless playbook config generator (playbook)** playbook `network_profile_wireless_playbook_config_generator` **Network settings playbook config generator (playbook)** playbook `network_settings_playbook_config_generator` **PnP playbook config generator (playbook)** playbook `pnp_playbook_config_generator` **Provision playbook config generator (playbook)** playbook `provision_playbook_config_generator` **RMA playbook config generator (playbook)** playbook `rma_playbook_config_generator` **SDA extranet policies playbook config generator (playbook)** playbook `sda_extranet_policies_playbook_config_generator` **SDA fabric devices playbook config generator (playbook)** playbook `sda_fabric_devices_playbook_config_generator` **SDA fabric multicast playbook config generator (playbook)** playbook `sda_fabric_multicast_playbook_config_generator` **SDA fabric sites zones playbook config generator (playbook)** playbook `sda_fabric_sites_zones_playbook_config_generator` **SDA fabric transits playbook config generator (playbook)** playbook `sda_fabric_transits_playbook_config_generator` **SDA fabric virtual networks playbook config generator (playbook)** playbook `sda_fabric_virtual_networks_playbook_config_generator` **SDA host port migration playbook config generator (playbook)** playbook `sda_host_port_migration_playbook_config_generator` **SDA host port onboarding playbook config generator (playbook)** playbook `sda_host_port_onboarding_playbook_config_generator` **Site playbook config generator (playbook)** playbook `site_playbook_config_generator` **Tags playbook config generator (playbook)** playbook `tags_playbook_config_generator` **Template playbook config generator (playbook)** playbook `template_playbook_config_generator` **User role playbook config generator (playbook)** playbook `user_role_playbook_config_generator` **Wired campus automation playbook config generator (playbook)** playbook `wired_campus_automation_playbook_config_generator` **Wireless design playbook config generator (playbook)** playbook `wireless_design_playbook_config_generator` **PnP playbook** `pnp_playbook_config_generator` **Provision playbook** `provision_playbook_config_generator` **RMA playbook** `rma_playbook_config_generator` **Accesspoint config generator (role)** role `accesspoint_config_generator` **Accesspoint location config generator (role)** role `accesspoint_location_config_generator` **Application policy config generator (role)** role `application_policy_config_generator` **Assurance device health score settings config generator (role)** role `assurance_device_health_score_settings_config_generator` **Assurance issue config generator (role)** role `assurance_issue_config_generator` **Backup and restore config generator (role)** role `backup_and_restore_config_generator` **Device credential config generator (role)** role `device_credential_config_generator` **Discovery config generator (role)** role `discovery_config_generator` **Events and notifications config generator (role)** role `events_and_notifications_config_generator` **Inventory config generator (role)** role `inventory_config_generator` **ISE radius integration config generator (role)** role `ise_radius_integration_config_generator` **Network profile switching config generator (role)** role `network_profile_switching_config_generator` **Network profile wireless config generator (role)** role `network_profile_wireless_config_generator` **Network settings config generator (role)** role `network_settings_config_generator` **PnP config generator (role)** role `pnp_config_generator` **Provision config generator (role)** role `provision_config_generator` **RMA config generator (role)** role `rma_config_generator` **SDA extranet policies config generator (role)** role `sda_extranet_policies_config_generator` **SDA fabric devices config generator (role)** role `sda_fabric_devices_config_generator` **SDA fabric multicast config generator (role)** role `sda_fabric_multicast_config_generator` **SDA fabric sites zones config generator (role)** role `sda_fabric_sites_zones_config_generator` **SDA fabric transits config generator (role)** role `sda_fabric_transits_config_generator` **SDA fabric virtual networks config generator (role)** role `sda_fabric_virtual_networks_config_generator` **SDA host port migration config generator (role)** role `sda_host_port_migration_config_generator` **SDA host port onboarding config generator (role)** role `sda_host_port_onboarding_config_generator` **Site config generator (role)** role `site_config_generator` **Tags config generator (role)** role `tags_config_generator` **Template config generator (role)** role `template_config_generator` **User role config generator (role)** role `user_role_config_generator` **Wired campus automation config generator (role)** role `wired_campus_automation_config_generator` **Wireless design config generator (role)** role `wireless_design_config_generator` **SDA extranet policies playbook** `sda_extranet_policies_playbook_config_generator` **SDA fabric devices playbook** `sda_fabric_devices_playbook_config_generator` **SDA fabric multicast playbook** `sda_fabric_multicast_playbook_config_generator` **SDA fabric sites zones playbook** `sda_fabric_sites_zones_playbook_config_generator` **SDA fabric transits playbook** `sda_fabric_transits_playbook_config_generator` **SDA fabric virtual networks playbook** `sda_fabric_virtual_networks_playbook_config_generator` **SDA host port migration playbook** `sda_host_port_migration_playbook_config_generator` **SDA host port onboarding playbook** `sda_host_port_onboarding_playbook_config_generator` **Site playbook** `site_playbook_config_generator` **Tags playbook** `tags_playbook_config_generator` **Template playbook** `template_playbook_config_generator` **User role playbook** `user_role_playbook_config_generator` **Wired campus automation playbook** `wired_campus_automation_playbook_config_generator` **Wireless design playbook** `wireless_design_playbook_config_generator`

Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Design the site hierarchy, floors and maps

Description	Create areas, buildings and floors, import and export floor maps, place planned access points and connect location servers such as CMX and Spaces.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Areas** `areas`, `areas_info` **Assign device to site** `assign_device_to_site` **Assign to site apply** `assign_to_site_apply` **Buildings** `buildings`, `buildings_info` **Buildings planned access points** `buildings_planned_access_points_info` **Floors** `floors`, `floors_info` **Floors floor ID access point positions** `floors_floor_id_access_point_positions_info` **Floors floor ID access point positions bulk change** `floors_floor_id_access_point_positions_bulk_change` **Floors floor ID access point positions count** `floors_floor_id_access_point_positions_count_info` **Floors floor ID planned access point positions** `floors_floor_id_planned_access_point_positions_info` **Floors floor ID planned access point positions assign access point positions** `floors_floor_id_planned_access_point_positions_assign_access_point_positions` **Floors floor ID planned access point positions bulk** `floors_floor_id_planned_access_point_positions_bulk` **Floors floor ID planned access point positions bulk change** `floors_floor_id_planned_access_point_positions_bulk_change` **Floors floor ID planned access point positions count** `floors_floor_id_planned_access_point_positions_count_info` **Floors floor ID planned access point positions ID** `floors_floor_id_planned_access_point_positions_id` **Floors settings** `floors_settings`, `floors_settings_info` **Floors upload image** `floors_upload_image` **Location servers CMX servers** `location_servers_cmx_servers`, `location_servers_cmx_servers_info` **Location servers CMX servers certificate** `location_servers_cmx_servers_certificate_info` **Location servers CMX servers count** `location_servers_cmx_servers_count_info` **Location servers spaces** `location_servers_spaces`, `location_servers_spaces_info` **Location servers spaces accounts** `location_servers_spaces_accounts_info` **Location servers spaces accounts count** `location_servers_spaces_accounts_count_info` **Location servers spaces activate via CCO credentials with existing account create** `location_servers_spaces_activate_via_cco_credentials_with_existing_account_create` **Location servers spaces activate via CCO credentials with new ac** `location_servers_spaces_activate_via_cco_credentials_with_ne` **Location servers spaces activate via one time token create** `location_servers_spaces_activate_via_one_time_token_create` **Location servers spaces regions** `location_servers_spaces_regio` **Location servers spaces regions count** `location_servers_spaces_` **Maps export** `maps_export` **Maps import** `maps_import` **Maps import perform** `maps_import_perform` **Maps import start** `maps_import_start` **Maps import status** `maps_import_status_info` **Maps supported access points** `maps_supported_access_points_in` **Network devices assigned to site** `network_devices_assigned_to_s` **Network devices assigned to site count** `network_devices_assigne` **Network devices assigned to site ID** `network_devices_assigned_t` **Network devices not assigned to site** `network_devices_not_assig` **Network devices not assigned to site count** `network_devices_not_assigned_to_site_count_info` **Network devices unassign from site apply** `network_devices_unass` **Planned access points** `planned_access_points`, `planned_access_p` **Site workflow manager (playbook)** playbook `site_workflow_manage` **Site (role)** role `site` **Site** `site`, `site_info` **Site count** `site_count_info` **Site count v2** `site_count_v2_info` **Site delete** `site_delete` **Site membership** `site_membership_info` **Site update** `site_update` **Site v2** `site_v2_info` **Site workflow manager** `site_workflow_manager` **Sites** `sites_info` **Sites bulk** `sites_bulk` **Sites count** `sites_count_info` **Topology site** `topology_site_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the recommended declarative way to manage a feature.

Use Case: Configure network settings and IP address pools

Description	Define global and per-site network settings (AAA, DHCP, DNS, NTP, banner, time zone, telemetry), global and site IP pools, IPAM integration and SNMP propert
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Global pool** `global_pool`, `global_pool_info` **IPAM global IP address pools** `ipam_global_ip_address_pools`, `ipam_global_ip_address_pools_info` **IPAM global IP address pools count** `ipam_global_ip_address_pools_count_info` **IPAM global IP address pools global IP address pool ID subpools** `ipam_global_ip_address_pools_global_ip_address_pool_id_subpools_info` **IPAM global IP address pools global IP address pool ID subpools count** `ipam_global_ip_address_pools_global_ip_address_pool_id_subpools_count_info` **IPAM server setting** `ipam_server_setting`, `ipam_server_setting_info` **IPAM site IP address pools** `ipam_site_ip_address_pools`, `ipam_site_ip_address_pools_info` **IPAM site IP address pools count** `ipam_site_ip_address_pools_count_info` **Network** `network_info` **Network create** `network_create` **Network settings workflow manager** `network_settings_workflow_manager` **Network update** `network_update` **Network v2** `network_v2`, `network_v2_info` **Global pool (playbook)** playbook `global_pool` **Network settings workflow manager (playbook)** playbook `network_settings_workflow_manager` **Reserve IP subpool (playbook)** playbook `reserve_ip_subpool` **Reserve IP subpool** `reserve_ip_subpool`, `reserve_ip_subpool_info` **Network settings (role)** role `network_settings` **Sites AAA settings** `sites_aaa_settings`, `sites_aaa_settings_info` **Sites banner settings** `sites_banner_settings`, `sites_banner_settings_inf` **Sites DHCP settings** `sites_dhcp_settings`, `sites_dhcp_settings_info` **Sites DNS settings** `sites_dns_settings`, `sites_dns_settings_info` **Sites image distribution settings** `sites_image_distribution_settings`, `sites_image_distribution_settings_info` **Sites location server settings** `sites_location_server_settings`, `sites_location_server_settings_info` **Sites NTP settings** `sites_ntp_settings`, `sites_ntp_settings_info` **Sites telemetry settings** `sites_telemetry_settings`, `sites_telemetry_settings_info` **Sites time zone settings** `sites_time_zone_settings`, `sites_time_zone_settings_info` **SNMP properties** `snmp_properties`, `snmp_properties_info` **Telemetry settings apply** `telemetry_settings_apply`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Manage device credentials and certificates

Description	Create CLI, SNMP, HTTP and NETCONF credentials, assign them to sites, and import trusted and authentication certificates.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Authentication import certificate** `authentication_import_certificate` **Authentication import certificate PKCS12** `authentication_import_certificate_p12` **CLI credential** `cli_credential` **Credential to site by siteid create v2** `credential_to_site_by_siteid_create_v2` **Device credential** `device_credential`, `device_credential_info` **Device credential workflow manager** `device_credential_workflow_manager` **Global credential** `global_credential_info` **Global credential delete** `global_credential_delete` **Global credential update** `global_credential_update` **Global credential v2** `global_credential_v2`, `global_credential_v2_info` **Global credentials** `global_credentials`, `global_credentials_info` **Global credentials count** `global_credentials_count_info` **HTTP read credential** `http_read_credential` **HTTP write credential** `http_write_credential` **NETCONF credential** `netconf_credential` **Authentication import (playbook)** playbook `authentication_import` **Device credential workflow manager (playbook)** playbook `device_credential_workflow_manager` **Global credentials (playbook)** playbook `global_credentials` **Global credentials info (playbook)** playbook `global_credentials_info` **Device credential (role)** role `device_credential` **Site assign credential** `site_assign_credential` **Sites device credentials** `sites_device_credentials`, `sites_device_credentials_info` **Sites device credentials apply** `sites_device_credentials_apply` **Sites device credentials status** `sites_device_credentials_status_info` **SNMPv2 read community credential** `snmpv2_read_community_credential` **SNMPv2 write community credential** `snmpv2_write_community_credential` **SNMPv3 credential** `snmpv3_credential` **Trusted certificates import** `trusted_certificates_import`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Manage administrators, roles and ISE integration

Description	Manage users, roles and permissions, external authentication servers, ISE integration and API authentication tokens.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Access groups** `access_groups`, `access_groups_info` **Access groups count** `access_groups_count_info` **Auth token create** `auth_token_create` **Authentication policy servers** `authentication_policy_servers`, `authentication_policy_servers_info` **Integrate ISE** `integrate_ise` **ISE integration status** `ise_integration_status_info` **ISE radius integration workflow manager** `ise_radius_integration_workflow_manager` **ISE radius integration workflow manager (playbook)** playbook `ise_radius_integration_workflow_manager` **User info (playbook)** playbook `user_info` **User role workflow manager (playbook)** playbook `user_role_workflow_manager` **ISE radius integration (role)** role `ise_radius_integration` **User role (role)** role `user_role` **Role permissions** `role_permissions_info` **Roles** `roles`, `roles_info` **Roles permissions** `roles_permissions_info` **Roles v2** `roles_v2`, `roles_v2_info` **User** `user`, `user_info` **User role workflow manager** `user_role_workflow_manager` **Users external authentication** `users_external_authentication`, `users_external_authentication_info` **Users external servers** `users_external_servers_info` **Users external servers AAA attribute** `users_external_servers_aaa_attribute`, `users_external_servers_aaa_attribute_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Discover devices and onboard them with Plug and Play

Description	Run discovery jobs and ranges, and add, claim, authorise and reset devices through Plug and Play workflows and virtual accounts.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Discovery** `discovery`, `discovery_info` **Discovery count** `discovery_count_info` **Discovery device** `discovery_device_info` **Discovery device count** `discovery_device_count_info` **Discovery device range** `discovery_device_range_info` **Discovery job** `discovery_job_info` **Discovery range** `discovery_range_info` **Discovery range delete** `discovery_range_delete` **Discovery summary** `discovery_summary_info` **Discovery workflow manager** `discovery_workflow_manager` **Discoverys** `discoverys`, `discoverys_info` **Discoverys count** `discoverys_count_info` **Discoverys jobs** `discoverys_jobs`, `discoverys_jobs_info` **Discoverys jobs count** `discoverys_jobs_count_info` **Discoverys jobs discovered network devices** `discoverys_jobs_discovered_network_devices_info` **Discoverys jobs discovered network devices count** `discoverys_jobs_discovered_network_devices_count_info` **Discoverys jobs stop create** `discoverys_jobs_stop_create` **Discoverys jobs summarys** `discoverys_jobs_summarys_info` **PnP Workflow Manager Playbook (playbook)** playbook `PnP_Workflow_Manager_Playbook` **Discovery (playbook)** playbook `discovery` **Discovery workflow manager (playbook)** playbook `discovery_workflow_manager` **PnP device (playbook)** playbook `pnp_device` **PnP workflow manager (playbook)** playbook `pnp_workflow_manager` **PnP device** `pnp_device`, `pnp_device_info` **PnP device authorize** `pnp_device_authorize` **PnP device claim** `pnp_device_claim` **PnP device claim to site** `pnp_device_claim_to_site` **PnP device config preview** `pnp_device_config_preview` **PnP device count** `pnp_device_count_info` **PnP device history** `pnp_device_history_info` **PnP device import** `pnp_device_import` **PnP device reset** `pnp_device_reset` **PnP device unclaim** `pnp_device_unclaim` **PnP global settings** `pnp_global_settings`, `pnp_global_settings_info` **PnP network devices** `pnp_network_devices`, `pnp_network_devices_info` **PnP network devices claim create** `pnp_network_devices_claim_create` **PnP network devices count** `pnp_network_devices_count_info` **PnP server profile update** `pnp_server_profile_update` **PnP smart account domains** `pnp_smart_account_domains_info` **PnP virtual account add** `pnp_virtual_account_add` **PnP virtual account deregister** `pnp_virtual_account_deregister` **PnP virtual account devices sync** `pnp_virtual_account_devices_sync` **PnP virtual account sync result** `pnp_virtual_account_sync_result_info` **PnP virtual accounts** `pnp_virtual_accounts_info` **PnP workflow** `pnp_workflow`, `pnp_workflow_info` **PnP workflow count** `pnp_workflow_count_info` **PnP workflow manager** `pnp_workflow_manager` **Discovery (role)** role `discovery` **PnP (role)** role `pnp`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Manage device inventory, interfaces and topology

Description	Query, add, update, export, resync and delete network devices, and read their interfaces, modules, VLANs, chassis details and Layer 2/3 topology.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Device details** `device_details_info` **Device family identifiers details** `device_family_identifiers_details_info` **Device interface** `device_interface_info` **Device interface by IP** `device_interface_by_ip_info` **Device interface count** `device_interface_count_info` **Device interface isis** `device_interface_isis_info` **Device interface ospf** `device_interface_ospf_info` **Intent network devices query** `intent_network_devices_query` **Intent network devices query count** `intent_network_devices_query_count` **Interface** `interface_info` **Interface network device** `interface_network_device_info` **Interface network device detail** `interface_network_device_detail_info` **Interface network device range** `interface_network_device_range_info` **Interface operation create** `interface_operation_create` **Interface update** `interface_update` **Catalyst Center dynamic inventory** inventory `catalystcenter` **Inventory workflow manager** `inventory_workflow_manager` **Network device** `network_device`, `network_device_info` **Network device by IP** `network_device_by_ip_info` **Network device by serial number** `network_device_by_serial_number_info` **Network device chassis details** `network_device_chassis_details_info` **Network device count** `network_device_count_info` **Network device equipment** `network_device_equipment_info` **Network device export** `network_device_export` **Network device functional capability** `network_device_functional_capability_info` **Network device global polling interval** `network_device_global_polling_interval_info` **Network device interface neighbor** `network_device_interface_neighbor_info` **Network device interface PoE** `network_device_interface_poe_info` **Network device lexicographically sorted** `network_device_lexicographically_sorted_info` **Network device linecard details** `network_device_linecard_details_info` **Network device maintenance schedules** `network_device_maintenance_schedules`, `network_device_maintenance_schedules_info` **Network device maintenance schedules count** `network_device_maintenance_schedules_count_info` **Network device management address update** `network_device_management_address_update` **Network device meraki organization** `network_device_meraki_organization_info` **Network device module** `network_device_module_info` **Network device module count** `network_device_module_count_info` **Network device PoE** `network_device_poe_info` **Network device polling interval** `network_device_polling_interval_info` **Network device range** `network_device_range_info` **Network device register for WSA** `network_device_register_for_wsa_info` **Network device settings SSH key settings** `network_device_settings_ssh_key_settings`, `network_device_settings_ssh_key_settings_info` **Network device stack details** `network_device_stack_details_info` **Network device summary** `network_device_summary_info` **Network device supervisor card details** `network_device_supervisor_card_details_info` **Network device sync** `network_device_sync` **Network device update role** `network_device_update_role` **Network device user defined field** `network_device_user_defined_field`, `network_device_user_defined_field_info` **Network device user defined field delete** `network_device_user_defined_field_delete` **Network device user defined field update** `network_device_user_defined_field_update` **Network device VLAN** `network_device_vlan_info` **Network device wireless LAN** `network_device_wireless_lan_info` **Network devices** `network_devices_info` **Network devices accept SSH key change create** `network_devices_accept_ssh_key_change_create` **Network devices count** `network_devices_count_info` **Network devices create** `network_devices_create` **Network devices delete with cleanup** `network_devices_delete_with_cleanup` **Network devices delete without cleanup** `network_devices_delete_without_cleanup` **Network devices device controllability settings** `network_devices_device_controllability_settings`, `network_devices_device_controllability_settings_info` **Network devices export credentials create** `network_devices_export_credentials_create` **Network devices info workflow manager** `network_devices_info_workflow_manager` **Network devices intent** `network_devices_intent_info` **Network devices intent count** `network_devices_intent_count_info` **Network devices interfaces query** `network_devices_interfaces_query` **Network devices members associations** `network_devices_members_associations_info` **Network devices members associations count** `network_devices_members_associations_count_info` **Network devices members associations query** `network_devices_members_associations_query` **Network devices query** `network_devices_query` **Network devices query count** `network_devices_query_count` **Network devices resync interval settings** `network_devices_resync_interval_settings` **Network devices resync interval settings ID** `network_devices_resync_interval_settings_id`, `network_devices_resync_interval_settings_id_info` **Network devices resync interval settings override** `network_devices_resync_interval_settings_override` **Network devices update** `network_devices_update` **Network devices update create** `network_devices_update_create` **Network devices validate device create** `network_devices_validate_device_create` **Inventory workflow manager (playbook)** playbook `inventory_workflow_manager` **Network device info (playbook)** playbook `network_device_info` **Network device update role (playbook)** playbook `network_device_update_role` **Network devices info workflow manager (playbook)** playbook `network_devices_info_workflow_manager` **Inventory (role)** role `inventory` **Network devices info (role)** role `network_devices_info` **Topology layer 2** `topology_layer_2_info` **Topology layer 3** `topology_layer_3_info` **Topology physical** `topology_physical_info` **Topology VLAN details** `topology_vlan_details_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Run commands and back up device configurations

Description	Run commands on devices through the command runner, retrieve and archive running configurations, and export device configuration files.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Command runner run command** `command_runner_run_command` **Device configs backup workflow manager** `device_configs_backup_workflow_manager` **Device configurations export** `device_configurations_export` **Catalyst Center command runner keywords** `dna_command_runner_keywords_info` **Execute suggested actions commands** `execute_suggested_actions_commands` **Network device config** `network_device_config_info` **Network device config archive** `network_device_config_archive_info` **Network device config count** `network_device_config_count_info` **Network device config files** `network_device_config_files_info` **Network device config files count** `network_device_config_files_count_info` **Network device config files ID download masked** `network_device_config_files_id_download_masked` **Network device config files ID download unmasked** `network_device_config_files_id_download_unmasked` **Network device config task** `network_device_config_task_info` **Network device config write memory** `network_device_config_write_memory` **Network device custom prompt** `network_device_custom_prompt`, `network_device_custom_prompt_info` **Command runner run command (playbook)** playbook `command_runner_run_command` **Configs backup (playbook)** playbook `configs_backup` **Device configs backup workflow manager (playbook)** playbook `device_configs_backup_workflow_manager` **Download device cmd runner output (playbook)** playbook `download_device_cmd_runner_output` **Network device custom prompt info (playbook)** playbook `network_device_custom_prompt_info` **Device configs backup (role)** role `device_configs_backup`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Replace and refresh devices

Description	Mark faulty devices for replacement, deploy replacement workflows and orchestrate switch refresh batches in SD-Access fabrics.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Device replacement** `device_replacement`, `device_replacement_info` **Device replacement count** `device_replacement_count_info` **Device replacement deploy** `device_replacement_deploy` **Network device replacements** `network_device_replacements_info` **Network device replacements workflow deploy create** `network_device_replacements_workflow_deploy_create` **RMA workflow manager (playbook)** playbook `rma_workflow_manager` **Switch refresh SDA fabric edge cleanup old (playbook)** playbook `switch_refresh_sda_fabric_edge_cleanup_old` **Switch refresh SDA fabric edge prepare (playbook)** playbook `switch_refresh_sda_fabric_edge_prepare` **RMA workflow manager** `rma_workflow_manager` **RMA (role)** role `rma` **Switch refresh SDA fabric edge (role)** role `switch_refresh_sda_fabric_edge`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Manage software images and upgrades

Description	Import, tag as golden, distribute and activate software images, check readiness, and work with software releases and optional packages.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Download software release create** `download_software_release_create` **Download software release delete** `download_software_release_delete` **Download software release update** `download_software_release_update` **Golden image create** `golden_image_create` **Golden tag image delete** `golden_tag_image_delete` **Golden tag image details** `golden_tag_image_details_info` **Images** `images_info` **Images addon images** `images_addon_images_info` **Images addon images count** `images_addon_images_count_info` **Images CCO sync** `images_cco_sync` **Images count** `images_count_info` **Images delete** `images_delete` **Images distribution server settings** `images_distribution_server_settings`, `images_distribution_server_settings_info` **Images download** `images_download` **Images ID sites site ID tag golden** `images_id_sites_site_id_tag_golden` **Images ID sites site ID untag golden** `images_id_sites_site_id_untag_golden` **Images site wise product names** `images_site_wise_product_names`, `images_site_wise_product_names_info` **Images site wise product names count** `images_site_wise_product_names_count_info` **Images transfer protocol settings** `images_transfer_protocol_settings_info` **Install optional packages create** `install_optional_packages_create` **Installed release** `installed_release_info` **Network device image updates** `network_device_image_updates_info` **Network device image updates count** `network_device_image_updates_count_info` **Network device images** `network_device_images_info` **Network device images activate bulk** `network_device_images_activate_bulk` **Network device images count** `network_device_images_count_info` **Network device images distribute bulk** `network_device_images_distribute_bulk` **Network device images ID activate** `network_device_images_id_activate` **Network device images ID distribute** `network_device_images_id_distribute` **Network device images ID readiness checks** `network_device_images_id_readiness_checks` **Network device images validation results** `network_device_images_validation_results_info` **Network device images validation results count** `network_device_images_validation_results_count_info` **Network device images validations** `network_device_images_validations`, `network_device_images_validations_info` **Network device images validations count** `network_device_images_validations_count_info` **Network device inventory insight link mismatch** `network_device_inventory_insight_link_mismatch_info` **Import image file (playbook)** playbook `import_image_file` **SWIM import local (playbook)** playbook `swim_import_local` **SWIM workflow manager (playbook)** playbook `swim_workflow_manager` **Product names** `product_names_info` **Product names count** `product_names_count_info` **Product series** `product_series_info` **Product series count** `product_series_count_info` **Releases** `releases_info` **Releases release summary** `releases_release_summary_info` **SWIM (role)** role `swim` **Site wise images summary** `site_wise_images_summary_info` **Site wise product names** `site_wise_product_names_info` **Site wise product names count** `site_wise_product_names_count_info` **Software management executions** `software_management_executions_info` **SWIM image details** `swim_image_details_info` **SWIM import local** `swim_import_local` **SWIM import via URL** `swim_import_via_url` **SWIM trigger activation** `swim_trigger_activation` **SWIM trigger distribution** `swim_trigger_distribution` **SWIM workflow manager** `swim_workflow_manager` **Uninstall optional packages create** `uninstall_optional_packages_create` **Upgrade software release create** `upgrade_software_release_create`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Manage licenses and Smart Account registration

Description	Register and deregister devices, review license usage and terms, and change virtual accounts.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**License deregister** `license_deregister` **License device count** `license_device_count_info` **License device deregistration** `license_device_deregistration` **License device license details** `license_device_license_details_info` **License device license summary** `license_device_license_summary_info` **License device registration** `license_device_registration` **License last operation status** `license_last_operation_status_info` **License register** `license_register` **License renew** `license_renew` **License setting** `license_setting`, `license_setting_info` **License smart account details** `license_smart_account_details_info` **License status** `license_status_info` **License term details** `license_term_details_info` **License usage** `license_usage_info` **License usage count** `license_usage_count_info` **License usage details** `license_usage_details_info` **License virtual account change** `license_virtual_account_change` **License virtual account details** `license_virtual_account_details_info` **Network device licenses** `network_device_licenses`, `network_device_licenses_info` **Network device licenses count** `network_device_licenses_count_info` **Network device licenses network device ID port channels** `network_device_licenses_network_device_id_port_channels_info` **Network device licenses network device ID port channels count** `network_device_licenses_network_device_id_port_channels_count_info` **Network device licenses query count create** `network_device_licenses_query_count_create` **Network device licenses query create** `network_device_licenses_query_create` **Network device licenses summary** `network_device_licenses_summary_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Create and deploy configuration templates

Description	Create, version, import, export, preview and deploy configuration template projects and templates to devices.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Configuration template** `configuration_template`, `configuration_template_info` **Configuration template clone** `configuration_template_clone` **Configuration template create** `configuration_template_create` **Configuration template deploy** `configuration_template_deploy` **Configuration template deploy status** `configuration_template_deploy_status_info` **Configuration template deploy v2** `configuration_template_deploy_v2` **Configuration template export project** `configuration_template_export_project` **Configuration template export template** `configuration_template_export_template` **Configuration template import project** `configuration_template_import_project` **Configuration template import template** `configuration_template_import_template` **Configuration template project** `configuration_template_project`, `configuration_template_project_info` **Configuration template version** `configuration_template_version_info` **Configuration template version create** `configuration_template_version_create` **Configuration template version create (playbook)** playbook `configuration_template_version_create` **Template (playbook)** playbook `template` **Template workflow manager (playbook)** playbook `template_workflow_manager` **Projects** `projects`, `projects_info` **Projects count** `projects_count_info` **Projects details** `projects_details_info` **Template (role)** role `template` **Template preview** `template_preview` **Template workflow manager** `template_workflow_manager` **Templates details** `templates_details_info` **Templates template ID network profiles for sites** `templates_template_id_network_profiles_for_sites`, `templates_template_id_network_profiles_for_sites_info` **Templates template ID network profiles for sites bulk create** `templates_template_id_network_profiles_for_sites_bulk_create` **Templates template ID network profiles for sites bulk delete** `templates_template_id_network_profiles_for_sites_bulk_delete` **Templates template ID network profiles for sites count** `templates_template_id_network_profiles_for_sites_count_info` **Templates template ID versions** `templates_template_id_versions_info` **Templates template ID versions commit** `templates_template_id_versions_commit` **Templates template ID versions count** `templates_template_id_versions_count_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Provision devices and apply network profiles

Description	Provision and re-provision wired and wireless devices, remediate issues and associate network profiles (switching and wireless) with sites.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Associate site to network profile** `associate_site_to_network_profile` **Disassociate site to network profile** `disassociate_site_to_network_profile` **Network devices issues remediation provision** `network_devices_issues_remediation_provision` **Network devices network profiles for sites** `network_devices_network_profiles_for_sites_info` **Network devices network profiles for sites count** `network_devices_network_profiles_for_sites_count_info` **Network devices network profiles for sites delete** `network_devices_network_profiles_for_sites_delete` **Network profile switching workflow manager** `network_profile_switching_workflow_manager` **Network profile wireless workflow manager** `network_profile_wireless_workflow_manager` **Network profiles for sites profile ID templates** `network_profiles_for_sites_profile_id_templates_info` **Network profiles for sites profile ID templates count** `network_profiles_for_sites_profile_id_templates_count_info` **Network profiles for sites site assignments** `network_profiles_for_sites_site_assignments`, `network_profiles_for_sites_site_assignments_info` **Network profiles for sites site assignments bulk create** `network_profiles_for_sites_site_assignments_bulk_create` **Network profiles for sites site assignments bulk delete** `network_profiles_for_sites_site_assignments_bulk_delete` **Network profiles for sites site assignments count** `network_profiles_for_sites_site_assignments_count_info` **Device provision workflow (playbook)** playbook `device_provision_workflow` **Example roles playbook (playbook)** playbook `example_roles_playbook` **Network profile switching workflow manager (playbook)** playbook `network_profile_switching_workflow_manager` **Network profile wireless workflow manager (playbook)** playbook `network_profile_wireless_workflow_manager` **Provision workflow manager (playbook)** playbook `provision_workflow_manager` **Provision workflow manager** `provision_workflow_manager` **Provisioning settings** `provisioning_settings`, `provisioning_settings_info` **Network profile switching (role)** role `network_profile_switching` **Network profile wireless (role)** role `network_profile_wireless` **Provision (role)** role `provision` **Sites profile assignments** `sites_profile_assignments_info` **Sites profile assignments count** `sites_profile_assignments_count_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Automate wired campus switch configuration

Description	Define, validate and deploy intended Layer 2, Layer 3, port, security and network-setting features on switches and review what is deployed.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Create wired network devices ID config features intended layer2** `create_wired_network_devices_id_config_features_intended_layer2`
	Wired campus automation workflow manager (playbook) playbook `wired_campus_automation_workflow_manager`
	Provision switches configs intended deploy create `provision_switches_configs_intended_deploy_create`
	Provision switches configs intended device deployments `provision_switches_configs_intended_device_deployments_info`
	Wired campus automation (role) role `wired_campus_automation`
	Switches configs deployed layer2 `switches_configs_deployed_layer2_info`
	Switches configs deployed layer2 count `switches_configs_deployed_layer2_count_info`
	Switches configs deployed layer3 `switches_configs_deployed_layer3_info`
	Switches configs deployed layer3 count `switches_configs_deployed_layer3_count_info`
	Switches configs deployed network settings `switches_configs_deployed_network_settings_info`
	Switches configs deployed network settings count `switches_configs_deployed_network_settings_count_info`
	Switches configs deployed port `switches_configs_deployed_port_info`
	Switches configs deployed port count `switches_configs_deployed_port_count_info`
	Switches configs deployed security `switches_configs_deployed_security_info`
	Switches configs deployed security count `switches_configs_deployed_security_count_info`
	Switches configs disable create `switches_configs_disable_create`
	Switches configs enable create `switches_configs_enable_create`
	Switches configs intended layer2 `switches_configs_intended_layer2_info`
	Switches configs intended layer2 count `switches_configs_intended_layer2_count_info`
	Switches configs intended layer2 create `switches_configs_intended_layer2_create`
	Switches configs intended layer2 delete `switches_configs_intended_layer2_delete`
	Switches configs intended layer2 update `switches_configs_intended_layer2_update`
	Switches configs intended layer3 `switches_configs_intended_layer3_info`
	Switches configs intended layer3 count `switches_configs_intended_layer3_count_info`
	Switches configs intended layer3 create `switches_configs_intended_layer3_create`
	Switches configs intended layer3 delete `switches_configs_intended_layer3_delete`
	Switches configs intended layer3 update `switches_configs_intended_layer3_update`
	Switches configs intended network settings `switches_configs_intended_network_settings_info`
	Switches configs intended network settings count `switches_configs_intended_network_settings_count_info`
	Switches configs intended network settings create `switches_configs_intended_network_settings_create`
	Switches configs intended network settings delete `switches_configs_intended_network_settings_delete`
	Switches configs intended network settings update `switches_configs_intended_network_settings_update`
	Switches configs intended port `switches_configs_intended_port_info`
	Switches configs intended port convert create `switches_configs_intended_port_convert_create`
	Switches configs intended port count `switches_configs_intended_port_count_info`
	Switches configs intended port create `switches_configs_intended_port_create`
	Switches configs intended port delete `switches_configs_intended_port_delete`
	Switches configs intended port update `switches_configs_intended_port_update`
		Switches configs intended
		Switches configs intended
		Switches configs intended
		Switches configs intended
		Switches configs intended
		Switches configs intended
		Switches configs status c
		Switches configs support
		Switches configs support
		Switches configs support
		Switches configs support
		Switches configs support
		Update wired network dev
		`update_wired_network_dev`
		Wired campus automation
		Wired network devices ID
		`wired_network_devices_id`
		Wired network devices ID
		`wired_network_devices_id`
		Wired network devices ID
		Wired network devices ID
		`wired_network_devices_id`
		`wired_network_devices_id`
		Wired network devices ID
		`wired_network_devices_id`
		Wired network devices ID
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`

Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the

Use Case: Configure industrial and IoT ring networking

Description	Configure REP, MRP and PRP rings, and Industrial Ethernet fabric extension upgrades.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**IoT fabric PEN upgrade configuration models create** `iot_fabric_pen_upgrade_configuration_models_create` **IoT fabric PEN upgrade configuration models delete** `iot_fabric_pen_upgrade_configuration_models_delete` **IoT fabric PEN upgrade configuration models deploy create** `iot_fabric_pen_upgrade_configuration_models_deploy_create` **IoT fabric PEN upgrade configuration models network device status details** `iot_fabric_pen_upgrade_configuration_models_network_device_status_details_info` **IoT fabric PEN upgrade configuration models network devices config** `iot_fabric_pen_upgrade_configuration_models_network_devices_config`, `iot_fabric_pen_upgrade_configuration_models_network_devices_config_info` **IoT fabric PEN upgrade deploy create** `iot_fabric_pen_upgrade_deploy_create` **IoT fabric PRP** `iot_fabric_prp_info` **IoT fabric PRP configuration models create** `iot_fabric_prp_configuration_models_create` **IoT fabric PRP configuration models delete** `iot_fabric_prp_configuration_models_delete` **IoT fabric PRP configuration models delete create** `iot_fabric_prp_configuration_models_delete_create` **IoT fabric PRP configuration models deploy create** `iot_fabric_prp_configuration_models_deploy_create` **IoT fabric PRP configuration models network device status details** `iot_fabric_prp_configuration_models_network_device_status_details_info`	**IoT fabric PRP configuration models network devices config** `iot_fabric_prp_configuration_models_network_devices_config`, `iot_fabric_prp_configuration_models_network_devices_config_info` **IoT fabric PRP configuration models update create** `iot_fabric_prp_configuration_models_update_create` **IoT fabric PRP count** `iot_fabric_prp_count_info` **IoT fabric PRP deploy create** `iot_fabric_prp_deploy_create` **IoT fabric PRP deploy delete** `iot_fabric_prp_deploy_delete` **IoT fabric PRP deploy update** `iot_fabric_prp_deploy_update` **IoT fabric REP rings** `iot_fabric_rep_rings` **IoT fabric REP rings delete** `iot_fabric_rep_rings_delete` **IoT network devices network device ID MRP rings** `iot_network_devices_network_device_id_mrp_rings_info` **IoT network devices network device ID MRP rings count** `iot_network_devices_network_device_id_mrp_rings_count_info` **IoT network devices network device ID MRP rings ID members** `iot_network_devices_network_device_id_mrp_rings_id_members_info` **IoT network devices network device ID MRP rings ID members count** `iot_network_devices_network_device_id_mrp_rings_id_members_coun` **IoT non fabric REP rings** `iot_non_fabric_rep_rings` **IoT non fabric REP rings delete** `iot_non_fabric_rep_rings_delete` **IoT REP rings ID** `iot_rep_rings_id_info` **IoT REP rings query** `iot_rep_rings_query` **IoT REP rings query count** `iot_rep_rings_query_count`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the are the recommended declarative way to manage a feature.

Use Case: Automate the LAN underlay with LAN Automation

Description	Start, update and stop LAN Automation sessions, review their status and logs, and manage the port channels they create.

Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**LAN automation count** `lan_automation_count_info` **LAN automation create** `lan_automation_create` **LAN automation delete** `lan_automation_delete` **LAN automation log** `lan_automation_log_info` **LAN automation log by serial number** `lan_automation_log_by_serial_number_info` **LAN automation port channels** `lan_automation_port_channels`, `lan_automation_port_channels_info` **LAN automation port channels ID add link** `lan_automation_port_channels_id_add_link` **LAN automation port channels ID remove link** `lan_automation_port_channels_id_remove_link` **LAN automation sessions** `lan_automation_sessions_info` **LAN automation status** `lan_automation_status_info` **LAN automation update** `lan_automation_update` **LAN automation update device** `lan_automation_update_device` **LAN automation update v2** `lan_automation_update_v2` **LAN automation v2** `lan_automation_v2` **LAN automation workflow manager** `lan_automation_workflow_manager` **LAN automation workflow manager (playbook)** playbook `lan_automation_workflow_manager` **LAN automation (role)** role `lan_automation`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Build SD-Access fabric sites, devices and transits

Description	Create fabric sites and zones, add border, control-plane and edge devices, configure transits and Layer 2 handoffs, and apply pending fabric events.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Fabric devices info workflow manager** `fabric_devices_info_workflow_manager` **Fabric site health summaries** `fabric_site_health_summaries_info` **Fabric site health summaries count** `fabric_site_health_summaries_count_info` **Fabric site health summaries ID trend analytics** `fabric_site_health_summaries_id_trend_analytics_info` **Fabric summary** `fabric_summary_info` **Fabric devices info workflow manager (playbook)** playbook `fabric_devices_info_workflow_manager` **SDA fabric (playbook)** playbook `sda_fabric` **SDA fabric devices workflow manager (playbook)** playbook `sda_fabric_devices_workflow_manager` **SDA fabric multicast workflow manager (playbook)** playbook `sda_fabric_multicast_workflow_manager` **SDA fabric site (playbook)** playbook `sda_fabric_site` **SDA fabric sites zones workflow manager (playbook)** playbook `sda_fabric_sites_zones_workflow_manager` **SDA fabric transits workflow manager (playbook)** playbook `sda_fabric_transits_workflow_manager` **SDA fabric virtual networks workflow manager (playbook)** playbook `sda_fabric_virtual_networks_workflow_manager` **Fabric devices info (role)** role `fabric_devices_info` **SDA fabric devices (role)** role `sda_fabric_devices` **SDA fabric multicast (role)** role `sda_fabric_multicast` **SDA fabric sites zones (role)** role `sda_fabric_sites_zones` **SDA fabric transits (role)** role `sda_fabric_transits` **SDA fabric virtual networks (role)** role `sda_fabric_virtual_networks` **SDA authentication profiles** `sda_authentication_profiles`, `sda_authentication_profiles_info` **SDA device** `sda_device_info` **SDA device role** `sda_device_role_info` **SDA fabric authentication profile** `sda_fabric_authentication_profile`, `sda_fabric_authentication_profile_info` **SDA fabric border device** `sda_fabric_border_device`, `sda_fabric_border_device_info` **SDA fabric control plane device** `sda_fabric_control_plane_device`, `sda_fabric_control_plane_device_info` **SDA fabric devices** `sda_fabric_devices`, `sda_fabric_devices_info` **SDA fabric devices count** `sda_fabric_devices_count_info` **SDA fabric devices layer2 handoffs** `sda_fabric_devices_layer2_handoffs`, `sda_fabric_devices_layer2_handoffs_info` **SDA fabric devices layer2 handoffs count** `sda_fabric_devices_layer2_handoffs_count_info` **SDA fabric devices layer2 handoffs IP transits** `sda_fabric_devices_layer2_handoffs_ip_transits`, `sda_fabric_devices_layer2_handoffs_ip_transits_info` **SDA fabric devices layer2 handoffs IP transits count** `sda_fabric_devices_layer2_handoffs_ip_transits_count_info` **SDA fabric devices layer2 handoffs SDA transits** `sda_fabric_devices_layer2_handoffs_sda_transits`, `sda_fabric_devices_layer2_handoffs_sda_transits_info` **SDA fabric devices layer2 handoffs SDA transits count** `sda_fabric_devices_layer2_handoffs_sda_transits_count_info` **SDA fabric devices workflow manager** `sda_fabric_devices_workflow_manager` **SDA fabric edge device** `sda_fabric_edge_device`, `sda_fabric_edge_device_info` **SDA fabric multicast workflow manager** `sda_fabric_multicast_workflow_manager` **SDA fabric site** `sda_fabric_site`, `sda_fabric_site_info` **SDA fabric sites** `sda_fabric_sites`, `sda_fabric_sites_info` **SDA fabric sites count** `sda_fabric_sites_count_info` **SDA fabric sites zones workflow manager** `sda_fabric_sites_zones_workflow_manager` **SDA fabric transits workflow manager** `sda_fabric_transits_workflow_manager` **SDA fabric virtual networks workflow manager** `sda_fabric_virtual_networks_workflow_manager` **SDA fabric zones** `sda_fabric_zones`, `sda_fabric_zones_info` **SDA fabric zones count** `sda_fabric_zones_count_info` **SDA pending fabric events** `sda_pending_fabric_events_info` **SDA pending fabric events apply** `sda_pending_fabric_events_apply` **SDA provision device** `sda_provision_device`, `sda_provision_device_info` **SDA provision devices** `sda_provision_devices`, `sda_provision_devices_info` **SDA provision devices count** `sda_provision_devices_count_info` **SDA site member member** `sda_site_member_member_info` **SDA transit networks** `sda_transit_networks`, `sda_transit_networks_info` **SDA transit networks count** `sda_transit_networks_count_info` **SDA transit networks delete** `sda_transit_networks_delete` **Transit network health summaries** `transit_network_health_summaries_info` **Transit network health summaries count** `transit_network_health_summaries_count_info` **Transit network health summaries ID trend analytics** `transit_network_health_summaries_id_trend_analytics_info` **Transit peer network** `transit_peer_network`, `transit_peer_network_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Configure fabric virtual networks, gateways and policies

Description	Manage Layer 2 and Layer 3 virtual networks, anycast gateways, VLAN-to-SSID mappings, extranet policies and fabric multicast.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Business SDA host onboarding SSID ippool** `business_sda_hostonboarding_ssid_ippool`, `business_sda_hostonboarding_ssid_ippool_info` **Business SDA virtual network summary** `business_sda_virtual_network_summary_info` **SDA anycast gateways (playbook)** playbook `sda_anycast_gateways` **SDA extranet policies workflow manager (playbook)** playbook `sda_extranet_policies_workflow_manager` **SDA layer3 virtual networks (playbook)** playbook `sda_layer3_virtual_networks` **SDA virtual network v2 (playbook)** playbook `sda_virtual_network_v2` **SDA extranet policies (role)** role `sda_extranet_policies` **SDA anycast gateways** `sda_anycast_gateways`, `sda_anycast_gateways_info` **SDA anycast gateways count** `sda_anycast_gateways_count_info` **SDA extranet policies** `sda_extranet_policies`, `sda_extranet_policies_info` **SDA extranet policies count** `sda_extranet_policies_count_info` **SDA extranet policies workflow manager** `sda_extranet_policies_workflow_manager` **SDA fabrics VLAN to SSIDs** `sda_fabrics_vlan_to_ssids_info` **SDA fabrics VLAN to SSIDs count** `sda_fabrics_vlan_to_ssids_count_info` **SDA fabrics VLAN to SSIDs fabric ID** `sda_fabrics_vlan_to_ssids_fabric_id`, `sda_fabrics_vlan_to_ssids_fabric_id_info` **SDA fabrics VLAN to SSIDs fabric ID count** `sda_fabrics_vlan_to_ssids_fabric_id_count_info` **SDA layer2 virtual networks** `sda_layer2_virtual_networks`, `sda_layer2_virtual_networks_info` **SDA layer2 virtual networks count** `sda_layer2_virtual_networks_count_info` **SDA layer3 virtual networks** `sda_layer3_virtual_networks`, `sda_layer3_virtual_networks_info` **SDA layer3 virtual networks count** `sda_layer3_virtual_networks_count_info` **SDA layer3 virtual networks delete** `sda_layer3_virtual_networks_delete` **SDA multicast** `sda_multicast`, `sda_multicast_info` **SDA multicast v1** `sda_multicast_v1`, `sda_multicast_v1_info` **SDA multicast virtual networks** `sda_multicast_virtual_networks`, `sda_multicast_virtual_networks_info` **SDA multicast virtual networks count** `sda_multicast_virtual_networks_count_info` **SDA virtual network** `sda_virtual_network`, `sda_virtual_network_info` **SDA virtual network IP pool** `sda_virtual_network_ip_pool`, `sda_virtual_network_ip_pool_info` **SDA virtual network v2** `sda_virtual_network_v2`, `sda_virtual_network_v2_info` **Virtual network health summaries** `virtual_network_health_summaries_info` **Virtual network health summaries count** `virtual_network_health_summaries_count_info` **Virtual network health summaries ID trend analytics** `virtual_network_health_summaries_id_trend_analytics_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Onboard hosts and ports to the fabric

Description	Assign ports for users, access points and devices, configure port channels and onboard or migrate host ports.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**SDA host port onboarding workflow manager (playbook)** playbook `sda_host_port_onboarding_workflow_manager` **SDA host port onboarding (role)** role `sda_host_port_onboarding` **SDA host port onboarding workflow manager** `sda_host_port_onboarding_workflow_manager` **SDA port assignment for access point** `sda_port_assignment_for_access_point`, `sda_port_assignment_for_access_point_info` **SDA port assignment for user device** `sda_port_assignment_for_user_device`, `sda_port_assignment_for_user_device_info` **SDA port assignments** `sda_port_assignments`, `sda_port_assignments_info` **SDA port assignments count** `sda_port_assignments_count_info` **SDA port channels** `sda_port_channels`, `sda_port_channels_info` **SDA port channels count** `sda_port_channels_count_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Design wireless profiles, SSIDs and RF settings

Description	Create SSIDs, wireless profiles, RF profiles, AP profiles, interfaces, power profiles, anchor groups and wireless policies, and apply them to sites.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Fabrics fabric ID switch wireless setting** `fabrics_fabric_id_switch_wireless_setting`, `fabrics_fabric_id_switch_wireless_setting_info` **Fabrics fabric ID switch wireless setting reload** `fabrics_fabric_id_switch_wireless_setting_reload` **Fabrics fabric ID wireless multicast** `fabrics_fabric_id_wireless_multicast`, `fabrics_fabric_id_wireless_multicast_info` **Feature templates wireless AAA radius attributes configurations** `feature_templates_wireless_aaa_radius_attributes_configurations`, `feature_templates_wireless_aaa_radius_attributes_configurations_info` **Feature templates wireless advanced SSID configurations** `feature_templates_wireless_advanced_ssid_configurations`, `feature_templates_wireless_advanced_ssid_configurations_info` **Feature templates wireless clean air configurations** `feature_templates_wireless_clean_air_configurations`, `feature_templates_wireless_clean_air_configurations_info` **Feature templates wireless 802.11ax configurations** `feature_templates_wireless_dot11ax_configurations`, `feature_templates_wireless_dot11ax_configurations_info` **Feature templates wireless 802.11be status configurations** `feature_templates_wireless_dot11be_status_configurations`, `feature_templates_wireless_dot11be_status_configurations_info` **Feature templates wireless event driven rrm configurations** `feature_templates_wireless_event_driven_rrm_configurations`, `feature_templates_wireless_event_driven_rrm_configurations_info` **Feature templates wireless flex connect configurations** `feature_templates_wireless_flex_connect_configurations`, `feature_templates_wireless_flex_connect_configurations_info` **Feature templates wireless multicast configurations** `feature_templates_wireless_multicast_configurations`, `feature_templates_wireless_multicast_configurations_info` **Feature templates wireless rrm fra configurations** `feature_templates_wireless_rrm_fra_configurations`, `feature_templates_wireless_rrm_fra_configurations_info` **Feature templates wireless rrm general configurations** `feature_templates_wireless_rrm_general_configurations`, `feature_templates_wireless_rrm_general_configurations_info` **Feature templates wireless summary** `feature_templates_wireless_summary_info` **Wireless design workflow manager (playbook)** playbook `wireless_design_workflow_manager` **Wireless enterprise SSID (playbook)** playbook `wireless_enterprise_ssid` **Wireless design (role)** role `wireless_design` **Sites site ID wireless settings flex connect AAA override** `sites_site_id_wireless_settings_flex_connect_aaa_override`, `sites_site_id_wireless_settings_flex_connect_aaa_override_info` **Sites site ID wireless settings flex connect native VLAN** `sites_site_id_wireless_settings_flex_connect_native_vlan`, `sites_site_id_wireless_settings_flex_connect_native_vlan_info` **Sites site ID wireless settings SSIDs ID update** `sites_site_id_wireless_settings_ssids_id_update` **Sites wireless settings SSIDs** `sites_wireless_settings_ssids`, `sites_wireless_settings_ssids_info` **Sites wireless settings SSIDs count** `sites_wireless_settings_ssids_count_info` **Wireless design workflow manager** `wireless_design_workflow_manager` **Wireless dynamic interface** `wireless_dynamic_interface`, `wireless_dynamic_interface_info` **Wireless enterprise SSID** `wireless_enterprise_ssid`, `wireless_enterprise_ssid_info` **Wireless policies access control list provision** `wireless_policies_access_control_list_provision`, `wireless_policies_access_control_list_provision_info` **Wireless policies access control list provision count** `wireless_policies_access_control_list_provision_count_info` **Wireless policies IP access control lists contract** `wireless_policies_ip_access_control_lists_contract`, `wireless_policies_ip_access_control_lists_contract_info` **Wireless policies IP access control lists contract count** `wireless_policies_ip_access_control_lists_contract_count_info` **Wireless policies IP access control lists network group** `wireless_policies_ip_access_control_lists_network_group`, `wireless_policies_ip_access_control_lists_network_group_info` **Wireless policies IP access control lists network group count** `wireless_policies_ip_access_control_lists_network_group_count_info` **Wireless policies IP access control lists protocol** `wireless_policies_ip_access_control_lists_protocol`, `wireless_policies_ip_access_control_lists_protocol_info` **Wireless profile** `wireless_profile`, `wireless_profile_info` **Wireless profiles** `wireless_profiles`, `wireless_profiles_info` **Wireless profiles count** `wireless_profiles_count_info` **Wireless profiles ID policy tags bulk** `wireless_profiles_id_policy_tags_bulk` **Wireless profiles ID policy tags count** `wireless_profiles_id_policy_tags_count_info` **Wireless profiles ID policy tags policy tag ID** `wireless_profiles_id_policy_tags_policy_tag_id`, `wireless_profiles_id_policy_tags_policy_tag_id_info` **Wireless profiles ID site tags** `wireless_profiles_id_site_tags`, `wireless_profiles_id_site_tags_info` **Wireless profiles ID site tags bulk** `wireless_profiles_id_site_tags_bulk` **Wireless profiles ID site tags count** `wireless_profiles_id_site_tags_count_info` **Wireless profiles policy tags** `wireless_profiles_policy_tags_info` **Wireless PSK override** `wireless_psk_override` **Wireless RF profile** `wireless_rf_profile`, `wireless_rf_profile_info` **Wireless settings anchor groups** `wireless_settings_anchor_groups`, `wireless_settings_anchor_groups_info` **Wireless settings anchor groups count** `wireless_settings_anchor_groups_count_info` **Wireless settings AP authorization lists** `wireless_settings_ap_authorization_lists`, `wireless_settings_ap_authorization_lists_info` **Wireless settings AP authorization lists count** `wireless_settings_ap_authorization_lists_count_info` **Wireless settings AP profiles** `wireless_settings_ap_profiles`, `wireless_settings_ap_profiles_info` **Wireless settings AP profiles count** `wireless_settings_ap_profiles_count_info` **Wireless settings certificate renewal profiles** `wireless_settings_certificate_renewal_profiles`, `wireless_settings_certificate_renewal_profiles_info` **Wireless settings certificate renewal profiles count** `wireless_settings_certificate_renewal_profiles_count_info` **Wireless settings 802.11be profiles** `wireless_settings_dot11be_profiles`, `wireless_settings_dot11be_profiles_info` **Wireless settings 802.11be profiles count** `wireless_settings_dot11be_profiles_count_info` **Wireless settings interfaces** `wireless_settings_interfaces`, `wireless_settings_interfaces_info` **Wireless settings interfaces count** `wireless_settings_interfaces_count_info` **Wireless settings network device ID assign anchor managed AP locations** `wireless_settings_network_device_id_assign_anchor_managed_ap_locations` **Wireless settings power profiles** `wireless_settings_power_profiles`, `wireless_settings_power_profiles_info` **Wireless settings power profiles count** `wireless_settings_power_profiles_count_info` **Wireless settings pre auth ACLs** `wireless_settings_pre_auth_acls`, `wireless_settings_pre_auth_acls_info` **Wireless settings pre auth ACLs count** `wireless_settings_pre_auth_acls_count_info` **Wireless settings remote teleworkers** `wireless_settings_remote_teleworkers`, `wireless_settings_remote_teleworkers_info` **Wireless settings remote teleworkers count** `wireless_settings_remote_teleworkers_count_info` **Wireless settings RF profiles** `wireless_settings_rf_profiles`, `wireless_settings_rf_profiles_info` **Wireless settings RF profiles count** `wireless_settings_rf_profiles_count_info` **Wireless settings SSIDs override at sites** `wireless_settings_ssids_override_at_sites_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Provision wireless controllers and access points

Description	Provision controllers and access points, manage AP configuration and reboot, mobility groups, managed AP locations, mesh neighbours and certificate renewal.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Accesspoint configuration details by task ID** `accesspoint_configuration_details_by_task_id_info` **Accesspoint location workflow manager** `accesspoint_location_workflow_manager` **Accesspoint workflow manager** `accesspoint_workflow_manager` **Business SDA wireless controller create** `business_sda_wireless_controller_create` **Business SDA wireless controller delete** `business_sda_wireless_controller_delete` **Device reboot apreboot** `device_reboot_apreboot`, `device_reboot_apreboot_info` **Network devices WLC redundancy slots** `network_devices_wlc_redundancy_slots_info` **Accesspoint location workflow manager (playbook)** playbook `accesspoint_location_workflow_manager` **Accesspoint workflow manager (playbook)** playbook `accesspoint_workflow_manager` **Accesspoint (role)** role `accesspoint` **Accesspoint location (role)** role `accesspoint_location` **Settings AP PnP location** `settings_ap_pnp_location`, `settings_ap_pnp_location_info` **System settings AP PnP location** `system_settings_ap_pnp_location`, `system_settings_ap_pnp_location_info` **Wireless accespoint configuration** `wireless_accespoint_configuration` **Wireless access points configurations** `wireless_access_points_configurations`, `wireless_access_points_configurations_info` **Wireless access points configurations count** `wireless_access_points_configurations_count_info` **Wireless access points configurations status** `wireless_access_points_configurations_status_info` **Wireless access points factory reset request provision** `wireless_access_points_factory_reset_request_provision` **Wireless access points factory reset request status** `wireless_access_points_factory_reset_request_status_info` **Wireless access points provision** `wireless_access_points_provision` **Wireless accesspoint configuration count** `wireless_accesspoint_configuration_count_info` **Wireless accesspoint configuration create** `wireless_accesspoint_configuration_create` **Wireless accesspoint configuration summary** `wireless_accesspoint_configuration_summary_info` **Wireless controllers anchor capable devices** `wireless_controllers_anchor_capable_devices_info` **Wireless controllers anchor managed AP locations** `wireless_controllers_anchor_managed_ap_locations_info` **Wireless controllers assign managed AP locations** `wireless_controllers_assign_managed_ap_locations` **Wireless controllers certificate renewal create** `wireless_controllers_certificate_renewal_create` **Wireless controllers managed AP locations count** `wireless_controllers_managed_ap_locations_count_info` **Wireless controllers mesh AP neighbours** `wireless_controllers_mesh_ap_neighbours_info` **Wireless controllers mesh AP neighbours count** `wireless_controllers_mesh_ap_neighbours_count_info` **Wireless controllers network device ID AP authorization lists** `wireless_controllers_network_device_id_ap_authorization_lists_info` **Wireless controllers network device ID provision status** `wireless_controllers_network_device_id_provision_status_info` **Wireless controllers primary managed AP locations** `wireless_controllers_primary_managed_ap_locations_info` **Wireless controllers provision** `wireless_controllers_provision` **Wireless controllers secondary managed AP locations** `wireless_controllers_secondary_managed_ap_locations_info` **Wireless controllers site tags** `wireless_controllers_site_tags_info` **Wireless controllers SSID details** `wireless_controllers_ssid_details_info` **Wireless controllers SSID details count** `wireless_controllers_ssid_details_count_info` **Wireless controllers supported device versions** `wireless_controllers_supported_device_versions_info` **Wireless controllers wireless mobility groups** `wireless_controllers_wireless_mobility_groups_info` **Wireless controllers wireless mobility groups count** `wireless_controllers_wireless_mobility_groups_count_info` **Wireless controllers wireless mobility groups mobility provision** `wireless_controllers_wireless_mobility_groups_mobility_provision` **Wireless controllers wireless mobility groups mobility reset** `wireless_controllers_wireless_mobility_groups_mobility_reset` **Wireless provision access point** `wireless_provision_access_point` **Wireless provision device create** `wireless_provision_device_create` **Wireless provision device update** `wireless_provision_device_update` **Wireless provision SSID create provision** `wireless_provision_ssid_create_provision` **Wireless provision SSID delete reprovision** `wireless_provision_ssid_delete_reprovision`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Define intended wireless controller configuration

Description	Create and deploy intended AP join, power, calendar and mesh profiles, Flex, mDNS, RF, SSID and tag configuration on wireless controllers.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Wireless controllers intended access points AP join profiles**
	`wireless_controllers_intended_access_points_ap_join_profiles`,
	`wireless_controllers_intended_access_points_ap_join_profiles_info`
	Wireless controllers intended access points AP join profiles calendar configs
	`wireless_controllers_intended_access_points_ap_join_profiles_calendar_configs`,
	`wireless_controllers_intended_access_points_ap_join_profiles_calendar_configs_info`
	Wireless controllers intended access points AP join profiles count
	`wireless_controllers_intended_access_points_ap_join_profiles_count_info`
	Wireless controllers intended access points AP join profiles halo BLE configs
	`wireless_controllers_intended_access_points_ap_join_profiles_halo_ble_configs_info`
	Wireless controllers intended access points AP join profiles ICAP client mac address configs
	`wireless_controllers_intended_access_points_ap_join_profiles_icap_client_mac_address_configs`,
	`wireless_controllers_intended_access_points_ap_join_profiles_icap_client_mac_address_configs_info`
	Wireless controllers intended access points AP join profiles ICAP mac client configs
	`wireless_controllers_intended_access_points_ap_join_profiles_icap_mac_client_configs`,
	`wireless_controllers_intended_access_points_ap_join_profiles_icap_mac_client_configs_info`
	Wireless controllers intended access points AP join profiles QoS map configs
	`wireless_controllers_intended_access_points_ap_join_profiles_qos_map_configs`,
	`wireless_controllers_intended_access_points_ap_join_profiles_qos_map_configs_info`
	Wireless controllers intended access points AP join profiles QoS map DSCP to UPs
	`wireless_controllers_intended_access_points_ap_join_profiles_qos_map_dscp_to_ups`,
	`wireless_controllers_intended_access_points_ap_join_profiles_qos_map_dscp_to_ups_info`
	Wireless controllers intended access points AP power profiles
	`wireless_controllers_intended_access_points_ap_power_profiles`,
	`wireless_controllers_intended_access_points_ap_power_profiles_info`
	Wireless controllers intended access points AP power profiles AP PPE configs
	`wireless_controllers_intended_access_points_ap_power_profiles_ap_ppe_configs`,
	`wireless_controllers_intended_access_points_ap_power_profiles_ap_ppe_configs_info`
	Wireless controllers intended access points AP power profiles count
	`wireless_controllers_intended_access_points_ap_power_profiles_count_info`
	Wireless controllers intended access points calendar profiles
	`wireless_controllers_intended_access_points_calendar_profiles`,
	`wireless_controllers_intended_access_points_calendar_profiles_info`
	Wireless controllers intended access points calendar profiles count
	`wireless_controllers_intended_access_points_calendar_profiles_count_info`
	Wireless controllers intended access points calendar profiles monthly configs
	`wireless_controllers_intended_access_points_calendar_profiles_monthly_configs`,
	`wireless_controllers_intended_access_points_calendar_profiles_monthly_configs_info`
	Wireless controllers intended access points calendar profiles weekly configs
	`wireless_controllers_intended_access_points_calendar_profiles_weekly_configs`,
	`wireless_controllers_intended_access_points_calendar_profiles_weekly_configs_info`
	Wireless controllers intended access points mesh profiles
	`wireless_controllers_intended_access_points_mesh_profiles`,
	`wireless_controllers_intended_access_points_mesh_profiles_info`
	Wireless controllers intended access points mesh profiles count
	`wireless_controllers_intended_access_points_mesh_profiles_count_info`
	Wireless controllers intended access points site tag configs
	`wireless_controllers_intended_access_points_site_tag_configs`,
	`wireless_controllers_intended_access_points_site_tag_configs_info`
	Wireless controllers intended access points site tag configs count
	`wireless_controllers_intended_access_points_site_tag_configs_count_info`
	Wireless controllers intended access points update create
	`wireless_controllers_intended_access_points_update_create`
	Wireless controllers intended deploy create `wireless_controllers_intended_deploy_create`
	Wireless controllers intended device deployments
	`wireless_controllers_intended_device_deployments_info`
	Wireless controllers intended flexes flex profiles
	`wireless_controllers_intended_flexes_flex_profiles_info`
	Wireless controllers intended flexes flex profiles count
	`wireless_controllers_intended_flexes_flex_profiles_count_info`
	Wireless controllers intended flexes flex profiles if name VLAN ID configs
	`wireless_controllers_intended_flexes_flex_profiles_if_name_vlan_id_configs_info`
	Wireless controllers intended flexes flex profiles local auth user configs
	`wireless_controllers_intended_flexes_flex_profiles_local_auth_user_configs_info`
	Wireless controllers intended flexes flex profiles policy ACL configs
	`wireless_controllers_intended_flexes_flex_profiles_policy_acl_configs_info`
	Wireless controllers intended flexes flex profiles umbrella profile configs
	`wireless_controllers_intended_flexes_flex_profiles_umbrella_profile_configs_info`
	Wireless controllers intended flexes update create `wireless_controllers_intended_flexes_update_create`
	Wireless controllers intended mDNS flex profiles
	`wireless_controllers_intended_mdns_flex_profiles_info`
	Wireless controllers intended mDNS flex profiles count
	`wireless_controllers_intended_mdns_flex_profiles_count_info`
	Wireless controllers intended mDNS gateways `wireless_controllers_intended_mdns_gateways_info`
	Wireless controllers intended mDNS gateways count
	`wireless_controllers_intended_mdns_gateways_count_info`
	Wireless controllers intended mDNS service definitions
	`wireless_controllers_intended_mdns_service_definitions_info`
	Wireless controllers intended mDNS service definitions count
	`wireless_controllers_intended_mdns_service_definitions_count_info`
	Wireless controllers intended mDNS service definitions mDNS service types
	`wireless_controllers_intended_mdns_service_definitions_mdns_service_types_info`
	Wireless controllers intended mDNS service policies
	`wireless_controllers_intended_mdns_service_policies_info`
		Wireless controllers intended mDNS service pol
		`wireless_controllers_intended_mdns_service_p`
		Wireless controllers intended mDNS service pol
		`wireless_controllers_intended_mdns_service_p`
		Wireless controllers intended mDNS service pol
		`wireless_controllers_intended_mdns_service_p`
		Wireless controllers intended mDNS update cre
		Wireless controllers intended mDNS wired filter
		`wireless_controllers_intended_mdns_wired_fil`
		Wireless controllers intended mDNS wired filter
		`wireless_controllers_intended_mdns_wired_fil`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended radio frequencies
		`wireless_controllers_intended_radio_frequenc`
		Wireless controllers intended SSIDs 802.11 be p
		`wireless_controllers_intended_ssids_dot11_be`
		`wireless_controllers_intended_ssids_dot11_be`
		Wireless controllers intended SSIDs 802.11 be p
		`wireless_controllers_intended_ssids_dot11_be`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy profi
		`wireless_controllers_intended_ssids_policy_p`
		`wireless_controllers_intended_ssids_policy_p`
		Wireless controllers intended SSIDs policy tags
		`wireless_controllers_intended_ssids_policy_t`
		Wireless controllers intended SSIDs policy tags
		`wireless_controllers_intended_ssids_policy_t`
		Wireless controllers intended SSIDs policy tags
		`wireless_controllers_intended_ssids_policy_t`
		`wireless_controllers_intended_ssids_policy_t`

	Wireless controllers intended SSIDs policy tags policy tag WLAN policies `wireless_controllers_intended_ssids_policy_tags_policy_tag_wlan_policies`, `wireless_controllers_intended_ssids_policy_tags_policy_tag_wlan_policies_info` **Wireless controllers intended SSIDs RLAN policies** `wireless_controllers_intended_ssids_rlan_policies`, `wireless_controllers_intended_ssids_rlan_policies_info` **Wireless controllers intended SSIDs RLAN policies count** `wireless_controllers_intended_ssids_rlan_policies_count_info` **Wireless controllers intended SSIDs RLAN profiles** `wireless_controllers_intended_ssids_rlan_profiles`, `wireless_controllers_intended_ssids_rlan_profiles_info` **Wireless controllers intended SSIDs RLAN profiles count** `wireless_controllers_intended_ssids_rlan_profiles_count_info` **Wireless controllers intended SSIDs update create** `wireless_controllers_intended_ssids_update_create` **Wireless controllers intended SSIDs WLAN profiles** `wireless_controllers_intended_ssids_wlan_profiles`, `wireless_controllers_intended_ssids_wlan_profiles_info` **Wireless controllers intended SSIDs WLAN profiles count** `wireless_controllers_intended_ssids_wlan_profiles_count_info` **Wireless controllers intended SSIDs WLAN profiles MPSK key configs** `wireless_controllers_intended_ssids_wlan_profiles_mpsk_key_configs`, `wireless_controllers_intended_ssids_wlan_profiles_mpsk_key_configs_info`	**Wireless controllers intended SSIDs WLAN prof** `wireless_controllers_intended_ssids_wlan_pro` `wireless_controllers_intended_ssids_wlan_pro` **Wireless controllers intended tag mappings AP** `wireless_controllers_intended_tag_mappings_a` **Wireless controllers intended tag mappings AP** `wireless_controllers_intended_tag_mappings_a` **Wireless controllers intended tag mappings AP** `wireless_controllers_intended_tag_mappings_a` `wireless_controllers_intended_tag_mappings_a` **Wireless controllers intended tag mappings AP** `wireless_controllers_intended_tag_mappings_a` **Wireless controllers intended tag mappings loca** `wireless_controllers_intended_tag_mappings_l` **Wireless controllers intended tag mappings loca** `wireless_controllers_intended_tag_mappings_l` **Wireless controllers intended tag mappings loca** `wireless_controllers_intended_tag_mappings_l` **Wireless controllers intended tag mappings upd** `wireless_controllers_intended_tag_mappings_u`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the manage a feature.

Use Case: Review deployed wireless controller configuration

Description	Read the configuration currently deployed on wireless controllers, including AP, Flex, mDNS, RF, SSID and tag mappings.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Wireless controllers deployed access points AP join profiles** wireless_controllers_deployed_access_points_ap_join_profiles_info **Wireless controllers deployed access points AP join profiles calendar configs** wireless_controllers_deployed_access_points_ap_join_profiles_calendar_configs_info **Wireless controllers deployed access points AP join profiles count** wireless_controllers_deployed_access_points_ap_join_profiles_count_info **Wireless controllers deployed access points AP join profiles halo BLE configs** wireless_controllers_deployed_access_points_ap_join_profiles_halo_ble_configs_info **Wireless controllers deployed access points AP join profiles ICAP client mac address configs** wireless_controllers_deployed_access_points_ap_join_profiles_icap_client_mac_address_configs_info **Wireless controllers deployed access points AP join profiles ICAP mac client configs** wireless_controllers_deployed_access_points_ap_join_profiles_icap_mac_client_configs_info **Wireless controllers deployed access points AP join profiles QoS map configs** wireless_controllers_deployed_access_points_ap_join_profiles_qos_map_configs_info **Wireless controllers deployed access points AP join profiles QoS map DSCP to UPs** wireless_controllers_deployed_access_points_ap_join_profiles_qos_map_dscp_to_ups_info **Wireless controllers deployed access points AP packet captures** wireless_controllers_deployed_access_points_ap_packet_captures_info **Wireless controllers deployed access points AP packet captures count** wireless_controllers_deployed_access_points_ap_packet_captures_count_info **Wireless controllers deployed access points AP power profiles** wireless_controllers_deployed_access_points_ap_power_profiles_info **Wireless controllers deployed access points AP power profiles AP PPE configs** wireless_controllers_deployed_access_points_ap_power_profiles_ap_ppe_configs_info **Wireless controllers deployed access points AP power profiles count** wireless_controllers_deployed_access_points_ap_power_profiles_count_info **Wireless controllers deployed access points AP trace profiles** wireless_controllers_deployed_access_points_ap_trace_profiles_info **Wireless controllers deployed access points AP trace profiles count** wireless_controllers_deployed_access_points_ap_trace_profiles_count_info **Wireless controllers deployed access points calendar profiles** wireless_controllers_deployed_access_points_calendar_profiles_info **Wireless controllers deployed access points calendar profiles count** wireless_controllers_deployed_access_points_calendar_profiles_count_info **Wireless controllers deployed access points calendar profiles monthly configs** wireless_controllers_deployed_access_points_calendar_profiles_monthly_configs_info **Wireless controllers deployed access points calendar profiles weekly configs** wireless_controllers_deployed_access_points_calendar_profiles_weekly_configs_info **Wireless controllers deployed access points mesh profiles** wireless_controllers_deployed_access_points_mesh_profiles_info **Wireless controllers deployed access points mesh profiles count** wireless_controllers_deployed_access_points_mesh_profiles_count_info **Wireless controllers deployed access points site tag configs** wireless_controllers_deployed_access_points_site_tag_configs_info **Wireless controllers deployed access points site tag configs count** wireless_controllers_deployed_access_points_site_tag_configs_count_info **Wireless controllers deployed flexes flex profiles** wireless_controllers_deployed_flexes_flex_profiles_info **Wireless controllers deployed flexes flex profiles count** wireless_controllers_deployed_flexes_flex_profiles_count_info **Wireless controllers deployed flexes flex profiles if name VLAN ID configs** wireless_controllers_deployed_flexes_flex_profiles_if_name_vlan_id_configs_info **Wireless controllers deployed flexes flex profiles local auth user configs** wireless_controllers_deployed_flexes_flex_profiles_local_auth_user_configs_info **Wireless controllers deployed flexes flex profiles policy ACL configs** wireless_controllers_deployed_flexes_flex_profiles_policy_acl_configs_info **Wireless controllers deployed flexes flex profiles umbrella profile configs** wireless_controllers_deployed_flexes_flex_profiles_umbrella_profile_configs_info **Wireless controllers deployed mDNS flex profiles** wireless_controllers_deployed_mdns_flex_profiles_info **Wireless controllers deployed mDNS flex profiles count** wireless_controllers_deployed_mdns_flex_profiles_count_info **Wireless controllers deployed mDNS gateways** wireless_controllers_deployed_mdns_gateways_info **Wireless controllers deployed mDNS gateways count** wireless_controllers_deployed_mdns_gateways_count_info **Wireless controllers deployed mDNS service definitions** wireless_controllers_deployed_mdns_service_definitions_info **Wireless controllers deployed mDNS service definitions count** wireless_controllers_deployed_mdns_service_definitions_count_info **Wireless controllers deployed mDNS service definitions mDNS service types** wireless_controllers_deployed_mdns_service_definitions_mdns_service_types_info **Wireless controllers deployed mDNS service policies** wireless_controllers_deployed_mdns_service_policies_info **Wireless controllers deployed mDNS service policies count** wireless_controllers_deployed_mdns_service_policies_count_info **Wireless controllers deployed mDNS service policies mDNS service policy ins** wireless_controllers_deployed_mdns_service_policies_mdns_service_policy_ins_info **Wireless controllers deployed mDNS service policies mDNS service policy outs** wireless_controllers_deployed_mdns_service_policies_mdns_service_policy_outs_info **Wireless controllers deployed mDNS wired filters** wireless_controllers_deployed_mdns_wired_filters_info **Wireless controllers deployed mDNS wired filters count** wireless_controllers_deployed_mdns_wired_filters_count_info	**Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed radio frequencies** wireless_controllers_deployed_radio_frequenc **Wireless controllers deployed SSIDs 802.11 be p** wireless_controllers_deployed_ssids_dot11_be **Wireless controllers deployed SSIDs 802.11 be p** wireless_controllers_deployed_ssids_dot11_be **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy prof** wireless_controllers_deployed_ssids_policy_p **Wireless controllers deployed SSIDs policy tags** wireless_controllers_deployed_ssids_policy_t **Wireless controllers deployed SSIDs policy tags** wireless_controllers_deployed_ssids_policy_t **Wireless controllers deployed SSIDs policy tags** wireless_controllers_deployed_ssids_policy_t **Wireless controllers deployed SSIDs RLAN poli** wireless_controllers_deployed_ssids_rlan_pol **Wireless controllers deployed SSIDs RLAN poli** wireless_controllers_deployed_ssids_rlan_pol **Wireless controllers deployed SSIDs RLAN prof** wireless_controllers_deployed_ssids_rlan_pro **Wireless controllers deployed SSIDs RLAN prof** wireless_controllers_deployed_ssids_rlan_pro **Wireless controllers deployed SSIDs WLAN prof** wireless_controllers_deployed_ssids_wlan_pro **Wireless controllers deployed SSIDs WLAN prof** wireless_controllers_deployed_ssids_wlan_pro **Wireless controllers deployed SSIDs WLAN prof** wireless_controllers_deployed_ssids_wlan_pro **Wireless controllers deployed SSIDs WLAN prof** wireless_controllers_deployed_ssids_wlan_pro **Wireless controllers deployed tag mappings AP** wireless_controllers_deployed_tag_mappings_a **Wireless controllers deployed tag mappings AP** wireless_controllers_deployed_tag_mappings_a **Wireless controllers deployed tag mappings AP** wireless_controllers_deployed_tag_mappings_a

	Wireless controllers deployed tag mappings AP tag configs count `wireless_controllers_deployed_tag_mappings_ap_tag_configs_count_info` **Wireless controllers deployed tag mappings location configs** `wireless_controllers_deployed_tag_mappings_location_configs_info`	**Wireless controllers deployed tag mappings loc** `wireless_controllers_deployed_tag_mappings_l` **Wireless controllers deployed tag mappings loc** `wireless_controllers_deployed_tag_mappings_l`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the manage a feature.	

Use Case: Manage application policy, QoS and application visibility

Description	Configure application policies, application sets, queuing profiles, QoS policy settings, traffic steering and application telemetry or CBAR on devices.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**App policy** `app_policy_info` **App policy default** `app_policy_default_info` **App policy intent create** `app_policy_intent_create` **App policy queuing profile** `app_policy_queuing_profile`, `app_policy_queuing_profile_info` **App policy queuing profile count** `app_policy_queuing_profile_count_info` **Application policy application set** `application_policy_application_set`, `application_policy_application_set_info` **Application policy application set count** `application_policy_application_set_count_info` **Application policy workflow manager** `application_policy_workflow_manager` **Application sets** `application_sets`, `application_sets_info` **Application sets count** `application_sets_count_info` **Application visibility network devices** `application_visibility_network_devices_info` **Application visibility network devices count** `application_visibility_network_devices_count_info` **Application visibility network devices disable app telemetry** `application_visibility_network_devices_disable_app_telemetry` **Application visibility network devices disable CBAR** `application_visibility_network_devices_disable_cbar` **Application visibility network devices enable app telemetry** `application_visibility_network_devices_enable_app_telemetry` **Application visibility network devices enable CBAR** `application_visibility_network_devices_enable_cbar` **Applications** `applications`, `applications_info` **Applications count** `applications_count_info` **Applications count v2** `applications_count_v2_info` **Applications v2** `applications_v2`, `applications_v2_info` **App policy queuing profile (playbook)** playbook `app_policy_queuing_profile` **Application policy workflow manager (playbook)** playbook `application_policy_workflow_manager` **Application set info (playbook)** playbook `application_set_info` **Application sets (playbook)** playbook `application_sets` **Applications (playbook)** playbook `applications` **QoS device interface** `qos_device_interface`, `qos_device_interface_info` **QoS device interface info count** `qos_device_interface_info_count_info` **QoS policy setting** `qos_policy_setting`, `qos_policy_setting_info` **Application policy (role)** role `application_policy` **Service provider** `service_provider`, `service_provider_info` **Service provider profile delete** `service_provider_profile_delete` **Service provider v2** `service_provider_v2`, `service_provider_v2_info` **Service provider profile delete v2** `sp_profile_delete_v2` **Traffic steering contracts** `traffic_steering_contracts`, `traffic_steering_contracts_info` **Traffic steering contracts count** `traffic_steering_contracts_count_info` **Traffic steering contracts top n analytics** `traffic_steering_contracts_top_n_analytics_info` **Traffic steering firewalls top n analytics** `traffic_steering_firewalls_top_n_analytics_info` **Traffic steering nodes top n analytics** `traffic_steering_nodes_top_n_analytics_info` **Traffic steering policys** `traffic_steering_policys`, `traffic_steering_policys_info` **Traffic steering policys bulk create** `traffic_steering_policys_bulk_create` **Traffic steering policys count** `traffic_steering_policys_count_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.	

Use Case: Monitor issues and health scores

Description	Query, resolve and ignore assurance issues, define custom issues and health score thresholds, and read device, site and system health.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Application health score definitions** `application_health_score_definitions`, `application_health_score_definitions_info` **Application health score definitions bulk update create** `application_health_score_definitions_bulk_update_create` **Application health score definitions count** `application_health_score_definitions_count_info` **Assurance device health score settings workflow manager** `assurance_device_health_score_settings_workflow_manager` **Assurance events** `assurance_events_info` **Assurance events child events** `assurance_events_child_events_info` **Assurance events count** `assurance_events_count_info` **Assurance events query** `assurance_events_query` **Assurance events query count** `assurance_events_query_count` **Assurance issue workflow manager** `assurance_issue_workflow_manager` **Assurance issues** `assurance_issues_info` **Assurance issues count** `assurance_issues_count_info` **Assurance issues ignore** `assurance_issues_ignore` **Assurance issues query** `assurance_issues_query` **Assurance issues query count** `assurance_issues_query_count` **Assurance issues resolve** `assurance_issues_resolve` **Assurance issues summary analytics** `assurance_issues_summary_analytics` **Assurance issues top n analytics** `assurance_issues_top_n_analytics` **Assurance issues trend analytics** `assurance_issues_trend_analytics` **Assurance issues update** `assurance_issues_update` **Assurance tasks** `assurance_tasks_info` **Assurance tasks count** `assurance_tasks_count_info` **Client health** `client_health_info` **Custom issue definitions** `custom_issue_definitions`, `custom_issue_definitions_info` **Custom issue definitions count** `custom_issue_definitions_count_info` **Device health** `device_health_info` **Catalyst Center health score definitions count** `dna_health_score_definitions_count_info` **Health score definitions** `health_score_definitions`, `health_score_definitions_info` **Health score definitions bulk update** `health_score_definitions_bulk_update` **Health score definitions count** `health_score_definitions_count_info` **Issue enrichment details** `issue_enrichment_details_info` **Issue enrichment details v3** `issue_enrichment_details_v3_info` **Issues** `issues_info` **Issues enrichment details** `issues_enrichment_details_info` **Assurance device health score settings workflow manager (playbook)** playbook `assurance_device_health_score_settings_workflow_manager` **Assurance issue workflow manager (playbook)** playbook `assurance_issue_workflow_manager` **Assurance device health score settings (role)** role `assurance_device_health_score_settings` **Assurance issue (role)** role `assurance_issue` **Site health** `site_health_info` **Site health summaries** `site_health_summaries_info` **Site health summaries count** `site_health_summaries_count_info` **Site health summaries ID trend analytics** `site_health_summaries_id_trend_analytics_info` **Site health summaries summary analytics** `site_health_summaries_summary_analytics`, `site_health_summaries_summary_analytics_info` **Site health summaries trend analytics** `site_health_summaries_trend_analytics_info` **Site KPI summaries** `site_kpi_summaries_info` **Site KPI summaries count** `site_kpi_summaries_count_info` **Site KPI summaries query** `site_kpi_summaries_query` **Site KPI summaries query count** `site_kpi_summaries_query_count` **Site KPI summaries summary analytics** `site_kpi_summaries_summary_analytics`, `site_kpi_summaries_summary_analytics_info` **Site KPI summaries top n analytics** `site_kpi_summaries_top_n_analytics`, `site_kpi_summaries_top_n_analytics_info` **Site KPI summaries trend analytics** `site_kpi_summaries_trend_analytics`, `site_kpi_summaries_trend_analytics_info` **System health** `system_health_info` **System health count** `system_health_count_info` **System issue definitions** `system_issue_definitions`, `system_issue_definitions_info` **System issue definitions count** `system_issue_definitions_count_info` **System performance** `system_performance_info` **System performance historical** `system_performance_historical_info` **Topology network health** `topology_network_health_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Analyse clients, users and applications

Description	Query client and user details, trends and enrichment, tracked clients, and network application and ThousandEyes test results.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Applications health** `applications_health_info` **Client detail** `client_detail_info` **Client enrichment details** `client_enrichment_details_info` **Client enrichment details v2** `client_enrichment_details_v2_info` **Client enrichment details v3** `client_enrichment_details_v3_info` **Client proximity** `client_proximity_info` **Clients** `clients_info` **Clients count** `clients_count_info` **Clients query** `clients_query` **Clients query count** `clients_query_count` **Clients summary analytics** `clients_summary_analytics` **Clients top n analytics** `clients_top_n_analytics` **Clients trend analytics** `clients_trend_analytics` **Clients trend analytics ID** `clients_trend_analytics_id` **Device enrichment details** `device_enrichment_details_info` **Device enrichment details v2** `device_enrichment_details_v2_info` **Device enrichment details v3** `device_enrichment_details_v3_info` **Excluded wireless clients** `excluded_wireless_clients`, `excluded_wireless_clients_info` **Excluded wireless clients bulk create** `excluded_wireless_clients_bulk_create` **Excluded wireless clients count** `excluded_wireless_clients_count_info` **Network applications** `network_applications_info` **Network applications clients** `network_applications_clients_info` **Network applications clients count** `network_applications_clients_count_info` **Network applications count** `network_applications_count_info` **Network applications ID trend analytics** `network_applications_id_trend_analytics` **Network applications summary analytics** `network_applications_summary_analytics` **Network applications trend analytics** `network_applications_trend_analytics` **ThousandEyes test results** `thousand_eyes_test_results_info` **ThousandEyes test results count** `thousand_eyes_test_results_count_info` **ThousandEyes test results path traces** `thousand_eyes_test_results_path_traces_info` **ThousandEyes test results path traces count** `thousand_eyes_test_results_path_traces_count_info` **ThousandEyes test results trend analytics** `thousand_eyes_test_results_trend_analytics_info` **Tracked clients** `tracked_clients`, `tracked_clients_info` **Tracked clients bulk create** `tracked_clients_bulk_create` **Tracked clients bulk delete create** `tracked_clients_bulk_delete_create` **Tracked clients bulk update create** `tracked_clients_bulk_update_create` **Tracked clients query create** `tracked_clients_query_create` **User enrichment details** `user_enrichment_details_info` **User enrichment details v2** `user_enrichment_details_v2_info` **User enrichment details v3** `user_enrichment_details_v3_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Monitor network services, interfaces and energy

Description	Analyse AAA, DHCP and DNS service performance, interface statistics, device KPIs, wireless controller statistics, fabric health and energy consumption.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**AAA services** `aaa_services_info` **AAA services count** `aaa_services_count_info` **AAA services ID trend analytics** `aaa_services_id_trend_analytics` **AAA services query** `aaa_services_query` **AAA services query count** `aaa_services_query_count` **AAA services summary analytics** `aaa_services_summary_analytics` **AAA services top n analytics** `aaa_services_top_n_analytics` **AAA services trend analytics** `aaa_services_trend_analytics` **DHCP services** `dhcp_services_info` **DHCP services count** `dhcp_services_count_info` **DHCP services ID trend analytics** `dhcp_services_id_trend_analytics` **DHCP services query** `dhcp_services_query` **DHCP services query count** `dhcp_services_query_count` **DHCP services summary analytics** `dhcp_services_summary_analytics` **DHCP services top n analytics** `dhcp_services_top_n_analytics` **DHCP services trend analytics** `dhcp_services_trend_analytics` **DNS services** `dns_services_info` **DNS services count** `dns_services_count_info` **DNS services ID trend analytics** `dns_services_id_trend_analytics` **DNS services query** `dns_services_query` **DNS services query count** `dns_services_query_count` **DNS services summary analytics** `dns_services_summary_analytics` **DNS services top n analytics** `dns_services_top_n_analytics` **DNS services trend analytics** `dns_services_trend_analytics` **Energy clients** `energy_clients_info` **Energy clients count** `energy_clients_count_info` **Energy clients query** `energy_clients_query` **Energy clients query count** `energy_clients_query_count` **Energy network devices** `energy_network_devices_info` **Energy network devices count** `energy_network_devices_count_info` **Energy network devices query** `energy_network_devices_query` **Energy network devices query count** `energy_network_devices_query_count` **Energy sites** `energy_sites_info` **Energy sites count** `energy_sites_count_info` **Energy sites query** `energy_sites_query`, `energy_sites_query_info` **Energy sites query count** `energy_sites_query_count`, `energy_sites_query_count_info` **Energy summary analytics** `energy_summary_analytics` **Energy trend analytics** `energy_trend_analytics` **Interfaces** `interfaces_info` **Interfaces count** `interfaces_count_info` **Interfaces ID trend analytics** `interfaces_id_trend_analytics` **Interfaces members associations** `interfaces_members_associations_info` **Interfaces members associations count** `interfaces_members_associations_count_info` **Interfaces members associations query** `interfaces_members_associations_query` **Interfaces query** `interfaces_query` **Interfaces query count** `interfaces_query_count` **Network devices process KPIs** `network_devices_process_kpis_info` **Network devices process KPIs count** `network_devices_process_kpis_count_info` **Network devices process KPIs query count create** `network_devices_process_kpis_query_count_create` **Network devices process KPIs query create** `network_devices_process_kpis_query_create` **Network devices process KPIs trend analytics create** `network_devices_process_kpis_trend_analytics_create` **Network devices summary analytics** `network_devices_summary_analytics` **Network devices top n analytics** `network_devices_top_n_analytics` **Network devices trend analytics** `network_devices_trend_analytics` **Network devices trend analytics ID** `network_devices_trend_analytics_id` **Wireless controllers stats** `wireless_controllers_stats_info` **Wireless controllers stats count** `wireless_controllers_stats_count_info` **Wireless controllers stats query count create** `wireless_controllers_stats_query_count_create` **Wireless controllers stats query create** `wireless_controllers_stats_query_create` **Wireless controllers stats trend analytics create** `wireless_controllers_stats_trend_analytics_create`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Troubleshoot with path trace, packet capture and sensors

Description	Run path traces, configure ICAP and wired packet capture, run diagnostic validations and manage sensor tests.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Assurance ICAP settings workflow manager** `assurance_icap_settings_workflow_manager` **Capture wired** `capture_wired`, `capture_wired_info` **Capture wired configuration models create** `capture_wired_configuration_models_create` **Capture wired configuration models delete** `capture_wired_configuration_models_delete` **Capture wired configuration models deploy create** `capture_wired_configuration_models_deploy_create` **Capture wired configuration models network device status details** `capture_wired_configuration_models_network_device_status_details_info` **Capture wired configuration models network devices config** `capture_wired_configuration_models_network_devices_config`, `capture_wired_configuration_models_network_devices_config_info` **Capture wired count** `capture_wired_count_info` **Capture wired deploy create** `capture_wired_deploy_create` **Diagnostic tasks ID** `diagnostic_tasks_id_info` **Diagnostic tasks ID detail** `diagnostic_tasks_id_detail_info` **Diagnostic validation sets** `diagnostic_validation_sets_info` **Diagnostic validation workflows** `diagnostic_validation_workflows`, `diagnostic_validation_workflows_info` **Diagnostic validation workflows count** `diagnostic_validation_workflows_count_info` **Flow analysis packet capture supported devices** `flow_analysis_packet_capture_supported_devices_info` **Flow analysis packet capture supported devices count** `flow_analysis_packet_capture_supported_devices_count_info` **ICAP capture files** `icap_capture_files_info` **ICAP capture files count** `icap_capture_files_count_info` **ICAP capture files ID download** `icap_capture_files_id_download_info` **ICAP clients ID** `icap_clients_id_stats` **ICAP radios ID** `icap_radios_id_stats` **ICAP settings** `icap_settings_info` **ICAP settings configuration models** `icap_settings_configuration_models` **ICAP settings configuration models ID delete deploy** `icap_settings_configuration_models_id_delete_deploy` **ICAP settings configuration models preview** `icap_settings_configuration_models_previe` **ICAP settings configuration models preview** `icap_settings_configuration_models_previe` **ICAP settings configuration models preview** `icap_settings_configuration_models_previe` **ICAP settings configuration models preview** `icap_settings_configuration_models_previe` `icap_settings_configuration_models_previe` **ICAP settings count** `icap_settings_count_in` **ICAP settings deploy** `icap_settings_deploy` **ICAP settings deploy ID delete deploy** `icap_s` **ICAP settings device deployments** `icap_sett` **ICAP settings device deployments count** `ica` **ICAP spectrum interference device reports** `i` **ICAP spectrum sensor reports** `icap_spectrur` **Path trace** `path_trace`, `path_trace_info` **Path trace workflow manager** `path_trace_wor` **Path traces** `path_traces`, `path_traces_info` **Path traces count** `path_traces_count_info` **Path traces result** `path_traces_result_info` **Assurance ICAP settings workflow manager** **Path trace workflow manager (playbook)** play **Assurance ICAP settings (role)** role `assuranc` **Path trace (role)** role `path_trace` **Sensor** `sensor`, `sensor_info` **Sensor test run** `sensor_test_run` **Sensor test template duplicate** `sensor_test_` **Sensor test template edit** `sensor_test_templa` **Wireless sensor test results** `wireless_sensor`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the r manage a feature.

Use Case: Check network compliance

Description	Define compliance policies, rules and site assignments, run compliance checks, and review and acknowledge violations.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Compliance device** `compliance_device`, `compliance_device_info` **Compliance device by ID** `compliance_device_by_id_info` **Compliance device details** `compliance_device_details_info` **Compliance device details count** `compliance_device_details_count_info` **Compliance device ID** `compliance_device_id_info` **Compliance device status count** `compliance_device_status_count_info` **Compliance network devices detail policys** `compliance_network_devices_detail_policys_info` **Compliance network devices detail policys violations** `compliance_network_devices_detail_policys_violations_info` **Compliance policys** `compliance_policys`, `compliance_policys_info` **Compliance policys count** `compliance_policys_count_info` **Compliance policys rules** `compliance_policys_rules`, `compliance_policys_rules_info` **Compliance policys rules conditions** `compliance_policys_rules_conditions`, `compliance_policys_rules_conditions_info` **Compliance policys rules conditions count** `compliance_policys_rules_conditions_count_info` **Compliance policys rules count** `compliance_policys_rules_count_info` **Compliance policys rules variables** `compliance_policys_rules_variables`, `compliance_policys_rules_variables_info` **Compliance policys rules variables count** `compliance_policys_rules_variables_count_info` **Compliance policys site assignments** `compliance_policys_site_assignments`, `compliance_policys_site_assignments_info` **Compliance policys sites rules variables** `compliance_policys_sites_rules_variables`, `compliance_policys_sites_rules_variables_info` **Compliance sites network devices** `compliance_sites_network_devices_info` **Compliance sites network devices count** `compliance_sites_network_devices_count_info` **Compliance sites network devices summary** `compliance_sites_network_devices_summary_info` **Compliance sites policys** `compliance_sites_policys_info` **Compliance sites policys count** `compliance_sites_policys_count_info` **Compliance sites policys violations** `compliance_sites_policys_violations_info` **Compliance sites policys violations count** `compliance_sites_policys_violations_count_info` **Compliance sites policys violations network devices** `compliance_sites_policys_violations_network_devices_info` **Compliance sites policys violations network devices count** `compliance_sites_policys_violations_network_devices_count_info` **Compliance sites violations** `compliance_sites_violations`, `compliance_sites_violations_info` **Compliance sites violations acknowledge bulk create** `compliance_sites_violations_acknowledge_bulk_create` **Compliance sites violations count** `compliance_sites_violations_count_info` **Compliance sites violations network devices** `compliance_sites_violations_network_devices`, `compliance_sites_violations_network_devices_info` **Compliance sites violations network devices count** `compliance_sites_violations_network_devices_count_info` **Compliance sites violations summary** `compliance_sites_violations_summary_info` **Network compliance workflow manager** `network_compliance_workflow_manager` **Network compliance workflow manager (playbook)** playbook `network_compliance_workflow_manager` **Network compliance (role)** role `network_compliance`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Track security advisories, bugs and field notices

Description	Scan for and review security advisories, software bugs, field notices and end-of-life status, and find insecure device configurations.

Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**EoX status device** `eox_status_device_info` **EoX status summary** `eox_status_summary_info` **Field notices results network devices** `field_notices_results_network_devices_info` **Field notices results network devices count** `field_notices_results_network_devices_count_info` **Field notices results network devices network device ID notices** `field_notices_results_network_devices_network_device_id_notices_info` **Field notices results network devices network device ID notices count** `field_notices_results_network_devices_network_device_id_notices_count_info` **Field notices results notices** `field_notices_results_notices_info` **Field notices results notices count** `field_notices_results_notices_count_info` **Field notices results notices ID network devices** `field_notices_results_notices_id_network_devices_info` **Field notices results notices ID network devices count** `field_notices_results_notices_id_network_devices_count_info` **Field notices results trend** `field_notices_results_trend_info` **Field notices results trend count** `field_notices_results_trend_count_info` **Field notices trials** `field_notices_trials`, `field_notices_trials_info` **Field notices trigger scan** `field_notices_trigger_scan` **Network bugs results bugs** `network_bugs_results_bugs_info` **Network bugs results bugs count** `network_bugs_results_bugs_count_info` **Network bugs results bugs ID network devices** `network_bugs_results_bugs_id_network_devices_info` **Network bugs results bugs ID network devices count** `network_bugs_results_bugs_id_network_devices_count_info` **Network bugs results network devices** `network_bugs_results_network_devices_info` **Network bugs results network devices count** `network_bugs_results_network_devices_count_info` **Network bugs results network devices network device ID bugs** `network_bugs_results_network_devices_network_device_id_bugs_info` **Network bugs results network devices network device ID bugs count** `network_bugs_results_network_devices_network_device_id_bugs_count_info` **Network bugs results trend** `network_bugs_results_trend_info` **Network bugs results trend count** `network_bugs_results_trend_` **Network bugs trials** `network_bugs_trials`, `network_bugs_trials_` **Network bugs trigger scan** `network_bugs_trigger_scan` **Network devices insecure configurations** `network_devices_inse` **Network devices insecure configurations count** `network_devices_insecure_configurations_count_info` **Security advisories** `security_advisories_info` **Security advisories devices** `security_advisories_devices_info` **Security advisories ids per device** `security_advisories_ids_pe` **Security advisories per device** `security_advisories_per_device` **Security advisories results advisories** `security_advisories_res` **Security advisories results advisories count** `security_advisories_results_advisories_count_info` **Security advisories results advisories ID network devices** `security_advisories_results_advisories_id_network_devices_` **Security advisories results advisories ID network devices coun** `security_advisories_results_advisories_id_network_devices_` **Security advisories results network devices** `security_advisories_results_network_devices_info` **Security advisories results network devices count** `security_advisories_results_network_devices_count_info` **Security advisories results network devices network device ID** `security_advisories_results_network_devices_network_device` **Security advisories results network devices network device ID** `security_advisories_results_network_devices_network_device` **Security advisories results trend** `security_advisories_results_` **Security advisories results trend count** `security_advisories_re` **Security advisories summary** `security_advisories_summary_in` **Security advisories trials** `security_advisories_trials`, `security` **Security advisories trigger scan** `security_advisories_trigger_`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the recommended declarative way to manage a feature.

Use Case: Detect rogue devices and secure endpoints

Description	Review threats and rogue devices, manage the rogue allowed list and containment, profile endpoints and set up security service insertion.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Endpoint analytics anc policies** `endpoint_analytics_anc_policies_info` **Endpoint analytics CMDB endpoints** `endpoint_analytics_cmdb_endpoints` **Endpoint analytics dictionaries** `endpoint_analytics_dictionaries_info` **Endpoint analytics endpoints** `endpoint_analytics_endpoints`, `endpoint_analytics_endpoints_info` **Endpoint analytics endpoints anc policy delete** `endpoint_analytics_endpoints_anc_policy_delete` **Endpoint analytics endpoints anc policy update** `endpoint_analytics_endpoints_anc_policy_update` **Endpoint analytics endpoints count** `endpoint_analytics_endpoints_count_info` **Endpoint analytics profiling rules** `endpoint_analytics_profiling_rules`, `endpoint_analytics_profiling_rules_info` **Endpoint analytics profiling rules bulk** `endpoint_analytics_profiling_rules_bulk` **Endpoint analytics profiling rules count** `endpoint_analytics_profiling_rules_count_info` **Endpoint analytics tasks** `endpoint_analytics_tasks_info` **Security rogue additional details** `security_rogue_additional_details` **Security rogue additional details count** `security_rogue_additional_details_count` **Security rogue wireless containment start** `security_rogue_wireless_containment_start` **Security rogue wireless containment status** `security_rogue_wireless_containment_status_info` **Security rogue wireless containment stop** `security_rogue_wireless_containment_stop` **Security service insertion fabric sites readiness** `security_service_insertion_fabric_sites_readiness_info` **Security service insertion fabric sites readiness virtual networks** `security_service_insertion_fabric_sites_readiness_virtual_networks_inf` **Security service insertion summaries** `security_service_insertion_summaries_info` **Security service insertion summaries count** `security_service_insertion_summaries_count_info` **Security service insertion system readiness** `security_service_insertion_system_readiness_info` **Security service insertions** `security_service_insertions`, `security_service_insertions_info` **Security service insertions count** `security_service_insertions_count_info` **Security threats details** `security_threats_details` **Security threats details count** `security_threats_details_count` **Security threats level** `security_threats_level_info` **Security threats rogue allowed** `security_threats_rogue_allowed_list` **Security threats rogue allowed list** `security_threats_rogue_allowed_list_info` **Security threats rogue allowed list count** `security_threats_rogue_allowed_list_count_info` **Security threats summary** `security_threats_summary` **Security threats type** `security_threats_type_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Configure events, notifications and ITSM integration

Description	Create event subscriptions and destinations (email, REST, syslog, SNMP, webhook, SFTP), read event series and audit logs, and integrate ITSM.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)

Key content	**Catalyst Center event SNMP config** `dna_event_snmp_config_info` **Event** `event_info` **Event API status** `event_api_status_info` **Event artifact** `event_artifact_info` **Event artifact count** `event_artifact_count_info` **Event config connector types** `event_config_connector_types_info` **Event count** `event_count_info` **Event destinations settings delete** `event_destinations_settings_delete` **Event email config** `event_email_config`, `event_email_config_info` **Event series** `event_series_info` **Event series audit logs** `event_series_audit_logs_info` **Event series audit logs parent records** `event_series_audit_logs_parent_records_info` **Event series audit logs summary** `event_series_audit_logs_summary_info` **Event series count** `event_series_count_info` **Event SFTP configurations** `event_sftp_configurations`, `event_sftp_configurations_info` **Event SNMP config create** `event_snmp_config_create` **Event SNMP config update** `event_snmp_config_update` **Event subscription** `event_subscription`, `event_subscription_info` **Event subscription count** `event_subscription_count_info` **Event subscription details email** `event_subscription_details_email_info` **Event subscription details REST** `event_subscription_details_rest_info` **Event subscription details syslog** `event_subscription_details_syslog_info` **Event subscription email** `event_subscription_email`, `event_subscription_email_info` **Event subscription REST** `event_subscription_rest`, `event_subscription_rest_info` **Event subscription syslog** `event_subscription_syslog`, `event_subscription_syslog_info` **Event syslog config** `event_syslog_config`, `event_syslog_config_info` **Event webhook** `event_webhook`, `event_webhook_info` **Events and notifications workflow manager** `events_and_notifications_workflow_manager` **Filter group aassociations delete** `filter_group_aassociations_delete` **Filter group associations** `filter_group_associations`, `filter_group_associations_info` **Filter group associations delete** `filter_group_associations_delete` **Filter groups** `filter_groups`, `filter_groups_info` **Integration settings instances ITSM** `integration_settings_instances_itsm`, `integration_settings_instances_itsm_info` **Integration settings ITSM instances** `integration_settings_itsm_instances_info` **Integration settings status** `integration_settings_status_info` **ITSM CMDB sync status** `itsm_cmdb_sync_status_info` **ITSM integration events failed** `itsm_integration_events_failed`, `itsm_integration_events_failed_info` **Event webhook create (playbook)** playbook `event_webhook_create` **Events and notifications workflow manager (playbook)** playbook `events_and_notifications_workflow_manager` **Events and notifications (role)** role `events_and_notifications`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Generate reports and track tasks

Description	Schedule and run reports, read report executions, and follow activities, tasks and their results.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Activities** `activities_info` **Activities activity ID triggered jobs** `activities_activity_id_triggered_jobs_info` **Activities activity ID triggered jobs count** `activities_activity_id_triggered_jobs_count_info` **Activities count** `activities_count_info` **Activities ID** `activities_id_info` **Flexible report content** `flexible_report_content_info` **Flexible report executions** `flexible_report_executions`, `flexible_report_executions_info` **Flexible report schedule** `flexible_report_schedule`, `flexible_report_schedule_info` **Flexible report schedules** `flexible_report_schedules_info` **Reports executions info (playbook)** playbook `reports_executions_info` **Reports workflow manager (playbook)** playbook `reports_workflow_manager` **Reports** `reports`, `reports_info` **Reports executions** `reports_executions_info` **Reports view group** `reports_view_group_info` **Reports view group view** `reports_view_group_view_info` **Reports workflow manager** `reports_workflow_manager` **Reports (role)** role `reports` **Task** `task_info` **Task count** `task_count_info` **Task operation** `task_operation_info` **Task tree** `task_tree_info` **Tasks** `tasks_info` **Tasks count** `tasks_count_info` **Tasks detail** `tasks_detail_info`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Back up, restore and administer the platform

Description	Back up and restore Catalyst Center, check disaster recovery, platform nodes, packages and connection mode, and manage files.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Backup and restore workflow manager** `backup_and_restore_workflow_manager` **Backup configuration** `backup_configuration`, `backup_configuration_info` **Backup NFS configurations** `backup_nfs_configurations`, `backup_nfs_configurations_info` **Backup restore executions** `backup_restore_executions_info` **Backup storages** `backup_storages_info` **Backups** `backups`, `backups_info` **Backups ID restore** `backups_id_restore` **Cisco IMCs** `cisco_imcs`, `cisco_imcs_info` **Connection mode setting** `connection_mode_setting`, `connection_mode_setting_info` **Disasterrecovery system operationstatus** `disasterrecovery_system_operationstatus_info` **Disasterrecovery system status** `disasterrecovery_system_status_info` **Catalyst Center packages** `dnac_packages_info` **Dnacaap management execution status** `dnacaap_management_execution_status_info` **File** `file_info` **File import** `file_import` **File namespaces** `file_namespaces_info` **Platform nodes configuration summary** `platform_nodes_configuration_summary_info` **Platform release summary** `platform_release_summary_info` **Backup and restore workflow manager (playbook)** playbook `backup_and_restore_workflow_manager` **Backup and restore (role)** role `backup_and_restore`

Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Use Case: Tag devices, interfaces and sites

Description	Create tags and manage tag membership for network devices and interfaces.
Collection	`cisco.catalystcenter` (Certified, Cisco, v2.10.2, updated 2026-08-11)
Key content	**Get tag members by ID (playbook)** playbook `get_tag_members_by_id` **Tag (playbook)** playbook `tag` **Tags workflow manager (playbook)** playbook `tags_workflow_manager` **Tags (role)** role `tags` **Tag** `tag`, `tag_info` **Tag count** `tag_count_info` **Tag member** `tag_member`, `tag_member_info` **Tag member count** `tag_member_count_info` **Tag member type** `tag_member_type_info` **Tag membership** `tag_membership` **Tags interfaces members associations** `tags_interfaces_members_associations_info` **Tags interfaces members associations bulk** `tags_interfaces_members_associations_bulk` **Tags interfaces members associations count** `tags_interfaces_members_associations_count_info` **Tags interfaces members associations query** `tags_interfaces_members_associations_query` **Tags network devices members associations** `tags_network_devices_members_associations_info` **Tags network devices members associations bulk** `tags_network_devices_members_associations_bulk` **Tags network devices members associations count** `tags_network_devices_members_associations_count_info` **Tags network devices members associations query** `tags_network_devices_members_associations_query` **Tags workflow manager** `tags_workflow_manager`
Notes	• Most modules are generated from the Catalyst Center API; names ending in _count, _query or _info read data, while *_workflow_manager modules and the matching roles are the recommended declarative way to manage a feature.

Cisco Nexus Dashboard Fabric Controller (NDFC/DCNM) `cisco.dcnm`

Certified Cisco · version 3.11.0 · updated 2026-04-13 · 28 modules → 6 use cases

Use Case: Manage fabrics, switches and links

Description	Create and configure NDFC fabrics and fabric groups, add or remove switches, and manage links, vPC pairs and maintenance mode.
Collection	`cisco.dcnm` (Certified, Cisco, v3.11.0, updated 2026-04-13)
Key content	**Fabrics** `dcnm_fabric` **Fabric groups** `dcnm_fabric_group` **Switch inventory** `dcnm_inventory` **Links** `dcnm_links` **vPC pairs** `dcnm_vpc_pair` **Maintenance mode** `dcnm_maintenance_mode` **Resources** `dcnm_resource_manager`

Use Case: Manage VXLAN networks, VRFs and interfaces

Description	Add, change and remove VRFs, networks and switch interfaces in a VXLAN fabric.
Collection	`cisco.dcnm` (Certified, Cisco, v3.11.0, updated 2026-04-13)
Key content	**VRFs** `dcnm_vrf` **Networks** `dcnm_network` **Interfaces** `dcnm_interface`

Use Case: Manage service insertion

Description	Configure service nodes, service policies and service route peerings in a managed fabric.
Collection	`cisco.dcnm` (Certified, Cisco, v3.11.0, updated 2026-04-13)
Key content	**Service nodes** `dcnm_service_node` **Service policies** `dcnm_service_policy` **Service route peerings** `dcnm_service_route_peering`

Use Case: Manage switch images, policies and templates

Description	Upload images, define image policies, upgrade Nexus switches, manage bootflash files, policies and templates.
Collection	`cisco.dcnm` (Certified, Cisco, v3.11.0, updated 2026-04-13)

Key content	**Image uploads** `dcnm_image_upload` **Image policies** `dcnm_image_policy` **Image upgrades** `dcnm_image_upgrade` **Bootflash files** `dcnm_bootflash` **Switch policies** `dcnm_policy` **Templates** `dcnm_template`

Use Case: Call the controller API and validate data

Description	Send REST requests to the controller, log messages and validate NDFC input data before applying it.
Collection	`cisco.dcnm` (Certified, Cisco, v3.11.0, updated 2026-04-13)
Key content	**REST API requests** `dcnm_rest` **Logging** `dcnm_log` **Interface data validation** `ndfc_interface_validate` **Inventory data validation** `ndfc_inventory_validate` **Network data validation** `ndfc_network_validate` **Port-channel member validation** `ndfc_pc_members_validate` **vPC pair validation** `ndfc_vpc_pair_validate` **Interface schemas** `schemas` **Test utilities** `tools`

Use Case: Run role test playbooks

Description	Example and test playbooks for the bootflash, fabric, image, interface, inventory, network, policy, vPC and VRF roles.
Collection	`cisco.dcnm` (Certified, Cisco, v3.11.0, updated 2026-04-13)
Key content	**Create bootflash files** playbook `roles.dcnm_bootflash.create_files` **Bootflash tests** playbook `roles.dcnm_bootflash.dcnm_tests` **Fabric tests** playbook `roles.dcnm_fabric.dcnm_tests` **Image policy tests** playbook `roles.dcnm_image_policy.dcnm_tests` **Image upgrade tests** playbook `roles.dcnm_image_upgrade.dcnm_tests` **Interface tests** playbook `roles.dcnm_interface.dcnm_tests` **Inventory tests** playbook `roles.dcnm_inventory.dcnm_tests` **Maintenance mode tests** playbook `roles.dcnm_maintenance_mode.dcnm_tests` **Network tests** playbook `roles.dcnm_network.dcnm_tests` **Policy tests** playbook `roles.dcnm_policy.dcnm_tests` **vPC pair tests** playbook `roles.dcnm_vpc_pair.dcnm_tests` **VRF tests** playbook `roles.dcnm_vrf.dcnm_tests` **NDFC interface tests** playbook `roles.ndfc_interface.ndfc_tests`

Cisco Catalyst Center (DNA Center) `cisco.dnac`

Certified Cisco · version 6.48.0 · updated 2026-02-19 · 1136 modules → 33 use cases

Use Case: Design the site hierarchy and network settings

Description	Create areas, buildings and floors, assign devices to sites, and set site-level network settings such as AAA, DHCP, DNS, NTP, banner, time zone, telemetry and image distribution.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Areas** `areas`, `areas_info` **Assign device to site** `assign_device_to_site` **Assign to site apply** `assign_to_site_apply` **Buildings** `buildings`, `buildings_info` **Floors** `floors`, `floors_info` **Floors settings** `floors_settings`, `floors_settings_info` **Floors upload image** `floors_upload_image` **Network** `network_info` **Network create** `network_create` **Network settings intent** `network_settings_intent` **Network settings playbook config generator** `network_settings_playbook_config_generator` **Network settings workflow manager** `network_settings_workflow_manager` **Network update** `network_update` **Network v2** `network_v2`, `network_v2_info` **Provisioning settings** `provisioning_settings`, `provisioning_settings_info` **Site** `site`, `site_info` **Site count** `site_count_info` **Site count v2** `site_count_v2_info` **Site delete** `site_delete` **Site intent** `site_intent` **Site membership** `site_membership_info` **Site playbook config generator** `site_playbook_config_generator` **Site update** `site_update` **Site v2** `site_v2_info` **Site workflow manager** `site_workflow_manager` **Sites** `sites_info` **Sites AAA settings** `sites_aaa_settings`, `sites_aaa_settings_info` **Sites banner settings** `sites_banner_settings`, `sites_banner_settings_info` **Sites bulk** `sites_bulk` **Sites count** `sites_count_info` **Sites DHCP settings** `sites_dhcp_settings`, `sites_dhcp_settings_info` **Sites DNS settings** `sites_dns_settings`, `sites_dns_settings_info` **Sites image distribution settings** `sites_image_distribution_settings`, `sites_image_distribution_settings_info` **Sites NTP settings** `sites_ntp_settings`, `sites_ntp_settings_info` **Sites telemetry settings** `sites_telemetry_settings`, `sites_telemetry_settings_info` **Sites time zone settings** `sites_time_zone_settings`, `sites_time_zone_settings_info` **SNMP properties** `snmp_properties`, `snmp_properties_info` **Telemetry settings apply** `telemetry_settings_apply` **Playbook: Network settings intent** playbook `network_settings_intent` **Playbook: Network settings playbook config generator** playbook `network_settings_playbook_config_generator` **Playbook: Network settings workflow manager** playbook `network_settings_workflow_manager` **Playbook: Site intent** playbook `site_intent` **Playbook: Site playbook config generator** playbook `site_playbook_config_generator` **Playbook: Site workflow manager** playbook `site_workflow_manager`

Use Case: Manage maps and access point placement

Description	Import and export maps, and place, plan and bulk-update access point positions on floors.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Buildings planned access points** `buildings_planned_access_points_info` **Floors floor ID access point positions** `floors_floor_id_access_point_positions_info` **Floors floor ID access point positions bulk change** `floors_floor_id_access_point_positions_bulk_change` **Floors floor ID access point positions count** `floors_floor_id_access_point_positions_count_info` **Floors floor ID planned access point positions** `floors_floor_id_planned_access_point_positions_info` **Floors floor ID planned access point positions assign access point positions** `floors_floor_id_planned_access_point_positions_assign_access_point_positions` **Floors floor ID planned access point positions bulk** `floors_floor_id_planned_access_point_positions_bulk` **Floors floor ID planned access point positions bulk change** `floors_floor_id_planned_access_point_positions_bulk_change` **Floors floor ID planned access point positions count** `floors_floor_id_planned_access_point_positions_count_info` **Floors floor ID planned access point positions ID** `floors_floor_id_planned_access_point_positions_id` **Maps export** `maps_export` **Maps import** `maps_import` **Maps import perform** `maps_import_perform` **Maps import start** `maps_import_start` **Maps import status** `maps_import_status_info` **Maps supported access points** `maps_supported_access_points_info` **Planned access points** `planned_access_points`, `planned_access_points_`

Use Case: Manage device credentials and certificates

Description	Create and assign CLI, SNMP, HTTP and NETCONF credentials globally and per site, export credentials, and import trusted and authentication certificates.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Auth token create** `auth_token_create` **Authentication import certificate** `authentication_import_certificate` **Authentication import certificate p12** `authentication_import_certificate_p12` **CLI credential** `cli_credential` **Credential to site by siteid create v2** `credential_to_site_by_siteid_create_v2` **Device credential** `device_credential`, `device_credential_info` **Device credential intent** `device_credential_intent` **Device credential playbook config generator** `device_credential_playbook_config_generator` **Device credential workflow manager** `device_credential_workflow_manager` **Global credential** `global_credential_info` **Global credential delete** `global_credential_delete` **Global credential update** `global_credential_update` **Global credential v2** `global_credential_v2`, `global_credential_v2_info` **Global credentials** `global_credentials`, `global_credentials_info` **Global credentials count** `global_credentials_count_info` **HTTP read credential** `http_read_credential` **HTTP write credential** `http_write_credential` **Netconf credential** `netconf_credential` **Network devices export credentials create** `network_devices_export_credentials_create` **Site assign credential** `site_assign_credential` **Sites device credentials** `sites_device_credentials`, `sites_device_credentials_info` **Sites device credentials apply** `sites_device_credentials_apply` **Sites device credentials status** `sites_device_credentials_status_info` **Snmpv2 read community credential** `snmpv2_read_community_credential` **Snmpv2 write community credential** `snmpv2_write_community_credential` **Snmpv3 credential** `snmpv3_credential` **Trusted certificates import** `trusted_certificates_import` **Playbook: Authentication import** playbook `authentication_import` **Playbook: Device credential intent** playbook `device_credential_intent` **Playbook: Device credential playbook config generator** playbook `device_credential_playbook_config_generator` **Playbook: Device credential workflow manager** playbook `device_credential_workflow_manager` **Playbook: Global credentials** playbook `global_credentials` **Playbook: Global credentials info** playbook `global_credentials_info`

Use Case: Manage IP address pools

Description	Create global and site IP address pools and subpools, reserve IP subpools and configure the IPAM server settings.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Global pool** `global_pool`, `global_pool_info` **Ipam global IP address pools** `ipam_global_ip_address_pools`, `ipam_global_ip_address_pools_info` **Ipam global IP address pools count** `ipam_global_ip_address_pools_count_info` **Ipam global IP address pools global IP address pool ID subpools** `ipam_global_ip_address_pools_global_ip_address_pool_id_subpools_info` **Ipam global IP address pools global IP address pool ID subpools count** `ipam_global_ip_address_pools_global_ip_address_pool_id_subpools_count_info` **Ipam server setting** `ipam_server_setting`, `ipam_server_setting_info` **Ipam site IP address pools** `ipam_site_ip_address_pools`, `ipam_site_ip_address_pools_info` **Ipam site IP address pools count** `ipam_site_ip_address_pools_count_info` **Reserve IP subpool** `reserve_ip_subpool`, `reserve_ip_subpool_info` **Playbook: Global pool** playbook `global_pool` **Playbook: Reserve IP subpool** playbook `reserve_ip_subpool`

Use Case: Discover network devices

Description	Define and run discovery jobs by IP range, CDP or LLDP, and review discovered devices and job results.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Discovery** `discovery`, `discovery_info` **Discovery count** `discovery_count_info` **Discovery device** `discovery_device_info` **Discovery device count** `discovery_device_count_info` **Discovery device range** `discovery_device_range_info` **Discovery intent** `discovery_intent` **Discovery job** `discovery_job_info` **Discovery playbook config generator** `discovery_playbook_config_generator` **Discovery range** `discovery_range_info` **Discovery range delete** `discovery_range_delete` **Discovery summary** `discovery_summary_info` **Discovery workflow manager** `discovery_workflow_manager` **Discoverys** `discoverys`, `discoverys_info` **Discoverys count** `discoverys_count_info` **Discoverys jobs** `discoverys_jobs`, `discoverys_jobs_info` **Discoverys jobs count** `discoverys_jobs_count_info` **Discoverys jobs discovered network devices** `discoverys_jobs_discovered_network_devices_info` **Discoverys jobs discovered network devices count** `discoverys_jobs_discovered_network_devices_count_info` **Discoverys jobs stop create** `discoverys_jobs_stop_create` **Discoverys jobs summarys** `discoverys_jobs_summarys_info` **Playbook: Discovery** playbook `discovery` **Playbook: Discovery intent** playbook `discovery_intent` **Playbook: Discovery playbook config generator** playbook `discovery_playbook_config_generator` **Playbook: Discovery workflow manager** playbook `discovery_workflow_manager`

Use Case: Manage device inventory and provision devices

Description	Query, add, update, sync and delete inventory devices, manage interfaces, roles, polling and user-defined fields, provision devices to sites and handle device replacement (RMA).
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Device interface** `device_interface_info` **Device interface by IP** `device_interface_by_ip_info` **Device interface count** `device_interface_count_info` **Device interface isis** `device_interface_isis_info` **Device interface OSPF** `device_interface_ospf_info` **Device replacement** `device_replacement`, `device_replacement_info` **Device replacement count** `device_replacement_count_info` **Device replacement deploy** `device_replacement_deploy` **Intent network devices query** `intent_network_devices_query` **Intent network devices query count** `intent_network_devices_query_count` **Interface** `interface_info` **Interface network device** `interface_network_device_info` **Interface network device detail** `interface_network_device_detail_info` **Interface network device range** `interface_network_device_range_info` **Interface operation create** `interface_operation_create` **Interface update** `interface_update` **Inventory intent** `inventory_intent` **Inventory playbook config generator** `inventory_playbook_config_generator` **Inventory workflow manager** `inventory_workflow_manager` **Network device** `network_device`, `network_device_info` **Network device by IP** `network_device_by_ip_info` **Network device by serial number** `network_device_by_serial_number_info` **Network device chassis details** `network_device_chassis_details_info` **Network device count** `network_device_count_info` **Network device custom prompt** `network_device_custom_prompt`, `network_device_custom_prompt_info` **Network device equipment** `network_device_equipment_info` **Network device functional capability** `network_device_functional_capability_info` **Network device global polling interval** `network_device_global_polling_interval_info` **Network device interface neighbor** `network_device_interface_neighbor_info` **Network device interface poe** `network_device_interface_poe_info` **Network device inventory insight link mismatch** `network_device_inventory_insight_link_mismatch_info` **Network device lexicographically sorted** `network_device_lexicographically_sorted_info` **Network device linecard details** `network_device_linecard_details_info` **Network device maintenance schedules** `network_device_maintenance_schedules`, `network_device_maintenance_schedules_info` **Network device maintenance schedules count** `network_device_maintenance_schedules_count_info` **Network device management address update** `network_device_management_address_update` **Network device meraki organization** `network_device_meraki_organization_info` **Network device module** `network_device_module_info` **Network device module count** `network_device_module_count_info` **Network device poe** `network_device_poe_info` **Network device polling interval** `network_device_polling_interval_info` **Network device range** `network_device_range_info` **Network device register for wsa** `network_device_register_for_wsa_info` **Network device replacements** `network_device_replacements_info` **Network device stack details** `network_device_stack_details_info` **Network device summary** `network_device_summary_info` **Network device supervisor card details** `network_device_supervisor_card_details_info` **Network device sync** `network_device_sync` **Network device update role** `network_device_update_role` **Network device user defined field** `network_device_user_defined_field`, `network_device_user_defined_field_info` **Network device user defined field delete** `network_device_user_defined_field_delete` **Network device user defined field update** `network_device_user_defined_field_update` **Network device VLAN** `network_device_vlan_info` **Network devices** `network_devices_info` **Network devices assigned to site** `network_devices_assigned_to_site_info` **Network devices assigned to site count** `network_devices_assigned_to_site_count_info` **Network devices assigned to site ID** `network_devices_assigned_to_site_id_info` **Network devices count** `network_devices_count_info` **Network devices create** `network_devices_create` **Network devices delete with cleanup** `network_devices_delete_with_cleanup` **Network devices delete without cleanup** `network_devices_delete_without_cleanup` **Network devices device controllability settings** `network_devices_device_controllability_settings`, `network_devices_device_controllability_settings_info` **Network devices info workflow manager** `network_devices_info_workflow_manager` **Network devices intent** `network_devices_intent_info` **Network devices intent count** `network_devices_intent_count_info` **Network devices interfaces query** `network_devices_interfaces_query` **Network devices not assigned to site** `network_devices_not_assigned_to_site_info` **Network devices not assigned to site count** `network_devices_not_assigned_to_site_count_info` **Network devices query** `network_devices_query` **Network devices query count** `network_devices_query_count` **Network devices resync interval settings** `network_devices_resync_interval_settings` **Network devices resync interval settings ID** `network_devices_resync_interval_settings_id`, `network_devices_resync_interval_settings_id_info` **Network devices resync interval settings override** `network_devices_resync_interval_settings_override` **Network devices unassign from site apply** `network_devices_unassign_from_site_apply` **Network devices update** `network_devices_update` **Network devices update create** `network_devices_update_create` **Network devices validate device create** `network_devices_validate_device_create` **Provision intent** `provision_intent` **Provision playbook config generator** `provision_playbook_config_generator` **Provision workflow manager** `provision_workflow_manager` **RMA playbook config generator** `rma_playbook_config_generator` **RMA workflow manager** `rma_workflow_manager` **Playbook: Device provision** playbook `device_provision` **Playbook: Device provision workflow** playbook `device_provision_workflow` **Playbook: Inventory device** playbook `inventory_device` **Playbook: Inventory playbook config generator** playbook `inventory_playbook_config_generator` **Playbook: Inventory workflow manager** playbook `inventory_workflow_manager` **Playbook: Network device custom prompt info** playbook `network_device_custom_prompt_info` **Playbook: Network device info** playbook `network_device_info` **Playbook: Network device update role** playbook `network_device_update_role` **Playbook: Network devices info workflow manager** playbook `network_devices_info_workflow_manager` **Playbook: Provision playbook config generator** playbook `provision_playbook_config_generator` **Playbook: Provision workflow manager** playbook `provision_workflow_manager` **Playbook: RMA playbook config generator** playbook `rma_playbook_config_generator` **Playbook: RMA workflow manager** playbook `rma_workflow_manager`

Use Case: Onboard devices with Plug and Play

Description	Import, claim, authorize, reset and unclaim PnP devices, manage PnP workflows, server profiles and smart or virtual accounts, and preview device configuration.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**PnP device** `pnp_device`, `pnp_device_info` **PnP device authorize** `pnp_device_authorize` **PnP device claim** `pnp_device_claim` **PnP device claim to site** `pnp_device_claim_to_site` **PnP device config preview** `pnp_device_config_preview` **PnP device count** `pnp_device_count_info` **PnP device history** `pnp_device_history_info` **PnP device import** `pnp_device_import` **PnP device reset** `pnp_device_reset` **PnP device unclaim** `pnp_device_unclaim` **PnP global settings** `pnp_global_settings`, `pnp_global_settings_info` **PnP intent** `pnp_intent` **PnP playbook config generator** `pnp_playbook_config_generator` **PnP server profile update** `pnp_server_profile_update` **PnP smart account domains** `pnp_smart_account_domains_info` **PnP virtual account add** `pnp_virtual_account_add` **PnP virtual account deregister** `pnp_virtual_account_deregister` **PnP virtual account devices sync** `pnp_virtual_account_devices_sync` **PnP virtual account sync result** `pnp_virtual_account_sync_result_info` **PnP virtual accounts** `pnp_virtual_accounts_info` **PnP workflow** `pnp_workflow`, `pnp_workflow_info` **PnP workflow count** `pnp_workflow_count_info` **PnP workflow manager** `pnp_workflow_manager` **System settings AP PnP location** `system_settings_ap_pnp_location`, `system_settings_ap_pnp_location_info` **Playbook: PnP** playbook `PnP` **Playbook: PnP device** playbook `pnp_device` **Playbook: PnP playbook config generator** playbook `pnp_playbook_config_generator` **Playbook: PnP workflow manager** playbook `pnp_workflow_manager` **Playbook: Template PnP intent** playbook `template_pnp_intent`

Use Case: Author configuration templates and manage device configuration

Description	Create template projects and templates, version, deploy, import and export them, and back up, export and download device configuration files.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Configuration template** `configuration_template`, `configuration_template_info` **Configuration template clone** `configuration_template_clone` **Configuration template create** `configuration_template_create` **Configuration template deploy** `configuration_template_deploy` **Configuration template deploy status** `configuration_template_deploy_status_info` **Configuration template deploy v2** `configuration_template_deploy_v2` **Configuration template export project** `configuration_template_export_project` **Configuration template export template** `configuration_template_export_template` **Configuration template import project** `configuration_template_import_project` **Configuration template import template** `configuration_template_import_template` **Configuration template project** `configuration_template_project`, `configuration_template_project_info` **Configuration template version** `configuration_template_version_info` **Configuration template version create** `configuration_template_version_create` **Device configs backup workflow manager** `device_configs_backup_workflow_manager` **Device configurations export** `device_configurations_export` **Network device config** `network_device_config_info` **Network device config archive** `network_device_config_archive_info` **Network device config count** `network_device_config_count_info` **Network device config files** `network_device_config_files_info` **Network device config files count** `network_device_config_files_count_info` **Network device config files ID download masked** `network_device_config_files_id_download_masked` **Network device config files ID download unmasked** `network_device_config_files_id_download_unmasked` **Network device config task** `network_device_config_task_info` **Network device config write memory** `network_device_config_write_memory` **Network device export** `network_device_export` **Projects** `projects`, `projects_info` **Projects count** `projects_count_info` **Projects details** `projects_details_info` **Template intent** `template_intent` **Template playbook config generator** `template_playbook_config_generator` **Template preview** `template_preview` **Template workflow manager** `template_workflow_manager` **Templates details** `templates_details_info` **Templates template ID versions** `templates_template_id_versions_info` **Templates template ID versions commit** `templates_template_id_versions_commit` **Templates template ID versions count** `templates_template_id_versions_count_info` **Playbook: Configuration template version create** playbook `configuration_template_version_create` **Playbook: Device configs backup workflow manager** playbook `device_configs_backup_workflow_manager` **Playbook: Template** playbook `template` **Playbook: Template playbook config generator** playbook `template_playbook_config_generator` **Playbook: Template workflow manager** playbook `template_workflow_manager` **Playbook: Configs backup** playbook `configs_backup`

Use Case: Manage network profiles and site assignments

Description	Create wireless and switching network profiles and associate them and their templates with sites.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Associate site to network profile** `associate_site_to_network_profile` **Disassociate site to network profile** `disassociate_site_to_network_profile` **Network devices network profiles for sites** `network_devices_network_profiles_for_sites_info` **Network devices network profiles for sites count** `network_devices_network_profiles_for_sites_count_info` **Network devices network profiles for sites delete** `network_devices_network_profiles_for_sites_delete` **Network profile switching playbook config generator** `network_profile_switching_playbook_config_generator` **Network profile switching workflow manager** `network_profile_switching_workflow_manager` **Network profile wireless playbook config generator** `network_profile_wireless_playbook_config_generator` **Network profile wireless workflow manager** `network_profile_wireless_workflow_manager` **Network profiles for sites profile ID templates** `network_profiles_for_sites_profile_id_templates_info` **Network profiles for sites profile ID templates count** `network_profiles_for_sites_profile_id_templates_count_info` **Network profiles for sites site assignments** `network_profiles_for_sites_site_assignments`, `network_profiles_for_sites_site_assignments_info` **Network profiles for sites site assignments bulk create** `network_profiles_for_sites_site_assignments_bulk_create` **Network profiles for sites site assignments bulk delete** `network_profiles_for_sites_site_assignments_bulk_delete` **Network profiles for sites site assignments count** `network_profiles_for_sites_site_assignments_count_info` **Sites profile assignments** `sites_profile_assignments_info` **Sites profile assignments count** `sites_profile_assignments_count_info` **Templates template ID network profiles for sites** `templates_template_id_network_profiles_for_sites`, `templates_template_id_network_profiles_for_sites_info` **Templates template ID network profiles for sites bulk create** `templates_template_id_network_profiles_for_sites_bulk_create` **Templates template ID network profiles for sites bulk delete** `templates_template_id_network_profiles_for_sites_bulk_delete` **Templates template ID network profiles for sites count** `templates_template_id_network_profiles_for_sites_count_info` **Playbook: Network profile switching playbook config generator** playbook `network_profile_switching_playbook_config_generator` **Playbook: Network profile switching workflow manager** playbook `network_profile_switching_workflow_manager` **Playbook: Network profile wireless playbook config generator** playbook `network_profile_wireless_playbook_config_generator` **Playbook: Network profile wireless workflow manager** playbook `network_profile_wireless_workflow_manager`

Use Case: Manage software images and upgrades

Description	Import and distribute software images, tag golden images, activate upgrades and optional packages, and review image and release information per site.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Download software release create** `download_software_release_create` **Download software release delete** `download_software_release_delete` **Download software release update** `download_software_release_update` **Golden image create** `golden_image_create` **Golden tag image delete** `golden_tag_image_delete` **Golden tag image details** `golden_tag_image_details_info` **Images** `images_info` **Images addon images** `images_addon_images_info` **Images addon images count** `images_addon_images_count_info` **Images cco sync** `images_cco_sync` **Images count** `images_count_info` **Images delete** `images_delete` **Images distribution server settings** `images_distribution_server_settings`, `images_distribution_server_settings_info` **Images download** `images_download` **Images ID sites site ID tag golden** `images_id_sites_site_id_tag_golden` **Images ID sites site ID untag golden** `images_id_sites_site_id_untag_golden` **Images site wise product names** `images_site_wise_product_names`, `images_site_wise_product_names_info` **Images site wise product names count** `images_site_wise_product_names_count_info` **Install optional packages create** `install_optional_packages_create` **Installed release** `installed_release_info` **Network device image updates** `network_device_image_updates_info` **Network device image updates count** `network_device_image_updates_count_info` **Network device images** `network_device_images_info` **Network device images activate bulk** `network_device_images_activate_bulk` **Network device images count** `network_device_images_count_info` **Network device images distribute bulk** `network_device_images_distribute_bulk` **Network device images ID activate** `network_device_images_id_activate` **Network device images ID distribute** `network_device_images_id_distribute` **Network device images ID readiness checks** `network_device_images_id_readiness_checks` **Network device images validation results** `network_device_images_validation_results_info` **Network device images validation results count** `network_device_images_validation_results_count_info` **Network device images validations** `network_device_images_validations`, `network_device_images_validations_info` **Network device images validations count** `network_device_images_validations_count_info` **Product names** `product_names_info` **Product names count** `product_names_count_info` **Product series** `product_series_info` **Product series count** `product_series_count_info` **Releases** `releases_info` **Releases release summary** `releases_release_summary_info` **Site wise images summary** `site_wise_images_summary_info` **Site wise product names** `site_wise_product_names_info` **Site wise product names count** `site_wise_product_names_count_info` **Software management executions** `software_management_executions_info` **Swim image details** `swim_image_details_info` **Swim import local** `swim_import_local` **Swim import via URL** `swim_import_via_url` **Swim intent** `swim_intent` **Swim trigger activation** `swim_trigger_activation` **Swim trigger distribution** `swim_trigger_distribution` **Swim workflow manager** `swim_workflow_manager` **Uninstall optional packages create** `uninstall_optional_packages_create` **Upgrade software release create** `upgrade_software_release_create` **Playbook: Import image file** playbook `import_image_file` **Playbook: Swim import local** playbook `swim_import_local` **Playbook: Swim intent** playbook `swim_intent` **Playbook: Swim workflow manager** playbook `swim_workflow_manager`

Use Case: Manage Smart Licensing

Description	Register and deregister devices, change virtual accounts and review license usage, status, terms and device license details.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**License deregister** `license_deregister` **License device count** `license_device_count_info` **License device deregistration** `license_device_deregistration` **License device license details** `license_device_license_details_info` **License device license summary** `license_device_license_summary_info` **License device registration** `license_device_registration` **License last operation status** `license_last_operation_status_info` **License register** `license_register` **License renew** `license_renew` **License setting** `license_setting`, `license_setting_info` **License smart account details** `license_smart_account_details_info` **License status** `license_status_info` **License term details** `license_term_details_info` **License usage details** `license_usage_details_info` **License virtual account change** `license_virtual_account_change` **License virtual account details** `license_virtual_account_details_info` **Network device licenses network device ID port channels** `network_device_licenses_network_device_id_port_channels_info` **Network device licenses network device ID port channels count** `network_device_licenses_network_device_id_port_channels_count_info` **Network device licenses summary** `network_device_licenses_summary_info`

Use Case: Check configuration compliance

Description	Define compliance policies, rules and conditions, assign them to sites, and review device compliance status and violations.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Compliance device** `compliance_device`, `compliance_device_info` **Compliance device by ID** `compliance_device_by_id_info` **Compliance device details** `compliance_device_details_info` **Compliance device details count** `compliance_device_details_count_info` **Compliance device ID** `compliance_device_id_info` **Compliance device status count** `compliance_device_status_count_info` **Compliance network devices detail policys** `compliance_network_devices_detail_policys_info` **Compliance network devices detail policys violations** `compliance_network_devices_detail_policys_violations_info` **Compliance policys** `compliance_policys`, `compliance_policys_info` **Compliance policys count** `compliance_policys_count_info` **Compliance policys rules** `compliance_policys_rules`, `compliance_policys_rules_info` **Compliance policys rules conditions** `compliance_policys_rules_conditions`, `compliance_policys_rules_conditions_info` **Compliance policys rules conditions count** `compliance_policys_rules_conditions_count_info` **Compliance policys rules count** `compliance_policys_rules_count_info` **Compliance policys rules variables** `compliance_policys_rules_variables`, `compliance_policys_rules_variables_info` **Compliance policys rules variables count** `compliance_policys_rules_variables_count_info` **Compliance policys site assignments** `compliance_policys_site_assignments`, `compliance_policys_site_assignments_info` **Compliance policys sites rules variables** `compliance_policys_sites_rules_variables`, `compliance_policys_sites_rules_variables_info` **Network compliance workflow manager** `network_compliance_workflow_manager` **Playbook: Network compliance workflow manager** playbook `network_compliance_workflow_manager`

Use Case: Track security advisories, bugs, field notices and end of life

Description	Run scans and review security advisories, software bugs, field notices and end-of-support (EoX) status for network devices.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**EoX status device** `eox_status_device_info` **EoX status summary** `eox_status_summary_info` **Field notices results network devices** `field_notices_results_network_devices_info` **Field notices results network devices count** `field_notices_results_network_devices_count_info` **Field notices results network devices network device ID notices** `field_notices_results_network_devices_network_device_id_notices_info` **Field notices results network devices network device ID notices count** `field_notices_results_network_devices_network_device_id_notices_count_info` **Field notices results notices** `field_notices_results_notices_info` **Field notices results notices count** `field_notices_results_notices_count_info` **Field notices results notices ID network devices** `field_notices_results_notices_id_network_devices_info` **Field notices results notices ID network devices count** `field_notices_results_notices_id_network_devices_count_info` **Field notices results trend** `field_notices_results_trend_info` **Field notices results trend count** `field_notices_results_trend_count_info` **Field notices trials** `field_notices_trials`, `field_notices_trials_info` **Field notices trigger scan** `field_notices_trigger_scan` **Network bugs results bugs** `network_bugs_results_bugs_info` **Network bugs results bugs count** `network_bugs_results_bugs_count_info` **Network bugs results bugs ID network devices** `network_bugs_results_bugs_id_network_devices_info` **Network bugs results bugs ID network devices count** `network_bugs_results_bugs_id_network_devices_count_info` **Network bugs results network devices** `network_bugs_results_network_devices_info` **Network bugs results network devices count** `network_bugs_results_network_devices_count_info` **Network bugs results network devices network device ID bugs** `network_bugs_results_network_devices_network_device_id_bugs_info` **Network bugs results network devices network device ID bugs** `network_bugs_results_network_devices_network_device_id_bug` **Network bugs results trend** `network_bugs_results_trend_info` **Network bugs results trend count** `network_bugs_results_trend_` **Network bugs trials** `network_bugs_trials`, `network_bugs_trials_` **Network bugs trigger scan** `network_bugs_trigger_scan` **Security advisories** `security_advisories_info` **Security advisories devices** `security_advisories_devices_info` **Security advisories IDs per device** `security_advisories_ids_pe` **Security advisories per device** `security_advisories_per_device` **Security advisories results advisories** `security_advisories_res` **Security advisories results advisories count** `security_advisories_results_advisories_count_info` **Security advisories results advisories ID network devices** `security_advisories_results_advisories_id_network_devices_` **Security advisories results advisories ID network devices coun** `security_advisories_results_advisories_id_network_devices_` **Security advisories results network devices** `security_advisories_results_network_devices_info` **Security advisories results network devices count** `security_advisories_results_network_devices_count_info` **Security advisories results network devices network device ID** `security_advisories_results_network_devices_network_device` **Security advisories results network devices network device ID** `security_advisories_results_network_devices_network_device` **Security advisories results trend** `security_advisories_results_` **Security advisories results trend count** `security_advisories_re` **Security advisories summary** `security_advisories_summary_int` **Security advisories trials** `security_advisories_trials`, `security` **Security advisories trigger scan** `security_advisories_trigger_`

Use Case: Detect rogue devices and security threats

Description	Review rogue and threat details, manage allowed rogue lists and start or stop rogue wireless containment.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Security rogue additional details** `security_rogue_additional_details` **Security rogue additional details count** `security_rogue_additional_details_count` **Security rogue wireless containment start** `security_rogue_wireless_containment_start` **Security rogue wireless containment status** `security_rogue_wireless_containment_status_info` **Security rogue wireless containment stop** `security_rogue_wireless_containment_stop` **Security threats details** `security_threats_details` **Security threats details count** `security_threats_details_count` **Security threats level** `security_threats_level_info` **Security threats rogue allowed** `security_threats_rogue_allowed_list` **Security threats rogue allowed list** `security_threats_rogue_allowed_list_info` **Security threats rogue allowed list count** `security_threats_rogue_allowed_list_count_info` **Security threats summary** `security_threats_summary` **Security threats type** `security_threats_type_info`

Use Case: Automate LAN and wired campus configuration

Description	Run LAN automation sessions and port channels, and manage Layer 2 wired features, configuration models and deployments on wired devices.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Create wired network devices ID config features intended layer2** `create_wired_network_devices_id_config_features_intended_layer2`
	LAN automation count `lan_automation_count_info`
	LAN automation create `lan_automation_create`
	LAN automation delete `lan_automation_delete`
	LAN automation log `lan_automation_log_info`
	LAN automation log by serial number `lan_automation_log_by_serial_number_info`
	LAN automation port channels `lan_automation_port_channels`, `lan_automation_port_channels_info`
	LAN automation port channels ID add link `lan_automation_port_channels_id_add_link`
	LAN automation port channels ID remove link `lan_automation_port_channels_id_remove_link`
	LAN automation sessions `lan_automation_sessions_info`
	LAN automation status `lan_automation_status_info`
	LAN automation update `lan_automation_update`
	LAN automation update device `lan_automation_update_device`
	LAN automation update v2 `lan_automation_update_v2`
	LAN automation v2 `lan_automation_v2`
	LAN automation workflow manager `lan_automation_workflow_manager`
	Update wired network devices ID config features intended layer2 `update_wired_network_devices_id_config_features_intended_layer2`
	Wired campus automation playbook config generator `wired_campus_automation_playbook_config_generator`
	Wired campus automation workflow manager `wired_campus_automation_workflow_manager`
	Wired network devices ID config features deployed layer2 feature `wired_network_devices_id_config_features_deployed_layer2_feature_info`
	Wired network devices ID config features deployed layer2 feature count `wired_network_devices_id_config_features_deployed_layer2_feature_count_info`
		Wired network devices ID
		Wired network devices ID
		`wired_network_devices_id`
		`wired_network_devices_id`
		Wired network devices ID
		`wired_network_devices_id`
		Wired network devices ID
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Wired network devices ne
		`wired_network_devices_ne`
		Playbook: LAN automatio
		Playbook: Wired campus
		Playbook: Wired campus

Use Case: Design SD-Access fabrics, transits and virtual networks

Description	Create fabric sites and zones, transits, Layer 2 and Layer 3 virtual networks, anycast gateways, multicast, extranet policies and authentication profiles.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Business SDA hostonboarding SSID ippool** `business_sda_hostonboarding_ssid_ippool`, `business_sda_hostonboarding_ssid_ippool_info` **Business SDA virtual network summary** `business_sda_virtual_network_summary_info` **Fabric summary** `fabric_summary_info` **SDA anycast gateways** `sda_anycast_gateways`, `sda_anycast_gateways_info` **SDA anycast gateways count** `sda_anycast_gateways_count_info` **SDA authentication profiles** `sda_authentication_profiles`, `sda_authentication_profiles_info` **SDA extranet policies** `sda_extranet_policies`, `sda_extranet_policies_info` **SDA extranet policies count** `sda_extranet_policies_count_info` **SDA extranet policies playbook config generator** `sda_extranet_policies_playbook_config_generator` **SDA extranet policies workflow manager** `sda_extranet_policies_workflow_manager` **SDA fabric authentication profile** `sda_fabric_authentication_profile`, `sda_fabric_authentication_profile_info` **SDA fabric multicast playbook config generator** `sda_fabric_multicast_playbook_config_generator` **SDA fabric multicast workflow manager** `sda_fabric_multicast_workflow_manager` **SDA fabric site** `sda_fabric_site`, `sda_fabric_site_info` **SDA fabric sites** `sda_fabric_sites`, `sda_fabric_sites_info` **SDA fabric sites count** `sda_fabric_sites_count_info` **SDA fabric sites zones playbook config generator** `sda_fabric_sites_zones_playbook_config_generator` **SDA fabric sites zones workflow manager** `sda_fabric_sites_zones_workflow_manager` **SDA fabric transits playbook config generator** `sda_fabric_transits_playbook_config_generator` **SDA fabric transits workflow manager** `sda_fabric_transits_workflow_manager` **SDA fabric virtual networks playbook config generator** `sda_fabric_virtual_networks_playbook_config_generator` **SDA fabric virtual networks workflow manager** `sda_fabric_virtual_networks_workflow_manager` **SDA fabric zones** `sda_fabric_zones`, `sda_fabric_zones_info` **SDA fabric zones count** `sda_fabric_zones_count_info` **SDA fabrics VLAN to SSIDs** `sda_fabrics_vlan_to_ssids_info` **SDA fabrics VLAN to SSIDs count** `sda_fabrics_vlan_to_ssids_count_info` **SDA fabrics VLAN to SSIDs fabric ID** `sda_fabrics_vlan_to_ssids_fabric_id`, `sda_fabrics_vlan_to_ssids_fabric_id_info` **SDA fabrics VLAN to SSIDs fabric ID count** `sda_fabrics_vlan_to_ssids_fabric_id_count_info` **SDA layer2 virtual networks** `sda_layer2_virtual_networks`, `sda_layer2_virtual_networks_info` **SDA layer2 virtual networks count** `sda_layer2_virtual_networks_count_info` **SDA layer3 virtual networks** `sda_layer3_virtual_networks`, `sda_layer3_virtual_networks_info` **SDA layer3 virtual networks count** `sda_layer3_virtual_networks_count_info` **SDA layer3 virtual networks delete** `sda_layer3_virtual_networks_delete` **SDA multicast** `sda_multicast`, `sda_multicast_info` **SDA multicast v1** `sda_multicast_v1`, `sda_multicast_v1_info` **SDA multicast virtual networks** `sda_multicast_virtual_networks`, `sda_multicast_virtual_networks_info` **SDA multicast virtual networks count** `sda_multicast_virtual_networks_count_info` **SDA transit networks** `sda_transit_networks`, `sda_transit_networks_info` **SDA transit networks count** `sda_transit_networks_count_info` **SDA transit networks delete** `sda_transit_networks_delete` **SDA virtual network** `sda_virtual_network`, `sda_virtual_network_info` **SDA virtual network IP pool** `sda_virtual_network_ip_pool`, `sda_virtual_network_ip_pool_info` **SDA virtual network v2** `sda_virtual_network_v2`, `sda_virtual_network_v2_info` **Transit peer network** `transit_peer_network`, `transit_peer_network_info` **Playbook: SDA anycast gateways** playbook `sda_anycast_gateways` **Playbook: SDA extranet policies playbook config generator** playbook `sda_extranet_policies_playbook_config_generator` **Playbook: SDA extranet policies workflow manager** playbook `sda_extranet_policies_workflow_manager` **Playbook: SDA fabric** playbook `sda_fabric` **Playbook: SDA fabric multicast playbook config generator** playbook `sda_fabric_multicast_playbook_config_generator` **Playbook: SDA fabric multicast workflow manager** playbook `sda_fabric_multicast_workflow_manager` **Playbook: SDA fabric site** playbook `sda_fabric_site` **Playbook: SDA fabric sites zones playbook config generator** playbook `sda_fabric_sites_zones_playbook_config_generator` **Playbook: SDA fabric sites zones workflow manager** playbook `sda_fabric_sites_zones_workflow_manager` **Playbook: SDA fabric transits playbook config generator** playbook `sda_fabric_transits_playbook_config_generator` **Playbook: SDA fabric transits workflow manager** playbook `sda_fabric_transits_workflow_manager` **Playbook: SDA fabric virtual networks playbook config generator** playbook `sda_fabric_virtual_networks_playbook_config_generator` **Playbook: SDA fabric virtual networks workflow manager** playbook `sda_fabric_virtual_networks_workflow_manager` **Playbook: SDA layer3 virtual networks** playbook `sda_layer3_virtual_networks` **Playbook: SDA virtual network v2** playbook `sda_virtual_network_v2`

Use Case: Provision SD-Access fabric devices and onboard hosts

Description	Add devices to fabrics with border, control plane and edge roles, configure Layer 2 handoffs, port assignments and port channels, and provision devices and wireless controllers.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Business SDA wireless controller create** `business_sda_wireless_controller_create` **Business SDA wireless controller delete** `business_sda_wireless_controller_delete` **Fabric devices info workflow manager** `fabric_devices_info_workflow_manager` **Fabrics fabric ID switch wireless setting** `fabrics_fabric_id_switch_wireless_setting`, `fabrics_fabric_id_switch_wireless_setting_info` **Fabrics fabric ID switch wireless setting reload** `fabrics_fabric_id_switch_wireless_setting_reload` **Fabrics fabric ID wireless multicast** `fabrics_fabric_id_wireless_multicast`, `fabrics_fabric_id_wireless_multicast_info` **SDA device** `sda_device_info` **SDA device role** `sda_device_role_info` **SDA fabric border device** `sda_fabric_border_device`, `sda_fabric_border_device_info` **SDA fabric control plane device** `sda_fabric_control_plane_device`, `sda_fabric_control_plane_device_info` **SDA fabric devices** `sda_fabric_devices`, `sda_fabric_devices_info` **SDA fabric devices count** `sda_fabric_devices_count_info` **SDA fabric devices layer2 handoffs** `sda_fabric_devices_layer2_handoffs`, `sda_fabric_devices_layer2_handoffs_info` **SDA fabric devices layer2 handoffs count** `sda_fabric_devices_layer2_handoffs_count_info` **SDA fabric devices layer2 handoffs IP transits** `sda_fabric_devices_layer2_handoffs_ip_transits`, `sda_fabric_devices_layer2_handoffs_ip_transits_info` **SDA fabric devices layer2 handoffs IP transits count** `sda_fabric_devices_layer2_handoffs_ip_transits_count_info` **SDA fabric devices layer2 handoffs SDA transits** `sda_fabric_devices_layer2_handoffs_sda_transits`, `sda_fabric_devices_layer2_handoffs_sda_transits_info` **SDA fabric devices layer2 handoffs SDA transits count** `sda_fabric_devices_layer2_handoffs_sda_transits_count_info` **SDA fabric devices playbook config generator** `sda_fabric_devices_playbook_config_generator` **SDA fabric devices workflow manager** `sda_fabric_devices_workflow_manager` **SDA fabric edge device** `sda_fabric_edge_device`, `sda_fabric_edge_device_info` **SDA host port onboarding playbook config generator** `sda_host_port_onboarding_playbook_config_generator` **SDA host port onboarding workflow manager** `sda_host_port_onboarding_workflow_manager` **SDA pending fabric events** `sda_pending_fabric_events_info` **SDA pending fabric events apply** `sda_pending_fabric_events_apply` **SDA port assignment for access point** `sda_port_assignment_for_access_point`, `sda_port_assignment_for_access_point_info` **SDA port assignment for user device** `sda_port_assignment_for_user_device`, `sda_port_assignment_for_user_device_info` **SDA port assignments** `sda_port_assignments`, `sda_port_assignments_info` **SDA port assignments count** `sda_port_assignments_count_info` **SDA port channels** `sda_port_channels`, `sda_port_channels_info` **SDA port channels count** `sda_port_channels_count_info` **SDA provision device** `sda_provision_device`, `sda_provision_device_info` **SDA provision devices** `sda_provision_devices`, `sda_provision_devices_info` **SDA provision devices count** `sda_provision_devices_count_info` **SDA site member member** `sda_site_member_member_info` **Playbook: Fabric devices info workflow manager** playbook `fabric_devices_info_workflow_manager` **Playbook: SDA fabric devices playbook config generator** playbook `sda_fabric_devices_playbook_config_generator` **Playbook: SDA fabric devices workflow manager** playbook `sda_fabric_devices_workflow_manager` **Playbook: SDA host port onboarding playbook config generator** playbook `sda_host_port_onboarding_playbook_config_generator` **Playbook: SDA host port onboarding workflow manager** playbook `sda_host_port_onboarding_workflow_manager`

Use Case: Design wireless profiles, SSIDs and RF settings

Description	Create wireless profiles, SSIDs, interfaces, RF, AP, power and 802.11be profiles, anchor groups and policy tags, and wireless feature templates.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Feature templates wireless AAA RADIUS attributes configurations** `feature_templates_wireless_aaa_radius_attributes_configurations`, `feature_templates_wireless_aaa_radius_attributes_configurations_info` **Feature templates wireless advanced SSID configurations** `feature_templates_wireless_advanced_ssid_configurations`, `feature_templates_wireless_advanced_ssid_configurations_info` **Feature templates wireless clean air configurations** `feature_templates_wireless_clean_air_configurations`, `feature_templates_wireless_clean_air_configurations_info` **Feature templates wireless dot11ax configurations** `feature_templates_wireless_dot11ax_configurations`, `feature_templates_wireless_dot11ax_configurations_info` **Feature templates wireless dot11be status configurations** `feature_templates_wireless_dot11be_status_configurations`, `feature_templates_wireless_dot11be_status_configurations_info` **Feature templates wireless event driven RRM configurations** `feature_templates_wireless_event_driven_rrm_configurations`, `feature_templates_wireless_event_driven_rrm_configurations_info` **Feature templates wireless flex connect configurations** `feature_templates_wireless_flex_connect_configurations`, `feature_templates_wireless_flex_connect_configurations_info` **Feature templates wireless multicast configurations** `feature_templates_wireless_multicast_configurations`, `feature_templates_wireless_multicast_configurations_info` **Feature templates wireless RRM fra configurations** `feature_templates_wireless_rrm_fra_configurations`, `feature_templates_wireless_rrm_fra_configurations_info` **Feature templates wireless RRM general configurations** `feature_templates_wireless_rrm_general_configurations`, `feature_templates_wireless_rrm_general_configurations_info` **Feature templates wireless summary** `feature_templates_wireless_summary_info` **Sites site ID wireless settings flex connect AAA override** `sites_site_id_wireless_settings_flex_connect_aaa_override`, `sites_site_id_wireless_settings_flex_connect_aaa_override_info` **Sites site ID wireless settings flex connect native VLAN** `sites_site_id_wireless_settings_flex_connect_native_vlan`, `sites_site_id_wireless_settings_flex_connect_native_vlan_info` **Sites site ID wireless settings SSIDs ID update** `sites_site_id_wireless_settings_ssids_id_update` **Sites wireless settings SSIDs** `sites_wireless_settings_ssids`, `sites_wireless_settings_ssids_info` **Sites wireless settings SSIDs count** `sites_wireless_settings_ssids_count_info` **Wireless design playbook config generator** `wireless_design_playbook_config_generator` **Wireless design workflow manager** `wireless_design_workflow_manager` **Wireless dynamic interface** `wireless_dynamic_interface`, `wireless_dynamic_interface_info` **Wireless enterprise SSID** `wireless_enterprise_ssid`, `wireless_enterprise_ssid_info` **Wireless profile** `wireless_profile`, `wireless_profile_info` **Wireless profiles** `wireless_profiles`, `wireless_profiles_info` **Wireless profiles count** `wireless_profiles_count_info` **Wireless profiles ID policy tags bulk** `wireless_profiles_id_policy_tags_bulk` **Wireless profiles ID policy tags count** `wireless_profiles_id_policy_tags_count_info` **Wireless profiles ID policy tags policy tag ID** `wireless_profiles_id_policy_tags_policy_tag_id`, `wireless_profiles_id_policy_tags_policy_tag_id_info` **Wireless profiles ID site tags** `wireless_profiles_id_site_tags`, `wireless_profiles_id_site_tags_info` **Wireless profiles ID site tags bulk** `wireless_profiles_id_site_tags_bulk` **Wireless profiles ID site tags count** `wireless_profiles_id_site_tags_count_info` **Wireless profiles policy tags** `wireless_profiles_policy_tags_info` **Wireless PSK override** `wireless_psk_override` **Wireless RF profile** `wireless_rf_profile`, `wireless_rf_profile_info` **Wireless settings anchor groups** `wireless_settings_anchor_groups`, `wireless_settings_anchor_groups_info` **Wireless settings anchor groups count** `wireless_settings_anchor_groups_count_info` **Wireless settings AP authorization lists** `wireless_settings_ap_authorization_lists`, `wireless_settings_ap_authorization_lists_info` **Wireless settings AP authorization lists count** `wireless_settings_ap_authorization_lists_count_info` **Wireless settings AP profiles** `wireless_settings_ap_profiles`, `wireless_settings_ap_profiles_info` **Wireless settings AP profiles count** `wireless_settings_ap_profiles_count_info` **Wireless settings certificate renewal profiles** `wireless_settings_certificate_renewal_profiles`, `wireless_settings_certificate_renewal_profiles_info` **Wireless settings certificate renewal profiles count** `wireless_settings_certificate_renewal_profiles_count_info` **Wireless settings dot11be profiles** `wireless_settings_dot11be_profiles`, `wireless_settings_dot11be_profiles_info` **Wireless settings dot11be profiles count** `wireless_settings_dot11be_profiles_count_info` **Wireless settings interfaces** `wireless_settings_interfaces`, `wireless_settings_interfaces_info` **Wireless settings interfaces count** `wireless_settings_interfaces_count_info` **Wireless settings network device ID assign anchor managed AP locations** `wireless_settings_network_device_id_assign_anchor_managed_ap_locations` **Wireless settings power profiles** `wireless_settings_power_profiles`, `wireless_settings_power_profiles_info` **Wireless settings power profiles count** `wireless_settings_power_profiles_count_info` **Wireless settings RF profiles** `wireless_settings_rf_profiles`, `wireless_settings_rf_profiles_info` **Wireless settings RF profiles count** `wireless_settings_rf_profiles_count_info` **Wireless settings SSIDs override at sites** `wireless_settings_ssids_override_at_sites_info` **Playbook: Wireless design playbook config generator** playbook `wireless_design_playbook_config_generator` **Playbook: Wireless design workflow manager** playbook `wireless_design_workflow_manager` **Playbook: Wireless enterprise SSID** playbook `wireless_enterprise_ssid`

Use Case: Provision wireless controllers and access points

Description	Provision controllers, access points and SSIDs, assign AP locations, configure mobility groups and AP authorization lists, and review access point configuration.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Accesspoint configuration details by task ID** `accesspoint_configuration_details_by_task_id_info` **Accesspoint location playbook config generator** `accesspoint_location_playbook_config_generator` **Accesspoint location workflow manager** `accesspoint_location_workflow_manager` **Accesspoint playbook config generator** `accesspoint_playbook_config_generator` **Accesspoint workflow manager** `accesspoint_workflow_manager` **Device reboot apreboot** `device_reboot_apreboot`, `device_reboot_apreboot_info` **Network device wireless LAN** `network_device_wireless_lan_info` **Wireless accespoint configuration** `wireless_accespoint_configuration` **Wireless access points factory reset request provision** `wireless_access_points_factory_reset_request_provision` **Wireless access points factory reset request status** `wireless_access_points_factory_reset_request_status_info` **Wireless access points provision** `wireless_access_points_provision` **Wireless accesspoint configuration count** `wireless_accesspoint_configuration_count_info` **Wireless accesspoint configuration create** `wireless_accesspoint_configuration_create` **Wireless accesspoint configuration summary** `wireless_accesspoint_configuration_summary_info` **Wireless controllers anchor capable devices** `wireless_controllers_anchor_capable_devices_info` **Wireless controllers anchor managed AP locations** `wireless_controllers_anchor_managed_ap_locations_info` **Wireless controllers assign managed AP locations** `wireless_controllers_assign_managed_ap_locations` **Wireless controllers certificate renewal create** `wireless_controllers_certificate_renewal_create` **Wireless controllers managed AP locations count** `wireless_controllers_managed_ap_locations_count_info` **Wireless controllers mesh AP neighbours** `wireless_controllers_mesh_ap_neighbours_info` **Wireless controllers mesh AP neighbours count** `wireless_controllers_mesh_ap_neighbours_count_info` **Wireless controllers network device ID AP authorization lists** `wireless_controllers_network_device_id_ap_authorization_lists_info` **Wireless controllers network device ID provision status** `wireless_controllers_network_device_id_provision_status_info` **Wireless controllers primary managed AP locations** `wireless_controllers_primary_managed_ap_locations_info` **Wireless controllers provision** `wireless_controllers_provision` **Wireless controllers secondary managed AP locations** `wireless_controllers_secondary_managed_ap_locations_info` **Wireless controllers site tags** `wireless_controllers_site_tags_info` **Wireless controllers SSID details** `wireless_controllers_ssid_details_info` **Wireless controllers SSID details count** `wireless_controllers_ssid_details_count_info` **Wireless controllers wireless mobility groups** `wireless_controllers_wireless_mobility_groups_info` **Wireless controllers wireless mobility groups count** `wireless_controllers_wireless_mobility_groups_count_info` **Wireless controllers wireless mobility groups mobility provision** `wireless_controllers_wireless_mobility_groups_mobility_provision` **Wireless controllers wireless mobility groups mobility reset** `wireless_controllers_wireless_mobility_groups_mobility_reset` **Wireless provision access point** `wireless_provision_access_point` **Wireless provision device create** `wireless_provision_device_create` **Wireless provision device update** `wireless_provision_device_update` **Wireless provision SSID create provision** `wireless_provision_ssid_create_provision` **Wireless provision SSID delete reprovision** `wireless_provision_ssid_delete_reprovision` **Playbook: Accesspoint location playbook config generator** playbook `accesspoint_location_playbook_config_generator` **Playbook: Accesspoint location workflow manager** playbook `accesspoint_location_workflow_manager` **Playbook: Accesspoint playbook config generator** playbook `accesspoint_playbook_config_generator` **Playbook: Accesspoint workflow manager** playbook `accesspoint_workflow_manager`

Use Case: Troubleshoot and test the network

Description	Run path trace, CLI commands and diagnostic validations, and create sensor and ThousandEyes tests and review their results.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Command runner run command** `command_runner_run_command` **Diagnostic tasks ID** `diagnostic_tasks_id_info` **Diagnostic tasks ID detail** `diagnostic_tasks_id_detail_info` **Diagnostic validation sets** `diagnostic_validation_sets_info` **Diagnostic validation workflows** `diagnostic_validation_workflows`, `diagnostic_validation_workflows_info` **Diagnostic validation workflows count** `diagnostic_validation_workflows_count_info` **Dna command runner keywords** `dna_command_runner_keywords_info` **Execute suggested actions commands** `execute_suggested_actions_commands` **Path trace** `path_trace`, `path_trace_info` **Path trace workflow manager** `path_trace_workflow_manager` **Sensor** `sensor`, `sensor_info` **Sensor test run** `sensor_test_run` **Sensor test template duplicate** `sensor_test_template_duplicate` **Sensor test template edit** `sensor_test_template_edit` **Thousand eyes test results** `thousand_eyes_test_results_info` **Thousand eyes test results count** `thousand_eyes_test_results_count_info` **Thousand eyes test results trend analytics** `thousand_eyes_test_results_trend_analytics_info` **Wireless sensor test results** `wireless_sensor_test_results_info` **Playbook: Command runner run command** playbook `command_runner_run_command` **Playbook: Download device cmd runner output** playbook `download_device_cmd_runner_output` **Playbook: Path trace workflow manager** playbook `path_trace_workflow_manager`

Use Case: Capture RF and packet data with Intelligent Capture

Description	Configure Intelligent Capture settings and deployments, and retrieve capture files, client and radio data and spectrum interference reports.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Assurance icap settings workflow manager** `assurance_icap_settings_workflow_manager` **Icap capture files** `icap_capture_files_info` **Icap capture files count** `icap_capture_files_count_info` **Icap capture files ID download** `icap_capture_files_id_download_info` **Icap clients ID** `icap_clients_id_stats` **Icap radios ID** `icap_radios_id_stats` **Icap settings** `icap_settings_info` **Icap settings configuration models** `icap_settings_configuration_models` **Icap settings configuration models ID delete deploy** `icap_settings_configuration_models_id_delete_deploy` **Icap settings configuration models preview activity ID** `icap_settings_configuration_models_preview_activity_id` **Icap settings configuration models preview activity ID deploy** `icap_settings_configuration_models_preview_activity_id_deploy` **Icap settings configuration models preview a** `icap_settings_configuration_models_previe` **Icap settings configuration models preview a** `icap_settings_configuration_models_previe` `icap_settings_configuration_models_previe` **Icap settings count** `icap_settings_count_inf` **Icap settings deploy** `icap_settings_deploy` **Icap settings deploy ID delete deploy** `icap_s` **Icap settings device deployments** `icap_setti` **Icap settings device deployments count** `icap` **Icap spectrum interference device reports** `ic` **Icap spectrum sensor reports** `icap_spectrum` **Playbook: Assurance icap settings workflow**

Use Case: Manage assurance issues, events and health score settings

Description	Query, resolve and ignore assurance issues and events, define custom issue definitions, and adjust health score definitions.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Assurance device health score settings playbook config generator** `assurance_device_health_score_settings_playbook_config_generator` **Assurance device health score settings workflow manager** `assurance_device_health_score_settings_workflow_manager` **Assurance events** `assurance_events_info` **Assurance events child events** `assurance_events_child_events_info` **Assurance events count** `assurance_events_count_info` **Assurance events query** `assurance_events_query` **Assurance events query count** `assurance_events_query_count` **Assurance issue playbook config generator** `assurance_issue_playbook_config_generator` **Assurance issue workflow manager** `assurance_issue_workflow_manager` **Assurance issues** `assurance_issues_info` **Assurance issues count** `assurance_issues_count_info` **Assurance issues ignore** `assurance_issues_ignore` **Assurance issues query** `assurance_issues_query` **Assurance issues query count** `assurance_issues_query_count` **Assurance issues resolve** `assurance_issues_resolve` **Assurance issues summary analytics** `assurance_issues_summary_analytics` **Assurance issues top n analytics** `assurance_issues_top_n_analytics` **Assurance issues trend analytics** `assurance_issues_trend_analytics` **Assurance issues update** `assurance_issues_update` **Assurance tasks** `assurance_tasks_info` **Assurance tasks count** `assurance_tasks_count_info` **Custom issue definitions** `custom_issue_definitions`, `custom_issue_definitions_info` **Custom issue definitions count** `custom_issue_definitions_count_info` **Dna health score definitions count** `dna_health_score_definitions_count_info` **Health score definitions** `health_score_definitions`, `health_score_definitions_info` **Health score definitions bulk update** `health_score_definitions_bulk_update` **Health score definitions count** `health_score_definitions_count_info` **Issue enrichment details** `issue_enrichment_details_info` **Issues** `issues_info` **Issues enrichment details** `issues_enrichment_details_info` **Network devices issues remediation provision** `network_devices_issues_remediation_provision` **System issue definitions** `system_issue_definitions`, `system_issue_definitions_info` **System issue definitions count** `system_issue_definitions_count_info` **Playbook: Assurance device health score settings playbook config generator** playbook `assurance_device_health_score_settings_playbook_config_generator` **Playbook: Assurance device health score settings workflow manager** playbook `assurance_device_health_score_settings_workflow_manager` **Playbook: Assurance issue playbook config generator** playbook `assurance_issue_playbook_config_generator` **Playbook: Assurance issue workflow manager** playbook `assurance_issue_workflow_manager`

Use Case: Monitor site, device and fabric health

Description	Query site, device, interface, fabric and transit health, site KPIs, trends and network topology.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Device details** `device_details_info` **Device enrichment details** `device_enrichment_details_info` **Device enrichment details v2** `device_enrichment_details_v2_info` **Device family identifiers details** `device_family_identifiers_details_info` **Device health** `device_health_info` **Fabric site health summaries** `fabric_site_health_summaries_info` **Fabric site health summaries count** `fabric_site_health_summaries_count_info` **Fabric site health summaries ID trend analytics** `fabric_site_health_summaries_id_trend_analytics_info` **Interfaces** `interfaces_info` **Interfaces count** `interfaces_count_info` **Interfaces ID trend analytics** `interfaces_id_trend_analytics` **Interfaces query** `interfaces_query` **Interfaces query count** `interfaces_query_count` **Network devices summary analytics** `network_devices_summary_analytics` **Network devices top n analytics** `network_devices_top_n_analytics` **Network devices trend analytics** `network_devices_trend_analytics` **Network devices trend analytics ID** `network_devices_trend_analytics_id` **Site health** `site_health_info` **Site health summaries** `site_health_summaries_info` **Site health summaries count** `site_health_summaries_count_info` **Site health summaries ID trend analytics** `site_health_summaries_id_trend_analytics_info` **Site health summaries summary analytics** `site_health_summaries_summary_analytics`, `site_health_summaries_summary_analytics_info` **Site health summaries trend analytics** `site_health_summaries_trend_analytics_info` **Site kpi summaries** `site_kpi_summaries_info` **Site kpi summaries count** `site_kpi_summaries_count_info` **Site kpi summaries query** `site_kpi_summaries_query` **Site kpi summaries query count** `site_kpi_summaries_query_count` **Site kpi summaries summary analytics** `site_kpi_summaries_summary_analytics`, `site_kpi_summaries_summary_analytics_info` **Site kpi summaries top n analytics** `site_kpi_summaries_top_n_analytics`, `site_kpi_summaries_top_n_analytics_info` **Site kpi summaries trend analytics** `site_kpi_summaries_trend_analytics`, `site_kpi_summaries_trend_analytics_info` **Topology layer 2** `topology_layer_2_info` **Topology layer 3** `topology_layer_3_info` **Topology network health** `topology_network_health_info` **Topology physical** `topology_physical_info` **Topology site** `topology_site_info` **Topology VLAN details** `topology_vlan_details_info` **Transit network health summaries** `transit_network_health_summaries_info` **Transit network health summaries count** `transit_network_health_summaries_count_info` **Transit network health summaries ID trend analytics** `transit_network_health_summaries_id_trend_analytics_info` **Virtual network health summaries** `virtual_network_health_summaries_info` **Virtual network health summaries count** `virtual_network_health_summaries_count_info` **Virtual network health summaries ID trend analytics** `virtual_network_health_summaries_id_trend_analytics_info`

Use Case: Monitor clients and energy usage

Description	Query client details, health and trends, user enrichment data, and energy use by client, device and site.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Client detail** `client_detail_info` **Client enrichment details** `client_enrichment_details_info` **Client enrichment details v2** `client_enrichment_details_v2_info` **Client health** `client_health_info` **Client proximity** `client_proximity_info` **Clients** `clients_info` **Clients count** `clients_count_info` **Clients query** `clients_query` **Clients query count** `clients_query_count` **Clients summary analytics** `clients_summary_analytics` **Clients top n analytics** `clients_top_n_analytics` **Clients trend analytics** `clients_trend_analytics` **Clients trend analytics ID** `clients_trend_analytics_id` **Energy clients** `energy_clients_info` **Energy clients count** `energy_clients_count_info` **Energy clients query** `energy_clients_query` **Energy clients query count** `energy_clients_query_count` **Energy network devices** `energy_network_devices_info` **Energy network devices count** `energy_network_devices_count_info` **Energy network devices query** `energy_network_devices_query` **Energy network devices query count** `energy_network_devices_query_count` **Energy sites** `energy_sites_info` **Energy sites count** `energy_sites_count_info` **Energy sites query** `energy_sites_query`, `energy_sites_query_info` **Energy sites query count** `energy_sites_query_count`, `energy_sites_query_count_info` **Energy summary analytics** `energy_summary_analytics` **Energy trend analytics** `energy_trend_analytics` **User enrichment details** `user_enrichment_details_info` **User enrichment details v2** `user_enrichment_details_v2_info`

Use Case: Monitor applications and network services

Description	Query application health and AAA, DHCP and DNS service performance, including summaries, top-N lists and trends.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**AAA services** `aaa_services_info` **AAA services count** `aaa_services_count_info` **AAA services ID trend analytics** `aaa_services_id_trend_analytics` **AAA services query** `aaa_services_query` **AAA services query count** `aaa_services_query_count` **AAA services summary analytics** `aaa_services_summary_analytics` **AAA services top n analytics** `aaa_services_top_n_analytics` **AAA services trend analytics** `aaa_services_trend_analytics` **Applications health** `applications_health_info` **DHCP services** `dhcp_services_info` **DHCP services count** `dhcp_services_count_info` **DHCP services ID trend analytics** `dhcp_services_id_trend_analytics` **DHCP services query** `dhcp_services_query` **DHCP services query count** `dhcp_services_query_count` **DHCP services summary analytics** `dhcp_services_summary_analytics` **DHCP services top n analytics** `dhcp_services_top_n_analytics` **DHCP services trend analytics** `dhcp_services_trend_analytics` **DNS services** `dns_services_info` **DNS services count** `dns_services_count_info` **DNS services ID trend analytics** `dns_services_id_trend_analytics` **DNS services query** `dns_services_query` **DNS services query count** `dns_services_query_count` **DNS services summary analytics** `dns_services_summary_analytics` **DNS services top n analytics** `dns_services_top_n_analytics` **DNS services trend analytics** `dns_services_trend_analytics` **Network applications** `network_applications_info` **Network applications count** `network_applications_count_info` **Network applications ID trend analytics** `network_applications_id_trend_analytics` **Network applications summary analytics** `network_applications_summary_analytics` **Network applications trend analytics** `network_applications_trend_analytics`

Use Case: Manage application policy, QoS and application visibility

Description	Define applications, application sets and application policies, queuing profiles and QoS settings, enable application telemetry and CBAR on devices, and manage service provider profiles.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**App policy** `app_policy_info` **App policy default** `app_policy_default_info` **App policy intent create** `app_policy_intent_create` **App policy queuing profile** `app_policy_queuing_profile`, `app_policy_queuing_profile_info` **App policy queuing profile count** `app_policy_queuing_profile_count_info` **Application policy application set** `application_policy_application_set`, `application_policy_application_set_info` **Application policy application set count** `application_policy_application_set_count_info` **Application policy playbook config generator** `application_policy_playbook_config_generator` **Application policy workflow manager** `application_policy_workflow_manager` **Application sets** `application_sets`, `application_sets_info` **Application sets count** `application_sets_count_info` **Application visibility network devices** `application_visibility_network_devices_info` **Application visibility network devices count** `application_visibility_network_devices_count_info` **Application visibility network devices disable app telemetry** `application_visibility_network_devices_disable_app_telemetry` **Application visibility network devices disable cbar** `application_visibility_network_devices_disable_cbar` **Application visibility network devices enable app telemetry** `application_visibility_network_devices_enable_app_telemetry` **Application visibility network devices enable cbar** `application_visibility_network_devices_enable_cbar` **Applications** `applications`, `applications_info` **Applications count** `applications_count_info` **Applications count v2** `applications_count_v2_info` **Applications v2** `applications_v2`, `applications_v2_info` **QoS device interface** `qos_device_interface`, `qos_device_interface_info` **QoS device interface info count** `qos_device_interface_info_count_info` **QoS policy setting** `qos_policy_setting`, `qos_policy_setting_info` **Service provider** `service_provider`, `service_provider_info` **Service provider profile delete** `service_provider_profile_delete` **Service provider v2** `service_provider_v2`, `service_provider_v2_info` **Sp profile delete v2** `sp_profile_delete_v2` **Playbook: Application policy playbook config generator** playbook `application_policy_playbook_config_generator` **Playbook: Application policy workflow manager** playbook `application_policy_workflow_manager` **Playbook: Application set info** playbook `application_set_info` **Playbook: Application sets** playbook `application_sets` **Playbook: Applications** playbook `applications`

Use Case: Configure events, notifications and webhooks

Description	Manage event subscriptions, email, REST, syslog and SNMP destinations and webhooks, and query events and audit logs.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Dna event SNMP config** `dna_event_snmp_config_info` **Event** `event_info` **Event API status** `event_api_status_info` **Event artifact** `event_artifact_info` **Event artifact count** `event_artifact_count_info` **Event config connector types** `event_config_connector_types_info` **Event count** `event_count_info` **Event email config** `event_email_config`, `event_email_config_info` **Event series** `event_series_info` **Event series audit logs** `event_series_audit_logs_info` **Event series audit logs parent records** `event_series_audit_logs_parent_records_info` **Event series audit logs summary** `event_series_audit_logs_summary_info` **Event series count** `event_series_count_info` **Event SNMP config create** `event_snmp_config_create` **Event SNMP config update** `event_snmp_config_update` **Event subscription** `event_subscription`, `event_subscription_info` **Event subscription count** `event_subscription_count_info` **Event subscription details email** `event_subscription_details_email_info` **Event subscription details rest** `event_subscription_details_rest_info` **Event subscription details syslog** `event_subscription_details_syslog_info` **Event subscription email** `event_subscription_email`, `event_subscription_email_info` **Event subscription rest** `event_subscription_rest`, `event_subscription_rest_info` **Event subscription syslog** `event_subscription_syslog`, `event_subscription_syslog_info` **Event syslog config** `event_syslog_config`, `event_syslog_config_info` **Event webhook** `event_webhook`, `event_webhook_info` **Events and notifications playbook config generator** `events_and_notifications_playbook_config_generator` **Events and notifications workflow manager** `events_and_notifications_workflow_manager` **Playbook: Event webhook create** playbook `event_webhook_create` **Playbook: Events and notifications playbook config generator** playbook `events_and_notifications_playbook_config_generator` **Playbook: Events and notifications workflow manager** playbook `events_and_notifications_workflow_manager`

Use Case: Integrate with ISE and ITSM

Description	Integrate Cisco ISE and RADIUS authentication servers, configure ITSM integration instances and review CMDB sync status.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Authentication policy servers** `authentication_policy_servers`, `authentication_policy_servers_info` **Integrate ISE** `integrate_ise` **Integration settings instances itsm** `integration_settings_instances_itsm`, `integration_settings_instances_itsm_info` **Integration settings itsm instances** `integration_settings_itsm_instances_info` **Integration settings status** `integration_settings_status_info` **ISE integration status** `ise_integration_status_info` **ISE RADIUS integration playbook config generator** `ise_radius_integration_playbook_config_generator` **ISE RADIUS integration workflow manager** `ise_radius_integration_workflow_manager` **Itsm cmdb sync status** `itsm_cmdb_sync_status_info` **Itsm integration events failed** `itsm_integration_events_failed`, `itsm_integration_events_failed_info` **Playbook: ISE RADIUS integration playbook config generator** playbook `ise_radius_integration_playbook_config_generator` **Playbook: ISE RADIUS integration workflow manager** playbook `ise_radius_integration_workflow_manager`

Use Case: Manage users, roles and external authentication

Description	Create users and roles with permissions, and configure external authentication servers and AAA attributes.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Role permissions** `role_permissions_info` **Roles** `roles`, `roles_info` **Roles permissions** `roles_permissions_info` **Roles v2** `roles_v2`, `roles_v2_info` **User** `user`, `user_info` **User role playbook config generator** `user_role_playbook_config_generator` **User role workflow manager** `user_role_workflow_manager` **Users external authentication** `users_external_authentication`, `users_external_authentication_info` **Users external servers** `users_external_servers_info` **Users external servers AAA attribute** `users_external_servers_aaa_attribute`, `users_external_servers_aaa_attribute_info` **Playbook: User info** playbook `user_info` **Playbook: User role playbook config generator** playbook `user_role_playbook_config_generator` **Playbook: User role workflow manager** playbook `user_role_workflow_manager`

Use Case: Track tasks and generate reports

Description	Query tasks, activities and execution status, and create, schedule and run flexible reports.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Activities** `activities_info` **Activities activity ID triggered jobs** `activities_activity_id_triggered_jobs_info` **Activities activity ID triggered jobs count** `activities_activity_id_triggered_jobs_count_info` **Activities count** `activities_count_info` **Activities ID** `activities_id_info` **Dnacaap management execution status** `dnacaap_management_execution_status_info` **Flexible report content** `flexible_report_content_info` **Flexible report executions** `flexible_report_executions`, `flexible_report_executions_info` **Flexible report schedule** `flexible_report_schedule`, `flexible_report_schedule_info` **Flexible report schedules** `flexible_report_schedules_info` **Reports** `reports`, `reports_info` **Reports executions** `reports_executions_info` **Reports view group** `reports_view_group_info` **Reports view group view** `reports_view_group_view_info` **Reports workflow manager** `reports_workflow_manager` **Task** `task_info` **Task count** `task_count_info` **Task operation** `task_operation_info` **Task tree** `task_tree_info` **Tasks** `tasks_info` **Tasks count** `tasks_count_info` **Tasks detail** `tasks_detail_info` **Playbook: Reports executions info** playbook `reports_executions_info` **Playbook: Reports workflow manager** playbook `reports_workflow_manager`

Use Case: Tag network resources

Description	Create tags and manage tag membership for devices and interfaces.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Interfaces members associations** `interfaces_members_associations_info` **Interfaces members associations count** `interfaces_members_associations_count_info` **Interfaces members associations query** `interfaces_members_associations_query` **Network devices members associations** `network_devices_members_associations_info` **Network devices members associations count** `network_devices_members_associations_count_info` **Network devices members associations query** `network_devices_members_associations_query` **Tag** `tag`, `tag_info` **Tag count** `tag_count_info` **Tag member** `tag_member`, `tag_member_info` **Tag member count** `tag_member_count_info` **Tag member type** `tag_member_type_info` **Tag membership** `tag_membership` **Tags interfaces members associations** `tags_interfaces_members_associations_info` **Tags interfaces members associations bulk** `tags_interfaces_members_associations_bulk` **Tags interfaces members associations count** `tags_interfaces_members_associations_count_info` **Tags interfaces members associations query** `tags_interfaces_members_associations_query` **Tags network devices members associations** `tags_network_devices_members_associations_info` **Tags network devices members associations bulk** `tags_network_devices_members_associations_bulk` **Tags network devices members associations count** `tags_network_devices_members_associations_count_info` **Tags network devices members associations query** `tags_network_devices_members_associations_query` **Tags playbook config generator** `tags_playbook_config_generator` **Tags workflow manager** `tags_workflow_manager` **Playbook: Get tag members by ID** playbook `get_tag_members_by_id` **Playbook: Tag** playbook `tag` **Playbook: Tags playbook config generator** playbook `tags_playbook_config_generator` **Playbook: Tags workflow manager** playbook `tags_workflow_manager`

Use Case: Manage IoT rings and endpoint analytics

Description	Manage REP and MRP ring information, endpoint analytics profiling rules, endpoints, dictionaries and ANC policies.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)
Key content	**Endpoint analytics anc policies** `endpoint_analytics_anc_policies_info` **Endpoint analytics cmdb endpoints** `endpoint_analytics_cmdb_endpoints` **Endpoint analytics dictionaries** `endpoint_analytics_dictionaries_info` **Endpoint analytics endpoints** `endpoint_analytics_endpoints`, `endpoint_analytics_endpoints_info` **Endpoint analytics endpoints anc policy delete** `endpoint_analytics_endpoints_anc_policy_delete` **Endpoint analytics endpoints anc policy update** `endpoint_analytics_endpoints_anc_policy_update` **Endpoint analytics endpoints count** `endpoint_analytics_endpoints_count_info` **Endpoint analytics profiling rules** `endpoint_analytics_profiling_rules`, `endpoint_analytics_profiling_rules_info` **Endpoint analytics profiling rules bulk** `endpoint_analytics_profiling_rules_bulk` **Endpoint analytics profiling rules count** `endpoint_analytics_profiling_rules_count_info` **Endpoint analytics tasks** `endpoint_analytics_tasks_info` **IoT fabric rep rings** `iot_fabric_rep_rings` **IoT fabric rep rings delete** `iot_fabric_rep_rings_delete` **IoT network devices network device ID mrp rings** `iot_network_devices_network_device_id_mrp_rings_info` **IoT network devices network device ID mrp rings count** `iot_network_devices_network_device_id_mrp_rings_count_info` **IoT network devices network device ID mrp rings ID members** `iot_network_devices_network_device_id_mrp_rings_id_members_info` **IoT network devices network device ID mrp rings ID members count** `iot_network_devices_network_device_id_mrp_rings_id_members_count_info` **IoT non fabric rep rings** `iot_non_fabric_rep_rings` **IoT non fabric rep rings delete** `iot_non_fabric_rep_rings_delete` **IoT rep rings ID** `iot_rep_rings_id_info` **IoT rep rings query** `iot_rep_rings_query` **IoT rep rings query count** `iot_rep_rings_query_count`

Use Case: Administer the Catalyst Center platform

Description	Back up and restore Catalyst Center, review system health, performance and disaster recovery status, and manage packages, files, access groups and filter groups.
Collection	`cisco.dnac` (Certified, Cisco, v6.48.0, updated 2026-02-19)

Key content	**Access groups** `access_groups`, `access_groups_info` **Access groups count** `access_groups_count_info` **Backup and restore playbook config generator** `backup_and_restore_playbook_config_generator` **Backup and restore workflow manager** `backup_and_restore_workflow_manager` **Backup configuration** `backup_configuration`, `backup_configuration_info` **Backup nfs configurations** `backup_nfs_configurations`, `backup_nfs_configurations_info` **Backup restore executions** `backup_restore_executions_info` **Backup storages** `backup_storages_info` **Backups** `backups`, `backups_info` **Backups ID restore** `backups_id_restore` **Cisco imcs** `cisco_imcs`, `cisco_imcs_info` **Connection mode setting** `connection_mode_setting`, `connection_mode_setting_info` **Disasterrecovery system operationstatus** `disasterrecovery_system_operationstatus_info` **Disasterrecovery system status** `disasterrecovery_system_status_info` **Dnac packages** `dnac_packages_info` **File** `file_info` **File import** `file_import` **File namespaces** `file_namespaces_info` **Filter group aassociations delete** `filter_group_aassociations_delete` **Filter group associations** `filter_group_associations`, `filter_group_associations_info` **Filter groups** `filter_groups`, `filter_groups_info` **Platform nodes configuration summary** `platform_nodes_configuration_summary_info` **Platform release summary** `platform_release_summary_info` **System health** `system_health_info` **System health count** `system_health_count_info` **System performance** `system_performance_info` **System performance historical** `system_performance_historical_info` **Playbook: Backup and restore playbook config generator** playbook `backup_and_restore_playbook_config_generator` **Playbook: Backup and restore workflow manager** playbook `backup_and_restore_workflow_manager`

Cisco IOS and IOS XE `cisco.ios`

Certified Cisco · version 11.6.0 · updated 2026-09-17 · 42 modules → 7 use cases

Use Case: Configure interfaces and Layer 2

Description	Configure physical and logical interfaces, VLANs, switchport settings, link aggregation and LLDP.
Collection	`cisco.ios` (Certified, Cisco, v11.6.0, updated 2026-09-17)
Key content	**Interfaces** `ios_interfaces` **Layer 2 interfaces** `ios_l2_interfaces` **Layer 3 interfaces** `ios_l3_interfaces` **VLANs** `ios_vlans` **Link aggregation (LAG) interfaces** `ios_lag_interfaces` **LACP global settings** `ios_lacp` **LACP interfaces** `ios_lacp_interfaces` **LLDP global settings** `ios_lldp_global` **LLDP interfaces** `ios_lldp_interfaces`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed, which allow configuration to be gathered and reconciled.

Use Case: Configure routing protocols

Description	Configure BGP, OSPFv2, OSPFv3 and static routes, with prefix lists and route maps to control route exchange.
Collection	`cisco.ios` (Certified, Cisco, v11.6.0, updated 2026-09-17)
Key content	**BGP global settings** `ios_bgp_global` **BGP address families** `ios_bgp_address_family` **OSPFv2** `ios_ospfv2` **OSPFv3** `ios_ospfv3` **OSPF interfaces** `ios_ospf_interfaces` **Static routes** `ios_static_routes` **Prefix lists** `ios_prefix_lists` **Route maps** `ios_route_maps`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed, which allow configuration to be gathered and reconciled.

Use Case: Configure first-hop redundancy and fast failure detection

Description	Configure HSRP on interfaces and BFD on interfaces and templates.
Collection	`cisco.ios` (Certified, Cisco, v11.6.0, updated 2026-09-17)
Key content	**HSRP interfaces** `ios_hsrp_interfaces` **BFD interfaces** `ios_bfd_interfaces` **BFD templates** `ios_bfd_templates`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed, which allow configuration to be gathered and reconciled.

Use Case: Configure VRFs and data center overlays

Description	Configure VRF definitions, their address families and interface membership, VXLAN VTEP interfaces and L2VPN EVPN.
Collection	`cisco.ios` (Certified, Cisco, v11.6.0, updated 2026-09-17)
Key content	**VRF definitions** `ios_vrf` **Global VRF settings** `ios_vrf_global` **VRF address families** `ios_vrf_address_family` **VRF interfaces** `ios_vrf_interfaces` **VXLAN VTEP interfaces** `ios_vxlan_vtep` **EVPN global settings** `ios_evpn_global` **EVPN EVIs** `ios_evpn_evi` **EVPN Ethernet segments** `ios_evpn_ethernet`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed, which allow configuration to be gathered and reconciled.

Use Case: Control traffic with ACLs

Description	Configure access control lists and apply them to interfaces.
Collection	`cisco.ios` (Certified, Cisco, v11.6.0, updated 2026-09-17)
Key content	**ACLs** `ios_acls` **ACL interfaces** `ios_acl_interfaces`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed, which allow configuration to be gathered and reconciled.

Use Case: Configure device management services

Description	Configure hostname, system attributes, banners, local users, logging, NTP, SNMP and service settings.
Collection	`cisco.ios` (Certified, Cisco, v11.6.0, updated 2026-09-17)
Key content	**Hostname** `ios_hostname` **System attributes** `ios_system` **Banners** `ios_banner` **Local users** `ios_user` **Logging** `ios_logging_global` **NTP** `ios_ntp_global` **SNMP server** `ios_snmp_server` **Services** `ios_service`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed, which allow configuration to be gathered and reconciled.

Use Case: Run commands, push configuration and gather facts

Description	Run show or exec commands, push or back up configuration sections, collect device facts and test reachability with ping.
Collection	`cisco.ios` (Certified, Cisco, v11.6.0, updated 2026-09-17)
Key content	**Commands** `ios_command` **Configuration sections** `ios_config` **Device facts** `ios_facts` **Ping** `ios_ping`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed, which allow configuration to be gathered and reconciled.

Cisco IOS-XR `cisco.iosxr`

Certified Cisco · version 13.0.0 · updated 2026-10-06 · 35 modules → 5 use cases

Use Case: Configure interfaces and link aggregation

Description	Manage interfaces, Layer 2 and Layer 3 settings, LAGs, LACP and LLDP.
Collection	`cisco.iosxr` (Certified, Cisco, v13.0.0, updated 2026-10-06)

Key content	**Interfaces** `iosxr_interfaces` **Layer 2 interfaces** `iosxr_l2_interfaces` **Layer 3 interfaces** `iosxr_l3_interfaces` **LAG interfaces** `iosxr_lag_interfaces` **LACP** `iosxr_lacp` **LACP interfaces** `iosxr_lacp_interfaces` **LLDP** `iosxr_lldp_global` **LLDP interfaces** `iosxr_lldp_interfaces`

Use Case: Configure routing and VRFs

Description	Manage BGP, OSPFv2 and OSPFv3, static routes, prefix lists, route maps and VRFs.
Collection	`cisco.iosxr` (Certified, Cisco, v13.0.0, updated 2026-10-06)
Key content	**BGP** `iosxr_bgp_global` **BGP address families** `iosxr_bgp_address_family` **BGP neighbor address families** `iosxr_bgp_neighbor_address_family` **BGP templates** `iosxr_bgp_templates` **OSPFv2** `iosxr_ospfv2` **OSPFv3** `iosxr_ospfv3` **OSPF interfaces** `iosxr_ospf_interfaces` **Static routes** `iosxr_static_routes` **Prefix lists** `iosxr_prefix_lists` **Route maps** `iosxr_route_maps` **VRFs** `iosxr_vrf_global` **VRF address families** `iosxr_vrf_address_family` **VRF interfaces** `iosxr_vrf_interfaces`

Use Case: Manage access control lists

Description	Configure ACLs and apply them to interfaces.
Collection	`cisco.iosxr` (Certified, Cisco, v13.0.0, updated 2026-10-06)
Key content	**ACLs** `iosxr_acls` **ACLs on interfaces** `iosxr_acl_interfaces`

Use Case: Administer the device

Description	Manage hostname, banners, system attributes, local users, NTP, logging, SNMP and NETCONF.
Collection	`cisco.iosxr` (Certified, Cisco, v13.0.0, updated 2026-10-06)
Key content	**System attributes** `iosxr_system` **Hostname** `iosxr_hostname` **Banners** `iosxr_banner` **Local users** `iosxr_user` **NTP** `iosxr_ntp_global` **Logging** `iosxr_logging_global` **SNMP** `iosxr_snmp_server` **NETCONF service** `iosxr_netconf`

Use Case: Run commands, push configuration and gather facts

Description	Run CLI commands, manage configuration sections, test reachability and collect device facts.
Collection	`cisco.iosxr` (Certified, Cisco, v13.0.0, updated 2026-10-06)
Key content	**Configuration** `iosxr_config` **CLI commands** `iosxr_command` **Reachability tests** `iosxr_ping` **Device facts** `iosxr_facts`

Cisco Meraki `cisco.meraki`

Certified Cisco · version 2.23.2 · updated 2026-04-07 · 714 modules → 24 use cases

Use Case: Manage organizations, administrators and API access

Description	Create and clone organizations, manage administrators, SAML single sign-on and login security, API keys and requests, branding and integrations, and review e features.

Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Administered identities me** `administered_identities_me_info` **Administered identities me API keys** `administered_identities_me_api_keys_info` **Administered identities me API keys generate** `administered_identities_me_api_keys_generate` **Administered identities me API keys revoke** `administered_identities_me_api_keys_revoke` **Organizations** `organizations`, `organizations_info` **Action batches (org)** `organizations_action_batches`, `organizations_action_batches_info` **Admins (org)** `organizations_admins`, `organizations_admins_info` **API requests (org)** `organizations_api_requests_info` **API requests overview (org)** `organizations_api_requests_overview_info` **API requests overview response codes by interval (org)** `organizations_api_requests_overview_response_codes_by_interval_info` **API rest provisioning pipelines jobs (org)** `organizations_api_rest_provisioning_pipelines_jobs_info` **API rest provisioning pipelines jobs overviews by pipeline (org)** `organizations_api_rest_provisioning_pipelines_jobs_overviews_by_pipeline_info` **Branding policies (org)** `organizations_branding_policies`, `organizations_branding_policies_info` **Branding policies priorities (org)** `organizations_branding_policies_priorities`, `organizations_branding_policies_priorities_info` **Claim (org)** `organizations_claim` **Clone (org)** `organizations_clone` **Early access features (org)** `organizations_early_access_features_info` **Early access features opt ins (org)** `organizations_early_access_features_opt_ins`, `organizations_early_access_features_opt_ins_info` **Integrations deployable (org)** `organizations_integrations_deployable` **Integrations deployed (org)** `organizations_integrations_deployed_in` **Login security (org)** `organizations_login_security`, `organizations_login_security_info` **Openapi spec (org)** `organizations_openapi_spec_info` **SAML (org)** `organizations_saml`, `organizations_saml_info` **SAML IdPs (org)** `organizations_saml_idps`, `organizations_saml_idps_` **SAML roles (org)** `organizations_saml_roles`, `organizations_saml_role` **Spaces integrate status (org)** `organizations_spaces_integrate_status` **Spaces integration remove (org)** `organizations_spaces_integration_r` **Playbook: What Org** playbook `What_Org` **Playbook: Who Am I** playbook `Who_Am_I` **Playbook: Administered identities me info** playbook `administered_identities_me_info` **Playbook: Administrerd identities me API keys generate** playbook `administrerd_identities_me_api_keys_generate` **Playbook: Organization** playbook `organization` **Playbook: Organization info** playbook `organization_info` **Playbook: Organization summary** playbook `organization_summary` **Playbook: Organizations admin** playbook `organizations_admin` **Playbook: Organizations admin copy** playbook `organizations_admin_c` **Playbook: Organizations admin info** playbook `organizations_admin_in` **Playbook: Organizations login security** playbook `organizations_login_` **Playbook: Organizations SAML IdPs** playbook `organizations_saml_idp` **Playbook: Test** playbook `test` **Playbook: Who am i** playbook `who_am_i`

Use Case: Configure networks, templates and monitoring settings

Description	Create, bind, split and combine networks, apply configuration templates, and set network-level SNMP, NetFlow, syslog and MQTT broker settings.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Networks** `networks`, `networks_info` **Bind** `networks_bind` **Campus gateway clusters** `networks_campus_gateway_clusters` **Devices syslog servers** `networks_devices_syslog_servers` **MQTT brokers** `networks_mqtt_brokers` **Netflow** `networks_netflow`, `networks_netflow_info` **Settings** `networks_settings`, `networks_settings_info` **SNMP** `networks_snmp`, `networks_snmp_info` **Split** `networks_split` **Syslog servers** `networks_syslog_servers`, `networks_syslog_servers_info` **Unbind** `networks_unbind` **Campus gateway clusters (org)** `organizations_campus_gateway_clusters_info` **Campus gateway devices uplinks local overrides by device (org)** `organizations_campus_gateway_devices_uplinks_local_overrides_by_device_info` **Config templates (org)** `organizations_config_templates`, `organizations_config_templates_info` **Devices syslog servers by network (org)** `organizations_devices_syslog_servers_by_network_info` **Devices syslog servers roles by network (org)** `organizations_devices_syslog_servers_roles_by_network_info` **Networks combine (org)** `organizations_networks_combine` **Networks moves (org)** `organizations_networks_moves`, `organizations_networks_moves_info` **SNMP (org)** `organizations_snmp`, `organizations_snmp_info` **Playbook: Create a Network** playbook `Create_a_Network` **Playbook: Networks** playbook `networks` **Playbook: Networks syslog servers** playbook `networks_syslog_servers` **Playbook: Organizations config templates** playbook `organizations_config_templates` **Playbook: Organizations config templates info** playbook `organizations_config_templates_info`

Use Case: Claim, inventory and manage devices

Description	Claim devices into organizations and networks, update device details and management interfaces, release or remove devices, and build dynamic inventory from Meraki.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)

Key content	**Devices** `devices`, `devices_info` **Blink LEDs (device)** `devices_blink_leds` **LLDP CDP (device)** `devices_lldp_cdp_info` **Management interface (device)** `devices_management_interface`, `devices_management_interface_info` **Devices claim** `networks_devices_claim` **Devices claim vMX** `networks_devices_claim_vmx` **Devices remove** `networks_devices_remove` **Devices (org)** `organizations_devices_info` **Devices controller migrations (org)** `organizations_devices_controller_migrations`, `organizations_devices_controller_migrations_info` **Devices details bulk update (org)** `organizations_devices_details_bulk_update` **Inventory claim (org)** `organizations_inventory_claim` **Inventory devices (org)** `organizations_inventory_devices_info` **Inventory devices EoX overview (org)** `organizations_inventory_devices_eox_overview_info` **Inventory onboarding cloud monitoring export events (org)** `organizations_inventory_onboarding_cloud_monitoring_export_events` **Inventory onboarding cloud monitoring imports (org)** `organizations_inventory_onboarding_cloud_monitoring_imports`, `organizations_inventory_onboarding_cloud_monitoring_imports_info` **Inventory onboarding cloud monitoring networks (org)** `organizations_inventory_onboarding_cloud_monitoring_networks_info` **Inventory onboarding cloud monitoring prepare (org)** `organizations_inventory_onboarding_cloud_monitoring_prepare` **Inventory orders claim (org)** `organizations_inventory_orders_claim` **Inventory orders preview (org)** `organizations_inventory_orders_preview` **Inventory release (org)** `organizations_inventory_release` **Meraki dynamic inventory** inventory `meraki` **Playbook: Bulk change DNS from file** playbook `Bulk_change_DNS_from_file` **Playbook: Get AP Serials** playbook `Get_AP_Serials` **Playbook: Get All Devices** playbook `Get_All_Devices` **Playbook: Get IP MGNT info** playbook `Get_IP_MGNT_info` **Playbook: Get IP MGNT info from file** playbook `Get_IP_MGNT_info_from_file` **Playbook: Set IP MGNT info** playbook `Set_IP_MGNT_info` **Playbook: Device blink LEDs** playbook `device_blink_leds` **Playbook: Device LLDP CDP** playbook `device_lldp_cdp` **Playbook: Devices** playbook `devices` **Playbook: Devices info** playbook `devices_info` **Playbook: Network devices claim** playbook `network_devices_claim` **Playbook: Networks devices claim 2** playbook `networks_devices_claim_2` **Playbook: Networks devices remove** playbook `networks_devices_remove` **Playbook: Organizations devices info** playbook `organizations_devices_info`

Use Case: Manage floor plans and device placement

Description	Create floor plans, place devices on them and run auto-locate jobs.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Floor plans** `networks_floor_plans`, `networks_floor_plans_info` **Floor plans auto locate jobs batch** `networks_floor_plans_auto_locate_jobs_batch` **Floor plans auto locate jobs cancel** `networks_floor_plans_auto_locate_jobs_cancel` **Floor plans auto locate jobs publish** `networks_floor_plans_auto_locate_jobs_publish` **Floor plans auto locate jobs recalculate** `networks_floor_plans_auto_locate_jobs_recalculate` **Floor plans devices batch update** `networks_floor_plans_devices_batch_update` **Floor plans auto locate devices (org)** `organizations_floor_plans_auto_locate_devices_info` **Floor plans auto locate statuses (org)** `organizations_floor_plans_auto_locate_statuses_info`

Use Case: Troubleshoot devices with live tools and packet capture

Description	Run on-demand ping, cable test, throughput, ARP, MAC and routing table lookups and port cycling on devices, and schedule or download packet captures.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Live tools ARP table (device)** `devices_live_tools_arp_table`, `devices_live_tools_arp_table_info` **Live tools cable test (device)** `devices_live_tools_cable_test`, `devices_live_tools_cable_test_info` **Live tools LEDs blink (device)** `devices_live_tools_leds_blink`, `devices_live_tools_leds_blink_info` **Live tools MAC table (device)** `devices_live_tools_mac_table`, `devices_live_tools_mac_table_info` **Live tools multicast routing (device)** `devices_live_tools_multicast_routing`, `devices_live_tools_multicast_routing_info` **Live tools ping (device)** `devices_live_tools_ping`, `devices_live_tools_ping_info` **Live tools ping device (device)** `devices_live_tools_ping_device`, `devices_live_tools_ping_device_info` **Live tools ports (device)** `devices_live_tools_ports_status` **Live tools ports cycle (device)** `devices_live_tools_ports_cycle`, `devices_live_tools_ports_cycle_info` **Live tools ports status (device)** `devices_live_tools_ports_status_info` **Live tools power usage (device)** `devices_live_tools_power_usage`, `devices_live_tools_power_usage_info` **Live tools routing table lookups (device)** `devices_live_tools_routing_table_lookups`, `devices_live_tools_routing_table_lookups_info` **Live tools routing table summaries (device)** `devices_live_tools_routing_table_summaries`, `devices_live_tools_routing_table_summaries_info` **Live tools throughput test (device)** `devices_live_tools_throughput_test`, `devices_live_tools_throughput_test_info` **Live tools wake on LAN (device)** `devices_live_tools_wake_on_lan`, `devices_live_tools_wake_on_lan_info` **Devices packet capture captures (org)** `organizations_devices_packet_capture_captures`, `organizations_devices_packet_capture_captures_info` **Devices packet capture captures bulk create (org)** `organizations_devices_packet_capture_captures_bulk_create` **Devices packet capture captures bulk delete (org)** `organizations_devices_packet_capture_captures_bulk_delete` **Devices packet capture captures download URL generate (org)** `organizations_devices_packet_capture_captures_download_url_generate` **Devices packet capture captures stop (org)** `organizations_devices_packet_capture_captures_stop` **Devices packet capture schedules (org)** `organizations_devices_packet_capture_schedules`, `organizations_devices_packet_capture_schedules_info` **Devices packet capture schedules reorder (org)** `organizations_devices_packet_capture_schedules_reorder` **Playbook: Devices live tools ping** playbook `devices_live_tools_ping`

Use Case: Manage licensing and subscriptions

Description	Claim, bind, assign, move and renew licenses and subscriptions, and review license and entitlement status.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)

Key content	**Administered licensing subscription entitlements** `administered_licensing_subscription_entitlements_info` **Administered licensing subscription subscriptions** `administered_licensing_subscription_subscriptions_info` **Administered licensing subscription subscriptions bind** `administered_licensing_subscription_subscriptions_bind` **Administered licensing subscription subscriptions claim** `administered_licensing_subscription_subscriptions_claim` **Administered licensing subscription subscriptions claim key validate** `administered_licensing_subscription_subscriptions_claim_key_validate` **Administered licensing subscription subscriptions compliance statuses** `administered_licensing_subscription_subscriptions_compliance_statuses_info` **Licenses (org)** `organizations_licenses`, `organizations_licenses_info` **Licenses assign seats (org)** `organizations_licenses_assign_seats` **Licenses move (org)** `organizations_licenses_move` **Licenses move seats (org)** `organizations_licenses_move_seats` **Licenses overview (org)** `organizations_licenses_overview_info` **Licenses renew seats (org)** `organizations_licenses_renew_seats` **Licensing coterm licenses (org)** `organizations_licensing_coterm_licenses_info` **Licensing coterm licenses move (org)** `organizations_licensing_coterm_licenses_move`

Use Case: Manage firmware upgrades

Description	Schedule, stage, defer and roll back firmware upgrades for networks and devices.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Firmware upgrades** `networks_firmware_upgrades`, `networks_firmware_upgrades_info` **Firmware upgrades rollbacks** `networks_firmware_upgrades_rollbacks` **Firmware upgrades staged events** `networks_firmware_upgrades_staged_events`, `networks_firmware_upgrades_staged_events_info` **Firmware upgrades staged events defer** `networks_firmware_upgrades_staged_events_defer` **Firmware upgrades staged events rollbacks** `networks_firmware_upgrades_staged_events_rollbacks` **Firmware upgrades staged groups** `networks_firmware_upgrades_staged_groups`, `networks_firmware_upgrades_staged_groups_info` **Firmware upgrades staged stages** `networks_firmware_upgrades_staged_stages`, `networks_firmware_upgrades_staged_stages_info` **Firmware upgrades (org)** `organizations_firmware_upgrades_info` **Firmware upgrades by device (org)** `organizations_firmware_upgrades_by_device_info`

Use Case: Manage group policies, policy objects and adaptive policy

Description	Define group policies, policy objects and global firewall rulesets, assign policies to clients and VLANs, and configure adaptive policy groups, ACLs and settings.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Clients policy** `networks_clients_policy`, `networks_clients_policy_info` **Group policies** `networks_group_policies`, `networks_group_policies_info` **Policies by client** `networks_policies_by_client_info` **Adaptive policy ACLs (org)** `organizations_adaptive_policy_acls`, `organizations_adaptive_policy_acls_info` **Adaptive policy groups (org)** `organizations_adaptive_policy_groups`, `organizations_adaptive_policy_groups_info` **Adaptive policy overview (org)** `organizations_adaptive_policy_overview_info` **Adaptive policy policies (org)** `organizations_adaptive_policy_policies`, `organizations_adaptive_policy_policies_info` **Adaptive policy settings (org)** `organizations_adaptive_policy_settings`, `organizations_adaptive_policy_settings_info` **Policies assignments by client (org)** `organizations_policies_assignments_by_client_info` **Policies global firewall application categories (org)** `organizations_policies_global_firewall_application_categories_info` **Policies global firewall rulesets (org)** `organizations_policies_global_firewall_rulesets`, `organizations_policies_global_firewall_rulesets_info` **Policies global firewall rulesets rules (org)** `organizations_policies_global_firewall_rulesets_rules`, `organizations_policies_global_firewall_rulesets_rules_info` **Policies global group policies (org)** `organizations_policies_global_group_policies`, `organizations_policies_global_group_policies_info` **Policies global group policies adaptive policy groups assign** `organizations_policies_global_group_policies_adaptive_po` **Policies global group policies adaptive policy groups assign** `organizations_policies_global_group_policies_adaptive_po` **Policies global group policies adaptive policy groups remove** `organizations_policies_global_group_policies_adaptive_po` **Policies global group policies appliance VLANs assign (org)** `organizations_policies_global_group_policies_appliance_v` **Policies global group policies appliance VLANs assignments** `organizations_policies_global_group_policies_appliance_v` **Policies global group policies appliance VLANs assignments** `organizations_policies_global_group_policies_appliance_v` **Policies global group policies appliance VLANs remove (org)** `organizations_policies_global_group_policies_appliance_v` **Policies global group policies firewall rulesets assignments (** `organizations_policies_global_group_policies_firewall_ru` `organizations_policies_global_group_policies_firewall_ru` **Policy objects (org)** `organizations_policy_objects`, `organiza` **Policy objects groups (org)** `organizations_policy_objects_g` `organizations_policy_objects_groups_info` **Playbook: Organizations adaptivePolicy ACLs** playbook `organ` **Playbook: Organizations adaptive policy groups** playbook `organizations_adaptive_policy_groups`

Use Case: Configure appliance firewall and content security

Description	Manage MX firewall rules, NAT and port forwarding, content filtering and intrusion and malware protection.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)

Key content	**Appliance content filtering** `networks_appliance_content_filtering`, `networks_appliance_content_filtering_info` **Appliance content filtering categories** `networks_appliance_content_filtering_categories_info` **Appliance firewall cellular firewall rules** `networks_appliance_firewall_cellular_firewall_rules`, `networks_appliance_firewall_cellular_firewall_rules_info` **Appliance firewall firewalled services** `networks_appliance_firewall_firewalled_services`, `networks_appliance_firewall_firewalled_services_info` **Appliance firewall inbound firewall rules** `networks_appliance_firewall_inbound_firewall_rules`, `networks_appliance_firewall_inbound_firewall_rules_info` **Appliance firewall L3 firewall rules** `networks_appliance_firewall_l3_firewall_rules`, `networks_appliance_firewall_l3_firewall_rules_info` **Appliance firewall L7 firewall rules** `networks_appliance_firewall_l7_firewall_rules`, `networks_appliance_firewall_l7_firewall_rules_info` **Appliance firewall L7 firewall rules application categories** `networks_appliance_firewall_l7_firewall_rules_application_categories_info` **Appliance firewall multicast forwarding** `networks_appliance_firewall_multicast_forwarding` **Appliance firewall one to many NAT rules** `networks_appliance_firewall_one_to_many_nat_rules`, `networks_appliance_firewall_one_to_many_nat_rules_info` **Appliance firewall one to one NAT rules** `networks_appliance_firewall_one_to_one_nat_rules`, `networks_appliance_firewall_one_to_one_nat_rules_info` **Appliance firewall port forwarding rules** `networks_appliance_firewall_port_forwarding_rules`, `networks_appliance_firewall_port_forwarding_rules_info` **Appliance firewall settings** `networks_appliance_firewall_settings`, `networks_appliance_firewall_settings_info` **Appliance security intrusion** `networks_appliance_security_intrusion`, `networks_appliance_security_intrusion_info` **Appliance security malware** `networks_appliance_security_malware`, `networks_appliance_security_malware_info` **Appliance firewall multicast forwarding by network (org)** `organizations_appliance_firewall_multicast_forwarding_by_network_inf` **Appliance security intrusion (org)** `organizations_appliance_security_intrusion`, `organizations_appliance_security_intrusion_info` **Playbook: Networks appliance firewall L7 firewall rules** playbook `networks_appliance_firewall_l7_firewall_rules`

Use Case: Configure appliance VLANs, ports, routing and uplinks

Description	Configure MX VLANs, LAN and L3 interfaces, ports, static routes, uplink settings, NAT, warm spare redundancy, DNS profiles and records, and VRF settings.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Appliance interfaces ports update (device)** `devices_appliance_interfaces_ports_update` **Appliance radio settings (device)** `devices_appliance_radio_settings`, `devices_appliance_radio_settings_info` **Appliance uplinks settings (device)** `devices_appliance_uplinks_settings`, `devices_appliance_uplinks_settings_info` **Appliance vMX authentication token (device)** `devices_appliance_vmx_authentication_token` **Appliance connectivity monitoring destinations** `networks_appliance_connectivity_monitoring_destinations`, `networks_appliance_connectivity_monitoring_destinations_info` **Appliance devices redundancy** `networks_appliance_devices_redundancy` **Appliance devices redundancy swap** `networks_appliance_devices_redundancy_swap` **Appliance interfaces L3** `networks_appliance_interfaces_l3` **Appliance ports** `networks_appliance_ports`, `networks_appliance_ports_info` **Appliance prefixes delegated statics** `networks_appliance_prefixes_delegated_statics`, `networks_appliance_prefixes_delegated_statics_info` **Appliance RF profiles** `networks_appliance_rf_profiles`, `networks_appliance_rf_profiles_info` **Appliance settings** `networks_appliance_settings`, `networks_appliance_settings_info` **Appliance single LAN** `networks_appliance_single_lan`, `networks_appliance_single_lan_info` **Appliance SSIDs** `networks_appliance_ssids`, `networks_appliance_ssids_info` **Appliance static routes** `networks_appliance_static_routes`, `networks_appliance_static_routes_info` **Appliance uplinks NAT** `networks_appliance_uplinks_nat` **Appliance VLANs** `networks_appliance_vlans`, `networks_appliance_vlans_info` **Appliance VLANs settings** `networks_appliance_vlans_settings`, `networks_appliance_vlans_settings_info` **Appliance warm spare** `networks_appliance_warm_spare`, `networks_appliance_warm_spare_info` **Appliance warm spare swap** `networks_appliance_warm_spare_swap` **Appliance devices interfaces L3 (org)** `organizations_appliance_devices_interfaces_l3_info` **Appliance devices interfaces ports by device (org)** `organizations_appliance_devices_interfaces_ports_by_device_info` **Appliance devices redundancy by network (org)** `organizations_appliance_devices_redundancy_by_network_info` **Appliance DNS local profiles (org)** `organizations_appliance_dns_local_profiles`, `organizations_appliance_dns_local_profiles_info` **Appliance DNS local profiles assignments (org)** `organizations_appliance_dns_local_profiles_assignments_info` **Appliance DNS local profiles assignments bulk create (org)** `organizations_appliance_dns_local_profiles_assignments_bulk_create` **Appliance DNS local profiles assignments bulk delete (org)** `organizations_appliance_dns_local_profiles_assignments_bulk_delete` **Appliance DNS local records (org)** `organizations_appliance_dns_local_records`, `organizations_appliance_dns_local_records_info` **Appliance DNS split profiles (org)** `organizations_appliance_dns_split_profiles`, `organizations_appliance_dns_split_profiles_info` **Appliance DNS split profiles assignments (org)** `organizations_appliance_dns_split_profiles_assignments_info` **Appliance DNS split profiles assignments bulk create (org)** `organizations_appliance_dns_split_profiles_assignments_bulk_create` **Appliance DNS split profiles assignments bulk delete (org)** `organizations_appliance_dns_split_profiles_assignments_bulk_delete` **Appliance routing VRFs settings (org)** `organizations_appliance_routing_vrfs_settings`, `organizations_appliance_routing_vrfs_settings_info` **Appliance uplinks NAT by network (org)** `organizations_appliance_uplinks_nat_by_network_info` **Playbook: Devices appliance uplinks settings** playbook `devices_appliance_uplinks_settings` **Playbook: Mx deployment** playbook `mx_deployment` **Playbook: Networks appliance static routes** playbook `networks_appliance_static_routes` **Playbook: Networks appliance VLANs** playbook `networks_appliance_vlans`

Use Case: Configure site-to-site VPN, SD-WAN and traffic shaping

Description	Configure auto VPN, BGP, third-party VPN peers, SD-WAN internet policies, traffic shaping rules and uplink selection, and VPN firewall rules.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)

Key content	**Appliance SD-WAN internet policies** `networks_appliance_sdwan_internet_policies` **Appliance traffic shaping** `networks_appliance_traffic_shaping`, `networks_appliance_traffic_shaping_info` **Appliance traffic shaping custom performance classes** `networks_appliance_traffic_shaping_custom_performance_classes`, `networks_appliance_traffic_shaping_custom_performance_classes_info` **Appliance traffic shaping rules** `networks_appliance_traffic_shaping_rules`, `networks_appliance_traffic_shaping_rules_info` **Appliance traffic shaping uplink bandwidth** `networks_appliance_traffic_shaping_uplink_bandwidth`, `networks_appliance_traffic_shaping_uplink_bandwidth_info` **Appliance traffic shaping uplink selection** `networks_appliance_traffic_shaping_uplink_selection`, `networks_appliance_traffic_shaping_uplink_selection_info` **Appliance traffic shaping VPN exclusions** `networks_appliance_traffic_shaping_vpn_exclusions` **Appliance VPN BGP** `networks_appliance_vpn_bgp`, `networks_appliance_vpn_bgp_info` **Appliance VPN site to site VPN** `networks_appliance_vpn_site_to_site_vpn`, `networks_appliance_vpn_site_to_site_vpn_info` **Traffic shaping application categories** `networks_traffic_shaping_application_categories_info` **Traffic shaping DSCP tagging options** `networks_traffic_shaping_dscp_tagging_options_info` **Appliance traffic shaping VPN exclusions by network (org)** `organizations_appliance_traffic_shaping_vpn_exclusions_by_network_info` **Appliance VPN site to site IPsec peers SLAs (org)** `organizations_appliance_vpn_site_to_site_ipsec_peers_slas`, `organizations_appliance_vpn_site_to_site_ipsec_peers_slas_info` **Appliance VPN third party VPN peers (org)** `organizations_appliance_vpn_third_party_vpn_peers`, `organizations_appliance_vpn_third_party_vpn_peers_info` **Appliance VPN VPN firewall rules (org)** `organizations_appliance_vpn_vpn_firewall_rules`, `organizations_appliance_vpn_vpn_firewall_rules_info` **Playbook: Networks appliance traffic shaping rules** playbook `networks_appliance_traffic_shaping_rules` **Playbook: Organizations appliance VPN third party vpnpeers** playbook `organizations_appliance_vpn_third_party_vpnpeers`

Use Case: Integrate with SASE, Umbrella and XDR

Description	Connect sites to SASE regions and connectors, attach or detach SASE sites, manage Cisco Umbrella integration, and enable XDR integrations.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Appliance umbrella account connect** `networks_appliance_umbrella_account_connect` **Appliance umbrella account disconnect** `networks_appliance_umbrella_account_disconnect` **Appliance umbrella domains exclusions** `networks_appliance_umbrella_domains_exclusions` **Appliance umbrella policies add** `networks_appliance_umbrella_policies_add` **Appliance umbrella policies remove** `networks_appliance_umbrella_policies_remove` **Appliance umbrella protection** `networks_appliance_umbrella_protection` **Integrations XDR networks (org)** `organizations_integrations_xdr_networks_info` **Integrations XDR networks disable (org)** `organizations_integrations_xdr_networks_disable` **Integrations XDR networks enable (org)** `organizations_integrations_xdr_networks_enable` **SASE connectors (org)** `organizations_sase_connectors_info` **SASE connectors batch create (org)** `organizations_sase_connectors_batch_create` **SASE connectors batch delete (org)** `organizations_sase_connectors_batch_delete` **SASE integrations (org)** `organizations_sase_integrations`, `organizations_sase_integrations_info` **SASE networks eligible (org)** `organizations_sase_networks_eligible_info` **SASE regions (org)** `organizations_sase_regions_info` **SASE sites (org)** `organizations_sase_sites`, `organizations_sase_sites_info` **SASE sites attach (org)** `organizations_sase_sites_attach` **SASE sites connectivity history by site (org)** `organizations_sase_sites_connectivity_history_by_site_info` **SASE sites connectivity overview (org)** `organizations_sase_sites_connectivity_overview_info` **SASE sites detach (org)** `organizations_sase_sites_detach`

Use Case: Configure wireless SSIDs and radio settings

Description	Configure SSIDs and their security, firewall, VPN, schedule and traffic-shaping options, RF profiles, Air Marshal, Bluetooth, Zigbee, location scanning and wirele profiles.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)

Key content	**Wireless alternate management interface IPv6 (device)** `devices_wireless_alternate_management_interface_ipv6` **Wireless bluetooth settings (device)** `devices_wireless_bluetooth_settings`, `devices_wireless_bluetooth_settings_info` **Wireless electronic shelf label (device)** `devices_wireless_electronic_shelf_label`, `devices_wireless_electronic_shelf_label_info` **Wireless radio settings (device)** `devices_wireless_radio_settings`, `devices_wireless_radio_settings_info` **Wireless zigbee enrollments (device)** `devices_wireless_zigbee_enrollments`, `devices_wireless_zigbee_enrollments_info` **Wireless air marshal rules** `networks_wireless_air_marshal_rules` **Wireless air marshal settings** `networks_wireless_air_marshal_settings` **Wireless alternate management interface** `networks_wireless_alternate_management_interface`, `networks_wireless_alternate_management_interface_info` **Wireless billing** `networks_wireless_billing`, `networks_wireless_billing_info` **Wireless bluetooth settings** `networks_wireless_bluetooth_settings`, `networks_wireless_bluetooth_settings_info` **Wireless electronic shelf label** `networks_wireless_electronic_shelf_label`, `networks_wireless_electronic_shelf_label_info` **Wireless electronic shelf label configured devices** `networks_wireless_electronic_shelf_label_configured_devices_info` **Wireless ethernet ports profiles** `networks_wireless_ethernet_ports_profiles`, `networks_wireless_ethernet_ports_profiles_info` **Wireless ethernet ports profiles assign** `networks_wireless_ethernet_ports_profiles_assign` **Wireless ethernet ports profiles set default** `networks_wireless_ethernet_ports_profiles_set_default` **Wireless location scanning** `networks_wireless_location_scanning` **Wireless radio RRM** `networks_wireless_radio_rrm` **Wireless RF profiles** `networks_wireless_rf_profiles`, `networks_wireless_rf_profiles_info` **Wireless settings** `networks_wireless_settings`, `networks_wireless_settings_info` **Wireless SSIDs** `networks_wireless_ssids`, `networks_wireless_ssids_info` **Wireless SSIDs bonjour forwarding** `networks_wireless_ssids_bonjour_forwarding`, `networks_wireless_ssids_bonjour_forwarding_info` **Wireless SSIDs device type group policies** `networks_wireless_ssids_device_type_group_policies`, `networks_wireless_ssids_device_type_group_policies_info` **Wireless SSIDs EAP override** `networks_wireless_ssids_eap_override`, `networks_wireless_ssids_eap_override_info` **Wireless SSIDs firewall L3 firewall rules** `networks_wireless_ssids_firewall_l3_firewall_rules`, `networks_wireless_ssids_firewall_l3_firewall_rules_info` **Wireless SSIDs firewall L7 firewall rules** `networks_wireless_ssids_firewall_l7_firewall_rules`, `networks_wireless_ssids_firewall_l7_firewall_rules_info` **Wireless SSIDs hotspot20** `networks_wireless_ssids_hotspot20`, `networks_wireless_ssids_hotspot20_info` **Wireless SSIDs identity PSKs** `networks_wireless_ssids_identity_psks`, `networks_wireless_ssids_identity_psks_info` **Wireless SSIDs open roaming** `networks_wireless_ssids_open_roami` **Wireless SSIDs schedules** `networks_wireless_ssids_schedules`, `networks_wireless_ssids_schedules_info` **Wireless SSIDs traffic shaping rules** `networks_wireless_ssids_traffic_shaping_rules`, `networks_wireless_ssids_traffic_shaping_rules_info` **Wireless SSIDs VPN** `networks_wireless_ssids_vpn`, `networks_wireless_ssids_vpn_info` **Wireless zigbee** `networks_wireless_zigbee` **Wireless air marshal rules (org)** `organizations_wireless_air_marsha` **Wireless air marshal settings by network (org)** `organizations_wireless_air_marshal_settings_by_network_info` **Wireless devices provisioning deployments (org)** `organizations_wireless_devices_provisioning_deployments`, `organizations_wireless_devices_provisioning_deployments_info` **Wireless devices RadSec certificates authorities (org)** `organizations_wireless_devices_radsec_certificates_authorities` `organizations_wireless_devices_radsec_certificates_authorities` **Wireless devices RadSec certificates authorities CRLs (org)** `organizations_wireless_devices_radsec_certificates_authorities` **Wireless devices RadSec certificates authorities CRLs deltas (org)** `organizations_wireless_devices_radsec_certificates_authorities` **Wireless location scanning by network (org)** `organizations_wireless_location_scanning_by_network_info` **Wireless location scanning receivers (org)** `organizations_wireless_location_scanning_receivers`, `organizations_wireless_location_scanning_receivers_info` **Wireless MQTT settings (org)** `organizations_wireless_mqtt_setting` `organizations_wireless_mqtt_settings_info` **Wireless radio auto RF channels recalculate (org)** `organizations_wireless_radio_auto_rf_channels_recalculate` **Wireless radio RRM by network (org)** `organizations_wireless_radio_rrm_by_network_info` **Wireless RF profiles assignments by device (org)** `organizations_wireless_rf_profiles_assignments_by_device_info` **Wireless SSIDs firewall isolation allowlist entries (org)** `organizations_wireless_ssids_firewall_isolation_allowlist_entr` `organizations_wireless_ssids_firewall_isolation_allowlist_entr` **Wireless SSIDs open roaming by network (org)** `organizations_wireless_ssids_open_roaming_by_network_info` **Wireless zigbee by network (org)** `organizations_wireless_zigbee_b` **Wireless zigbee devices (org)** `organizations_wireless_zigbee_devi` `organizations_wireless_zigbee_devices_info` **Wireless zigbee disenrollments (org)** `organizations_wireless_zigbe` `organizations_wireless_zigbee_disenrollments_info` **Wireless zigbee door locks (org)** `organizations_wireless_zigbee_do` `organizations_wireless_zigbee_door_locks_info` **Playbook: Networks wireless SSIDs identityPsks** playbook `networks_wireless_ssids_identityPsks` **Playbook: Wifi deploy one** playbook `wifi_deploy_one` **Playbook: Wifi deployment** playbook `wifi_deployment` **Playbook: Wireless SSIDs** playbook `wireless_ssids`

Use Case: Monitor wireless performance and controllers

Description	Query wireless connection, latency, channel utilization, signal quality and usage statistics, access point and controller status, and packet loss.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)

Key content	**Wireless connection stats (device)** `devices_wireless_connection_stats_info` **Wireless latency stats (device)** `devices_wireless_latency_stats_info` **Wireless status (device)** `devices_wireless_status_info` **Wireless channel utilization history** `networks_wireless_channel_utilization_history_info` **Wireless client count history** `networks_wireless_client_count_history_info` **Wireless clients connection stats** `networks_wireless_clients_connection_stats_info` **Wireless clients latency stats** `networks_wireless_clients_latency_stats_info` **Wireless connection stats** `networks_wireless_connection_stats_info` **Wireless data rate history** `networks_wireless_data_rate_history_info` **Wireless devices connection stats** `networks_wireless_devices_connection_stats_info` **Wireless failed connections** `networks_wireless_failed_connections_info` **Wireless latency history** `networks_wireless_latency_history_info` **Wireless latency stats** `networks_wireless_latency_stats_info` **Wireless mesh statuses** `networks_wireless_mesh_statuses_info` **Wireless signal quality history** `networks_wireless_signal_quality_history_info` **Wireless usage history** `networks_wireless_usage_history_info` **Wireless clients overview by device (org)** `organizations_wireless_clients_overview_by_device_info` **Wireless controller availabilities change history (org)** `organizations_wireless_controller_availabilities_change_history_info` **Wireless controller clients overview history by device by interval (org)** `organizations_wireless_controller_clients_overview_history_by_device_by_interval_info` **Wireless controller connections (org)** `organizations_wireless_controller_connections_info` **Wireless controller devices interfaces L2 by device (org)** `organizations_wireless_controller_devices_interfaces_l2_by_device_info` **Wireless controller devices interfaces L2 statuses change history by device (org)** `organizations_wireless_controller_devices_interfaces_l2_statuses_change_history_by_device_info` **Wireless controller devices interfaces L2 usage history by interval (org)** `organizations_wireless_controller_devices_interfaces_l2_usage_history_by_interval_info` **Wireless controller devices interfaces L3 by device (org)** `organizations_wireless_controller_devices_interfaces_l3_by_device_info` **Wireless controller devices interfaces L3 statuses change history by device (org)** `organizations_wireless_controller_devices_interfaces_l3_statuses_change_history_by_device_info` **Wireless controller devices interfaces L3 usage history by interval (org)** `organizations_wireless_controller_devices_interfaces_l3_usage_history_by_interval_info` **Wireless controller devices interfaces packets over** `organizations_wireless_controller_devices_inter` **Wireless controller devices interfaces usage histor** `organizations_wireless_controller_devices_inter` **Wireless controller devices redundancy failover his** `organizations_wireless_controller_devices_redun` **Wireless controller devices redundancy statuses (o** `organizations_wireless_controller_devices_redun` **Wireless controller devices system utilization histo** `organizations_wireless_controller_devices_syste` **Wireless controller overview by device (org)** `organizations_wireless_controller_overview_by_d` **Wireless devices channel utilization by device (org)** `organizations_wireless_devices_channel_utilizat` **Wireless devices channel utilization by network (or** `organizations_wireless_devices_channel_utilizat` **Wireless devices channel utilization history by devi** `organizations_wireless_devices_channel_utilizat` **Wireless devices channel utilization history by netv** `organizations_wireless_devices_channel_utilizat` **Wireless devices ethernet statuses (org)** `organizat` **Wireless devices packet loss by client (org)** `organizations_wireless_devices_packet_loss_by_c` **Wireless devices packet loss by device (org)** `organizations_wireless_devices_packet_loss_by_d` **Wireless devices packet loss by network (org)** `organizations_wireless_devices_packet_loss_by_n` **Wireless devices power mode history (org)** `organiz` **Wireless devices system CPU load history (org)** `organizations_wireless_devices_system_cpu_load_` **Wireless devices wireless controllers by device (or** `organizations_wireless_devices_wireless_control` **Wireless SSIDs statuses by device (org)** `organizati`

Use Case: Configure switch ports, stacks and VLAN profiles

Description	Configure switch ports, port schedules, link aggregation, storm control, STP, MTU, stacks, VLAN profiles and switch profiles in configuration templates, and clone switches.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Switch ports (device)** `devices_switch_ports`, `devices_switch_ports_info` **Switch ports cycle (device)** `devices_switch_ports_cycle` **Switch ports statuses (device)** `devices_switch_ports_statuses_info` **Switch warm spare (device)** `devices_switch_warm_spare`, `devices_switch_warm_spare_info` **Switch alternate management interface** `networks_switch_alternate_management_interface`, `networks_switch_alternate_management_interface_info` **Switch link aggregations** `networks_switch_link_aggregations`, `networks_switch_link_aggregations_info` **Switch MTU** `networks_switch_mtu`, `networks_switch_mtu_info` **Switch port schedules** `networks_switch_port_schedules`, `networks_switch_port_schedules_info` **Switch settings** `networks_switch_settings`, `networks_switch_settings_info` **Switch stacks** `networks_switch_stacks`, `networks_switch_stacks_info` **Switch stacks add** `networks_switch_stacks_add` **Switch stacks remove** `networks_switch_stacks_remove` **Switch storm control** `networks_switch_storm_control`, `networks_switch_storm_control_info` **Switch STP** `networks_switch_stp`, `networks_switch_stp_info` **VLAN profiles** `networks_vlan_profiles`, `networks_vlan_profiles_info` **VLAN profiles assignments by device** `networks_vlan_profiles_assignments_by_device_info` **VLAN profiles assignments reassign** `networks_vlan_profiles_assignments_reassign` **Config templates switch profiles (org)** `organizations_config_templates_switch_profiles_info` **Config templates switch profiles ports (org)** `organizations_config_templates_switch_profiles_ports`, `organizations_config_templates_switch_profiles_ports_info` **Switch devices clone (org)** `organizations_switch_devices_clone` **Switch ports by switch (org)** `organizations_switch_ports_by_switch_info` **Switch ports clients overview by device (org)** `organizations_switch_ports_clients_overview_by_device_info` **Switch ports overview (org)** `organizations_switch_ports_overview_info` **Switch ports statuses by switch (org)** `organizations_switch_ports_statuses_by_switch_info` **Switch ports topology discovery by device (org)** `organizations_switch_ports_topology_discovery_by_device_info` **Switch ports usage history by device by interval (org)** `organizations_switch_ports_usage_history_by_device_by_interval_info` **Playbook: Device switch ports** playbook `device_switch_ports` **Playbook: Ms playbook** playbook `ms_playbook` **Playbook: Switch port config** playbook `switch_port_config`

Use Case: Configure switch routing, DHCP, QoS and access control

Description	Configure switch routing interfaces, static routes, OSPF, multicast, DHCP server policy and ARP inspection, access policies and ACLs, and QoS and DSCP-to-C mappings.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)

Key content	**Switch routing interfaces (device)** `devices_switch_routing_interfaces`, `devices_switch_routing_interfaces_info` **Switch routing interfaces DHCP (device)** `devices_switch_routing_interfaces_dhcp`, `devices_switch_routing_interfaces_dhcp_info` **Switch routing static routes (device)** `devices_switch_routing_static_routes`, `devices_switch_routing_static_routes_info` **Switch access control lists** `networks_switch_access_control_lists`, `networks_switch_access_control_lists_info` **Switch access policies** `networks_switch_access_policies`, `networks_switch_access_policies_info` **Switch DHCP server policy** `networks_switch_dhcp_server_policy`, `networks_switch_dhcp_server_policy_info` **Switch DHCP server policy ARP inspection trusted servers** `networks_switch_dhcp_server_policy_arp_inspection_trusted_servers`, `networks_switch_dhcp_server_policy_arp_inspection_trusted_servers_info` **Switch DHCP server policy ARP inspection warnings by device** `networks_switch_dhcp_server_policy_arp_inspection_warnings_by_device_info` **Switch DHCP v4 servers seen** `networks_switch_dhcp_v4_servers_seen_info` **Switch DSCP to CoS mappings** `networks_switch_dscp_to_cos_mappings`, `networks_switch_dscp_to_cos_mappings_info` **Switch QoS rules order** `networks_switch_qos_rules_order`, `networks_switch_qos_rules_order_info` **Switch routing multicast** `networks_switch_routing_multicast`, `networks_switch_routing_multicast_info` **Switch routing multicast rendezvous points** `networks_switch_routing_multicast_rendezvous_points`, `networks_switch_routing_multicast_rendezvous_points_info` **Switch routing OSPF** `networks_switch_routing_ospf`, `networks_switch_routing_ospf_info` **Switch stacks routing interfaces** `networks_switch_stacks_routing_interfa` `networks_switch_stacks_routing_interfaces_info` **Switch stacks routing interfaces DHCP** `networks_switch_stacks_routing_interfaces_dhcp`, `networks_switch_stacks_routing_interfaces_dhcp_info` **Switch stacks routing static routes** `networks_switch_stacks_routing_static_routes`, `networks_switch_stacks_routing_static_routes_info` **Playbook: Devices switch routing interfaces** playbook `devices_switch_routing_interfaces` **Playbook: Networks switch DSCP to CoS mappings** playbook `networks_switch_dscp_to_cos_mappings` **Playbook: Networks switch QoS rules order** playbook `networks_switch_qos_rules_order`

Use Case: Manage cameras and video analytics

Description	Configure camera video settings, quality and retention, wireless profiles, custom analytics and boundaries, camera roles and permissions, and retrieve snapshots, clips and video links.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Camera analytics live (device)** `devices_camera_analytics_live_info` **Camera clip (device)** `devices_camera_clip_info` **Camera custom analytics (device)** `devices_camera_custom_analytics`, `devices_camera_custom_analytics_info` **Camera generate snapshot (device)** `devices_camera_generate_snapshot` **Camera quality and retention (device)** `devices_camera_quality_and_retention`, `devices_camera_quality_and_retention_info` **Camera sense (device)** `devices_camera_sense`, `devices_camera_sense_info` **Camera video link (device)** `devices_camera_video_link_info` **Camera video settings (device)** `devices_camera_video_settings`, `devices_camera_video_settings_info` **Camera wireless profiles (device)** `devices_camera_wireless_profiles`, `devices_camera_wireless_profiles_info` **Camera quality retention profiles** `networks_camera_quality_retention_profiles`, `networks_camera_quality_retention_profiles_info` **Camera wireless profiles** `networks_camera_wireless_profiles`, `networks_camera_wireless_profiles_info` **Camera boundaries areas by device (org)** `organizations_camera_boundaries_areas_by_device_info` **Camera boundaries lines by device (org)** `organizations_camera_boundaries_lines_by_device_info` **Camera custom analytics artifacts (org)** `organizations_camera_custom_analytics_artifacts`, `organizations_camera_custom_analytics_artifacts_info` **Camera detections history by boundary by interval (org)** `organizations_camera_detections_history_by_boundary_by_interval_info` **Camera permissions (org)** `organizations_camera_permissions_info` **Camera roles (org)** `organizations_camera_roles`, `organizations_camera_roles_info` **Playbook: Mv playbook** playbook `mv_playbook` **Playbook: Organizations camera custom analytics artifacts** playbook `organizations_camera_custom_analytics_artifacts`

Use Case: Manage cellular gateways and eSIMs

Description	Configure cellular gateway LAN, DHCP, subnet pools, uplinks and port forwarding, manage SIMs, eSIM inventory and service providers, and review cellular data
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Cellular gateway LAN (device)** `devices_cellular_gateway_lan`, `devices_cellular_gateway_lan_info` **Cellular gateway port forwarding rules (device)** `devices_cellular_gateway_port_forwarding_rules`, `devices_cellular_gateway_port_forwarding_rules_info` **Cellular geolocations (device)** `devices_cellular_geolocations` **Cellular SIMs (device)** `devices_cellular_sims`, `devices_cellular_sims_info` **Cellular uplinks bands masks update (device)** `devices_cellular_uplinks_bands_masks_update` **Cellular gateway connectivity monitoring destinations** `networks_cellular_gateway_connectivity_monitoring_destinations`, `networks_cellular_gateway_connectivity_monitoring_destinations_info` **Cellular gateway DHCP** `networks_cellular_gateway_dhcp`, `networks_cellular_gateway_dhcp_info` **Cellular gateway subnet pool** `networks_cellular_gateway_subnet_pool`, `networks_cellular_gateway_subnet_pool_info` **Cellular gateway uplink** `networks_cellular_gateway_uplink`, `networks_cellular_gateway_uplink_info` **Cellular gateway eSIMs inventory (org)** `organizations_cellular_gateway_esims_inventory`, `organizations_cellular_gateway_esims_inventory_info` **Cellular gateway eSIMs service providers (org)** `organizations_cellular_gateway_esims_service_providers_info` **Cellular gateway eSIMs service providers accounts (org)** `organizations_cellular_gateway_esims_service_providers_accounts`, `organizations_cellular_gateway_esims_service_providers_accounts_info` **Cellular gateway eSIMs service providers accounts comm** `organizations_cellular_gateway_esims_service_provider` **Cellular gateway eSIMs service providers accounts rate p** `organizations_cellular_gateway_esims_service_provider` **Cellular gateway eSIMs swap (org)** `organizations_cellula` **Cellular gateway uplink statuses (org)** `organizations_cell` **Devices cellular data devices (org)** `organizations_devices` **Devices cellular data profiles (org)** `organizations_devices` `organizations_devices_cellular_data_profiles_info` **Devices cellular data profiles assignments (org)** `organizations_devices_cellular_data_profiles_assignme` **Devices cellular data profiles assignments batch create (o** `organizations_devices_cellular_data_profiles_assignme` **Devices cellular data profiles assignments bulk delete (or** `organizations_devices_cellular_data_profiles_assignme` **Devices cellular data usage by device (org)** `organizations_devices_cellular_data_usage_by_device_i` **Devices cellular data usage history by device by interval (** `organizations_devices_cellular_data_usage_history_by_` **Devices cellular geolocations (org)** `organizations_device` **Devices cellular uplinks bands by device (org)** `organizations_devices_cellular_uplinks_bands_by_devic` **Devices cellular uplinks towers by device (org)** `organizations_devices_cellular_uplinks_towers_by_devi`

Use Case: Manage environmental sensors

Description	Configure sensor alert profiles, relationships and MQTT brokers, send sensor commands, and read current and historical sensor readings.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Sensor commands (device)** `devices_sensor_commands`, `devices_sensor_commands_info` **Sensor relationships (device)** `devices_sensor_relationships`, `devices_sensor_relationships_info` **Sensor alerts current overview by metric** `networks_sensor_alerts_current_overview_by_metric_info` **Sensor alerts overview by metric** `networks_sensor_alerts_overview_by_metric_info` **Sensor alerts profiles** `networks_sensor_alerts_profiles`, `networks_sensor_alerts_profiles_info` **Sensor MQTT brokers** `networks_sensor_mqtt_brokers`, `networks_sensor_mqtt_brokers_info` **Sensor relationships** `networks_sensor_relationships_info` **Sensor gateways connections latest (org)** `organizations_sensor_gateways_connections_latest_info` **Sensor readings history (org)** `organizations_sensor_readings_history_info` **Sensor readings latest (org)** `organizations_sensor_readings_latest_info`

Use Case: Manage mobile devices with Systems Manager

Description	Manage enrolled Systems Manager devices, users, profiles, target groups, apps and trusted access, run lock, wipe and reboot commands, and configure APNs and VPP accounts.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**SM bypass activation lock attempts** `networks_sm_bypass_activation_lock_attempts`, `networks_sm_bypass_activation_lock_attempts_info` **SM devices** `networks_sm_devices_info` **SM devices cellular usage history** `networks_sm_devices_cellular_usage_history_info` **SM devices certs** `networks_sm_devices_certs_info` **SM devices checkin** `networks_sm_devices_checkin` **SM devices connectivity** `networks_sm_devices_connectivity_info` **SM devices desktop logs** `networks_sm_devices_desktop_logs_info` **SM devices device command logs** `networks_sm_devices_device_command_logs_info` **SM devices device profiles** `networks_sm_devices_device_profiles_info` **SM devices fields** `networks_sm_devices_fields` **SM devices install apps** `networks_sm_devices_install_apps` **SM devices lock** `networks_sm_devices_lock` **SM devices modify tags** `networks_sm_devices_modify_tags` **SM devices move** `networks_sm_devices_move` **SM devices network adapters** `networks_sm_devices_network_adapters_info` **SM devices performance history** `networks_sm_devices_performance_history_info` **SM devices reboot** `networks_sm_devices_reboot` **SM devices refresh details** `networks_sm_devices_refresh_details` **SM devices security centers** `networks_sm_devices_security_centers_info` **SM devices shutdown** `networks_sm_devices_shutdown` **SM devices unenroll** `networks_sm_devices_unenroll` **SM devices uninstall apps** `networks_sm_devices_uninstall_apps` **SM devices wipe** `networks_sm_devices_wipe` **SM devices WLAN lists** `networks_sm_devices_wlan_lists_info` **SM profiles** `networks_sm_profiles_info` **SM target groups** `networks_sm_target_groups`, `networks_sm_target_groups_info` **SM trusted access configs** `networks_sm_trusted_access_configs_info` **SM user access devices** `networks_sm_user_access_devices_info` **SM user access devices delete** `networks_sm_user_access_devices_delete` **SM users** `networks_sm_users_info` **SM users device profiles** `networks_sm_users_device_profiles_info` **SM users softwares** `networks_sm_users_softwares_info` **SM admins roles (org)** `organizations_sm_admins_roles`, `organizations_sm_admins_roles_info` **SM APNs cert (org)** `organizations_sm_apns_cert_info` **SM sentry policies assignments (org)** `organizations_sm_sentry_policies_assignments` **SM sentry policies assignments by network (org)** `organizations_sm_sentry_policies_assignments_by_network_info` **SM VPP accounts (org)** `organizations_sm_vpp_accounts_info`

Use Case: Manage clients, guest access and privacy data

Description	Provision clients, authorize splash page access, configure splash themes and assets, manage Meraki authentication users and Bluetooth clients, and handle PII requests and keys.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Bluetooth clients** `networks_bluetooth_clients_info` **Clients** `networks_clients_info` **Clients provision** `networks_clients_provision` **Clients splash authorization** `networks_clients_splash_authorization_status` **Clients splash authorization status** `networks_clients_splash_authorization_status_info` **Meraki auth users** `networks_meraki_auth_users`, `networks_meraki_auth_users_info` **PII PII keys** `networks_pii_pii_keys_info` **PII requests** `networks_pii_requests_info` **PII requests delete** `networks_pii_requests_delete` **PII SM devices for key** `networks_pii_sm_devices_for_key_info` **PII SM owners for key** `networks_pii_sm_owners_for_key_info` **Wireless SSIDs splash settings** `networks_wireless_ssids_splash_settings`, `networks_wireless_ssids_splash_settings_info` **Splash assets (org)** `organizations_splash_assets`, `organizations_splash_assets_info` **Splash themes (org)** `organizations_splash_themes`, `organizations_splash_themes_info` **Splash themes assets (org)** `organizations_splash_themes_assets`

Use Case: Configure alerts, webhooks and events

Description	Configure alert settings and profiles, webhook HTTP servers and payload templates, review events and assurance alerts, and dismiss or restore alerts.

Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Alerts history** `networks_alerts_history_info` **Alerts settings** `networks_alerts_settings`, `networks_alerts_settings_info` **Events** `networks_events_info` **Events event types** `networks_events_event_types_info` **Webhooks HTTP servers** `networks_webhooks_http_servers`, `networks_webhooks_http_servers_info` **Webhooks payload templates** `networks_webhooks_payload_templates`, `networks_webhooks_payload_templates_info` **Webhooks webhook tests** `networks_webhooks_webhook_tests_info` **Alerts profiles (org)** `organizations_alerts_profiles` **Assurance alerts (org)** `organizations_assurance_alerts_info` **Assurance alerts dismiss (org)** `organizations_assurance_alerts_dismiss` **Assurance alerts overview (org)** `organizations_assurance_alerts_overview_info` **Assurance alerts overview by network (org)** `organizations_assurance_alerts_overview_by_network_info` **Assurance alerts overview by type (org)** `organizations_assurance_alerts_overview_by_type_info` **Assurance alerts overview historical (org)** `organizations_assurance_alerts_overview_historical_info` **Assurance alerts restore (org)** `organizations_assurance_alerts_restore` **Assurance alerts taxonomy categories (org)** `organizations_assurance_alerts_taxonomy_categories_info` **Assurance alerts taxonomy types (org)** `organizations_assurance_alerts_taxonomy_types_info` **Webhooks callbacks statuses (org)** `organizations_webhooks_callbacks_statuses_info` **Webhooks logs (org)** `organizations_webhooks_logs_info`

Use Case: Monitor device status, uplinks and availability

Description	Query device availability, status, provisioning, uplink status and usage, power modules and memory usage across the organization.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Appliance performance (device)** `devices_appliance_performance_info` **Appliance devices ports transceivers readings history by device (org)** `organizations_appliance_devices_ports_transceivers_readings_history_by_device_info` **Appliance interfaces packets overviews by device (org)** `organizations_appliance_interfaces_packets_overviews_by_device_info` **Appliance uplinks statuses overview (org)** `organizations_appliance_uplinks_statuses_overview_info` **Appliance uplinks usage by network (org)** `organizations_appliance_uplinks_usage_by_network_info` **Devices availabilities (org)** `organizations_devices_availabilities_info` **Devices availabilities change history (org)** `organizations_devices_availabilities_change_history_info` **Devices overview by model (org)** `organizations_devices_overview_by_model_info` **Devices power modules statuses by device (org)** `organizations_devices_power_modules_statuses_by_device_info` **Devices provisioning statuses (org)** `organizations_devices_prov` **Devices statuses (org)** `organizations_devices_statuses_info` **Devices statuses overview (org)** `organizations_devices_statuse` **Devices system memory usage history by interval (org)** `organizations_devices_system_memory_usage_history_by_interv` **Devices uplinks addresses by device (org)** `organizations_devices_uplinks_addresses_by_device_info` **Devices uplinks loss and latency (org)** `organizations_devices_uplinks_loss_and_latency_info` **Uplinks statuses (org)** `organizations_uplinks_statuses_info` **Playbook: Device statuses** playbook `device_statuses`

Use Case: Report on clients, applications and network usage

Description	Search clients and review their bandwidth, review top clients, applications, devices and SSIDs by usage, and query traffic analysis, application health and link-layer topology.
Collection	`cisco.meraki` (Certified, Cisco, v2.23.2, updated 2026-04-07)
Key content	**Clients overview** `networks_clients_overview_info` **Health alerts** `networks_health_alerts_info` **Insight applications health by time** `networks_insight_applications_health_by_time_info` **Topology link layer** `networks_topology_link_layer_info` **Traffic analysis** `networks_traffic_analysis`, `networks_traffic_analysis_info` **Clients bandwidth usage history (org)** `organizations_clients_bandwidth_usage_history_info` **Clients overview (org)** `organizations_clients_overview_info` **Clients search (org)** `organizations_clients_search_info` **Insight applications (org)** `organizations_insight_applications_info` **Insight monitored media servers (org)** `organizations_insight_monitored_media_servers`, `organizations_insight_monitored_media_servers_info` **Summary top appliances by utilization (org)** `organizations_summary_top_appliances_by_utilization_info` **Summary top applications by usage (org)** `organizations_summary_top_applications_by_usage_info` **Summary top applications categories by usage (org)** `organizations_summary_top_applications_categories_by_usage_info` **Summary top clients by usage (org)** `organizations_summary_top_clients_by_usage_info` **Summary top clients manufacturers by usage (org)** `organizations_summary_top_clients_manufacturers_by_usage_info` **Summary top devices by usage (org)** `organizations_summary_top_devices_by_usage_info` **Summary top devices models by usage (org)** `organizations_summary_top_devices_models_by_usage_info` **Summary top networks by status (org)** `organizations_summary_top_networks_by_status_info` **Summary top SSIDs by usage (org)** `organizations_summary_top_ssids_by_usage_info` **Summary top switches by energy usage (org)** `organizations_summary_top_switches_by_energy_usage_info` **Playbook: Organizations clients search info** playbook `organizations_clients_search_info`

Cisco Nexus Dashboard Orchestrator (ACI Multi-Site) `cisco.mso`

Certified · Cisco · version 2.13.0 · updated 2026-03-16 · 111 modules → 10 use cases

Use Case: Manage multi-site schemas and templates

Description	Create, clone, validate and migrate schemas and templates, and track the orchestrator version or call its REST API directly.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)

Key content	**Direct REST API access** `mso_rest` **Orchestrator version** `mso_version` **Schemas** `mso_schema` **Schema cloning** `mso_schema_clone` **Schema templates** `mso_schema_template` **Template cloning** `mso_schema_template_clone` **BD and EPG migration between templates** `mso_schema_template_migrate` **Schema validation** `mso_schema_validate` **Templates** `ndo_template`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Deploy templates to sites

Description	Push templates to ACI sites and check the objects affected before deployment.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**Pre-deployment checks** `mso_schema_template_deploy_status` **Template deployment** `ndo_template_deploy` **Sites in schemas** `mso_schema_site`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Manage tenant networking: VRFs, bridge domains and subnets

Description	Define VRFs, bridge domains, their subnets and DHCP policies in schema templates and at the site level.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**Template VRFs** `mso_schema_template_vrf` **VRF rendezvous points** `mso_schema_template_vrf_rp` **Template bridge domains** `mso_schema_template_bd` **Template BD subnets** `mso_schema_template_bd_subnet` **BD DHCP policies** `ndo_schema_template_bd_dhcp_policy` **Site-local VRFs** `mso_schema_site_vrf` **Site-local bridge domains** `mso_schema_site_bd` **Site-local BD subnets** `mso_schema_site_bd_subnet` **Site-local BD L3Outs** `mso_schema_site_bd_l3out`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Manage application profiles and endpoint groups

Description	Create application network profiles and EPGs with their subnets, uSeg attributes, annotations, domains and static bindings.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**Template application profiles** `mso_schema_template_anp` **Template EPGs** `mso_schema_template_anp_epg` **Template EPG subnets** `mso_schema_template_anp_epg_subnet` **Template EPG uSeg attributes** `mso_schema_template_anp_epg_useg_attribute` **EPG annotations** `mso_schema_template_anp_epg_annotation` **Site-local application profiles** `mso_schema_site_anp` **Site-local EPGs** `mso_schema_site_anp_epg` **Site EPG domains** `mso_schema_site_anp_epg_domain` **Site EPG static leafs** `mso_schema_site_anp_epg_staticleaf` **Site EPG static ports** `mso_schema_site_anp_epg_staticport` **Site EPG subnets** `mso_schema_site_anp_epg_subnet` **Site EPG uSeg attributes** `mso_schema_site_anp_epg_useg_attribute`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Manage contracts, filters and service chaining

Description	Define contracts and filters between EPGs, external EPGs and VRFs, and insert L4-L7 service graphs and device clusters.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)

Key content	**EPG contracts** `mso_schema_template_anp_epg_contract` **Intra-EPG contracts** `mso_schema_template_anp_epg_intra_epg_contract` **Contract filters** `mso_schema_template_contract_filter` **Filter entries** `mso_schema_template_filter_entry` **VRF contracts** `mso_schema_template_vrf_contract` **External EPG contracts** `mso_schema_template_external_epg_contract` **Template service graphs** `mso_schema_template_service_graph` **Template contract service graphs** `mso_schema_template_contract_service_graph` **Site service graphs** `mso_schema_site_service_graph` **Site contract service graphs** `mso_schema_site_contract_service_graph` **Contract service chaining** `ndo_schema_template_contract_service_chain` **Service device clusters** `ndo_service_device_cluster`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Connect to external networks with L3Outs and external EPGs

Description	Define external EPGs and L3Outs in schemas and L3Out templates, including interfaces, SVIs, BGP peers and static routes.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**Template external EPGs** `mso_schema_template_external_epg` **External EPGs (legacy)** `mso_schema_template_externalepg` **External EPG subnets** `mso_schema_template_external_epg_subnet` **Site external EPGs** `mso_schema_site_external_epg` **Template L3Outs** `mso_schema_template_l3out` **Site L3Outs** `mso_schema_site_l3out` **L3Out templates** `ndo_l3out_template` **L3Out annotations** `ndo_l3out_annotation` **L3 domains** `ndo_l3_domain` **Routed interfaces** `ndo_l3out_routed_interface` **Routed sub-interfaces** `ndo_l3out_routed_sub_interface` **SVI interfaces** `ndo_l3out_svi_interface` **Floating SVI interfaces** `ndo_l3out_floating_svi_interface` **Floating SVI path attributes** `ndo_l3out_floating_svi_interface_path_attributes` **Secondary IP addresses** `ndo_l3out_secondary_ip` **BGP peers** `ndo_l3out_bgp_peer` **Node static routes** `ndo_l3out_node_static_route` **Static route next hops** `ndo_l3out_node_static_route_next_hop` **Node group policies** `ndo_l3out_node_group_policy` **Node routing policies** `ndo_l3out_node_routing_policy` **Interface group policies** `ndo_l3out_interface_group_policy` **Interface routing policies** `ndo_l3out_interface_routing_policy`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Control routing with route maps and match rules

Description	Manage route map policies, match rules and set rules, BGP peer prefix policies and multicast route maps.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**Route control route maps** `ndo_route_map_policy_route_control` **Route map contexts** `ndo_route_map_policy_route_control_context` **Multicast route maps** `ndo_route_map_policy_multicast` **Match rule policies** `ndo_match_rule_policy` **Match prefix lists** `ndo_match_rule_prefix` **Match community terms** `ndo_match_rule_community_term` **Set rule policies** `ndo_set_rule_policy` **BGP peer prefix policies** `ndo_tenant_bgp_peer_prefix_policy`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Define tenant protocol, QoS and telemetry policies

Description	Manage tenant policy templates for IGMP and MLD snooping, custom QoS, NetFlow and SPAN, plus endpoint tag policies.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**IGMP interface policies** `ndo_tenant_igmp_interface_policy` **IGMP snooping policies** `ndo_tenant_igmp_snooping_policy` **MLD snooping policies** `ndo_tenant_mld_snooping_policy` **Custom QoS policies** `ndo_tenant_custom_qos_policy` **NetFlow exporters** `ndo_tenant_netflow_exporter` **NetFlow monitors** `ndo_tenant_netflow_monitor` **NetFlow records** `ndo_tenant_netflow_record` **Tenant SPAN sessions** `ndo_tenant_span_session` **Endpoint IP tag policies** `ndo_endpoint_ip_tag_policy` **Endpoint MAC tag policies** `ndo_endpoint_mac_tag_policy` **IPSLA monitoring policies** `ndo_ipsla_monitoring_policy` **IPSLA track lists** `ndo_ipsla_track_list`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Configure fabric access: domains, VLAN pools and interfaces

Description	Manage physical domains, VLAN pools, and physical, port-channel and vPC interfaces and FEX devices in fabric policy templates.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**Physical domains** `ndo_physical_domain` **VLAN pools** `ndo_vlan_pool` **Physical interfaces** `ndo_physical_interface` **Port channel interfaces** `ndo_port_channel_interface` **Virtual port channel interfaces** `ndo_virtual_port_channel_interface` **Interface policy groups** `ndo_interface_setting` **FEX devices** `ndo_fex_device` **MACsec policies** `ndo_macsec_policy` **SyncE interface policies** `ndo_synce_interface_policy`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Use Case: Configure fabric-wide policies: pods, nodes, time, QoS and monitoring

Description	Manage pod and node profiles and settings, NTP and PTP, MCP, QoS class policies and fabric SPAN sessions.
Collection	`cisco.mso` (Certified, Cisco, v2.13.0, updated 2026-03-16)
Key content	**Pod profiles** `ndo_pod_profile` **Pod settings** `ndo_pod_settings` **Node profiles** `ndo_node_profile` **Node settings** `ndo_node_setting` **NTP policies** `ndo_ntp_policy` **PTP policies** `ndo_ptp_policy` **PTP policy profiles** `ndo_ptp_policy_profiles` **MCP global policy** `ndo_mcp_global_policy` **QoS class policies** `ndo_qos_class_policy` **QoS DSCP-CoS translation policies** `ndo_qos_dscp_cos_translation_policy` **Fabric SPAN sessions** `ndo_fabric_span_session` **Fabric SPAN sources** `ndo_fabric_span_session_source` **Fabric SPAN source filters** `ndo_fabric_span_session_source_filter` **DHCP option policies** `ndo_dhcp_option_policy` **DHCP relay policies** `ndo_dhcp_relay_policy`
Notes	• Modules are prefixed mso_ (legacy schema-based API) and ndo_ (template-based API of newer Nexus Dashboard Orchestrator releases).

Cisco Nexus Dashboard `cisco.nd`

Certified Cisco · version 1.5.0 · updated 2026-03-15 · 67 modules → 9 use cases

Use Case: Create and manage VXLAN and external fabrics

Description	Create and manage iBGP, eBGP, AI/ML, campus and external connectivity fabrics, and group fabrics into VXLAN fabric groups.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**iBGP VXLAN fabrics** `nd_manage_fabric_ibgp_vxlan` **eBGP VXLAN fabrics** `nd_manage_fabric_ebgp_vxlan` **AI/ML iBGP VXLAN fabrics** `nd_manage_fabric_ai_ibgp_vxlan` **AI/ML eBGP VXLAN fabrics** `nd_manage_fabric_ai_ebgp_vxlan` **Campus iBGP VXLAN fabrics** `nd_manage_fabric_campus_ibgp_vxlan` **External connectivity fabrics** `nd_manage_fabric_external` **VXLAN fabric groups (MSD)** `nd_manage_fabric_group_vxlan` **Fabric group members** `nd_manage_fabric_group_members`

Use Case: Manage switches, links and fabric resources

Description	Add switches to fabrics, manage links, vPC pairs, ToR associations, policies, policy groups and resource pools, and set maintenance mode.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**Switches** `nd_manage_switches` **Links** `nd_manage_links` **vPC pairs** `nd_manage_vpc_pair` **Access and ToR switches** `nd_manage_tor` **Policies** `nd_manage_policy` **Policy groups** `nd_manage_policy_group` **Resource manager** `nd_manage_resource_manager` **Maintenance mode** `nd_maintenance_mode`

Use Case: Provision networks, VRFs and routing policy

Description	Define networks, VRFs and L3Outs, and manage ACLs, prefix lists, route maps and BGP community lists on fabrics.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**Networks** `nd_manage_networks` **VRFs** `nd_manage_vrfs` **L3Outs** `nd_manage_l3out` **ACLs** `nd_manage_acl` **Prefix lists** `nd_manage_prefix_list` **Route maps** `nd_manage_route_map` **BGP community lists** `nd_manage_community_list` **BGP extended community lists** `nd_manage_extended_community_list`

Use Case: Configure switch interfaces

Description	Manage Ethernet, port-channel, vPC, loopback, SVI and subinterface definitions, singly or together in one workflow.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**Ethernet access interfaces** `nd_interface_ethernet_access` **Routed Ethernet interfaces** `nd_interface_ethernet_routed` **Ethernet trunk host interfaces** `nd_interface_ethernet_trunk_host` **Access port channels** `nd_interface_port_channel_access` **Routed port channels** `nd_interface_port_channel_routed` **Trunk port channels** `nd_interface_port_channel_trunk_host` **vPC access interfaces** `nd_interface_vpc_access` **vPC trunk interfaces** `nd_interface_vpc_trunk_host` **Loopback interfaces** `nd_interface_loopback` **SVI interfaces** `nd_interface_svi` **Managed subinterfaces** `nd_interface_subinterface_managed` **Unmanaged subinterfaces** `nd_interface_subinterface_unmanaged` **Multi-family interface workflow** `nd_interfaces_workflow`

Use Case: Upgrade fabric software

Description	Stage, validate and apply software updates to switches through fabric update groups.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**Fabric update groups** `nd_fabric_update_group` **Update preparation and validation** `nd_fabric_prepare_update`

Use Case: Run assurance, compliance and pre-change analysis

Description	Run Nexus Dashboard Insights compliance analysis, delta analysis, instant assurance and pre-change analysis, and capture snapshots.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**Compliance analysis** `nd_compliance_analysis` **Communication requirements** `nd_compliance_requirement_communication` **Import configuration requirements** `nd_compliance_requirement_config_import` **Manual configuration requirements** `nd_compliance_requirement_config_manual` **Snapshot configuration requirements** `nd_compliance_requirement_config_snapshot` **Template configuration requirements** `nd_compliance_requirement_config_template` **Delta analysis** `nd_delta_analysis` **Instant assurance analysis** `nd_instant_assurance_analysis` **Pre-change analysis jobs** `nd_pcv` **Pre-change compliance** `nd_pcv_compliance` **Pre-change delta analysis** `nd_pcv_delta_analysis` **Fabric snapshots** `nd_snapshot`

Use Case: Monitor flows and policy usage

Description	Configure flow rules and interface flow rules and retrieve policy CAM hit counts.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**Flow rules** `nd_flow_rules` **Interface flow rules** `nd_interface_flow_rules` **Policy CAM hit counts** `nd_policy_cam_statistics_hit_counts`

Use Case: Administer Nexus Dashboard

Description	Set up the dashboard, manage sites, clusters and federation, users, API keys, backups, restores and packaged services.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**Initial setup** `nd_setup` **Sites** `nd_site` **Cluster routes** `nd_cluster_config_route` **Multi-cluster federation** `nd_federation_member` **Local users** `nd_local_user` **API keys** `nd_api_key` **Backups** `nd_backup` **Backup restore** `nd_backup_restore` **Service packages** `nd_service` **Service instances** `nd_service_instance` **Version** `nd_version`

Use Case: Call the REST API and log messages

Description	Call the Nexus Dashboard REST API directly and log messages from playbooks.
Collection	`cisco.nd` (Certified, Cisco, v1.5.0, updated 2026-03-15)
Key content	**REST API** `nd_rest` **Playbook logging** `nd_log`

Cisco NX-OS `cisco.nxos`

Certified Cisco · version 13.0.0 · updated 2026-09-17 · 75 modules → 10 use cases

Use Case: Configure interfaces, VLANs and Layer 2

Description	Configure physical and logical interfaces, VLANs, link aggregation (LACP), LLDP, UDLD and VTP.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**Interfaces** `nxos_interfaces` **Layer 2 interfaces** `nxos_l2_interfaces` **Layer 3 interfaces** `nxos_l3_interfaces` **VLANs** `nxos_vlans` **Link aggregation (LAG) interfaces** `nxos_lag_interfaces` **LACP global settings** `nxos_lacp` **LACP interfaces** `nxos_lacp_interfaces` **LLDP global settings** `nxos_lldp_global` **LLDP interfaces** `nxos_lldp_interfaces` **UDLD global settings** `nxos_udld` **UDLD interfaces** `nxos_udld_interface` **VTP domain** `nxos_vtp_domain` **VTP password** `nxos_vtp_password` **VTP version** `nxos_vtp_version`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Configure routing protocols and route policy

Description	Configure BGP, OSPFv2, OSPFv3 and static routes, with prefix lists and route maps to control route exchange.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**BGP global settings** `nxos_bgp_global` **BGP address families** `nxos_bgp_address_family` **BGP neighbor address families** `nxos_bgp_neighbor_address_family` **BGP templates** `nxos_bgp_templates` **OSPFv2** `nxos_ospfv2` **OSPFv3** `nxos_ospfv3` **OSPF interfaces** `nxos_ospf_interfaces` **Static routes** `nxos_static_routes` **Prefix lists** `nxos_prefix_lists` **Route maps** `nxos_route_maps`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Configure VRFs and multicast

Description	Define VRFs and their address families and interfaces, and configure PIM, IGMP and IGMP snooping for multicast.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**VRFs** `nxos_vrf_global` **VRF address families** `nxos_vrf_address_family` **VRF address family settings** `nxos_vrf_af` **VRF interfaces** `nxos_vrf_interfaces` **PIM** `nxos_pim` **PIM interfaces** `nxos_pim_interface` **PIM static RP addresses** `nxos_pim_rp_address` **IGMP** `nxos_igmp` **IGMP interfaces** `nxos_igmp_interface` **IGMP snooping** `nxos_igmp_snooping`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Build VXLAN EVPN fabrics and vPC

Description	Configure VXLAN VTEPs and VNIs, the EVPN control plane, anycast gateway MAC, and virtual port channels.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**VXLAN VTEP (NVE)** `nxos_vxlan_vtep` **VXLAN VNI members** `nxos_vxlan_vtep_vni` **EVPN control plane** `nxos_evpn_global` **EVPN VNIs** `nxos_evpn_vni` **Anycast gateway MAC** `nxos_overlay_global` **vPC domain** `nxos_vpc` **vPC interfaces** `nxos_vpc_interface`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Provide first-hop redundancy and fast failure detection

Description	Configure HSRP, VRRP and BFD for gateway redundancy and rapid failure detection.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**HSRP interfaces** `nxos_hsrp_interfaces` **VRRP** `nxos_vrrp` **BFD global settings** `nxos_bfd_global` **BFD interfaces** `nxos_bfd_interfaces`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Control access and device security

Description	Manage ACLs and their interface bindings, AAA servers, local users and login banners.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**ACLs** `nxos_acls` **ACL interface bindings** `nxos_acl_interfaces` **AAA server global settings** `nxos_aaa_server` **AAA server hosts** `nxos_aaa_server_host` **Local users** `nxos_user` **Banners** `nxos_banner`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Configure device services and management

Description	Set hostname and system attributes, enable features, and configure NTP, SNMP, logging, NX-API and telemetry.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**Hostname** `nxos_hostname` **System attributes** `nxos_system` **Features** `nxos_feature` **NTP** `nxos_ntp_global` **SNMP server** `nxos_snmp_server` **Logging** `nxos_logging_global` **NX-API** `nxos_nxapi` **Telemetry** `nxos_telemetry`

Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Configure Fibre Channel on MDS switches

Description	Configure Fibre Channel interfaces, VSANs, zones and zonesets, and device aliases on Cisco MDS storage switches.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**Fibre Channel interfaces** `nxos_fc_interfaces` **VSANs** `nxos_vsan` **Zones and zonesets** `nxos_zone_zoneset` **Device aliases** `nxos_devicealias`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Upgrade, maintain and roll back devices

Description	Install OS images and RPM patches, copy files, take snapshots and checkpoints, roll back, reboot, and place switches in or out of maintenance mode with GIR.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**OS install and boot options** `nxos_install_os` **RPM patches** `nxos_rpm` **File copy** `nxos_file_copy` **Snapshots** `nxos_snapshot` **Checkpoints and rollback** `nxos_rollback` **Reboot** `nxos_reboot` **Graceful insertion and removal** `nxos_gir` **GIR profiles** `nxos_gir_profile_management`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Use Case: Run commands, gather facts and test reachability

Description	Run arbitrary commands, manage configuration sections, gather device facts and test reachability with ping.
Collection	`cisco.nxos` (Certified, Cisco, v13.0.0, updated 2026-09-17)
Key content	**Commands** `nxos_command` **Configuration sections** `nxos_config` **Facts** `nxos_facts` **Ping** `nxos_ping`
Notes	• Resource modules support states such as merged, replaced, overridden, deleted, gathered, rendered and parsed.

Dell Enterprise SONiC `dellemc.enterprise_sonic`

Certified Dell Technologies · version 4.1.0 · updated 2026-01-27 · 90 modules → 10 use cases

Use Case: Configure Layer 2: interfaces, VLANs and link aggregation

Description	Manage interface attributes, VLANs and VLAN mapping, access and trunk assignments, LAGs, MAC, STP, port breakout and port groups.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**Interfaces** `sonic_interfaces` **Layer 2 interfaces** `sonic_l2_interfaces` **VLANs** `sonic_vlans` **VLAN mappings** `sonic_vlan_mapping` **LAG interfaces** `sonic_lag_interfaces` **MAC settings** `sonic_mac` **Spanning tree** `sonic_stp` **Port breakout** `sonic_port_breakout` **Port groups** `sonic_port_group` **Port security** `sonic_pms` **Link state tracking** `sonic_lst` **Layer 2 protocol tunneling** `sonic_br_l2pt` **MCLAG domains** `sonic_mclag` **Power over Ethernet** `sonic_poe`

Use Case: Configure Layer 3 addressing, VRFs and static routing

Description	Set up IP addressing, VRFs, static routes, ECMP, IP neighbours, VRRP, IPv6 router advertisement and DHCP relay and snooping.

Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**Layer 3 interfaces** `sonic_l3_interfaces` **VRFs** `sonic_vrfs` **Static routes** `sonic_static_routes` **ECMP load sharing** `sonic_ecmp_load_share` **Adaptive routing and switching** `sonic_ars` **IP neighbour settings** `sonic_ip_neighbor` **Interface IP neighbour settings** `sonic_ip_neighbor_interfaces` **IPv6 router advertisement** `sonic_ipv6_router_advertisement` **VRRP** `sonic_vrrp` **DHCP relay** `sonic_dhcp_relay` **DHCP snooping** `sonic_dhcp_snooping`

Use Case: Configure dynamic routing: BGP, OSPF and PIM

Description	Manage BGP sessions, address families and communities, OSPFv2 and OSPFv3, BFD and multicast PIM.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**BGP global** `sonic_bgp` **BGP address families** `sonic_bgp_af` **BGP neighbours** `sonic_bgp_neighbors` **BGP neighbour address families** `sonic_bgp_neighbors_af` **BGP AS path lists** `sonic_bgp_as_paths` **BGP communities** `sonic_bgp_communities` **BGP extended communities** `sonic_bgp_ext_communities` **OSPFv2 global** `sonic_ospfv2` **OSPFv2 areas** `sonic_ospf_area` **OSPFv2 interfaces** `sonic_ospfv2_interfaces` **OSPFv3 global** `sonic_ospfv3` **OSPFv3 areas** `sonic_ospfv3_area` **OSPFv3 interfaces** `sonic_ospfv3_interfaces` **PIM global** `sonic_pim_global` **PIM interfaces** `sonic_pim_interfaces` **BFD** `sonic_bfd`

Use Case: Control routing policy

Description	Maintain prefix lists and route maps used to filter and modify routes.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**Prefix lists** `sonic_prefix_lists` **Route maps** `sonic_route_maps`

Use Case: Build data centre overlays with VXLAN and EVPN

Description	Configure VXLAN tunnels, EVPN ESI multihoming and BGP L3 fabrics.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**VXLANs** `sonic_vxlans` **EVPN ESI multihoming** `sonic_evpn_esi_multihome` **BGP Clos fabric** playbook `bgp_l3_fabric.clos-fabric`

Use Case: Apply ACLs, flow-based services and control-plane protection

Description	Manage Layer 2 and Layer 3 ACLs and their interface bindings, FBS policies and CoPP.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**Layer 2 ACLs** `sonic_l2_acls` **Layer 3 ACLs** `sonic_l3_acls` **ACL interface bindings** `sonic_acl_interfaces` **FBS classifiers** `sonic_fbs_classifiers` **FBS groups** `sonic_fbs_groups` **FBS policies** `sonic_fbs_policies` **FBS interfaces** `sonic_fbs_interfaces` **Control-plane policing** `sonic_copp`

Use Case: Configure QoS, RoCE and lossless Ethernet

Description	Manage QoS buffers, maps, schedulers, WRED, PFC and RoCE settings, DCBx and drop counters.

Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**QoS buffers** `sonic_qos_buffer` **QoS interfaces** `sonic_qos_interfaces` **QoS maps** `sonic_qos_maps` **QoS PFC** `sonic_qos_pfc` **QoS schedulers** `sonic_qos_scheduler` **QoS WRED profiles** `sonic_qos_wred` **RoCE QoS** `sonic_roce` **DCBx** `sonic_dcbx` **Drop counters** `sonic_drop_counter`

Use Case: Secure device access and authentication

Description	Manage AAA, users, RADIUS, TACACS, LDAP, MFA, login lockout, SSH, PKI and FIPS.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**AAA** `sonic_aaa` **Users** `sonic_users` **RADIUS servers** `sonic_radius_server` **TACACS servers** `sonic_tacacs_server` **LDAP** `sonic_ldap` **Multi-factor authentication** `sonic_mfa` **Login lockout** `sonic_login_lockout` **SSH client** `sonic_ssh` **SSH server** `sonic_ssh_server` **PKI** `sonic_pki` **FIPS** `sonic_fips`

Use Case: Configure monitoring, time and discovery

Description	Set up logging, NTP, PTP, sFlow, port mirroring, LLDP and network policy.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**Logging** `sonic_logging` **NTP** `sonic_ntp` **PTP global** `sonic_ptp_default_ds` **PTP ports** `sonic_ptp_port_ds` **sFlow** `sonic_sflow` **Port mirroring** `sonic_mirroring` **LLDP global** `sonic_lldp_global` **LLDP interfaces** `sonic_lldp_interfaces` **Network policy** `sonic_network_policy`

Use Case: Administer devices and run commands

Description	Set system parameters, management servers, software images, send CLI commands or REST calls, manage configuration sections and use example playbooks.
Collection	`dellemc.enterprise_sonic` (Certified, Dell Technologies, v4.1.0, updated 2026-01-27)
Key content	**System settings** `sonic_system` **Management servers** `sonic_mgmt_servers` **Image management** `sonic_image_management` **CLI commands** `sonic_command` **Configuration sections** `sonic_config` **REST operations** `sonic_api` **SONiC action plugin** `sonic`, `sonic_facts` **Interface naming** playbook `common_examples.interface_naming` **Management VRF** playbook `common_examples.mgmt_vrf_config` **AAA example** playbook `common_examples.sonic_aaa` **REST API example** playbook `common_examples.sonic_api` **BGP example** playbook `common_examples.sonic_bgp` **BGP communities example** playbook `common_examples.sonic_bgp_communities` **CLI command example** playbook `common_examples.sonic_command` **Configuration example** playbook `common_examples.sonic_config` **Facts example** playbook `common_examples.sonic_facts` **Interfaces example** playbook `common_examples.sonic_interfaces_config` **System example** playbook `common_examples.sonic_system` **VXLAN example** playbook `common_examples.sonic_vxlans_config`

Dell EMC SmartFabric OS10 `dellemc.os10`

Certified Dell Technologies · version 1.2.7 · updated 2025-01-09 · 11 modules → 7 use cases

Use Case: Run commands and manage device configuration

Description	Run commands on OS10 switches, manage configuration sections and push a backup running configuration to the device.
Collection	`dellemc.os10` (Certified, Dell Technologies, v1.2.7, updated 2025-01-09)

Key content	**Run commands** `os10_command` **Configuration sections** `os10_config` **Action plugin documentation stub** `os10`, `os10_facts` **Push backup configuration** role `os10_copy_config` **Structured CLI output** role `os10_template` **TextFSM parser** `textfsm_parser` **XML to JSON conversion** `base_xml_to_dict`
Notes	• Most configuration is delivered as roles that render templates and push them with os10_config; the role names map one to one to OS10 feature areas.

Use Case: Configure interfaces, VLANs and Layer 2

Description	Configure interfaces, VLANs, link aggregation, LLDP, spanning tree, VLT and uplink settings.
Collection	`dellemc.os10` (Certified, Dell Technologies, v1.2.7, updated 2025-01-09)
Key content	**Interfaces** role `os10_interface` **VLANs** role `os10_vlan` **Link aggregation (LAG)** role `os10_lag` **LLDP** role `os10_lldp` **Spanning tree** role `os10_xstp` **VLT** role `os10_vlt` **Uplink failure detection** role `os10_uplink`
Notes	• Most configuration is delivered as roles that render templates and push them with os10_config; the role names map one to one to OS10 feature areas.

Use Case: Configure routing and Layer 3 services

Description	Configure BGP, VRFs, ECMP, VRRP, BFD, prefix lists and route maps.
Collection	`dellemc.os10` (Certified, Dell Technologies, v1.2.7, updated 2025-01-09)
Key content	**BGP** role `os10_bgp` **VRFs** role `os10_vrf` **ECMP groups** role `os10_ecmp` **VRRP** role `os10_vrrp` **BFD** role `os10_bfd` **Prefix lists** role `os10_prefix_list` **Route maps** role `os10_route_map`
Notes	• Most configuration is delivered as roles that render templates and push them with os10_config; the role names map one to one to OS10 feature areas.

Use Case: Configure security, access and QoS policy

Description	Configure AAA, users, access control lists, flow monitoring, IPv6 RA guard and QoS attributes.
Collection	`dellemc.os10` (Certified, Dell Technologies, v1.2.7, updated 2025-01-09)
Key content	**AAA** role `os10_aaa` **Local users** role `os10_users` **Access control lists** role `os10_acl` **ACL flow-based monitoring** role `os10_flow_monitor` **IPv6 RA guard** role `os10_raguard` **QoS** role `os10_qos`
Notes	• Most configuration is delivered as roles that render templates and push them with os10_config; the role names map one to one to OS10 feature areas.

Use Case: Administer the switch system and software

Description	Configure system attributes, DNS, NTP, logging and SNMP, and install or upgrade OS10 software images.
Collection	`dellemc.os10` (Certified, Dell Technologies, v1.2.7, updated 2025-01-09)
Key content	**System attributes** role `os10_system` **DNS** role `os10_dns` **NTP** role `os10_ntp` **Logging** role `os10_logging` **SNMP** role `os10_snmp` **Software image upgrade** role `os10_image_upgrade`
Notes	• Most configuration is delivered as roles that render templates and push them with os10_config; the role names map one to one to OS10 feature areas.

Use Case: Validate the network and fabric state

Description	Validate BGP neighbors, MTU, VLT peers and wiring, and collect system network information from all switches in the fabric.
Collection	`dellemc.os10` (Certified, Dell Technologies, v1.2.7, updated 2025-01-09)
Key content	**Network validation** role `os10_network_validation` **Fabric summary** role `os10_fabric_summary` **System network summary** `show_system_network_summary` **BGP neighbor validation** `bgp_validate` **MTU validation** `mtu_validate` **VLT validation** `vlt_validate` **Wiring validation** `wiring_validate`
Notes	• Most configuration is delivered as roles that render templates and push them with os10_config; the role names map one to one to OS10 feature areas.

Use Case: Deploy data center fabrics

Description	Deploy a CLOS leaf-spine fabric with eBGP or a VXLAN EVPN fabric across OS10 switches.
Collection	`dellemc.os10` (Certified, Dell Technologies, v1.2.7, updated 2025-01-09)
Key content	**VXLAN and EVPN** role `os10_vxlan` **CLOS fabric with eBGP** playbook `clos_fabric_ebgp.datacenter` **VXLAN EVPN fabric** playbook `vxlan_evpn.datacenter`
Notes	• Most configuration is delivered as roles that render templates and push them with os10_config; the role names map one to one to OS10 feature areas.

F5 BIG-IP and BIG-IQ `f5networks.f5_bigip`

Certified F5 Networks · version 3.14.0 · updated 2025-07-03 · 58 modules → 7 use cases

Use Case: Deploy application services with declarative automation

Description	Deploy AS3, FAST, Device Onboarding, Telemetry Streaming and Cloud Failover declarations to BIG-IP and BIG-IQ, and manage iControl LX packages.
Collection	`f5networks.f5_bigip` (Certified, F5 Networks, v3.14.0, updated 2025-07-03)
Key content	**AS3 on BIG-IP** `bigip_as3_deploy` **AS3 on BIG-IQ** `bigiq_as3_deploy` **FAST applications** `bigip_fast_application` **FAST template sets** `bigip_fast_template` **Declarative Onboarding on BIG-IP** `bigip_do_deploy` **Declarative Onboarding on BIG-IQ** `bigiq_do_deploy` **Telemetry Streaming** `bigip_ts_deploy` **Cloud Failover Extension** `bigip_cfe_deploy` **iControl LX packages** `bigip_lx_package`

Use Case: Protect applications with WAF and access policy

Description	Import and export Advanced WAF and ASM policies and custom attack signatures, manage WAF policies and APM access policies, and security HTTP, log and SSH proxy profiles.
Collection	`f5networks.f5_bigip` (Certified, F5 Networks, v3.14.0, updated 2025-07-03)
Key content	**Advanced WAF policies** `bigip_awaf_policy` **Custom attack signatures** `bigip_awaf_custom_attack_signatures` **ASM policy import** `bigip_asm_policy_import` **ASM policy export** `bigip_asm_policy_fetch` **APM policy import** `bigip_apm_policy_import` **APM policy export** `bigip_apm_policy_fetch` **Security HTTP profiles** `bigip_security_http_profile` **Security log profiles** `bigip_security_log_profile` **SSH proxy profiles** `bigip_security_ssh_profile` **SSH proxy keys** `bigip_security_ssh_profile_keys` **SSH proxy rules** `bigip_security_ssh_profile_rules`

Use Case: Orchestrate SSL inspection with SSL Orchestrator

Description	Configure SSL Orchestrator topologies, SSL settings, authentication, resolvers, security policies and service chains, and attach inline security services.

Collection	`f5networks.f5_bigip` (Certified, F5 Networks, v3.14.0, updated 2025-07-03)
Key content	**Topologies** `bigip_sslo_config_topology` **SSL configuration** `bigip_sslo_config_ssl` **Authentication objects** `bigip_sslo_config_authentication` **DNS resolver** `bigip_sslo_config_resolver` **Security policies** `bigip_sslo_config_policy` **Service chains** `bigip_sslo_config_service_chain` **O365 URL updater** `bigip_sslo_config_o365` **Utility functions** `bigip_sslo_config_utility` **HTTP services** `bigip_sslo_service_http` **ICAP services** `bigip_sslo_service_icap` **Layer 2 services** `bigip_sslo_service_layer2` **Layer 3 services** `bigip_sslo_service_layer3` **TAP services** `bigip_sslo_service_tap` **SWG service** `bigip_sslo_service_swg` **O365 tenant restriction** `bigip_sslo_service_o365_tr` **Off-box AWAF service** `bigip_sslo_service_offbox_awaf` **On-box WAF service** `bigip_sslo_service_waf_onbox`

Use Case: Manage SSL certificates and keys

Description	Create certificate signing requests and import or delete SSL keys, certificates and PKCS12 bundles.
Collection	`f5networks.f5_bigip` (Certified, F5 Networks, v3.14.0, updated 2025-07-03)
Key content	**Certificate signing requests** `bigip_ssl_csr` **Keys and certificates** `bigip_ssl_key_cert` **PKCS12 bundles** `bigip_ssl_pkcs12`

Use Case: Upgrade software and back up configuration

Description	Upload and install software images, and upload, install, fetch and remove UCS archives.
Collection	`f5networks.f5_bigip` (Certified, F5 Networks, v3.14.0, updated 2025-07-03)
Key content	**Software images** `bigip_software_image` **Software installation** `bigip_software_install` **UCS archives** `bigip_ucs` **UCS fetch** `bigip_ucs_fetch`

Use Case: Run commands and administer devices

Description	Run TMSH and Bash commands, manage configuration and advanced routing sections, config-sync, QKview diagnostics, vCMP guests, and gather device information.
Collection	`f5networks.f5_bigip` (Certified, F5 Networks, v3.14.0, updated 2025-07-03)
Key content	**TMSH and Bash commands** `bigip_command` **Configuration sections** `bigip_config` **Advanced routing configuration** `bigip_imish_config` **Config-sync actions** `bigip_configsync_action` **QKview diagnostics** `bigip_qkview` **vCMP guests** `bigip_vcmp_guest` **BIG-IP device information** `bigip_device_info`

Use Case: Manage BIG-IP fleets and licences with BIG-IQ

Description	Discover BIG-IP devices through BIG-IQ and manage registration key and utility licence pools and assignments.
Collection	`f5networks.f5_bigip` (Certified, F5 Networks, v3.14.0, updated 2025-07-03)
Key content	**BIG-IQ device information** `bigiq_device_info` **Device discovery** `bigiq_device_discovery` **Registration key pools** `bigiq_regkey_pool` **Registration key licences** `bigiq_regkey_license` **Registration key assignments** `bigiq_regkey_license_assignment` **Utility licences** `bigiq_utility_license` **Utility licence assignments** `bigiq_utility_license_assignment`

F5 BIG-IP `f5networks.f5_eda_security`

Validated F5 Networks · version 1.0.1 · updated 2024-12-05 · 0 modules → 1 use cases

Use Case: Respond to threats on F5 BIG-IP

Description	Switch ASM policy enforcement mode and update ASM policies with blocked IPs on BIG-IP classic and BIG-IP Next from Event-Driven Ansible.
Collection	`f5networks.f5_eda_security` (Validated, F5 Networks, v1.0.1, updated 2024-12-05)

Key content	**Set blocking mode (classic)** playbook `bigip-classic.awaf-blocking` **Set transparent mode (classic)** playbook `bigip-classic.awaf-transparent` **Block IPs (classic)** playbook `bigip-classic.block-ips` **Block IPs (Next)** playbook `bigip-next.block-ips`

F5 BIG-IP and BIG-IQ `f5networks.f5_modules`

Certified F5 Networks · version 1.44.0 · updated 2026-09-08 · 179 modules → 14 use cases

Use Case: Publish applications through virtual servers and pools

Description	Create virtual servers and addresses, pools, members and nodes, SNAT, traffic policies, iRules, data groups and iApp or LX packages for local traffic management.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Virtual servers** `bigip_virtual_server` **Virtual addresses** `bigip_virtual_address` **Pools** `bigip_pool` **Pool members** `bigip_pool_member` **Nodes** `bigip_node` **SNAT pools** `bigip_snat_pool` **SNAT translations** `bigip_snat_translation` **LTM policies** `bigip_policy` **LTM policy rules** `bigip_policy_rule` **iRules** `bigip_irule` **Data groups** `bigip_data_group` **iApp services** `bigip_iapp_service` **iApp templates** `bigip_iapp_template` **LX packages** `bigip_lx_package` **Global LTM settings** `bigip_ltm_global`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Tune traffic with profiles and persistence

Description	Configure HTTP, TCP, UDP, SSL, FTP, SIP, DNS and analytics profiles, persistence, timer and service policies, and message-routing for generic message protocols.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Analytics profiles** `bigip_profile_analytics` **Client SSL profiles** `bigip_profile_client_ssl` **Server SSL profiles** `bigip_profile_server_ssl` **DNS profiles** `bigip_profile_dns` **Fast L4 profiles** `bigip_profile_fastl4` **FTP profiles** `bigip_profile_ftp` **HTTP profiles** `bigip_profile_http` **HTTP/2 profiles** `bigip_profile_http2` **HTTP compression profiles** `bigip_profile_http_compression` **OneConnect profiles** `bigip_profile_oneconnect` **Cookie persistence profiles** `bigip_profile_persistence_cookie` **Source address persistence profiles** `bigip_profile_persistence_src_addr` **Universal persistence profiles** `bigip_profile_persistence_universal` **SIP profiles** `bigip_profile_sip` **TCP profiles** `bigip_profile_tcp` **UDP profiles** `bigip_profile_udp` **Timer policies** `bigip_timer_policy` **Service policies** `bigip_service_policy` **Message routing peers** `bigip_message_routing_peer` **Message routing protocols** `bigip_message_routing_protocol` **Message routing routes** `bigip_message_routing_route` **Message routing routers** `bigip_message_routing_router` **Message routing transports** `bigip_message_routing_transport_config`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Monitor application health

Description	Define LTM health monitors for HTTP, HTTPS, TCP, UDP, ICMP, DNS, FTP, LDAP, MySQL, Oracle, SMTP, SNMP and external checks.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)

Key content	**DNS monitors** `bigip_monitor_dns` **External monitors** `bigip_monitor_external` **FTP monitors** `bigip_monitor_ftp` **Gateway ICMP monitors** `bigip_monitor_gateway_icmp` **HTTP monitors** `bigip_monitor_http` **HTTPS monitors** `bigip_monitor_https` **ICMP monitors** `bigip_monitor_icmp` **LDAP monitors** `bigip_monitor_ldap` **MySQL monitors** `bigip_monitor_mysql` **Oracle monitors** `bigip_monitor_oracle` **SMTP monitors** `bigip_monitor_smtp` **SNMP DCA monitors** `bigip_monitor_snmp_dca` **TCP monitors** `bigip_monitor_tcp` **TCP echo monitors** `bigip_monitor_tcp_echo` **TCP half-open monitors** `bigip_monitor_tcp_half_open` **UDP monitors** `bigip_monitor_udp`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Distribute traffic globally with DNS and GTM

Description	Configure data centers, GTM servers, pools, wide IPs and monitors, DNS listeners and zones, resolvers and topology-based routing.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Global GTM settings** `bigip_gtm_global` **Data centers** `bigip_gtm_datacenter` **GTM servers** `bigip_gtm_server` **GTM virtual servers** `bigip_gtm_virtual_server` **GTM pools** `bigip_gtm_pool` **GTM pool members** `bigip_gtm_pool_member` **Wide IPs** `bigip_gtm_wide_ip` **DNS listeners** `bigip_gtm_dns_listener` **Topology regions** `bigip_gtm_topology_region` **Topology records** `bigip_gtm_topology_record` **GTM BIG-IP monitors** `bigip_gtm_monitor_bigip` **GTM external monitors** `bigip_gtm_monitor_external` **GTM FirePass monitors** `bigip_gtm_monitor_firepass` **GTM HTTP monitors** `bigip_gtm_monitor_http` **GTM HTTPS monitors** `bigip_gtm_monitor_https` **GTM TCP monitors** `bigip_gtm_monitor_tcp` **GTM TCP half-open monitors** `bigip_gtm_monitor_tcp_half_open` **DNS zones** `bigip_dns_zone` **DNS nameservers** `bigip_dns_nameserver` **DNS resolvers** `bigip_dns_resolver` **DNS cache resolvers** `bigip_dns_cache_resolver`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Protect applications with AFM firewall and DoS controls

Description	Define AFM firewall policies, rules, address and port lists, schedules, DoS profiles and vectors, and firewall logging profiles.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Firewall policies** `bigip_firewall_policy` **Firewall rules** `bigip_firewall_rule`, `bigip_firewall_rule_list` **Address lists** `bigip_firewall_address_list` **Port lists** `bigip_firewall_port_list` **Schedules** `bigip_firewall_schedule` **Global rules** `bigip_firewall_global_rules` **DoS profiles** `bigip_firewall_dos_profile` **DoS attack vectors** `bigip_firewall_dos_vector` **Firewall log profiles** `bigip_firewall_log_profile` **Network firewall log settings** `bigip_firewall_log_profile_network`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Secure web applications and remote access with ASM and APM

Description	Import, export and manage ASM web application firewall policies and DoS settings, and import APM policies, access control lists and network access resources.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)

Key content	**ASM policies** `bigip_asm_policy_manage` **ASM policy imports** `bigip_asm_policy_import` **ASM policy exports** `bigip_asm_policy_fetch` **ASM server technologies** `bigip_asm_policy_server_technology` **ASM signature sets** `bigip_asm_policy_signature_set` **ASM advanced settings** `bigip_asm_advanced_settings` **ASM DoS application settings** `bigip_asm_dos_application` **APM policy imports** `bigip_apm_policy_import` **APM policy exports** `bigip_apm_policy_fetch` **APM ACLs** `bigip_apm_acl` **APM network access resources** `bigip_apm_network_access`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Configure network connectivity and routing

Description	Set up VLANs, self IPs, trunks, interfaces, route domains, routes, tunnels, IPsec, advanced routing and CGNAT pools.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**VLANs** `bigip_vlan` **Self IPs** `bigip_selfip` **Trunks** `bigip_trunk` **Physical interfaces** `bigip_interface` **Route domains** `bigip_routedomain` **Static routes** `bigip_static_route` **Management routes** `bigip_management_route` **Advanced routing configuration** `bigip_imish_config` **Global network settings** `bigip_network_globals` **Tunnels** `bigip_tunnel` **IPsec IKE peers** `bigip_ike_peer` **IPsec policies** `bigip_ipsec_policy` **IPsec traffic selectors** `bigip_traffic_selector` **CGNAT LSN pools** `bigip_cgnat_lsn_pool`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Manage certificates and keys

Description	Import and delete SSL certificates and keys, create CSRs, configure OCSP and renew device certificates.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**SSL certificates** `bigip_ssl_certificate` **SSL keys** `bigip_ssl_key` **Key and certificate pairs** `bigip_ssl_key_cert` **Certificate signing requests** `bigip_ssl_csr` **OCSP configurations** `bigip_ssl_ocsp` **Device certificates** `bigip_device_certificate`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Administer users and authentication

Description	Manage local users, partitions, password policy, remote authentication through LDAP and RADIUS, and remote user defaults and roles.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Users** `bigip_user` **Partitions** `bigip_partition` **Password policy** `bigip_password_policy` **System authentication** `bigip_device_auth` **LDAP authentication** `bigip_device_auth_ldap` **RADIUS authentication** `bigip_device_auth_radius` **RADIUS servers** `bigip_device_auth_radius_server` **Remote roles** `bigip_remote_role` **Remote user defaults** `bigip_remote_user`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Build high-availability clusters

Description	Establish device trust, device groups, traffic groups, HA groups and config-sync between BIG-IP systems.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Device trust** `bigip_device_trust` **HA connectivity** `bigip_device_connectivity` **Device groups** `bigip_device_group` **Device group members** `bigip_device_group_member` **Traffic groups** `bigip_device_traffic_group` **HA groups** `bigip_device_ha_group` **Config-sync actions** `bigip_configsync_action`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: License, provision and upgrade devices

Description	Install licenses, provision modules, create vCMP guests, install software images and manage UCS archives, QKviews and files.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Device licenses** `bigip_device_license` **Module provisioning** `bigip_provision` **Module provisioning (asynchronous)** `bigip_provision_async` **vCMP guests** `bigip_vcmp_guest` **Software images** `bigip_software_image` **Software installation** `bigip_software_install` **Software update settings** `bigip_software_update` **UCS archives** `bigip_ucs` **UCS downloads** `bigip_ucs_fetch` **QKviews** `bigip_qkview` **Datastore files** `bigip_file_copy`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Configure system settings, logging and monitoring

Description	Set hostname, DNS, NTP, SSH, HTTPD, SMTP, SNMP, syslog and log destinations, and tune system database variables, CLI scripts and configuration sections.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**Hostname** `bigip_hostname` **System DNS** `bigip_device_dns` **NTP servers** `bigip_device_ntp` **SSH daemon** `bigip_device_sshd` **HTTPD settings** `bigip_device_httpd` **SMTP settings** `bigip_smtp` **SNMP settings** `bigip_snmp` **SNMP communities** `bigip_snmp_community` **SNMP traps** `bigip_snmp_trap` **System syslog** `bigip_device_syslog` **Remote syslog** `bigip_remote_syslog` **Log destinations** `bigip_log_destination` **Log publishers** `bigip_log_publisher` **System database variables** `bigip_sys_db` **Global system settings** `bigip_sys_global` **TMM daemon logging** `bigip_sys_daemon_log_tmm` **CLI aliases** `bigip_cli_alias` **CLI scripts** `bigip_cli_script` **Configuration sections** `bigip_config`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Run commands and gather device facts

Description	Collect device information, run TMSH or BASH commands and wait for a device to be ready before continuing.
Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**BIG-IP device information** `bigip_device_info` **BIG-IQ device information** `bigiq_device_info` **TMSH and BASH commands** `bigip_command` **Wait for device condition** `bigip_wait` **File path helper filter** filter `markdev`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

Use Case: Manage BIG-IQ applications, devices and licenses

Description	Deploy applications through BIG-IQ, discover BIG-IP devices, and manage registration key and utility license pools and assignments.

Collection	`f5networks.f5_modules` (Certified, F5 Networks, v1.44.0, updated 2026-09-08)
Key content	**HTTP applications** `bigiq_application_http` **HTTPS offload applications** `bigiq_application_https_offload` **HTTPS WAF applications** `bigiq_application_https_waf` **FastHTTP applications** `bigiq_application_fasthttp` **FastL4 TCP applications** `bigiq_application_fastl4_tcp` **FastL4 UDP applications** `bigiq_application_fastl4_udp` **Device discovery** `bigiq_device_discovery` **Registration key pools** `bigiq_regkey_pool` **Registration key licenses** `bigiq_regkey_license` **Registration key license assignments** `bigiq_regkey_license_assignment` **Utility licenses** `bigiq_utility_license` **Utility license assignments** `bigiq_utility_license_assignment` **BIG-IQ license lookup** lookup `bigiq_license` **License hopper lookup** lookup `license_hopper`
Notes	• Modules cover BIG-IP (bigip_*) and BIG-IQ (bigiq_*) through their iControl REST interfaces; most are idempotent and can be run against a device or a cluster member.

F5 BIG-IP `f5networks.f5_platform_modernization`

Validated F5 Networks · version 1.0.0 · updated 2026-01-26 · 0 modules → 3 use cases

Use Case: Migrate standalone BIG-IP devices

Description	Back up a source BIG-IP, move configuration and UCS files to the destination, and run the standalone migration in stages or in full.
Collection	`f5networks.f5_platform_modernization` (Validated, F5 Networks, v1.0.0, updated 2026-01-26)
Key content	**Backup** role `backup` **Source BIG-IP** role `f5_source` **Destination BIG-IP** role `f5_destination` **Full standalone migration** playbook `standalone.migrate_standalone_full` **Standalone migration part 1** playbook `standalone.migrate_standalone_pt1` **Standalone migration part 2** playbook `standalone.migrate_standalone_pt2`

Use Case: Migrate high availability pairs

Description	Migrate combined or sequential HA pairs in stages, including waiting for UCS restore.
Collection	`f5networks.f5_platform_modernization` (Validated, F5 Networks, v1.0.0, updated 2026-01-26)
Key content	**Combined HA part 1** playbook `high_availability.migrate_ha_combined_pt1` **Combined HA part 2** playbook `high_availability.migrate_ha_combined_pt2` **Sequential HA part 1** playbook `high_availability.migrate_ha_sequential_pt1` **Sequential HA part 2** playbook `high_availability.migrate_ha_sequential_pt2` **Sequential HA part 3** playbook `high_availability.migrate_ha_sequential_pt3` **UCS restore wait** playbook `high_availability.ha_test_code`

Use Case: Prepare and clean up test applications

Description	Create AS3 test applications at scale and remove all tenants.
Collection	`f5networks.f5_platform_modernization` (Validated, F5 Networks, v1.0.0, updated 2026-01-26)
Key content	**AS3 test applications** playbook `test_code.00_pre-setup_apps` **Remove tenants** playbook `test_code.00_delete_apps`

F5OS (rSeries and VELOS) `f5networks.f5os`

Certified F5 Networks · version 1.20.0 · updated 2025-08-14 · 50 modules → 7 use cases

Use Case: Manage tenants and VELOS partitions

Description	Deploy and manage tenants and their images and console access on rSeries and VELOS, and manage VELOS chassis partitions and their high availability.
Collection	`f5networks.f5os` (Certified, F5 Networks, v1.20.0, updated 2025-08-14)

Key content	**Tenants** `f5os_tenant` **Tenant images** `f5os_tenant_image` **Tenant console access** `f5os_tenant_console_enable` **Tenant wait** `f5os_tenant_wait` **VELOS partitions** `velos_partition` **Partition high availability** `velos_partition_ha_config` **Partition images** `velos_partition_image` **Partition password** `velos_partition_change_password` **Partition wait** `velos_partition_wait`

Use Case: Configure ports, VLANs and Layer 2

Description	Configure interfaces, port groups, LAGs, VLANs, LLDP, spanning tree, forwarding database entries and management interfaces.
Collection	`f5networks.f5os` (Certified, F5 Networks, v1.20.0, updated 2025-08-14)
Key content	**Interfaces** `f5os_interface` **Port groups** `f5os_portgroup` **LAGs** `f5os_lag` **VLANs** `f5os_vlan` **LLDP** `f5os_lldp_config` **Spanning tree** `f5os_stp_config` **Forwarding database** `f5os_fdb` **rSeries management interfaces** `rseries_management_interfaces` **VELOS controller management interfaces** `velos_controller_management_interfaces`

Use Case: Configure quality of service

Description	Configure QoS mappings, meter groups and traffic priorities.
Collection	`f5networks.f5os` (Certified, F5 Networks, v1.20.0, updated 2025-08-14)
Key content	**QoS mappings** `f5os_qos_mapping` **QoS meter groups** `f5os_qos_meter_group` **QoS traffic priorities** `f5os_qos_traffic_priority`

Use Case: Manage users, authentication and certificates

Description	Manage users and passwords, LDAP and external authentication, login policy, client certificate authentication, TLS certificates, allowed IPs, appliance mode and the primary key.
Collection	`f5networks.f5os` (Certified, F5 Networks, v1.20.0, updated 2025-08-14)
Key content	**Users and roles** `f5os_user` **User passwords** `f5os_user_password_change` **Authentication settings** `f5os_auth` **LDAP configuration** `f5os_auth_ldap` **Authentication server groups** `f5os_auth_server` **Login policy** `f5os_login_policy` **Client certificate authentication** `f5os_client_cert_auth` **TLS certificate and key** `f5os_tls_cert_key` **TLS certificate and key import** `f5os_import_tls_cert_key` **Allowed IPs** `f5os_allowed_ips` **Appliance mode** `f5os_appliance_mode` **Primary key** `f5os_primarykey`

Use Case: Configure system services

Description	Configure system settings, DNS, NTP, proxy, banners, SNMP, logging and audit logging, and license activation.
Collection	`f5networks.f5os` (Certified, F5 Networks, v1.20.0, updated 2025-08-14)
Key content	**System settings** `f5os_system` **DNS** `f5os_dns` **NTP servers** `f5os_ntp_server` **Proxy server** `f5os_proxy_server` **Login banner and MoTD** `f5os_banner` **SNMP** `f5os_snmp` **Logging** `f5os_logging` **Audit logging** `f5os_audit_log` **Licensing** `f5os_license`

Use Case: Upgrade software and protect configuration

Description	Import and install system images, back up and restore configuration, and generate qkview diagnostics.
Collection	`f5networks.f5os` (Certified, F5 Networks, v1.20.0, updated 2025-08-14)
Key content	**System image import** `f5os_system_image_import` **System software install** `f5os_system_image_install` **Configuration backup** `f5os_config_backup` **Configuration restore** `f5os_config_restore` **Qkview generation** `f5os_qkview`

Use Case: Gather facts and health

Description	Collect device facts, device information and system health.
Collection	`f5networks.f5os` (Certified, F5 Networks, v1.20.0, updated 2025-08-14)
Key content	**Facts** `f5os_facts` **Device information** `f5os_device_info` **System health** `f5os_system_health_info`

F5 BIG-IP Next `f5networks.next`

Certified F5 Networks · version 1.5.0 · updated 2024-12-05 · 17 modules → 3 use cases

Use Case: Deploy and onboard BIG-IP Next instances

Description	Deploy instances on F5OS and VMware, discover them in Central Manager, manage providers and licenses.
Collection	`f5networks.next` (Certified, F5 Networks, v1.5.0, updated 2024-12-05)
Key content	**F5OS deployments** `cm_next_deploy_f5os` **VMware deployments** `cm_next_deploy_vmware` **Instance discovery** `cm_next_discover` **Providers** `cm_next_provider` **Licenses** `cm_next_license` **Device information** `cm_device_info`

Use Case: Configure applications, certificates and high availability

Description	Deploy AS3 declarations, manage certificates, files, high availability, failover and global resiliency groups.
Collection	`f5networks.next` (Certified, F5 Networks, v1.5.0, updated 2024-12-05)
Key content	**AS3 declarations** `cm_next_as3_deploy` **Certificate creation** `cm_ssl_certificate_create` **Certificate import** `cm_ssl_certificate_import` **Instance files** `cm_next_files` **Central Manager files** `cm_files` **High availability** `cm_next_ha` **HA failover** `cm_next_ha_failover` **Global resiliency groups** `cm_next_global_resiliency_group`

Use Case: Upgrade and back up

Description	Upgrade instances and back up and restore Central Manager and instance configuration.
Collection	`f5networks.next` (Certified, F5 Networks, v1.5.0, updated 2024-12-05)
Key content	**Instance upgrades** `cm_next_upgrade` **Central Manager backup and restore** `cm_backup_restore` **Instance backup and restore** `cm_next_backup_restore`

FRRouting `frr.frr`

Certified Red Hat · version 2.0.2 · updated 2023-04-14 · 2 modules → 1 use cases

Use Case: Configure BGP and gather facts on FRR

Description	Configure global BGP settings and collect facts from FRR devices.
Collection	`frr.frr` (Certified, Red Hat, v2.0.2, updated 2023-04-14)

Key content	**Global BGP** `frr_bgp` **Facts** `frr_facts`

Gluware Control `gluware_inc.control`

Certified Gluware · version 2.0.3 · updated 2026-01-15 · 14 modules → 3 use cases

Use Case: Manage devices and inventory

Description	Manage device lifecycle, discover and update device attributes and build inventory from Gluware Control.
Collection	`gluware_inc.control` (Certified, Gluware, v2.0.3, updated 2026-01-15)
Key content	**Devices** `glu_device`, `glu_device_facts` **Device attribute discovery** `glu_run_discover_device_attributes` **Device attribute updates** `glu_update_device_attributes` **Gluware inventory** inventory `glu_devices`

Use Case: Audit, back up and restore configuration

Description	Run configuration audits, capture configuration to detect drift, take backups, list them and restore devices.
Collection	`gluware_inc.control` (Certified, Gluware, v2.0.3, updated 2026-01-15)
Key content	**Configuration audits** `glu_audit_config` **Batch configuration audits** `glu_audit_config_batch` **Configuration capture** `glu_capture_config` **Backups** `glu_backup` **Backup list** `glu_get_backups` **Restores** `glu_restore`

Use Case: Provision and run workflows

Description	Preview and provision configuration models and run RPA workflows on one or many devices.
Collection	`gluware_inc.control` (Certified, Gluware, v2.0.3, updated 2026-01-15)
Key content	**Config modeling preview** `glu_cm_preview` **Config modeling provision** `glu_cm_provision` **RPA workflows** `glu_rpa_workflow` **Batch RPA workflows** `glu_rpa_workflow_batch`

H3C Comware 7 `h3c_open.comware`

Certified H3C · version 1.0.2 · updated 2024-01-11 · 73 modules → 9 use cases

Use Case: Configure interfaces, VLANs and Layer 2

Description	Configure interface state, MTU and IP addressing, switchport and VLAN settings, port channels, LACP, LLDP, STP and DLDP.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**Interfaces** `comware_interface` **IP interfaces** `comware_ipinterface` **MTU and jumbo frames** `comware_mtu` **Switchport settings** `comware_switchport` **VLANs** `comware_vlan` **Port channels (LAG)** `comware_portchannel` **LACP** `comware_lacp` **LLDP timers** `comware_lldp` **LLDP global settings** `comware_lldp_global` **LLDP interfaces** `comware_lldp_interface` **LLDP neighbors** `comware_neighbors` **Spanning tree** `comware_stp` **Interface spanning tree** `comware_iface_stp` **DLDP** `comware_dldp` **Interface state check** `comware_intfstate`

Use Case: Configure routing and VRFs

Description	Configure BGP, OSPF and IS-IS routing and VPN instances.

Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**BGP global settings** `comware_bgp_global` **BGP address families** `comware_bgp_af` **BGP groups** `comware_bgp_group` **OSPF** `comware_ospf` **OSPF interfaces** `comware_ospf_intf` **IS-IS global settings** `comware_isis_global` **IS-IS interfaces** `comware_isis_interface` **VPN instances** `comware_vpn_instance`

Use Case: Build VXLAN and EVPN fabrics

Description	Configure EVPN, L2VPN, VSIs and VXLAN tunnels and mappings.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**EVPN** `comware_evpn` **L2VPN global settings** `comware_l2vpn_global` **VSIs** `comware_vsi` **VSI interfaces** `comware_vsi_intf` **VXLAN mappings** `comware_vxlan` **VXLAN service instances** `comware_vxlan_svc_instance` **VXLAN tunnels** `comware_vxlan_tunnel`

Use Case: Stack switches and provide gateway redundancy

Description	Manage IRF stacking membership and ports, VRRP, and BFD failure detection.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**IRF members** `comware_irf_members` **IRF ports** `comware_irf_ports` **VRRP** `comware_vrrp` **VRRP global settings** `comware_vrrp_global` **BFD** `comware_bfd`

Use Case: Control access, authentication and users

Description	Configure ACLs, AAA, RADIUS and HWTACACS schemes, local users, and FTP service.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**ACLs** `comware_acl` **AAA** `comware_aaa` **RADIUS schemes** `comware_radius` **HWTACACS schemes** `comware_hwtacacs` **Local users** `comware_local_user` **FTP service** `comware_ftp`

Use Case: Configure multicast and time services

Description	Configure IGMP multicast and NTP time synchronisation.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**IGMP** `comware_igmp` **NTP** `comware_ntp`

Use Case: Monitor, log and collect telemetry

Description	Configure SNMP, syslog and info-center logging, sFlow, NetStream and telemetry streaming, and collect diagnostic logs.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)

Key content	**SNMP communities** `comware_snmp_community` **SNMP groups** `comware_snmp_group` **SNMP target hosts** `comware_snmp_target_host` **SNMP users** `comware_snmp_user` **Syslog global settings** `comware_syslog_global` **Log output rules** `comware_log_source` **Log hosts** `comware_loghost` **Diagnostic log collection** `comware_log` **sFlow** `comware_sflow` **sFlow interfaces** `comware_sflow_intf` **NetStream** `comware_netstream` **Telemetry streaming** `comware_tele_stream` **Telemetry flow groups** `comware_teleflowgroup_global` **Telemetry flow trace** `comware_telemetryflowtrace` **NETCONF service** `comware_netconf`

Use Case: Upgrade devices and manage configuration

Description	Install OS images and patches, load licenses, set startup files, save, back up, compare and roll back configuration, reboot or factory-reset devices.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**OS install** `comware_install_os` **Patches** `comware_patch` **Licenses** `comware_license` **Startup files** `comware_startup` **File copy** `comware_file_copy` **Save configuration** `comware_save` **Configuration backup** `comware_config` **Install configuration** `comware_install_config` **Configuration compare** `comware_compare` **Configuration rollback** `comware_rollback` **Reboot** `comware_reboot` **Factory reset** `comware_clean_erase`

Use Case: Run commands and gather facts

Description	Run CLI commands, gather device facts and test reachability with ping.
Collection	`h3c_open.comware` (Certified, H3C, v1.0.2, updated 2024-01-11)
Key content	**CLI commands** `comware_command` **Facts** `comware_facts` **Ping** `comware_ping`

Infoblox NIOS `infoblox.nios_modules`

Certified Infoblox · version 1.8.0 · updated 2025-01-29 · 35 modules → 6 use cases

Use Case: Manage DNS zones, views and name server groups

Description	Create DNS views and zones, and define authoritative, delegation, forwarding, stub and forward/stub name server groups.
Collection	`infoblox.nios_modules` (Certified, Infoblox, v1.8.0, updated 2025-01-29)
Key content	**DNS zones** `nios_zone` **DNS views** `nios_dns_view` **Authoritative name server groups** `nios_nsgroup` **Delegation groups** `nios_nsgroup_delegation` **Forwarding member groups** `nios_nsgroup_forwardingmember` **Forward/stub server groups** `nios_nsgroup_forwardstubserver` **Stub member groups** `nios_nsgroup_stubmember` **Zone creation sample** playbook `create_zone` **Zone deletion sample** playbook `delete_zone` **DNS view creation sample** playbook `create_dns_view` **DNS view deletion sample** playbook `delete_dnsview` **Delegation group sample** playbook `create_nsgroup_delegation` **Forwarding member group sample** playbook `create_nsgroup_forwardingmember` **Forward/stub server group sample** playbook `create_nsgroup_forwardstubserver` **Stub member group sample** playbook `create_nsgroup_stubmember`
Notes	• Module names have the nios_ prefix omitted; sample playbooks are listed with the object they create, update or delete.

Use Case: Manage DNS records

Description	Add, update and remove A, AAAA, CNAME, host, MX, NAPTR, PTR, SRV and TXT records.
Collection	`infoblox.nios_modules` (Certified, Infoblox, v1.8.0, updated 2025-01-29)
Key content	**A records** `nios_a_record` **AAAA records** `nios_aaaa_record` **CNAME records** `nios_cname_record` **Host records** `nios_host_record` **MX records** `nios_mx_record` **NAPTR records** `nios_naptr_record` **PTR records** `nios_ptr_record` **SRV records** `nios_srv_record` **TXT records** `nios_txt_record` **A record creation sample** playbook `create_a_record` **A record update sample** playbook `update_a_record` **A record deletion by name sample** playbook `delete_a_record_by_name` **CNAME record creation sample** playbook `create_cname_record` **CNAME record deletion sample** playbook `delete_cname_record` **MX record creation sample** playbook `create_mx_record` **MX record deletion sample** playbook `delete_mx_record` **TXT record creation sample** playbook `create_txt_record` **TXT record deletion sample** playbook `delete_txt_record` **Host record deletion sample** playbook `delete_host_record` **Host record alias idempotency sample** playbook `host_record_aliases_issue160` **Host record duplicate-name sample** playbook `host_record_issue108` **Host record next-IP allocation sample** playbook `host_record_nextip_issue63`
Notes	• Module names have the nios_ prefix omitted; sample playbooks are listed with the object they create, update or delete.

Use Case: Manage IPAM: networks, ranges, fixed addresses and VLANs

Description	Maintain network views, networks, DHCP ranges, fixed addresses, VLANs and extensible attributes.
Collection	`infoblox.nios_modules` (Certified, Infoblox, v1.8.0, updated 2025-01-29)
Key content	**Network views** `nios_network_view` **Networks** `nios_network` **DHCP ranges** `nios_range` **DHCP fixed addresses** `nios_fixed_address` **VLANs** `nios_vlan` **Extensible attributes** `nios_extensible_attribute` **Network creation sample** playbook `create_network` **Network deletion sample** playbook `delete_network` **Network view creation sample** playbook `create_network_view` **Network view deletion sample** playbook `delete_network_view` **Extensible attribute sample** playbook `create_extensible_attribute` **Extensible attribute inheritance sample** playbook `extattrs_inheritance`
Notes	• Module names have the nios_ prefix omitted; sample playbooks are listed with the object they create, update or delete.

Use Case: Manage DNS traffic control (DTC)

Description	Define DTC servers, pools, LBDNs, topologies and HTTP, ICMP, PDP, SIP, SNMP and TCP health monitors.
Collection	`infoblox.nios_modules` (Certified, Infoblox, v1.8.0, updated 2025-01-29)
Key content	**DTC servers** `nios_dtc_server` **DTC pools** `nios_dtc_pool` **DTC LBDNs** `nios_dtc_lbdn` **DTC topologies** `nios_dtc_topology` **HTTP monitors** `nios_dtc_monitor_http` **ICMP monitors** `nios_dtc_monitor_icmp` **PDP monitors** `nios_dtc_monitor_pdp` **SIP monitors** `nios_dtc_monitor_sip` **SNMP monitors** `nios_dtc_monitor_snmp` **TCP monitors** `nios_dtc_monitor_tcp` **DTC server sample** playbook `create_dtc_server` **DTC server (WAPI 2.14) sample** playbook `nios_dtc_server_wapi_2_14_example` **DTC pool sample** playbook `create_dtc_pool` **DTC LBDN sample** playbook `create_dtc_lbdn` **DTC topology creation sample** playbook `create_dtc_topology` **DTC topology deletion sample** playbook `delete_dtc_topology` **HTTP monitor creation sample** playbook `create_dtc_monitor_http` **HTTP monitor deletion sample** playbook `delete_dtc_monitor_http` **ICMP monitor creation sample** playbook `create_dtc_monitor_icmp` **ICMP monitor deletion sample** playbook `delete_dtc_monitor_icmp` **PDP monitor creation sample** playbook `create_dtc_monitor_pdp` **PDP monitor deletion sample** playbook `delete_dtc_monitor_pdp` **SIP monitor creation sample** playbook `create_dtc_monitor_sip` **SIP monitor deletion sample** playbook `delete_dtc_monitor_sip` **SNMP monitor creation sample** playbook `create_dtc_monitor_snmp` **SNMP monitor deletion sample** playbook `delete_dtc_monitor_snmp` **TCP monitor creation sample** playbook `create_dtc_monitor_tcp` **TCP monitor deletion sample** playbook `delete_dtc_monitor_tcp`
Notes	• Module names have the nios_ prefix omitted; sample playbooks are listed with the object they create, update or delete.

Use Case: Administer the Grid

Description	Manage Grid members and admin users and restart Grid services.
Collection	`infoblox.nios_modules` (Certified, Infoblox, v1.8.0, updated 2025-01-29)
Key content	**Grid members** `nios_member` **Admin users** `nios_adminuser` **Service restarts** `nios_restartservices` **Member update sample** playbook `update_member` **Admin user sample** playbook `create_adminuser` **Service restart sample** playbook `restart_services`

Notes	• Module names have the nios_ prefix omitted; sample playbooks are listed with the object they create, update or delete.

Use Case: Query NIOS and allocate next-available resources

Description	Use NIOS as dynamic inventory and look up objects, the next free IP, network or VLAN ID.
Collection	`infoblox.nios_modules` (Certified, Infoblox, v1.8.0, updated 2025-01-29)
Key content	**NIOS inventory** inventory `nios_inventory` **Object lookup** lookup `nios_lookup` **Next available IP** lookup `nios_next_ip` **Next available network** lookup `nios_next_network` **Next available VLAN ID** lookup `nios_next_vlan_id`
Notes	• Module names have the nios_ prefix omitted; sample playbooks are listed with the object they create, update or delete.

Infoblox Universal DDI `infoblox.universal_ddi`

Certified Infoblox · version 1.1.0 · updated 2025-12-03 · 87 modules → 6 use cases

Use Case: Manage IP address space (IPAM)

Description	Create IP spaces, address blocks, subnets, ranges and addresses, and query the next available block, subnet or IP.
Collection	`infoblox.universal_ddi` (Certified, Infoblox, v1.1.0, updated 2025-12-03)
Key content	**IP addresses** `ipam_address`, `ipam_address_info` **Address blocks** `ipam_address_block`, `ipam_address_block_info` **Federated blocks** `ipam_federation_federated_block`, `ipam_federation_federated_block_info` **Federated realms** `ipam_federation_federated_realm`, `ipam_federation_federated_realm_info` **IPAM hosts** `ipam_host`, `ipam_host_info` **IP spaces** `ipam_ip_space`, `ipam_ip_space_info` **Next available address block** `ipam_next_available_address_block_info` **Next available IP** `ipam_next_available_ip_info` **Next available subnet** `ipam_next_available_subnet_info` **DHCP ranges** `ipam_range`, `ipam_range_info` **Subnets** `ipam_subnet`, `ipam_subnet_info`

Use Case: Manage authoritative and forward DNS

Description	Create DNS views, authoritative and forward zones, delegations and resource records, and copy DNS objects between views.
Collection	`infoblox.universal_ddi` (Certified, Infoblox, v1.1.0, updated 2025-12-03)
Key content	**Authoritative zones** `dns_auth_zone`, `dns_auth_zone_info` **Zone delegations** `dns_delegation`, `dns_delegation_info` **Forward zones** `dns_forward_zone`, `dns_forward_zone_info` **DNS resource records** `dns_record`, `dns_record_info` **DNS views** `dns_view`, `dns_view_info` **DNS view bulk copy** `dns_view_bulk_copy`

Use Case: Manage DNS and DHCP servers and options

Description	Configure DNS and DHCP server profiles, name server groups, ACLs, HA groups, DHCP hosts, fixed addresses, DHCP options and TSIG and Kerberos keys.
Collection	`infoblox.universal_ddi` (Certified, Infoblox, v1.1.0, updated 2025-12-03)
Key content	**Fixed addresses** `dhcp_fixed_address`, `dhcp_fixed_address_info` **DHCP HA groups** `dhcp_ha_group`, `dhcp_ha_group_info` **DHCP hosts** `dhcp_host`, `dhcp_host_info` **DHCP option codes** `dhcp_option_code`, `dhcp_option_code_info` **DHCP option groups** `dhcp_option_group`, `dhcp_option_group_info` **DHCP option spaces** `dhcp_option_space`, `dhcp_option_space_info` **DHCP config profiles (servers)** `dhcp_server`, `dhcp_server_info` **Named ACLs** `dns_acl`, `dns_acl_info` **Authoritative name server groups** `dns_auth_nsg`, `dns_auth_nsg_info` **Forward name server groups** `dns_forward_nsg`, `dns_forward_nsg_info` **DNS hosts** `dns_host`, `dns_host_info` **DNS config profiles (servers)** `dns_server`, `dns_server_info` **Kerberos keys** `kerberos_key_info` **TSIG keys** `tsig_key`, `tsig_key_info`

Use Case: Manage DNS traffic control (DTC)

Description	Define DTC servers, pools, LBDNs, policies and health checks to steer DNS answers by service health.
Collection	`infoblox.universal_ddi` (Certified, Infoblox, v1.1.0, updated 2025-12-03)
Key content	**HTTP health checks** `dtc_health_check_http`, `dtc_health_check_http_info` **ICMP health checks** `dtc_health_check_icmp`, `dtc_health_check_icmp_info` **SNMP health checks** `dtc_health_check_snmp`, `dtc_health_check_snmp_info` **TCP health checks** `dtc_health_check_tcp`, `dtc_health_check_tcp_info` **LBDNs** `dtc_lbdn`, `dtc_lbdn_info` **DTC policies** `dtc_policy`, `dtc_policy_info` **DTC pools** `dtc_pool`, `dtc_pool_info` **DTC servers** `dtc_server`, `dtc_server_info` **SNMP user security models** `dtc_snmp_user_security_model`, `dtc_snmp_user_security_model_info`

Use Case: Manage Universal DDI infrastructure and anycast

Description	Register infrastructure hosts and services with join tokens, configure anycast and discover cloud providers.
Collection	`infoblox.universal_ddi` (Certified, Infoblox, v1.1.0, updated 2025-12-03)
Key content	**Anycast configurations** `anycast_config`, `anycast_config_info` **Anycast hosts** `anycast_host`, `anycast_host_info` **Cloud discovery providers** `cloud_discovery_providers`, `cloud_discovery_providers_info` **Infrastructure hosts** `infra_host`, `infra_host_info` **Host activation join tokens** `infra_join_token`, `infra_join_token_info` **Infrastructure services** `infra_service`, `infra_service_info`

Use Case: Query Universal DDI objects

Description	Look up Universal DDI objects from playbooks using the lookup plugin.
Collection	`infoblox.universal_ddi` (Certified, Infoblox, v1.1.0, updated 2025-12-03)
Key content	**Universal DDI lookup plugin** lookup `universal_ddi_lookup`

Juniper Apstra `juniper.apstra`

Certified Juniper Networks · version 1.1.0 · updated 2026-07-31 · 40 modules → 5 use cases

Use Case: Design the fabric

Description	Manage logical devices, rack types, templates, configlets, property sets, tags, interface maps and resource pools used to design a data center fabric.
Collection	`juniper.apstra` (Certified, Juniper Networks, v1.1.0, updated 2026-07-31)
Key content	**Logical devices, rack types and templates** `design` **Configlets** `configlets` **Property sets** `property_set` **Tags** `tag` **Interface maps** `interface_map` **Resource pools** `resource_pools` **Resource groups** `resource_group`

Use Case: Build and operate blueprints

Description	Create and roll back blueprints, apply fabric settings and cabling, and collect configuration, health and reports.
Collection	`juniper.apstra` (Certified, Juniper Networks, v1.1.0, updated 2026-07-31)
Key content	**Blueprints** `blueprint` **Fabric settings** `fabric_settings` **Cabling maps** `cabling_map` **Rollback and revisions** `rollback` **Rendered configuration** `blueprint_config` **Anomalies and build errors** `blueprint_health` **Blueprint reports** `blueprint_report` **Intent-based analytics probes** `iba_probes`

Use Case: Provide tenant networking and connectivity

Description	Manage security zones, virtual networks, connectivity templates, routing policies, external gateways, interconnect gateways, floating IPs, generic systems and endpoint policies.

Collection	`juniper.apstra` (Certified, Juniper Networks, v1.1.0, updated 2026-07-31)
Key content	**Security zones and tenants** `security_zone` **Virtual networks** `virtual_network` **Connectivity templates** `connectivity_template` **Connectivity template assignments** `connectivity_template_assignment` **Routing policies** `routing_policy` **External EVPN gateways** `external_gateway` **Interconnect domains and gateways** `interconnect_gateway` **Floating IPs** `floating_ip` **Generic systems** `generic_systems` **Endpoint policies** `endpoint_policy` **Virtual infrastructure managers** `virtual_infra_manager`

Use Case: Onboard and upgrade devices

Description	Onboard devices with system agents and ZTP, manage devices, and upgrade device operating systems.
Collection	`juniper.apstra` (Certified, Juniper Networks, v1.1.0, updated 2026-07-31)
Key content	**System agents** `system_agents` **ZTP configuration** `ztp_config` **ZTP devices** `ztp_device` **Device management** `device_management` **NOS images** `os_images` **Device OS upgrades** `os_upgrade` **Upgrade groups** `upgrade_group`

Use Case: Administer the Apstra platform

Description	Authenticate, manage users, roles, AAA servers, and IP allow and deny lists, and gather facts.
Collection	`juniper.apstra` (Certified, Juniper Networks, v1.1.0, updated 2026-07-31)
Key content	**Authentication** `authenticate` **Apstra facts** `apstra_facts` **RBAC users** `rbac_user` **RBAC roles** `rbac_roles` **AAA (RADIUS) servers** `aaa_server` **IP allow list** `allowed_list` **IP deny list** `banned_list`

Juniper Junos `juniper.device`

Certified Juniper Networks · version 2.0.1 · updated 2025-11-19 · 53 modules → 7 use cases

Use Case: Configure interfaces, VLANs and Layer 2

Description	Configure interfaces, Layer 2 and Layer 3 settings, VLANs, link aggregation, LACP and LLDP.
Collection	`juniper.device` (Certified, Juniper Networks, v2.0.1, updated 2025-11-19)
Key content	**Interfaces** `junos_interfaces` **Layer 2 interfaces** `junos_l2_interfaces` **Layer 3 interfaces** `junos_l3_interfaces` **VLANs** `junos_vlans` **Link aggregation interfaces** `junos_lag_interfaces` **LACP global settings** `junos_lacp` **LACP interfaces** `junos_lacp_interfaces` **LLDP global settings** `junos_lldp_global` **LLDP interfaces** `junos_lldp_interfaces`

Use Case: Configure routing and VRFs

Description	Configure BGP, OSPFv2, OSPFv3, static routes, prefix lists, routing options, routing instances and VRFs.
Collection	`juniper.device` (Certified, Juniper Networks, v2.0.1, updated 2025-11-19)

Key content	**BGP global settings** `junos_bgp_global` **BGP address families** `junos_bgp_address_family` **OSPFv2** `junos_ospfv2` **OSPFv3** `junos_ospfv3` **OSPF interfaces** `junos_ospf_interfaces` **Static routes** `junos_static_routes` **Prefix lists** `junos_prefix_lists` **Routing options** `junos_routing_options` **Routing instances** `junos_routing_instances` **VRFs** `junos_vrf`

Use Case: Secure the network with ACLs and firewall policy

Description	Configure ACLs, SRX security policies and zones, and chassis clusters.
Collection	`juniper.device` (Certified, Juniper Networks, v2.0.1, updated 2025-11-19)
Key content	**ACLs** `junos_acls` **ACL interfaces** `junos_acl_interfaces` **Security policies** `junos_security_policies` **Global security policy settings** `junos_security_policies_global` **Security zones** `junos_security_zones` **SRX chassis clusters** `srx_cluster`

Use Case: Administer the device

Description	Manage hostname, system attributes, users, banners, NTP, SNMP, logging and the NETCONF service.
Collection	`juniper.device` (Certified, Juniper Networks, v2.0.1, updated 2025-11-19)
Key content	**Hostname** `junos_hostname` **Hostname (tbr)** `tbrdevice_hostname` **System attributes** `junos_system` **Local users** `junos_user` **Banners** `junos_banner` **NTP** `junos_ntp_global` **SNMP server** `junos_snmp_server` **Logging** `junos_logging_global` **NETCONF service** `junos_netconf`

Use Case: Run commands, RPCs and manage configuration

Description	Run CLI commands and NETCONF RPCs, load and commit configuration, retrieve data with PyEZ tables, gather facts, and connect through PyEZ.
Collection	`juniper.device` (Certified, Juniper Networks, v2.0.1, updated 2025-11-19)
Key content	**CLI commands** `command` **Junos CLI commands** `junos_command` **Arbitrary commands** `junos`, `junos_facts` **NETCONF RPCs** `rpc` **Junos RPCs** `junos_rpc` **Configuration** `config` **Junos configuration** `junos_config` **PyEZ tables and views** `table` **Facts** `facts` **Device facts (tbr)** `tbrdevice_facts` **PyEZ connection** connection `pyez`

Use Case: Upgrade software and operate the system

Description	Install Junos software and packages, copy files, and reboot, halt or otherwise operate the system.
Collection	`juniper.device` (Certified, Juniper Networks, v2.0.1, updated 2025-11-19)
Key content	**Software installation** `software` **Packages** `junos_package` **File copy** `file_copy` **System operations** `system`

Use Case: Test and validate network state

Description	Test reachability with ping and path MTU discovery, and run JSNAPy snapshot tests with formatted results.
Collection	`juniper.device` (Certified, Juniper Networks, v2.0.1, updated 2025-11-19)

Key content	**Ping** `ping` **Junos ping** `junos_ping` **Path MTU discovery** `pmtud` **JSNAPy tests** `jsnapy` **JSNAPy result output** callback `jsnapy`

Juniper Junos `junipernetworks.junos`

Certified Red Hat · version 11.1.1 · updated 2026-03-09 · 38 modules → 5 use cases

Use Case: Configure interfaces, VLANs and link aggregation

Description	Manage interfaces, Layer 2 and Layer 3 settings, VLANs, LAGs, LACP and LLDP.
Collection	`junipernetworks.junos` (Certified, Red Hat, v11.1.1, updated 2026-03-09)
Key content	**Interfaces** `junos_interfaces` **Layer 2 interfaces** `junos_l2_interfaces` **Layer 3 interfaces** `junos_l3_interfaces` **VLANs** `junos_vlans` **LAG interfaces** `junos_lag_interfaces` **LACP** `junos_lacp` **LACP interfaces** `junos_lacp_interfaces` **LLDP** `junos_lldp_global` **LLDP interfaces** `junos_lldp_interfaces`

Use Case: Configure routing

Description	Manage BGP, OSPFv2 and OSPFv3, static routes, routing options, routing instances, VRFs and prefix lists.
Collection	`junipernetworks.junos` (Certified, Red Hat, v11.1.1, updated 2026-03-09)
Key content	**BGP** `junos_bgp_global` **BGP address families** `junos_bgp_address_family` **OSPFv2** `junos_ospfv2` **OSPFv3** `junos_ospfv3` **OSPF interfaces** `junos_ospf_interfaces` **Static routes** `junos_static_routes` **Routing options** `junos_routing_options` **Routing instances** `junos_routing_instances` **VRFs** `junos_vrf` **Prefix lists** `junos_prefix_lists`

Use Case: Manage security policies and ACLs

Description	Configure firewall filters, ACLs, security zones and security policies.
Collection	`junipernetworks.junos` (Certified, Red Hat, v11.1.1, updated 2026-03-09)
Key content	**ACLs** `junos_acls` **ACLs on interfaces** `junos_acl_interfaces` **Security zones** `junos_security_zones` **Security policies** `junos_security_policies` **Global security policy settings** `junos_security_policies_global`

Use Case: Administer the device

Description	Manage hostname, banners, system settings, local users, NTP, logging, SNMP, NETCONF and packages.
Collection	`junipernetworks.junos` (Certified, Red Hat, v11.1.1, updated 2026-03-09)
Key content	**System attributes** `junos_system` **Hostname** `junos_hostname` **Banners** `junos_banner` **Local users** `junos_user` **NTP** `junos_ntp_global` **Logging** `junos_logging_global` **SNMP** `junos_snmp_server` **NETCONF service** `junos_netconf` **Packages** `junos_package`

Use Case: Run commands, push configuration and gather facts

Description	Run CLI commands and NETCONF RPCs, load configuration, test reachability and collect device facts.
Collection	`junipernetworks.junos` (Certified, Red Hat, v11.1.1, updated 2026-03-09)
Key content	**Configuration** `junos_config` **CLI commands** `junos_command` **NETCONF RPCs** `junos_rpc` **Reachability tests** `junos_ping` **Device facts** `junos_facts`

Kentik `kentik.ansible_eda`

Certified · Kentik · version 1.0.7 · updated 2024-03-25 · 0 modules → 1 use cases

Use Case: Respond to Kentik alerts

Description	Receive Kentik webhook events in Event-Driven Ansible.
Collection	`kentik.ansible_eda` (Certified, Kentik, v1.0.7, updated 2024-03-25)
Key content	**Webhook event source** eda event source `kentik_webhook` **Example playbook** playbook `example_playbook`

Lantronix LMOS `lantronix.lmos`

Certified · lantronix · version 1.0.4 · updated 2026-08-31 · 1 modules → 1 use cases

Use Case: Gather facts and validate Lantronix LMOS devices

Description	Collect facts from LMOS devices and run an IQOQ validation.
Collection	`lantronix.lmos` (Certified, lantronix, v1.0.4, updated 2026-08-31)
Key content	**Device facts** `lmos_facts` **IQOQ validation** playbook `iqoq_validate`

Lantronix Percepxion and SLC9000 `lantronix.oob`

Certified · lantronix · version 1.2.0 · updated 2026-08-31 · 21 modules → 6 use cases

Use Case: Onboard devices and organize the Percepxion fleet

Description	Register and assign devices, query the device inventory, organize devices with tags, smart groups and projects, and write a dynamic inventory from Percepxion.
Collection	`lantronix.oob` (Certified, lantronix, v1.2.0, updated 2026-08-31)
Key content	**Platform facts** `percepxion_facts` **Device import** `percepxion_import_devices` **Device inventory** `percepxion_devices` **Device tags** `percepxion_device_tags` **Smart groups** `percepxion_smart_groups` **Project assignment** `percepxion_projects` **Fleet inventory role** role `oob_fleet_inventory` **Platform facts playbook** playbook `percepxion.percepxion_facts` **Device import playbook** playbook `percepxion.percepxion_import_devices` **Device inventory playbook** playbook `percepxion.percepxion_devices` **Device tag playbook** playbook `percepxion.percepxion_device_tags` **Smart group playbook** playbook `percepxion.percepxion_smart_groups` **Project assignment playbook** playbook `percepxion.percepxion_projects`

Use Case: Configure, schedule and monitor through Percepxion

Description	Manage configuration content, scheduled job groups and event rules, and query telemetry and audit logs.
Collection	`lantronix.oob` (Certified, lantronix, v1.2.0, updated 2026-08-31)

Key content	**Configuration content** `percepxion_config` **Scheduled job groups** `percepxion_jobs` **Event rules** `percepxion_event_rules` **Telemetry** `percepxion_telemetry` **Audit logs** `percepxion_audit_logs` **Configuration playbook** playbook `percepxion.percepxion_config` **Job group playbook** playbook `percepxion.percepxion_jobs` **Event rule playbook** playbook `percepxion.percepxion_event_rules` **Telemetry playbook** playbook `percepxion.percepxion_telemetry` **Audit log playbook** playbook `percepxion.percepxion_audit_logs`

Use Case: Control user access

Description	Manage Percepxion user accounts and bulk suspend or restore, manage local accounts on SLC9000 devices, and lock down access outside an allowlist.
Collection	`lantronix.oob` (Certified, lantronix, v1.2.0, updated 2026-08-31)
Key content	**Percepxion users** `percepxion_users` **SLC9000 sysadmin account** `slc_users` **User management role** role `oob_user_management` **Percepxion user playbook** playbook `percepxion.percepxion_users` **SLC9000 user playbook** playbook `slc.slc_users` **Access lockdown playbook** playbook `lockdown`

Use Case: Manage firmware across the fleet

Description	Check firmware compliance and trigger firmware updates on Percepxion-managed fleets and on SLC9000 devices.
Collection	`lantronix.oob` (Certified, lantronix, v1.2.0, updated 2026-08-31)
Key content	**Fleet firmware** `percepxion_firmware` **SLC9000 firmware** `slc_firmware` **Firmware audit role** role `oob_firmware_audit` **Fleet firmware playbook** playbook `percepxion.percepxion_firmware` **SLC9000 firmware playbook** playbook `slc.slc_firmware`

Use Case: Configure SLC9000 console servers

Description	Gather facts, manage running configuration, ethernet interfaces and system identity, query serial ports and attached devices, and enforce hostname, NTP and syslog baselines.
Collection	`lantronix.oob` (Certified, lantronix, v1.2.0, updated 2026-08-31)
Key content	**SLC9000 facts** `slc_facts` **Running configuration** `slc_config` **Ethernet interfaces** `slc_network` **System identity and reboot** `slc_system` **Serial device ports** `slc_device_ports` **Devices on serial ports** `slc_managed_devices` **Baseline configuration role** role `oob_baseline_config` **Facts playbook** playbook `slc.slc_facts` **Configuration playbook** playbook `slc.slc_config` **Network playbook** playbook `slc.slc_network` **System identity playbook** playbook `slc.slc_system` **Serial port playbook** playbook `slc.slc_device_ports` **Managed device playbook** playbook `slc.slc_managed_devices`

Use Case: Run end-to-end out-of-band workflows

Description	Onboard SLC9000 devices, snapshot fleet health, confirm the out-of-band path during incidents, and audit out-of-band infrastructure security.
Collection	`lantronix.oob` (Certified, lantronix, v1.2.0, updated 2026-08-31)
Key content	**Day-1 SLC9000 onboarding** playbook `combo.day1_slc_onboarding` **Fleet onboarding** playbook `combo.new_device_fleet_onboarding` **Fleet health snapshot** playbook `combo.fleet_health_snapshot` **Incident response path check** playbook `combo.incident_response_oob` **Security audit** playbook `combo.security_audit`

NetBox `netbox.netbox`

Certified Netbox Labs Inc. · version 3.21.0 · updated 2025-03-12 · 90 modules → 9 use cases

Use Case: Model sites, locations, racks and tenants

Description	Define regions, sites, locations, racks and rack roles, tenants, contacts and manufacturers.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**Regions** `netbox_region` **Site groups** `netbox_site_group` **Sites** `netbox_site` **Locations** `netbox_location` **Rack groups** `netbox_rack_group` **Rack roles** `netbox_rack_role` **Racks** `netbox_rack` **Tenant groups** `netbox_tenant_group` **Tenants** `netbox_tenant` **Contact groups** `netbox_contact_group` **Contact roles** `netbox_contact_role` **Contacts** `netbox_contact` **Contact assignments** `netbox_contact_assignment` **Manufacturers** `netbox_manufacturer`

Use Case: Manage devices and hardware components

Description	Create devices, types, roles and platforms with their interfaces, ports, bays, modules and inventory items, and connect them with cables.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**Devices** `netbox_device` **Device types** `netbox_device_type` **Device roles** `netbox_device_role` **Platforms** `netbox_platform` **Virtual chassis** `netbox_virtual_chassis` **Inventory items** `netbox_inventory_item` **Inventory item roles** `netbox_inventory_item_role` **Device interfaces** `netbox_device_interface` **Interface templates** `netbox_device_interface_template` **Front ports** `netbox_front_port` **Front port templates** `netbox_front_port_template` **Rear ports** `netbox_rear_port` **Rear port templates** `netbox_rear_port_template` **Console ports** `netbox_console_port` **Console port templates** `netbox_console_port_template` **Console server ports** `netbox_console_server_port` **Console server port templates** `netbox_console_server_port_template` **Device bays** `netbox_device_bay` **Device bay templates** `netbox_device_bay_template` **Modules** `netbox_module` **Module types** `netbox_module_type` **Module bays** `netbox_module_bay` **Cables** `netbox_cable`

Use Case: Manage power infrastructure

Description	Track power panels, feeds, ports and outlets and their templates.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**Power panels** `netbox_power_panel` **Power feeds** `netbox_power_feed` **Power ports** `netbox_power_port` **Power port templates** `netbox_power_port_template` **Power outlets** `netbox_power_outlet` **Power outlet templates** `netbox_power_outlet_template`

Use Case: Manage IP addressing and VLANs

Description	Maintain RIRs, aggregates, ASNs, prefixes, IP and MAC addresses, VRFs, route targets, VLANs and FHRP groups.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)

Key content	**RIRs** `netbox_rir` **Aggregates** `netbox_aggregate` **ASNs** `netbox_asn` **Prefixes** `netbox_prefix` **IP addresses** `netbox_ip_address` **MAC addresses** `netbox_mac_address` **IPAM roles** `netbox_ipam_role` **VRFs** `netbox_vrf` **Route targets** `netbox_route_target` **VLAN groups** `netbox_vlan_group` **VLANs** `netbox_vlan` **FHRP groups** `netbox_fhrp_group` **FHRP group assignments** `netbox_fhrp_group_assignment` **Services** `netbox_service` **Service templates** `netbox_service_template`

Use Case: Track circuits, tunnels and overlays

Description	Record providers, provider networks, circuits and terminations, tunnels and L2VPNs.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**Providers** `netbox_provider` **Provider networks** `netbox_provider_network` **Circuit types** `netbox_circuit_type` **Circuits** `netbox_circuit` **Circuit terminations** `netbox_circuit_termination` **Tunnel groups** `netbox_tunnel_group` **Tunnels** `netbox_tunnel` **L2VPNs** `netbox_l2vpn` **L2VPN terminations** `netbox_l2vpn_termination`

Use Case: Model virtualisation and wireless

Description	Track clusters, virtual machines, their interfaces and disks, and wireless LANs and links.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**Cluster types** `netbox_cluster_type` **Cluster groups** `netbox_cluster_group` **Clusters** `netbox_cluster` **Virtual machines** `netbox_virtual_machine` **VM interfaces** `netbox_vm_interface` **Virtual disks** `netbox_virtual_disk` **Wireless LAN groups** `netbox_wireless_lan_group` **Wireless LANs** `netbox_wireless_lan` **Wireless links** `netbox_wireless_link`

Use Case: Customise NetBox and drive automation

Description	Add custom fields and links, tags, config contexts and templates, export templates, data sources, journal entries and webhooks.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**Custom fields** `netbox_custom_field` **Custom field choice sets** `netbox_custom_field_choice_set` **Custom links** `netbox_custom_link` **Tags** `netbox_tag` **Config contexts** `netbox_config_context` **Config templates** `netbox_config_template` **Export templates** `netbox_export_template` **Data sources** `netbox_data_source` **Journal entries** `netbox_journal_entry` **Webhooks** `netbox_webhook`

Use Case: Administer users and permissions

Description	Manage users, groups, API tokens and object permissions.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**Users** `netbox_user` **User groups** `netbox_user_group` **API tokens** `netbox_token` **Permissions** `netbox_permission`

Use Case: Use NetBox as inventory and lookup source

Description	Build dynamic inventory from NetBox and query its objects from playbooks.
Collection	`netbox.netbox` (Certified, Netbox Labs Inc., v3.21.0, updated 2025-03-12)
Key content	**NetBox inventory** inventory `nb_inventory` **NetBox lookup** lookup `nb_lookup`

Citrix NetScaler ADC `netscaler.adc`

Certified NetScaler · version 2.17.0 · updated 2026-07-15 · 976 modules → 33 use cases

Use Case: Configure load balancing virtual servers and policies

Description	Create load balancing virtual servers, their policy and service bindings, persistence, groups, routes and parameters.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Lb action** `lbaction` **LB global to lb policy binding** `lbglobal_lbpolicy_binding` **LB group** `lbgroup` **LB group to Load Balancing Virtual Server binding** `lbgroup_lbvserver_binding` **LB parameter** `lbparameter` **Persistence session** `lbpersistentsessions` **Lb policy** `lbpolicy` **Lb policy label** `lbpolicylabel` **Lb policy label to lb policy binding** `lbpolicylabel_lbpolicy_binding` **LB profile** `lbprofile` **LB route** `lbroute` **LB route6** `lbroute6` **SIP parameters** `lbsipparameters` **Load Balancing Virtual Server** `lbvserver` **Load Balancing Virtual Server to Analytics profile binding** `lbvserver_analyticsprofile_binding` **Load Balancing Virtual Server to AppFlow policy binding** `lbvserver_appflowpolicy_binding` **Load Balancing Virtual Server to application firewall policy binding** `lbvserver_appfwpolicy_binding` **Load Balancing Virtual Server to AppQoS policy binding** `lbvserver_appqoepolicy_binding` **Load Balancing Virtual Server to ns log policy binding** `lbvserver_auditnslogpolicy_binding` **Load Balancing Virtual Server to system log policy binding** `lbvserver_auditsyslogpolicy_binding` **Load Balancing Virtual Server to authorization policy binding** `lbvserver_authorizationpolicy_binding` **Load Balancing Virtual Server to Bot policy binding** `lbvserver_botpolicy_binding` **Load Balancing Virtual Server to Integrated Cache policy binding** `lbvserver_cachepolicy_binding` **Load Balancing Virtual Server to compression policy binding** `lbvserver_cmppolicy_binding` **Load Balancing Virtual Server to ContentInspection policy binding** `lbvserver_contentinspectionpolicy_binding` **Load Balancing Virtual Server to dns64 policy binding** `lbvserver_dnspolicy64_binding` **Load Balancing Virtual Server to Front end optimization policy binding** `lbvserver_feopolicy_binding` **Load Balancing Virtual Server to lb policy binding** `lbvserver_lbpolicy_binding` **Load Balancing Virtual Server to responder policy binding** `lbvserver_responderpolicy_binding` **Load Balancing Virtual Server to rewrite policy binding** `lbvserver_rewritepolicy_binding` **Load Balancing Virtual Server to service binding** `lbvserver_service_binding` **Load Balancing Virtual Server to service group binding** `lbvserver_servicegroup_binding` **Load Balancing Virtual Server to Spillover policy binding** `lbvserver_spilloverpolicy_binding` **Load Balancing Virtual Server to TM traffic policy binding** `lbvserver_tmtrafficpolicy_binding` **Load Balancing Virtual Server to URL Transformation policy binding** `lbvserver_transformpolicy_binding` **Load Balancing Virtual Server to videooptimization detectionpolicy binding** `lbvserver_videooptimizationdetectionpolicy_binding` **Load Balancing Virtual Server to videooptimization pacingpolicy binding** `lbvserver_videooptimizationpacingpolicy_binding` **Virtual server** `uservserver` **Virtual server** `vserver`

Use Case: Define backend servers, services and health monitors

Description	Define servers, services and service groups, health monitors, weighted metric tables, database load balancing profiles and Kafka clusters that load balancers distribute traffic to.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**DB profile** `dbdbprofile` **DB monitors** `dbsmonitors` **DB user** `dbuser` **Kafka cluster** `kafkacluster` **Kafka cluster to service group binding** `kafkacluster_servicegroup_binding` **Metric table** `lbmetrictable` **Metric table to metric binding** `lbmetrictable_metric_binding` **Monitor** `lbmonitor` **Monitor to metric binding** `lbmonitor_metric_binding` **Monitor to certificate key binding** `lbmonitor_sslcertkey_binding` **Web log manager** `lbwlm` **Web log manager to Load Balancing Virtual Server binding** `lbwlm_lbvserver_binding` **Server** `server` **Service** `service` **Service to monitor binding** `service_lbmonitor_binding` **Service group** `servicegroup` **Service group to monitor binding** `servicegroup_lbmonitor_binding` **Service group to servicegroupmember binding** `servicegroup_servicegroupmember_binding`

Use Case: Configure global server load balancing and geolocation

Description	Configure GSLB sites, services, service groups and virtual servers, LDNS entries and geolocation data for multi-site traffic distribution.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**Gslb config** `gslbconfig` **LDNS entry** `gslbldnsentries` **LDNS entry** `gslbldnsentry` **GSLB parameter** `gslbparameter` **GSLB service** `gslbservice` **GSLB service to DNS view binding** `gslbservice_dnsview_binding` **GSLB service to monitor binding** `gslbservice_lbmonitor_binding` **GSLB service group** `gslbservicegroup` **GSLB service group to gslbservicegroupmember binding** `gslbservicegroup_gslbservicegroupmember_binding` **GSLB service group to monitor binding** `gslbservicegroup_lbmonitor_binding` **GSLB site** `gslbsite` **Global Server Load Balancing Virtual Server** `gslbvserver` **Global Server Load Balancing Virtual Server to domain binding** `gslbvserver_domain_binding` **Global Server Load Balancing Virtual Server to GSLB service binding** `gslbvserver_gslbservice_binding` **Global Server Load Balancing Virtual Server to GSLB service group binding** `gslbvserver_gslbservicegroup_binding` **Global Server Load Balancing Virtual Server to lb policy binding** `gslbvserver_lbpolicy_binding` **Global Server Load Balancing Virtual Server to Spillover policy binding** `gslbvserver_spilloverpolicy_binding` **Location** `location` **Location data** `locationdata` **Location file** `locationfile` **Location file6** `locationfile6` **Location parameter** `locationparameter`

Use Case: Configure content switching and cache redirection

Description	Route requests with content switching virtual servers, actions and policies, and with cache redirection virtual servers and policies.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Cache redirection policy** `crpolicy` **CR virtual server** `crvserver` **CR virtual server to Analytics profile binding** `crvserver_analyticsprofile_binding` **CR virtual server to AppFlow policy binding** `crvserver_appflowpolicy_binding` **CR virtual server to application firewall policy binding** `crvserver_appfwpolicy_binding` **CR virtual server to AppQoS policy binding** `crvserver_appqoepolicy_binding` **CR virtual server to Integrated Cache policy binding** `crvserver_cachepolicy_binding` **CR virtual server to compression policy binding** `crvserver_cmppolicy_binding` **CR virtual server to cache redirection policy binding** `crvserver_crpolicy_binding` **CR virtual server to content-switching policy binding** `crvserver_cspolicy_binding` **CR virtual server to Front end optimization policy binding** `crvserver_feopolicy_binding` **CR virtual server to ICA policy binding** `crvserver_icapolicy_binding` **CR virtual server to Load Balancing Virtual Server binding** `crvserver_lbvserver_binding` **CR virtual server to map policy binding** `crvserver_policymap_binding` **CR virtual server to responder policy binding** `crvserver_responderpolicy_binding` **CR virtual server to rewrite policy binding** `crvserver_rewritepolicy_binding` **CR virtual server to Spillover policy binding** `crvserver_spilloverpolicy_binding` **Content Switching action** `csaction` **CS parameter** `csparameter` **Content-switching policy** `cspolicy` **CS policy label** `cspolicylabel` **CS policy label to content-switching policy binding** `cspolicylabel_cspolicy_binding` **CS virtual server** `csvserver` **CS virtual server to Analytics profile binding** `csvserver_analyticsprofile_binding` **CS virtual server to AppFlow policy binding** `csvserver_appflowpolicy_binding` **CS virtual server to application firewall policy binding** `csvserver_appfwpolicy_binding` **CS virtual server to AppQoS policy binding** `csvserver_appqoepolicy_binding` **CS virtual server to ns log policy binding** `csvserver_auditnslogpolicy_binding` **CS virtual server to system log policy binding** `csvserver_auditsyslogpolicy_binding` **CS virtual server to authorization policy binding** `csvserver_authorizationpolicy_binding` **CS virtual server to Bot policy binding** `csvserver_botpolicy_binding` **CS virtual server to Integrated Cache policy binding** `csvserver_cachepolicy_binding` **CS virtual server to compression policy binding** `csvserver_cmppolicy_binding` **CS virtual server to ContentInspection policy binding** `csvserver_contentinspectionpolicy_binding` **CS virtual server to content-switching policy binding** `csvserver_cspolicy_binding` **CS virtual server to domain binding** `csvserver_domain_binding` **CS virtual server to Front end optimization policy binding** `csvserver_feopolicy_binding` **CS virtual server to Global Server Load Balancing Virtual Server binding** `csvserver_gslbvserver_binding` **CS virtual server to Load Balancing Virtual Server binding** `csvserver_lbvserver_binding` **CS virtual server to responder policy binding** `csvserver_responderpolicy_binding` **CS virtual server to rewrite policy binding** `csvserver_rewritepolicy_binding` **CS virtual server to Spillover policy binding** `csvserver_spilloverpolicy_binding` **CS virtual server to TM traffic policy binding** `csvserver_tmtrafficpolicy_binding` **CS virtual server to URL Transformation policy binding** `csvserver_transformpolicy_binding` **CS virtual server to VPN virtual server binding** `csvserver_vpnvserver_binding`

Use Case: Accelerate content with caching, compression and optimization

Description	Configure integrated caching, HTTP compression, front-end optimization, video optimization and AppQoE policies and actions.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**AppQoS action** `appqoeaction` **AppQoE custom response page** `appqoecustomresp` **QOS parameter** `appqoeparameter` **AppQoS policy** `appqoepolicy` **Integrated Cache content group** `cachecontentgroup` **Forward proxy** `cacheforwardproxy` **CACHE global to Integrated Cache policy binding** `cacheglobal_cachepolicy_binding` **Cache object** `cacheobject` **Cache parameter** `cacheparameter` **Integrated Cache policy** `cachepolicy` **Cache policy label** `cachepolicylabel` **Cache policy label to Integrated Cache policy binding** `cachepolicylabel_cachepolicy_binding` **Cache selector** `cacheselector` **Compression action** `cmpaction` **CMP global to compression policy binding** `cmpglobal_cmppolicy_binding` **CMP parameter** `cmpparameter` **Compression policy** `cmppolicy` **Compression policy label** `cmppolicylabel` **Compression policy label to compression policy binding** `cmppolicylabel_cmppolicy_binding` **Front end optimization action** `feoaction` **FEO global to Front end optimization policy binding** `feoglobal_feopo` **FEO parameter** `feoparameter` **Front end optimization policy** `feopolicy` **HTTP request/response band** `protocolhttpband` **Videooptimization detectionaction** `videooptimizationdetectionactio` **Videooptimization detectionpolicy** `videooptimizationdetectionpolic` **Videooptimization detection policy label** `videooptimizationdetectior` **Videooptimization detection policy label to videooptimization detecti binding** `videooptimizationdetectionpolicylabel_videooptimizationdetectio` **Videooptimizationglobaldetection to videooptimization detectionpoli** `videooptimizationglobaldetection_videooptimizationdetectionpoli` **Videooptimizationglobalpacing to videooptimization pacingpolicy bi** `videooptimizationglobalpacing_videooptimizationpacingpolicy_bin` **Videooptimization pacingaction** `videooptimizationpacingaction` **Videooptimization pacingpolicy** `videooptimizationpacingpolicy` **Videooptimization pacing policy label** `videooptimizationpacingpolic` **Videooptimization pacing policy label to videooptimization pacingpo** `videooptimizationpacingpolicylabel_videooptimizationpacingpolic` **VideoOptimization parameter** `videooptimizationparameter`

Use Case: Rewrite, respond to and transform HTTP traffic

Description	Define rewrite, responder, transform and spillover actions, policies, policy labels and their global and virtual server bindings.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Responder action** `responderaction` **RESPONDER global to responder policy binding** `responderglobal_responderpolicy_binding` **Responder HTML page** `responderhtmlpage` **Responser parameter** `responderparam` **Responder policy** `responderpolicy` **Responder policy label** `responderpolicylabel` **Responder policy label to responder policy binding** `responderpolicylabel_responderpolicy_binding` **Rewrite action** `rewriteaction` **REWRITE global to rewrite policy binding** `rewriteglobal_rewritepolicy_binding` **Rewrite parameter** `rewriteparam` **Rewrite policy** `rewritepolicy` **Rewrite policy label** `rewritepolicylabel` **Rewrite policy label to rewrite policy binding** `rewritepolicylabel_rewritepolicy_binding` **Spillover action** `spilloveraction` **Spillover policy** `spilloverpolicy` **Transform action** `transformaction` **TRANSFORM global to URL Transformation policy binding** `transformglobal_transformpolicy_binding` **URL Transformation policy** `transformpolicy` **Transform policy label** `transformpolicylabel` **Transform policy label to URL Transformation policy binding** `transformpolicylabel_transformpolicy_binding` **URL Transformation profile** `transformprofile`

Use Case: Build policy expressions, data sets and content inspection

Description	Maintain shared policy building blocks such as expressions, data sets, pattern sets, string maps, callouts and URL sets, plus content inspection, ICAP, tunnel traffic policies, extensions, variables, service chains and WebAssembly modules.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Application** `application` **Content Inspection action** `contentinspectionaction` **Content Inspection callout** `contentinspectioncallout` **CONTENTINSPECTION global to ContentInspection policy binding** `contentinspectionglobal_contentinspectionpolicy_binding` **Contentinspection parameter** `contentinspectionparameter` **ContentInspection policy** `contentinspectionpolicy` **ContentInspection policy label** `contentinspectionpolicylabel` **ContentInspection policy label to ContentInspection policy binding** `contentinspectionpolicylabel_contentinspectionpolicy_binding` **ContentInspection profile** `contentinspectionprofile` **WASM profile** `contentinspectionwasmprofile` **AI GW profile** `nsaigwprofile` **Assignment** `nsassignment` **Extension** `nsextension` **ICAP profile** `nsicapprofile` **Service Function** `nsservicefunction` **Service Chain** `nsservicepath` **Service Chain to service Function binding** `nsservicepath_nsservicefunction_binding` **Variable** `nsvariable` **XML namespace** `nsxmlnamespace` **TYPE set** `policydataset` **TYPE set to value binding** `policydataset_value_binding` **Expression** `policyexpression` **HTTP callout** `policyhttpcallout` **Map policy** `policymap` **Policy parameter** `policyparam` **PAT set** `policypatset` **PAT set to pattern binding** `policypatset_pattern_binding` **Patset file** `policypatsetfile` **String map** `policystringmap` **String map to pattern binding** `policystringmap_pattern_binding` **Policytracing** `policytracing` **URL set** `policyurlset` **TUNNEL global to tunnel policy binding** `tunnelglobal_tunneltrafficpolicy_binding` **Tunnel policy** `tunneltrafficpolicy` **WASM module related files** `wasmfile` **WASM Module** `wasmmodule`

Use Case: Configure web application firewall profiles

Description	Define application firewall profiles and the security checks bound to them, such as SQL injection, cross-site scripting, command injection, URL and field checks.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**Application firewall profile** `appfwprofile` **Application firewall profile to configured confidential form fields binding** `appfwprofile_appfwconfidfield_binding` **Application firewall profile to blockkeyword binding** `appfwprofile_blockkeyword_binding` **Application firewall profile to bypasslist binding** `appfwprofile_bypasslist_binding` **Application firewall profile to cmdinjection binding** `appfwprofile_cmdinjection_binding` **Application firewall profile to contenttype binding** `appfwprofile_contenttype_binding` **Application firewall profile to cookieconsistency binding** `appfwprofile_cookieconsistency_binding` **Application firewall profile to creditcardnumber binding** `appfwprofile_creditcardnumber_binding` **Application firewall profile to crosssitescripting binding** `appfwprofile_crosssitescripting_binding` **Application firewall profile to csrftag binding** `appfwprofile_csrftag_binding` **Application firewall profile to denylist binding** `appfwprofile_denylist_binding` **Application firewall profile to denyurl binding** `appfwprofile_denyurl_binding` **Application firewall profile to excluderescontenttype binding** `appfwprofile_excluderescontenttype_binding` **Application firewall profile to fakeaccount binding** `appfwprofile_fakeaccount_binding` **Application firewall profile to fieldconsistency binding** `appfwprofile_fieldconsistency_binding` **Application firewall profile to fieldformat binding** `appfwprofile_fieldformat_binding` **Application firewall profile to fileuploadtype binding** `appfwprofile_fileuploadtype_binding` **Application firewall profile to grpcvalidation binding** `appfwprofile_grpcvalidation_binding` **Application firewall profile to jsonblockkeyword binding** `appfwprofile_jsonblockkeyword_binding` **Application firewall profile to jsoncmdurl binding** `appfwprofile_jsoncmdurl_binding` **Application firewall profile to jsondosurl binding** `appfwprofile_jsondosurl_binding` **Application firewall profile to jsonsqlurl binding** `appfwprofile_jsonsqlurl_binding` **Application firewall profile to jsonxssurl binding** `appfwprofile_jsonxssurl_binding` **Application firewall profile to logexpression binding** `appfwprofile_logexpression_binding` **Application firewall profile to restvalidation binding** `appfwprofile_restvalidation_binding` **Application firewall profile to safeobject binding** `appfwprofile_safeobject_binding` **Application firewall profile to sqlinjection binding** `appfwprofile_sqlinjection_binding` **Application firewall profile to starturl binding** `appfwprofile_starturl_binding` **Application firewall profile to trustedlearningclients binding** `appfwprofile_trustedlearningclients_binding` **Application firewall profile to xmlattachmenturl binding** `appfwprofile_xmlattachmenturl_binding` **Application firewall profile to xmldosurl binding** `appfwprofile_xmldosurl_binding` **Application firewall profile to xmlsqlinjection binding** `appfwprofile_xmlsqlinjection_binding` **Application firewall profile to xmlvalidationurl binding** `appfwprofile_xmlvalidationurl_binding` **Application firewall profile to xmlwsiurl binding** `appfwprofile_xmlwsiurl_binding` **Application firewall profile to xmlxss binding** `appfwprofile_xmlxss_binding`

Use Case: Manage application firewall policies, signatures and API protection

Description	Manage application firewall policies, labels, signatures, learning settings, content types and error pages, and define API profiles and specifications.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**API profile** `apiprofile` **API profile to API specification binding** `apiprofile_apispec_binding` **API specification** `apispec` **API specification file** `apispecfile` **Archive** `appfwarchive` **Configured confidential form fields** `appfwconfidfield` **Application firewall custom settings XML configuration** `appfwcustomsettings` **Application firewall form field type** `appfwfieldtype` **APPFW global to application firewall policy binding** `appfwglobal_appfwpolicy_binding` **APPFW global to ns log policy binding** `appfwglobal_auditnslogpolicy_binding` **APPFW global to system log policy binding** `appfwglobal_auditsyslogpolicy_binding` **GRRPC content type** `appfwgrpccontenttype` **GRRPC-Web-json content type** `appfwgrpcwebjsoncontenttype` **GRRPC-Web-text content type** `appfwgrpcwebtextcontenttype` **HTML error page** `appfwhtmlerrorpage` **JSON content type** `appfwjsoncontenttype` **JSON error page** `appfwjsonerrorpage` **Learning data** `appfwlearningdata` **Learning settings** `appfwlearningsettings` **Multipart form content type** `appfwmultipartformcontenttype` **Application firewall policy** `appfwpolicy` **Application firewall policy label** `appfwpolicylabel` **Application firewall policy label to application firewall policy binding** `appfwpolicylabel_appfwpolicy_binding` **GRPC protofile** `appfwprotofile` **AS settings** `appfwsettings` **Application firewall signatures XML configuration** `appfwsignatures` **Urlencoded form content type** `appfwurlencodedformcontenttype` **WSDL file** `appfwwsdl` **XML Content type** `appfwxmlcontenttype` **Xml error page** `appfwxmlerrorpage` **XML schema** `appfwxmlschema`

Use Case: Protect against bots

Description	Configure bot management profiles, policies, signatures, settings and IP reputation checks, with their rate limit, allow list, deny list and CAPTCHA bindings.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**BOT global to Bot policy binding** `botglobal_botpolicy_binding` **Bot policy** `botpolicy` **Bot policy label** `botpolicylabel` **Bot policy label to Bot policy binding** `botpolicylabel_botpolicy_binding` **Bot profile** `botprofile` **Bot profile to blacklist binding** `botprofile_blacklist_binding` **Bot profile to captcha binding** `botprofile_captcha_binding` **Bot profile to ipreputation binding** `botprofile_ipreputation_binding` **Bot profile to kmdetectionexpr binding** `botprofile_kmdetectionexpr_binding` **Bot profile to logexpression binding** `botprofile_logexpression_binding` **Bot profile to ratelimit binding** `botprofile_ratelimit_binding` **Bot profile to tps binding** `botprofile_tps_binding` **Bot profile to trapinsertionurl binding** `botprofile_trapinsertionurl_binding` **Bot profile to whitelist binding** `botprofile_whitelist_binding` **Bot engine settings** `botsettings` **Bot signatures** `botsignature` **Reputation service settings** `reputationsettings`

Use Case: Configure authentication virtual servers and policy labels

Description	Create authentication virtual servers and bind the authentication, authorization, session and other policies that apply to them, and maintain authentication policy labels.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Authentication policy label** `authenticationpolicylabel` **Authentication policy label to Authentication Policy binding** `authenticationpolicylabel_authenticationpolicy_binding` **Authentication virtual server** `authenticationvserver` **Authentication virtual server to ns log policy binding** `authenticationvserver_auditnslogpolicy_binding` **Authentication virtual server to system log policy binding** `authenticationvserver_auditsyslogpolicy_binding` **Authentication virtual server to CERT policy binding** `authenticationvserver_authenticationcertpolicy_binding` **Authentication virtual server to LDAP policy binding** `authenticationvserver_authenticationldappolicy_binding` **Authentication virtual server to LOCAL policy binding** `authenticationvserver_authenticationlocalpolicy_binding` **Authentication virtual server to authenticationloginschemapolicy binding** `authenticationvserver_authenticationloginschemapolicy_binding` **Authentication virtual server to Negotiate Policy binding** `authenticationvserver_authenticationnegotiatepolicy_binding` **Authentication virtual server to AAA OAuth IdentityProvider (IdP) policy binding** `authenticationvserver_authenticationoauthidppolicy_binding` **Authentication virtual server to Authentication Policy binding** `authenticationvserver_authenticationpolicy_binding` **Authentication virtual server to RADIUS policy binding** `authenticationvserver_authenticationradiuspolicy_binding` **Authentication virtual server to AAA Saml IdentityProvider (IdP) policy binding** `authenticationvserver_authenticationsamlidppolicy_binding` **Authentication virtual server to AAA Saml policy binding** `authenticationvserver_authenticationsamlpolicy_binding` **Authentication virtual server to SmartAccess policy binding** `authenticationvserver_authenticationsmartaccesspolicy_binding` **Authentication virtual server to TACACS policy binding** `authenticationvserver_authenticationtacacspolicy_binding` **Authentication virtual server to Web authentication policy binding** `authenticationvserver_authenticationwebauthpolicy_binding` **Authentication virtual server to Integrated Cache policy binding** `authenticationvserver_cachepolicy_binding` **Authentication virtual server to content-switching policy binding** `authenticationvserver_cspolicy_binding` **Authentication virtual server to responder policy binding** `authenticationvserver_responderpolicy_binding` **Authentication virtual server to rewrite policy binding** `authenticationvserver_rewritepolicy_binding` **Authentication virtual server to TM session policy binding** `authenticationvserver_tmsessionpolicy_binding` **Authentication virtual server to portaltheme binding** `authenticationvserver_vpnportaltheme_binding`

Use Case: Define authentication methods and identity provider integrations

Description	Define authentication actions and policies for LDAP, RADIUS, TACACS, SAML, OAuth, certificates, Negotiate, web authentication and login schemas, including SAML and OAuth identity provider profiles.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**ADFSProxy Profile** `authenticationadfsproxyprofile` **Authentication profile** `authenticationauthnprofile` **Captcha Action** `authenticationcaptchaaction` **CERT action** `authenticationcertaction` **CERT policy** `authenticationcertpolicy` **Citrix Authentication action** `authenticationcitrixauthaction` **Dfa authentication action** `authenticationdfaaction` **Dfa authentication policy** `authenticationdfapolicy` **Email entity** `authenticationemailaction` **Epa action** `authenticationepaaction` **LDAP action** `authenticationldapaction` **LDAP policy** `authenticationldappolicy` **LOCAL policy** `authenticationlocalpolicy` **Authenticationloginschema** `authenticationloginschema` **Authenticationloginschemapolicy** `authenticationloginschemapolicy` **Negotiate action** `authenticationnegotiateaction` **Negotiate Policy** `authenticationnegotiatepolicy` **No authentication action** `authenticationnoauthaction` **OAuth authentication action** `authenticationoauthaction` **AAA OAuth IdentityProvider (IdP) policy** `authenticationoauthidppolicy` **OAuth Identity Provider (IdP) profile** `authenticationoauthidpprofile` **Authentication Policy** `authenticationpolicy` **Protected user action** `authenticationprotecteduseraction` **Service details for sending push notifications** `authenticationpushservice` **RADIUS action** `authenticationradiusaction` **RADIUS policy** `authenticationradiuspolicy` **AAA Saml action** `authenticationsamlaction` **AAA Saml IdentityProvider (IdP) policy** `authenticationsamlidppolicy` **AAA Saml IdentityProvider (IdP) profile** `authenticationsamlidpprofile` **AAA Saml policy** `authenticationsamlpolicy` **SmartAccess policy** `authenticationsmartaccesspolicy` **SmartAccess profile** `authenticationsmartaccessprofile` **Storefront authentication action** `authenticationstorefrontauthaction` **TACACS action** `authenticationtacacsaction` **TACACS policy** `authenticationtacacspolicy` **Web authentication action** `authenticationwebauthaction` **Web authentication policy** `authenticationwebauthpolicy`

Use Case: Manage AAA users and groups

Description	Create AAA users and groups and bind session, authorization, audit and intranet access policies to them.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**AAA group** `aaagroup` **AAA group to AAA user binding** `aaagroup_aaauser_binding` **AAA group to ns log policy binding** `aaagroup_auditnslogpolicy_binding` **AAA group to system log policy binding** `aaagroup_auditsyslogpolicy_binding` **AAA group to authorization policy binding** `aaagroup_authorizationpolicy_binding` **AAA group to intranetip6 binding** `aaagroup_intranetip6_binding` **AAA group to intranetip binding** `aaagroup_intranetip_binding` **AAA group to TM session policy binding** `aaagroup_tmsessionpolicy_binding` **AAA group to SSLVPN intranet application binding** `aaagroup_vpnintranetapplication_binding` **AAA group to Secure Private Access Profile binding** `aaagroup_vpnsecureprivateaccessprofile_binding` **AAA group to VPN session policy binding** `aaagroup_vpnsessionpolicy_binding` **AAA group to VPN traffic policy binding** `aaagroup_vpntrafficpolicy_binding` **AAA group to VPN URL binding** `aaagroup_vpnurl_binding` **AAA group to VPN url policy binding** `aaagroup_vpnurlpolicy_binding` **AAA user** `aaauser` **AAA user to ns log policy binding** `aaauser_auditnslogpolicy_binding` **AAA user to system log policy binding** `aaauser_auditsyslogpolicy_binding` **AAA user to authorization policy binding** `aaauser_authorizationpolicy_binding` **AAA user to intranetip6 binding** `aaauser_intranetip6_binding` **AAA user to intranetip binding** `aaauser_intranetip_binding` **AAA user to TM session policy binding** `aaauser_tmsessionpolicy_binding` **AAA user to SSLVPN intranet application binding** `aaauser_vpnintranetapplication_binding` **AAA user to Secure Private Access Profile binding** `aaauser_vpnsecureprivateaccessprofile_binding` **AAA user to VPN session policy binding** `aaauser_vpnsessionpolicy_binding` **AAA user to VPN traffic policy binding** `aaauser_vpntrafficpolicy_binding` **AAA user to VPN URL binding** `aaauser_vpnurl_binding` **AAA user to VPN url policy binding** `aaauser_vpnurlpolicy_binding`

Use Case: Configure AAA parameters and pre-authentication

Description	Configure AAA global parameters, directory and OTP settings, SSO profiles, Kerberos constrained delegation accounts, sessions and pre-authentication checks.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Certificate parameter** `aaacertparams` **AAA global to pre authentication policy binding** `aaaglobal_aaapreauthenticationpolicy_binding` **AAA global to Negotiate action binding** `aaaglobal_authenticationnegotiateaction_binding` **Kerberos constrained delegation account** `aaakcdaccount` **LDAP parameter** `aaaldapparams` **AAA otpparameter** `aaaotpparameter` **AAA parameter** `aaaparameter` **Pre authentication action** `aaapreauthenticationaction` **Pre authentication parameter** `aaapreauthenticationparameter` **Pre authentication policy** `aaapreauthenticationpolicy` **AAA proxy parameter** `aaaproxyparam` **RADIUS parameter** `aaaradiusparams` **Active connection** `aaasession` **Aaa sso profile** `aaassoprofile` **Tacacs parameters** `aaatacacsparams` **RADIUS Node** `radiusnode`

Use Case: Control authorization, traffic management sessions and policy labels

Description	Define authorization policies and labels and traffic management session and traffic policies, actions and parameters.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Authorization policy** `authorizationpolicy` **Authorization policy label** `authorizationpolicylabel` **Authorization policy label to authorization policy binding** `authorizationpolicylabel_authorizationpolicy_binding` **Form sso action** `tmformssoaction` **TM global to ns log policy binding** `tmglobal_auditnslogpolicy_binding` **TM global to system log policy binding** `tmglobal_auditsyslogpolicy_binding` **TM global to TM session policy binding** `tmglobal_tmsessionpolicy_binding` **TM global to TM traffic policy binding** `tmglobal_tmtrafficpolicy_binding` **SAML sso action** `tmsamlssoprofile` **TM session action** `tmsessionaction` **Session parameter** `tmsessionparameter` **TM session policy** `tmsessionpolicy` **TM traffic action** `tmtrafficaction` **TM traffic policy** `tmtrafficpolicy`

Use Case: Configure Gateway virtual servers

Description	Create Gateway (VPN) virtual servers and bind authentication, session, traffic, portal theme, intranet and other policies to them.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**VPN virtual server** `vpnvserver` **VPN virtual server to pre authentication policy binding** `vpnvserver_aaapreauthenticationpolicy_binding` **VPN virtual server to Analytics profile binding** `vpnvserver_analyticsprofile_binding` **VPN virtual server to appcontroller binding** `vpnvserver_appcontroller_binding` **VPN virtual server to AppFlow policy binding** `vpnvserver_appflowpolicy_binding` **VPN virtual server to application firewall policy binding** `vpnvserver_appfwpolicy_binding` **VPN virtual server to ns log policy binding** `vpnvserver_auditnslogpolicy_binding` **VPN virtual server to system log policy binding** `vpnvserver_auditsyslogpolicy_binding` **VPN virtual server to CERT policy binding** `vpnvserver_authenticationcertpolicy_binding` **VPN virtual server to Dfa authentication policy binding** `vpnvserver_authenticationdfapolicy_binding` **VPN virtual server to LDAP policy binding** `vpnvserver_authenticationldappolicy_binding` **VPN virtual server to LOCAL policy binding** `vpnvserver_authenticationlocalpolicy_binding` **VPN virtual server to authenticationloginschemapolicy binding** `vpnvserver_authenticationloginschemapolicy_binding` **VPN virtual server to Negotiate Policy binding** `vpnvserver_authenticationnegotiatepolicy_binding` **VPN virtual server to AAA OAuth IdentityProvider (IdP) policy binding** `vpnvserver_authenticationoauthidppolicy_binding` **VPN virtual server to Authentication Policy binding** `vpnvserver_authenticationpolicy_binding` **VPN virtual server to RADIUS policy binding** `vpnvserver_authenticationradiuspolicy_binding` **VPN virtual server to AAA Saml IdentityProvider (IdP) policy binding** `vpnvserver_authenticationsamlidppolicy_binding` **VPN virtual server to AAA Saml policy binding** `vpnvserver_authenticationsamlpolicy_binding` **VPN virtual server to TACACS policy binding** `vpnvserver_authenticationtacacspolicy_binding` **VPN virtual server to Web authentication policy binding** `vpnvserver_authenticationwebauthpolicy_binding` **VPN virtual server to Integrated Cache policy binding** `vpnvserver_cachepolicy_binding` **VPN virtual server to content-switching policy binding** `vpnvserver_cspolicy_binding` **VPN virtual server to Front end optimization policy binding** `vpnvserver_feopolicy_binding` **VPN virtual server to ICA policy binding** `vpnvserver_icapolicy_binding` **VPN virtual server to intranetip6 binding** `vpnvserver_intranetip6_binding` **VPN virtual server to intranetip binding** `vpnvserver_intranetip_binding` **VPN virtual server to responder policy binding** `vpnvserver_responderpolicy_binding` **VPN virtual server to rewrite policy binding** `vpnvserver_rewritepolicy_binding` **VPN virtual server to secureprivateaccessurl binding** `vpnvserver_secureprivateaccessurl_binding` **VPN virtual server to sharefileserver binding** `vpnvserver_sharefileserver_binding` **VPN virtual server to staserver binding** `vpnvserver_staserver_binding` **VPN virtual server to Clientless VPN rewrite policy binding** `vpnvserver_vpnclientlessaccesspolicy_binding` **VPN virtual server to Epa profile binding** `vpnvserver_vpnepaprofile_binding` **VPN virtual server to EULA for vservers binding** `vpnvserver_vpneula_binding` **VPN virtual server to SSLVPN intranet application binding** `vpnvserver_vpnintranetapplication_binding` **VPN virtual server to Next Hop Server binding** `vpnvserver_vpnnexthopserver_binding` **VPN virtual server to portaltheme binding** `vpnvserver_vpnportaltheme_binding` **VPN virtual server to Secure Private Access Profile binding** `vpnvserver_vpnsecureprivateaccessprofile_binding` **VPN virtual server to VPN session policy binding** `vpnvserver_vpnsessionpolicy_binding` **VPN virtual server to VPN traffic policy binding** `vpnvserver_vpntrafficpolicy_binding` **VPN virtual server to VPN URL binding** `vpnvserver_vpnurl_binding` **VPN virtual server to VPN url policy binding** `vpnvserver_vpnurlpolicy_binding`

Use Case: Configure global Gateway bindings

Description	Bind authentication, session, traffic, clientless access and other policies and resources globally to the Gateway.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**VPN global to appcontroller binding** `vpnglobal_appcontroller_binding` **VPN global to application firewall policy binding** `vpnglobal_appfwpolicy_binding` **VPN global to ns log policy binding** `vpnglobal_auditnslogpolicy_binding` **VPN global to system log policy binding** `vpnglobal_auditsyslogpolicy_binding` **VPN global to CERT policy binding** `vpnglobal_authenticationcertpolicy_binding` **VPN global to LDAP policy binding** `vpnglobal_authenticationldappolicy_binding` **VPN global to LOCAL policy binding** `vpnglobal_authenticationlocalpolicy_binding` **VPN global to Negotiate Policy binding** `vpnglobal_authenticationnegotiatepolicy_binding` **VPN global to Authentication Policy binding** `vpnglobal_authenticationpolicy_binding` **VPN global to RADIUS policy binding** `vpnglobal_authenticationradiuspolicy_binding` **VPN global to AAA Saml policy binding** `vpnglobal_authenticationsamlpolicy_binding` **VPN global to TACACS policy binding** `vpnglobal_authenticationtacacspolicy_binding` **VPN global to domain binding** `vpnglobal_domain_binding` **VPN global to intranetip6 binding** `vpnglobal_intranetip6_binding` **VPN global to intranetip binding** `vpnglobal_intranetip_binding` **VPN global to secureprivateaccessurl binding** `vpnglobal_secureprivateaccessurl_binding` **VPN global to sharefileserver binding** `vpnglobal_sharefileserver_binding` **VPN global to certificate key binding** `vpnglobal_sslcertkey_binding` **VPN global to staserver binding** `vpnglobal_staserver_binding` **VPN global to Clientless VPN rewrite policy binding** `vpnglobal_vpnclientlessaccesspolicy_binding` **VPN global to EULA for vservers binding** `vpnglobal_vpneula_binding` **VPN global to SSLVPN intranet application binding** `vpnglobal_vpnintranetapplication_binding` **VPN global to Next Hop Server binding** `vpnglobal_vpnnexthopserver_binding` **VPN global to portaltheme binding** `vpnglobal_vpnportaltheme_binding` **VPN global to Secure Private Access Profile binding** `vpnglobal_vpnsecureprivateaccessprofile_binding` **VPN global to VPN session policy binding** `vpnglobal_vpnsessionpolicy_binding` **VPN global to VPN traffic policy binding** `vpnglobal_vpntrafficpolicy_binding` **VPN global to VPN URL binding** `vpnglobal_vpnurl_binding` **VPN global to VPN url policy binding** `vpnglobal_vpnurlpolicy_binding`

Use Case: Define Gateway policies, actions, profiles and ICA access

Description	Define Gateway session, traffic, URL and clientless access policies and actions, portal themes, EPA and SAML SSO profiles, ICA access settings and RDP and PCoIP profiles.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**Ica accessprofile** `icaaccessprofile` **Ica action** `icaaction` **ICA global to ICA policy binding** `icaglobal_icapolicy_binding` **Profile for Latency monitoring** `icalatencyprofile` **Config Parameters for NS ICA** `icaparameter` **ICA policy** `icapolicy` **RDP clientprofile** `rdpclientprofile` **Active rdp connections** `rdpconnections` **RDP serverprofile** `rdpserverprofile` **AlwyasON profile** `vpnalwaysonprofile` **Clientless VPN rewrite policy** `vpnclientlessaccesspolicy` **Clientless VPN rewrite profile** `vpnclientlessaccessprofile` **Epa profile** `vpnepaprofile` **EULA for vservers** `vpneula` **Form sso action** `vpnformssoaction` **Active ica connections** `vpnicaconnection` **SSLVPN intranet application** `vpnintranetapplication` **Next Hop Server** `vpnnexthopserver` **VPN parameter** `vpnparameter` **PCoIP connection** `vpnpcoipconnection` **PCoIP session profile** `vpnpcoipprofile` **PCoIP vserver profile** `vpnpcoipvserverprofile` **Portaltheme** `vpnportaltheme` **SAML sso action** `vpnsamlssoprofile` **Secure Private Access Profile** `vpnsecureprivateaccessprofile` **VPN session action** `vpnsessionaction` **VPN session policy** `vpnsessionpolicy` **VPN traffic action** `vpntrafficaction` **VPN traffic policy** `vpntrafficpolicy` **VPN URL** `vpnurl` **VPN url action** `vpnurlaction` **VPN url policy** `vpnurlpolicy`

Use Case: Manage SSL certificates, keys and CRLs

Description	Create, import and link certificates, keys, CA bundles, CRLs, OCSP responders, DH parameters and FIPS or HSM keys.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**CA certbundle** `sslcacertbundle` **Group of CA certificate-key pairs** `sslcacertgroup` **Group of CA certificate-key pairs to certificate key binding** `sslcacertgroup_sslcertkey_binding` **Cerificate** `sslcert` **Imported Certbundle** `sslcertbundle` **Imported Certfile** `sslcertfile` **Linked certificate** `sslcertificatechain` **Certificate key** `sslcertkey` **Certificate key to OCSP responser binding** `sslcertkey_sslocspresponder_binding` **Certkey bundle** `sslcertkeybundle` **Certificate request** `sslcertreq` **Certificate Revocation List** `sslcrl` **Imported crl files** `sslcrlfile` **Dh imported file** `ssldhfile` **Dh Parameter** `ssldhparam` **Dynamic Client Certificate Cache** `ssldynamicclientcertcache` **Ecdsa key** `sslecdsakey` **Fips** `sslfips` **FIPS key** `sslfipskey` **FIPsSIM source** `sslfipssimsource` **FIPS SIM Target** `sslfipssimtarget` **HPKE key** `sslhpkekey` **HSM key** `sslhsmkey` **Imported ssl key files** `sslkeyfile` **OCSP responser** `sslocspresponder` **Pkcs12** `sslpkcs12` **Pkcs8** `sslpkcs8` **RSA key** `sslrsakey` **WRAP key** `sslwrapkey`

Use Case: Configure SSL on virtual servers, services and profiles

Description	Configure SSL settings, ciphers, curves and certificate bindings on virtual servers, services and service groups, and define SSL profiles, policies, actions and global parameters.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**SSL action** `sslaction` **Cipher** `sslcipher` **Cipher to sslciphersuite binding** `sslcipher_sslciphersuite_binding` **SSL default profile** `ssldefaultprofile` **DTLS profile** `ssldtlsprofile` **Ech config** `sslechconfig` **SSL global to SSL policy binding** `sslglobal_sslpolicy_binding` **SSL logging Profile** `ssllogprofile` **SSL parameter** `sslparameter` **SSL policy** `sslpolicy` **SSL policy label** `sslpolicylabel` **SSL policy label to SSL policy binding** `sslpolicylabel_sslpolicy_binding` **SSL profile** `sslprofile` **SSL profile to ecccurve binding** `sslprofile_ecccurve_binding` **SSL profile to certificate key binding** `sslprofile_sslcertkey_binding` **SSL profile to cipher binding** `sslprofile_sslcipher_binding` **SSL profile to sslciphersuite binding** `sslprofile_sslciphersuite_binding` **SSL profile to ech config binding** `sslprofile_sslechconfig_binding` **SSL service** `sslservice` **SSL service to ecccurve binding** `sslservice_ecccurve_binding` **SSL service to CA certbundle binding** `sslservice_sslcacertbundle_binding` **SSL service to certificate key binding** `sslservice_sslcertkey_binding` **SSL service to cipher binding** `sslservice_sslcipher_binding` **SSL service to sslciphersuite binding** `sslservice_sslciphersuite_binding` **SSL service to SSL policy binding** `sslservice_sslpolicy_binding` **SSL service group** `sslservicegroup` **SSL service group to ecccurve binding** `sslservicegroup_ecccurve_binding` **SSL service group to CA certbundle binding** `sslservicegroup_sslcacertbundle_binding` **SSL service group to certificate key binding** `sslservicegroup_sslcertkey_binding` **SSL service group to cipher binding** `sslservicegroup_sslcipher_binding` **SSL service group to sslciphersuite binding** `sslservicegroup_sslciphersuite_binding` **SSL virtual server** `sslvserver` **SSL virtual server to ecccurve binding** `sslvserver_ecccurve_binding` **SSL virtual server to CA certbundle binding** `sslvserver_sslcacertbundle_binding` **SSL virtual server to certificate key binding** `sslvserver_sslcertkey_binding` **SSL virtual server to certkey bundle binding** `sslvserver_sslcertkeybundle_binding` **SSL virtual server to cipher binding** `sslvserver_sslcipher_binding` **SSL virtual server to sslciphersuite binding** `sslvserver_sslciphersuite_binding` **SSL virtual server to SSL policy binding** `sslvserver_sslpolicy_binding` **Zerotouch params** `sslzerotouchparam`

Use Case: Configure DNS records, zones and policies

Description	Manage DNS records, zones, views, name servers and keys, and DNS policies, actions, profiles and parameters.

Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**IPv6 address type record** `dnsaaaarec` **DNS action** `dnsaction` **Dns64 action** `dnsaction64` **Address type record** `dnsaddrec` **CAA record** `dnscaarec` **CNAME record** `dnscnamerec` **DNS global to DNS policy binding** `dnsglobal_dnspolicy_binding` **Dns key** `dnskey` **MX record** `dnsmxrec` **Name server** `dnsnameserver` **NAPTR record** `dnsnaptrrec` **Name server record** `dnsnsrec` **DNS parameter** `dnsparameter` **DNS policy** `dnspolicy` **Dns64 policy** `dnspolicy64` **Dns policy label** `dnspolicylabel` **Dns policy label to DNS policy binding** `dnspolicylabel_dnspolicy_binding` **DNS profile** `dnsprofile` **Proxy record** `dnsproxyrecords` **PTR record** `dnsptrrec` **SOA record** `dnssoarec` **Server record** `dnssrvrec` **Subnet cache** `dnssubnetcache` **DNS suffix** `dnssuffix` **SVCB/HTTPS service binding record** `dnssvcbrec` **TXT record** `dnstxtrec` **DNS view** `dnsview` **DNS zone** `dnszone`

Use Case: Configure interfaces, VLANs and network segmentation

Description	Configure interfaces, link aggregation, VLANs, VXLAN, bridge groups, LLDP, ARP, IPv6 neighbor discovery, traffic domains and admin partitions.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Arp** `arp` **Global arp parameters** `arpparam` **Bridge group** `bridgegroup` **Bridge group to ip6 binding** `bridgegroup_nsip6_binding` **Bridge group to nsip binding** `bridgegroup_nsip_binding` **Bridge group to "VLAN" binding** `bridgegroup_vlan_binding` **Bridge table entry** `bridgetable` **Channel** `channel` **Channel to interface binding** `channel_interface_binding` **"FIS"** `fis` **"FIS" to channel binding** `fis_channel_binding` **"FIS" to interface binding** `fis_interface_binding` **Interface** `interface` **"Interface Pair"** `interfacepair` **Ip v6** `ipv6` **Layer 2 related parameter** `l2param` **Layer 3 related parameter** `l3param` **Layer 4 related parameter** `l4param` **Link aggregation control protocol** `lacp` **Link set** `linkset` **Link set to channel binding** `linkset_channel_binding` **Link set to interface binding** `linkset_interface_binding` **Lldp neighbors** `lldpneighbors` **Lldp params** `lldpparam` **Nd6** `nd6` **Nd6 Router Advertisment configuration variables** `nd6ravariables` **Nd6 Router Advertisment configuration variables to on-link IPv6 global prefixes for Router Advertisment binding** `nd6ravariables_onlinkipv6prefix_binding` **Network bridge** `netbridge` **Network bridge to ip Tunnel binding** `netbridge_iptunnel_binding` **Network bridge to ip6 binding** `netbridge_nsip6_binding` **Network bridge to nsip binding** `netbridge_nsip_binding` **Network bridge to "VLAN" binding** `netbridge_vlan_binding` **Admin partition** `nspartition` **Admin partition to bridge group binding** `nspartition_bridgegroup_binding` **Admin partition to "VLAN" binding** `nspartition_vlan_binding` **Admin partition to "VXLAN" binding** `nspartition_vxlan_binding` **Traffic Domain** `nstrafficdomain` **Traffic Domain to bridge group binding** `nstrafficdomain_bridgegroup_binding` **Traffic Domain to "VLAN" binding** `nstrafficdomain_vlan_binding` **Traffic Domain to "VXLAN" binding** `nstrafficdomain_vxlan_binding` **On-link IPv6 global prefixes for Router Advertisment** `onlinkipv6prefix` **Precision Time Protocol** `ptp` **RSS key type** `rsskeytype` **"VLAN"** `vlan` **"VLAN" to channel binding** `vlan_channel_binding` **"VLAN" to interface binding** `vlan_interface_binding` **"VLAN" to link set binding** `vlan_linkset_binding` **"VLAN" to ip6 binding** `vlan_nsip6_binding` **"VLAN" to nsip binding** `vlan_nsip_binding` **"VXLAN"** `vxlan` **"VXLAN" to ip6 binding** `vxlan_nsip6_binding` **"VXLAN" to nsip binding** `vxlan_nsip_binding` **"VXLAN" to srcip binding** `vxlan_srcip_binding` **Vxlan vlan mapping** `vxlanvlanmap` **Vxlan vlan mapping to "VXLAN" binding** `vxlanvlanmap_vxlan_binding`

Use Case: Configure IP addresses, tunnels and routing

Description	Configure NetScaler IPs, IP sets, tunnels, static and policy-based routes, route maps, OSPF, BGP and BFD.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Bfdinterface** `bfdinterface` **BGP Router** `bgprouter` **Ip6 Tunnel** `ip6tunnel` **Ip6 tunnel parameter** `ip6tunnelparam` **Iproute** `iproute` **IPSEC paramter** `ipsecparameter` **IPSEC profile** `ipsecprofile` **Network ipset** `ipset` **Network ipset to ip6 binding** `ipset_nsip6_binding` **Network ipset to nsip binding** `ipset_nsip_binding` **Ip Tunnel** `iptunnel` **Ip tunnel parameter** `iptunnelparam` **Dhcp IP** `nsdhcpip` **Nsip** `nsip` **Ip6** `nsip6` **Policy Based Routing(PBR) entry** `nspbr` **PBR6 entry** `nspbr6` **Policy based routing** `nspbrs` **Source IP Mac Cache Table.** `nssourceroutecachetable` **Ospf6interface** `ospf6interface` **Ospf6router** `ospf6router` **OSPF Interface** `ospfinterface` **OSPF Router** `ospfrouter` **Route** `route` **Route 6** `route6` **Routemap** `routemap` **Dynamic routing config** `routerdynamicrouting`, `routerdynamicrouting_info`

Use Case: Configure NAT, carrier-grade NAT and subscriber management

Description	Configure RNAT, INAT, NAT64, network profiles, large scale NAT, MAP-E and PCP, and subscriber profiles and session interfaces.

Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Session forward** `forwardingsession` **Inbound nat** `inat` **INAT parameter** `inatparam` **IPSEC ALG profile** `ipsecalgprofile` **IPSEC ALG session** `ipsecalgsession` **LSN Application Attributes** `lsnappsattributes` **LSN Application Profile** `lsnappsprofile` **LSN Application Profile to LSN Application Attributes binding** `lsnappsprofile_lsnappsattributes_binding` **LSN Application Profile to port binding** `lsnappsprofile_port_binding` **Lsn client** `lsnclient` **Lsn client to network6 binding** `lsnclient_network6_binding` **Lsn client to network binding** `lsnclient_network_binding` **Lsn client to ACL6 entry binding** `lsnclient_nsacl6_binding` **Lsn client to ACL entry binding** `lsnclient_nsacl_binding` **LSN group** `lsngroup` **LSN group to IPSEC ALG profile binding** `lsngroup_ipsecalgprofile_binding` **LSN group to LSN Application Profile binding** `lsngroup_lsnappsprofile_binding` **LSN group to LSN HTTP header logging Profile binding** `lsngroup_lsnhttphdrlogprofile_binding` **LSN group to LSN logging Profile binding** `lsngroup_lsnlogprofile_binding` **LSN group to LSN pool binding** `lsngroup_lsnpool_binding` **LSN group to LSN RTSPALG Profile binding** `lsngroup_lsnrtspalgprofile_binding` **LSN group to LSN SIPALG Profile binding** `lsngroup_lsnsipalgprofile_binding` **LSN group to LSN Transport Profile binding** `lsngroup_lsntransportprofile_binding` **LSN group to server binding** `lsngroup_pcpserver_binding` **LSN HTTP header logging Profile** `lsnhttphdrlogprofile` **LSN ip6 Profile** `lsnip6profile` **LSN logging Profile** `lsnlogprofile` **LSN parameter** `lsnparameter` **LSN pool** `lsnpool` **LSN pool to lsnip binding** `lsnpool_lsnip_binding` **LSN RTSPALG Profile** `lsnrtspalgprofile` **LSN RTSPALG session** `lsnrtspalgsession` **Lsn session** `lsnsession` **LSN SIPALG call** `lsnsipalgcall` **LSN SIPALG Profile** `lsnsipalgprofile` **Static mapping** `lsnstatic` **LSN Transport Profile** `lsntransportprofile` **MAP-T Basic Mapping rule** `mapbmr` **MAP-T Basic Mapping rule to bmrv4network binding** `mapbmr_bmrv4network_binding` **MAP-T Default Mapping rule** `mapdmr` **MAP-T Map Domain** `mapdomain` **MAP-T Map Domain to MAP-T Basic Mapping rule binding** `mapdomain_mapbmr_binding` **Nat64 config** `nat64` **NAT64 parameter** `nat64param` **Network profile** `netprofile` **Network profile to natrule binding** `netprofile_natrule_binding` **Network profile to srcportset binding** `netprofile_srcportset_binding` **PCP Profile** `pcpprofile` **Server** `pcpserver` **RNAT configured route** `rnat` **IPv6 RNAT configured route** `rnat6` **IPv6 RNAT configured route to ip6 binding** `rnat6_nsip6_binding` **RNAT configured route to nsip binding** `rnat_nsip_binding` **RNAT configured route to retainsourceportset binding** `rnat_retainsourceportset_binding` **RNAT global to system log policy binding** `rnatglobal_auditsyslogpolicy_binding` **RNAT parameter** `rnatparam` **RNAT session** `rnatsession` **Gx interface Parameters** `subscribergxinterface` **Subscriber Params** `subscriberparam` **Subscriber Profile** `subscriberprofile` **RADIUS interface Parameters** `subscriberradiusinterface` **Subscriber sesions** `subscribersessions`

Use Case: Configure high availability and clustering

Description	Configure HA node pairs, failover and synchronisation, VRRP virtual router IDs, and cluster nodes and node groups.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Cluster** `cluster` **Files** `clusterfiles` **Cluster instance** `clusterinstance` **Cluster node** `clusternode` **Cluster node to routemonitor binding** `clusternode_routemonitor_binding` **Node group object type** `clusternodegroup` **Node group object type to authentication virtual server binding** `clusternodegroup_authenticationvserver_binding` **Node group object type to cluster node binding** `clusternodegroup_clusternode_binding` **Node group object type to CR virtual server binding** `clusternodegroup_crvserver_binding` **Node group object type to CS virtual server binding** `clusternodegroup_csvserver_binding` **Node group object type to GSLB site binding** `clusternodegroup_gslbsite_binding` **Node group object type to Global Server Load Balancing Virtual Server binding** `clusternodegroup_gslbvserver_binding` **Node group object type to Load Balancing Virtual Server binding** `clusternodegroup_lbvserver_binding` **Node group object type to limit Identifier binding** `clusternodegroup_nslimitidentifier_binding` **Node group object type to service binding** `clusternodegroup_service_binding` **Node group object type to identifier binding** `clusternodegroup_streamidentifier_binding` **Node group object type to VPN virtual server binding** `clusternodegroup_vpnvserver_binding` **Clusterpropstatus** `clusterpropstatus` **Sync** `clustersync` **Failover** `hafailover` **Files** `hafiles` **Node** `hanode` **Node to routemonitor6 binding** `hanode_routemonitor6_binding` **Node to routemonitor binding** `hanode_routemonitor_binding` **HA Secure Heartbeats parameter** `hasecureheartbeats` **Sync** `hasync` **Virtual Router ID** `vrid` **Virtual Router ID for IPv6** `vrid6` **Virtual Router ID for IPv6 to channel binding** `vrid6_channel_binding` **Virtual Router ID for IPv6 to interface binding** `vrid6_interface_binding` **Virtual Router ID for IPv6 to trackinterface binding** `vrid6_trackinterface_binding` **Virtual Router ID to channel binding** `vrid_channel_binding` **Virtual Router ID to interface binding** `vrid_interface_binding` **Virtual Router ID to trackinterface binding** `vrid_trackinterface_binding` **VR ID parameter** `vridparam`

Use Case: Control traffic with ACLs, rate limiting and surge protection

Description	Define extended and simple ACLs, access lists, limit identifiers and selectors, rate control and surge queue settings.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**Accesslist** `accesslist` **ACL entry** `nsacl` **ACL6 entry** `nsacl6` **ACL entry** `nsacls` **ACL6 entry** `nsacls6` **Limit Identifier** `nslimitidentifier` **Limit selector** `nslimitselector` **Limit sessions** `nslimitsessions` **Rate control** `nsratecontrol` **Simple ACL** `nssimpleacl` **Simple ACL6** `nssimpleacl6` **Surge queue** `nssurgeq`

Use Case: Configure audit logging and AppFlow export

Description	Configure audit message, syslog and nslog actions and policies, AppFlow collectors, actions and policies, analytics profiles and stream identifiers.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**ANALYTICS global to Analytics profile binding** `analyticsglobal_analyticsprofile_binding` **Analytics profile** `analyticsprofile` **AppFlow action** `appflowaction` **AppFlow action to Analytics profile binding** `appflowaction_analyticsprofile_binding` **AppFlow collector** `appflowcollector` **APPFLOW global to AppFlow policy binding** `appflowglobal_appflowpolicy_binding` **AppFlow parameter** `appflowparam` **AppFlow policy** `appflowpolicy` **AppFlow policy label** `appflowpolicylabel` **AppFlow policy label to AppFlow policy binding** `appflowpolicylabel_appflowpolicy_binding` **Message action** `auditmessageaction` **Ns log action** `auditnslogaction` **AUDITNSLOG global to ns log policy binding** `auditnslogglobal_auditnslogpolicy_binding` **Ns log parameters** `auditnslogparams` **Ns log policy** `auditnslogpolicy` **System log action** `auditsyslogaction` **AUDITSYSLOG global to system log policy binding** `auditsysloglobal_auditsyslogpolicy_binding` **System log parameters** `auditsyslogparams` **System log policy** `auditsyslogpolicy` **Callhome** `callhome` **AppflowCollector** `nsappflowcollector` **AppflowParam** `nsappflowparam` **Web log parameters** `nsweblogparam` **Identifier** `streamidentifier` **Identifier to Analytics profile binding** `streamidentifier_analyticsprofile_binding` **Selector** `streamselector` **Active connection** `streamsession` **Ulfd server** `ulfdserver`

Use Case: Configure SNMP and metrics monitoring

Description	Configure SNMP managers, communities, users, groups, views, alarms and traps, and metrics profiles for monitoring.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Metrics profile** `metricsprofile` **Metrics profile to authentication virtual server binding** `metricsprofile_authenticationvserver_binding` **Metrics profile to CR virtual server binding** `metricsprofile_crvserver_binding` **Metrics profile to CS virtual server binding** `metricsprofile_csvserver_binding` **Metrics profile to Global Server Load Balancing Virtual Server binding** `metricsprofile_gslbvserver_binding` **Metrics profile to Load Balancing Virtual Server binding** `metricsprofile_lbvserver_binding` **Metrics profile to service binding** `metricsprofile_service_binding` **Metrics profile to service group binding** `metricsprofile_servicegroup_binding` **Metrics profile to virtual server binding** `metricsprofile_uservserver_binding` **Metrics profile to VPN virtual server binding** `metricsprofile_vpnvserver_binding` **Stats** `nsstats` **Alarm** `snmpalarm` **Community** `snmpcommunity` **SNMP engine id** `snmpengineid` **SNMP group** `snmpgroup` **Manager** `snmpmanager` **SNMP mib** `snmpmib` **SNMP option** `snmpoption` **Snmp trap** `snmptrap` **Snmp trap to SNMP user binding** `snmptrap_snmpuser_binding` **SNMP user** `snmpuser` **View** `snmpview`

Use Case: Manage administrators, command policies and sessions

Description	Manage system users, groups, command policies, partition access, SSH keys, sessions and login or logout of an appliance.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**Change_password** `change_password` **Login to a NetScaler ADC node** `login` **Logout from a NetScaler ADC node** `logout` **Encryption key** `nsencryptionkey` **Default encryption parameters** `nsencryptionparams` **HMAC key** `nshmackey` **Systemadmuserinfo** `systemadmuserinfo` **Command policy** `systemcmdpolicy` **SYSTEM global to ns log policy binding** `systemglobal_auditnslogpolicy_binding` **SYSTEM global to system log policy binding** `systemglobal_auditsyslogpolicy_binding` **SYSTEM global to LDAP policy binding** `systemglobal_authenticationldappolicy_binding` **SYSTEM global to LOCAL policy binding** `systemglobal_authenticationlocalpolicy_binding` **SYSTEM global to Authentication Policy binding** `systemglobal_authenticationpolicy_binding` **SYSTEM global to RADIUS policy binding** `systemglobal_authenticationradiuspolicy_binding` **SYSTEM global to TACACS policy binding** `systemglobal_authenticationtacacspolicy_binding` **System group** `systemgroup` **System group to admin partition binding** `systemgroup_nspartition_binding` **System group to command policy binding** `systemgroup_systemcmdpolicy_binding` **System group to system user binding** `systemgroup_systemuser_binding` **System session** `systemsession` **Systemsshkey** `systemsshkey` **System user** `systemuser` **System user to admin partition binding** `systemuser_nspartition_binding` **System user to command policy binding** `systemuser_systemcmdpolicy_binding`

Use Case: Back up, configure and operate the appliance

Description	Save and back up configuration, manage restore points and files, install software, reboot or shut down, run ping and traceroute and manage the SSH connection.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Information** `endpointinfo` **Install** `install` **System config** `nsconfig` **Nsconfigview** `nsconfigview` **Migration operation** `nsmigration` **Rpc node** `nsrpcnode` **Nstrace operations** `nstrace` **Ping** `ping` **Ping6** `ping6` **Reboot** `reboot` **Login to a NetScaler ADC node** `save_config` **Shutdown** `shutdown` **Enable/Disable the autorestore feature use to create restorepoint** `systemautorestorefeature` **System Autosaveparam** `systemautosaveparam` **Backup Data for ns backup and restore** `systembackup` **File** `systemfile` **Systemnsbtracing** `systemnsbtracing` **Setting restorepoints for auto restore** `systemrestorepoint` **Traceroute** `traceroute` **Traceroute6** `traceroute6` **SSH connection to NetScaler ADC** connection `ssh_netscaler_adc`

Use Case: Tune appliance, TCP, HTTP and protocol settings

Description	Tune system, NetScaler mode and feature settings, TCP and HTTP profiles and parameters, timers, NTP, DHCP, DPS, QUIC and other protocol profiles.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**AppAlg Param** `appalgparam` **Dps parameter** `dpsparameter` **Parameter for extended memory used by LSN and Subscriber Store** `extendedmemoryparam` **File System Encryption Information** `filesystemencryption` **McpProfile** `mcpprofile` **Nschannelparam** `nschannelparam` **Console prompt** `nsconsoleloginprompt` **Cqaparam** `nscqaparam` **DHCP parameters** `nsdhcpparams` **Diameter Parameters** `nsdiameter` **Feature** `nsfeature` **Host name** `nshostname` **HTTP parameter** `nshttpparam` **HTTP profile** `nshttpprofile` **Keymanagerproxy** `nskeymanagerproxy` **Management related configuration** `nsmgmtparam` **Ns mode** `nsmode` **Nsnextgenapi** `nsnextgenapi` **Citrix ADC parameters** `nsparam` **Surge Protection parameter** `nsspparams` **Tcp buffer parameter** `nstcpbufparam` **Tcp parameters** `nstcpparam` **TCP profile** `nstcpprofile` **Timeout** `nstimeout` **Timer** `nstimer` **"VPX"** `nsvpxparam` **NTP parameter** `ntpparam` **NTP server** `ntpserver` **NTP sync** `ntpsync` **QUIC BRIDGE profile** `quicbridgeprofile` **Citrix ADC QUIC parameters** `quicparam` **QUIC profile** `quicprofile` **SMPP configuration parameters** `smppparam` **SMPP user** `smppuser` **Systemcpuparam** `systemcpuparam` **Systemextramgmtcpu** `systemextramgmtcpu` **Hardware errors** `systemhwerror` **Key encryption Key** `systemkek` **System parameter** `systemparameter` **Scalable Management Threads feature** `systemscalablemgmtthreads` **System signedExeReport** `systemsignedexereport` **User protocol** `userprotocol`

Use Case: Manage licensing and central management

Description	Manage license servers, proxies and offline LAS tokens, capacity, and Application Delivery Management (ADM) and central management settings.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)

Key content	**ADM parameter** `admparameter` **Aptlicense** `nsaptlicense` **Capacity** `nscapacity` **Centralmanagementserver** `nscentralmanagementserver` **Laslicense** `nslaslicense` **Nslaslicense_offline** `nslaslicense_offline` **Licenseparameters** `nslicenseparameters` **Licenseproxyserver** `nslicenseproxyserver` **Licenseserver** `nslicenseserver` **Testlicense** `nstestlicense`

Use Case: Integrate with public cloud and autoscaling

Description	Configure cloud profiles and parameters, cloud routes and tunnels, Azure and AWS integration, Azure Key Vault and autoscale profiles and policies.
Collection	`netscaler.adc` (Certified, NetScaler, v2.17.0, updated 2026-07-15)
Key content	**Azure Key Vault entity** `authenticationazurekeyvault` **Autoscale action** `autoscaleaction` **Autoscale policy** `autoscalepolicy` **Autoscale profile** `autoscaleprofile` **Azure Application** `azureapplication` **Azure Key Vault entity** `azurekeyvault` **Allowed ticket profile for NGS** `cloudallowedngsticketprofile` **Cloud awsParam** `cloudawsparam` **Cloud credentials** `cloudcredential` **Cloud api to push static routes to cloud** `cloudgcpstaticroutes` **Cloud ngsparameter** `cloudngsparameter` **Cloud parameter** `cloudparameter` **Cloud paramInternal** `cloudparaminternal` **Cloud profile** `cloudprofile` **Cloud routes** `cloudroutes` **Cloud service** `cloudservice` **Cloud trafficroutes** `cloudtrafficroutes` **Cloudtunnel parameter** `cloudtunnelparameter` **Cloud Tunnel virtual server** `cloudtunnelvserver` **Timer to Autoscale policy binding** `nstimer_autoscalepolicy_binding`

Network access control lists `network.acls`

Validated Red Hat · version 4.0.0 · updated 2025-01-17 · 0 modules → 1 use cases

Use Case: Manage ACL configuration and detect drift

Description	Gather and persist ACL facts, detect drift from the desired state, deploy changes and remediate drift across network platforms, and run health checks.
Collection	`network.acls` (Validated, Red Hat, v4.0.0, updated 2025-01-17)
Key content	**Gather facts** role `gather` **Persist facts as host variables** role `persist` **Detect configuration drift** role `detect` **Remediate drift** role `remediate` **Deploy configuration** role `deploy` **Health checks** role `health_checks` **Health check view filter** filter `health_check_view`

Network device backup `network.backup`

Validated Red Hat · version 5.1.0 · updated 2026-01-16 · 0 modules → 1 use cases

Use Case: Back up and restore network configurations

Description	Back up network configurations to a datastore and restore them across platforms such as IOS, IOS-XR, NX-OS, EOS and Junos.
Collection	`network.backup` (Validated, Red Hat, v5.1.0, updated 2026-01-16)
Key content	**Backup** role `backup` **Restore** role `restore`

Network device management `network.base`

Validated Red Hat · version 8.0.0 · updated 2025-09-03 · 0 modules → 2 use cases

Use Case: Gather and persist network facts

Description	Collect structured facts for network resources, list supported resource modules and save facts to YAML.
Collection	`network.base` (Validated, Red Hat, v8.0.0, updated 2025-09-03)

Key content	**Fact gathering** role `gather` **Supported resource modules** role `list` **Fact persistence** role `persist` **Resource manager** role `resource_manager`

Use Case: Configure, detect drift and remediate

Description	Apply configuration from host_vars, deploy changes, detect configuration drift and remediate it.
Collection	`network.base` (Validated, Red Hat, v8.0.0, updated 2025-09-03)
Key content	**Configuration** role `configure` **Deployment** role `deploy` **Drift detection** role `detect` **Drift remediation** role `remediate`

BGP `network.bgp`

Validated Red Hat · version 8.0.0 · updated 2025-09-03 · 0 modules → 1 use cases

Use Case: Manage BGP configuration and detect drift

Description	Gather and persist BGP facts, detect drift from the desired state, deploy changes and remediate drift across network platforms, and run health checks.
Collection	`network.bgp` (Validated, Red Hat, v8.0.0, updated 2025-09-03)
Key content	**Gather facts** role `gather` **Persist facts as host variables** role `persist` **Detect configuration drift** role `detect` **Remediate drift** role `remediate` **Deploy configuration** role `deploy` **Health checks** role `health_checks` **Health check view filter** filter `health_check_view`

Network device health checks `network.healthchecks`

Validated Red Hat · version 2.0.0 · updated 2025-08-26 · 0 modules → 3 use cases

Use Case: Check device resources and hardware health

Description	Monitor CPU, memory, filesystem usage, uptime, environmental conditions and crash files on network devices.
Collection	`network.healthchecks` (Validated, Red Hat, v2.0.0, updated 2025-08-26)
Key content	**CPU usage** role `cpu` **Memory usage** role `memory` **Filesystem usage** role `filesystem` **System uptime** role `uptime` **Environmental conditions** role `environment` **Crash files** role `crashfiles`

Use Case: Check interfaces, routing and reachability

Description	Check interface health, BGP neighbor sessions, OSPF resources and IP reachability by ping.
Collection	`network.healthchecks` (Validated, Red Hat, v2.0.0, updated 2025-08-26)
Key content	**Interfaces** role `interfaces` **BGP neighbors** role `bgp` **OSPF** role `ospf` **IP reachability** role `ip_reachability`

Use Case: Filter health check results

Description	Filter health check data by target for reporting.
Collection	`network.healthchecks` (Validated, Red Hat, v2.0.0, updated 2025-08-26)

Key content	**Health check view** filter `health_check_view` **Filesystem health check view** filter `filesystem_health_check_view` **Interfaces health check view** filter `interfaces_health_check_view` **OSPF health check view** filter `ospf_health_check_view` **Main filter** filter `main`

Network interface management `network.interfaces`

Validated Red Hat · version 7.0.0 · updated 2025-09-03 · 0 modules → 2 use cases

Use Case: Gather and persist interface facts

Description	Collect structured interface facts, list supported resource modules and save facts to YAML.
Collection	`network.interfaces` (Validated, Red Hat, v7.0.0, updated 2025-09-03)
Key content	**Fact gathering** role `gather` **Supported resource modules** role `list` **Fact persistence** role `persist`

Use Case: Deploy, detect drift and remediate

Description	Deploy interface changes, detect configuration drift, remediate it and run interface health checks.
Collection	`network.interfaces` (Validated, Red Hat, v7.0.0, updated 2025-09-03)
Key content	**Deployment** role `deploy` **Drift detection** role `detect` **Drift remediation** role `remediate` **Health checks** role `health_checks` **Health check view** filter `health_check_view`

Network device configuration and reporting `network.inventory_reports`

Validated Red Hat · version 2.0.0 · updated 2025-08-26 · 0 modules → 2 use cases

Use Case: Collect network facts and build reports

Description	Gather device facts, list supported resource modules, persist facts to YAML and generate HTML reports or CSV-ready data.
Collection	`network.inventory_reports` (Validated, Red Hat, v2.0.0, updated 2025-08-26)
Key content	**Fact gathering** role `gather` **Supported resource modules** role `list` **Fact persistence** role `persist` **HTML reports** role `generate_report` **CSV export data** filter `flatten_dict`

Use Case: Apply configuration and fix drift

Description	Apply configuration from host_vars, detect configuration drift and remediate it.
Collection	`network.inventory_reports` (Validated, Red Hat, v2.0.0, updated 2025-08-26)
Key content	**Configuration** role `configure` **Deployment from data store** role `deploy` **Drift detection** role `detect` **Drift remediation** role `remediate`

Red Hat OpenShift (SDN to OVN-Kubernetes migration) `network.offline_migration_sdn_to_ovnk`

Validated Red Hat · version 1.0.3 · updated 2025-07-10 · 25 modules → 4 use cases

Use Case: Check migration prerequisites

Description	Verify cluster access, client and version, node and operator health, CIDR ranges, network provider and policy mode before migrating.
Collection	`network.offline_migration_sdn_to_ovnk` (Validated, Red Hat, v1.0.3, updated 2025-07-10)

Key content	**Kubeconfig** `check_kubeconfig` **oc client** `check_oc_client` **Cluster-admin rights** `check_whoami` **OpenShift version** `get_ocp_version` **CIDR range conflicts** `check_cidr_ranges` **Node readiness** `check_nodes_ready` **NetworkPolicy isolation mode** `check_network_policy_mode` **CNI network provider** `check_network_provider` **Migration state** `check_network_migration` **Cluster operator health** `verify_cluster_operators_health` **Pre-checks role** role `prechecks` **Prechecks** playbook `prechecks-playbook` **Pre-migration checks** playbook `playbook-migration` **Pre-rollback checks** playbook `playbook-rollback`

Use Case: Migrate the cluster network from SDN to OVN-Kubernetes

Description	Switch the default network type, configure MTU, ports and subnets, and run the offline migration end to end.
Collection	`network.offline_migration_sdn_to_ovnk` (Validated, Red Hat, v1.0.3, updated 2025-07-10)
Key content	**Network type change** `change_network_type` **Network type trigger** `trigger_network_type` **Network settings** `configure_network_settings` **Automatic migration of egress IP, firewall and multicast** `disable_automatic_migration` **Primary NNCP** `delete_primary_nncp` **Migration field cleanup** `clean_migration_field` **Old network configuration cleanup** `manage_network_config` **Migration role** role `migration` **Migration** playbook `migration-playbook`

Use Case: Control machine config updates and reboots

Description	Pause and resume machine config pools, wait for MCO and the network operator, and reboot nodes.
Collection	`network.offline_migration_sdn_to_ovnk` (Validated, Red Hat, v1.0.3, updated 2025-07-10)
Key content	**Machine config pool pause** `patch_mcp_paused` **Machine config pool resume** `resume_mcp` **MCO update start** `wait_for_mco` **MCO completion** `wait_for_mco_completion` **Network operator progress** `wait_for_network_co` **Multus restart** `wait_multus_restart` **Machine config status** `verify_machine_config` **Node reboot** `reboot_nodes` **Node reboot role** role `reboot_nodes` **Node reboot** playbook `reboot-playbook`

Use Case: Validate after migration and roll back

Description	Run post-migration and post-rollback checks and roll the cluster back to the previous network provider.
Collection	`network.offline_migration_sdn_to_ovnk` (Validated, Red Hat, v1.0.3, updated 2025-07-10)
Key content	**Post-migration role** role `post_migration` **Post-rollback role** role `post_rollback` **Rollback role** role `rollback` **Post-migration** playbook `post_migration-playbook` **Post-rollback** playbook `post_rollback-playbook` **Rollback** playbook `rollback-playbook`

OSPF `network.ospf`

Validated Red Hat · version 7.0.0 · updated 2025-09-03 · 0 modules → 1 use cases

Use Case: Manage OSPF configuration and detect drift

Description	Gather and persist OSPF facts, detect drift from the desired state, deploy changes and remediate drift across network platforms, and run health checks.
Collection	`network.ospf` (Validated, Red Hat, v7.0.0, updated 2025-09-03)

Key content	**Gather facts** role `gather` **Persist facts as host variables** role `persist` **Detect configuration drift** role `detect` **Remediate drift** role `remediate` **Deploy configuration** role `deploy` **Health checks** role `health_checks` **Health check view filter** filter `health_check_view`

Network telemetry `network.telemetry`

Validated Red Hat · version 3.0.0 · updated 2024-06-13 · 0 modules → 2 use cases

Use Case: Manage telemetry configuration and detect drift

Description	Gather and persist telemetry settings, detect drift from the desired state, deploy changes and remediate drift on network devices.
Collection	`network.telemetry` (Validated, Red Hat, v3.0.0, updated 2024-06-13)
Key content	**Gather facts** role `gather` **Persist facts as host variables** role `persist` **Detect configuration drift** role `detect` **Remediate drift** role `remediate` **Deploy configuration** role `deploy`

Use Case: Collect and verify telemetry

Description	Deploy a Docker and Telegraf telemetry collector and check telemetry configuration health on Cisco NX-OS devices.
Collection	`network.telemetry` (Validated, Red Hat, v3.0.0, updated 2024-06-13)
Key content	**Telemetry collector** role `deploy_collector` **Health checks** role `health_checks`

VPN `network.vpn`

Validated Red Hat · version 6.0.0 · updated 2025-09-03 · 3 modules → 1 use cases

Use Case: Deploy and validate VPN configuration

Description	Deploy VPN settings to network appliances, validate them, and manage Azure local network gateways and virtual network gateway connections.
Collection	`network.vpn` (Validated, Red Hat, v6.0.0, updated 2025-09-03)
Key content	**Deploy VPN configuration** role `deploy` **Validate VPN configuration** role `validate` **Azure local network gateways** `azure_rm_localnetworkgateway` **Azure gateway connections** `azure_rm_virtualnetworkgatewayconnection`, `azure_rm_virtualnetworkgatewayconnection_info`

Nautobot (Network to Code) `networktocode.nautobot`

Certified · version 6.3.0 · updated 2026-09-17 · 90 modules → 10 use cases

Use Case: Model sites, locations and tenants

Description	Define location types and locations, tenants and groups, racks and rack groups, contacts, teams and manufacturers.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Location types** `location_type` **Locations** `location` **Tenant groups** `tenant_group` **Tenants** `tenant` **Rack groups** `rack_group` **Racks** `rack` **Contacts** `contact` **Teams** `team` **Manufacturers** `manufacturer`

Use Case: Manage devices and hardware components

Description	Create devices, types, families, platforms and software versions, with their interfaces, ports, bays, modules and inventory items.

Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Devices** `device` **Device types** `device_type` **Device families** `device_family` **Platforms** `platform` **Software versions** `software_version` **Device redundancy groups** `device_redundancy_group` **Virtual chassis** `virtual_chassis` **Virtual device contexts** `virtual_device_context` **Inventory items** `inventory_item` **Device interfaces** `device_interface` **Interface templates** `device_interface_template` **Front ports** `front_port` **Front port templates** `front_port_template` **Rear ports** `rear_port` **Rear port templates** `rear_port_template` **Console ports** `console_port` **Console port templates** `console_port_template` **Console server ports** `console_server_port` **Console server port templates** `console_server_port_template` **Device bays** `device_bay` **Device bay templates** `device_bay_template` **Modules** `module` **Module types** `module_type` **Module bays** `module_bay` **Module bay templates** `module_bay_template` **Cables** `cable`

Use Case: Manage power infrastructure

Description	Track power panels, feeds, ports and outlets and their templates.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Power panels** `power_panel` **Power feeds** `power_feed` **Power ports** `power_port` **Power port templates** `power_port_template` **Power outlets** `power_outlet` **Power outlet templates** `power_outlet_template`

Use Case: Manage IP addressing and VLANs

Description	Maintain namespaces, prefixes, IP addresses, RIRs, VRFs, route targets, VLANs and VLAN groups.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Namespaces** `namespace` **Prefixes** `prefix` **IP addresses** `ip_address` **RIRs** `rir` **VRFs** `vrf` **Route targets** `route_target` **VLANs** `vlan` **VLAN groups** `vlan_group` **Services** `service`

Use Case: Track circuits and providers

Description	Record providers, circuit types, circuits and circuit terminations.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Providers** `provider` **Circuit types** `circuit_type` **Circuits** `circuit` **Circuit terminations** `circuit_termination`

Use Case: Model virtualisation and cloud resources

Description	Track clusters, virtual machines and their interfaces, plus cloud accounts, networks, services and resource types.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Cluster types** `cluster_type` **Cluster groups** `cluster_group` **Clusters** `cluster` **Virtual machines** `virtual_machine` **VM interfaces** `vm_interface` **Cloud accounts** `cloud_account` **Cloud networks** `cloud_network` **Cloud services** `cloud_service` **Cloud resource types** `cloud_resource_type`

Use Case: Manage wireless and controller-managed devices

Description	Define controllers and managed device groups, wireless networks, radio profiles and supported data rates.

Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Controllers** `controller` **Controller managed device groups** `controller_managed_device_group` **Wireless networks** `wireless_network` **Radio profiles** `radio_profile` **Supported data rates** `supported_data_rate`

Use Case: Customise the data model and enforce data quality

Description	Add custom fields, statuses, roles, tags, relationships, metadata and dynamic groups, and enforce validation rules.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Custom fields** `custom_field` **Statuses** `status` **Roles** `role` **Tags** `tag` **Relationship associations** `relationship_association` **Metadata types** `metadata_type` **Object metadata** `object_metadata` **Dynamic groups** `dynamic_group` **Job buttons** `job_button` **Plugin objects** `plugin` **Min/max rules** `min_max_rule` **Regex rules** `regex_rule` **Required rules** `required_rule` **Unique rules** `unique_rule`

Use Case: Administer Nautobot access and secrets

Description	Manage users, groups, permissions, secrets and secrets groups, and the Nautobot server application.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**Users** `admin_user` **Groups** `admin_group` **Object permissions** `admin_permission` **Secrets** `secret` **Secrets groups** `secrets_group` **Nautobot server** `nautobot_server`

Use Case: Query Nautobot and use it as inventory

Description	Query data through GraphQL or lookups, build dynamic inventory and react to change events.
Collection	`networktocode.nautobot` (Certified, , v6.3.0, updated 2026-09-17)
Key content	**GraphQL query facts** `graphql_facts`, `graphql_info` **GraphQL string filter** filter `graphql` **Nautobot inventory** inventory `inventory` **GraphQL inventory** inventory `gql_inventory` **Nautobot lookup** lookup `lookup` **GraphQL lookup** lookup `lookup_graphql` **Change log events** eda event source `nautobot_changelog`

Nokia SR Linux `nokia.srlinux`

Certified Nokia · version 1.1.1 · updated 2026-08-12 · 4 modules → 1 use cases

Use Case: Configure and query Nokia SR Linux devices

Description	Run CLI commands, update, replace and delete configuration, validate it, and retrieve configuration or state.
Collection	`nokia.srlinux` (Certified, Nokia, v1.1.1, updated 2026-08-12)
Key content	**CLI commands** `cli` **Configuration** `config` **Validation** `validate` **Configuration and state retrieval** `get`

Open vSwitch `openvswitch.openvswitch`

Unknown · version · updated · 4 modules → 1 use cases

Use Case: Manage Open vSwitch bridges, ports and bonds

Description	Configure Open vSwitch bridges, ports, bonds and database settings.
Collection	`openvswitch.openvswitch` (Unknown,)
Key content	**Bridges** `openvswitch_bridge` **Ports** `openvswitch_port` **Bonds** `openvswitch_bond` **Database settings** `openvswitch_db`

Radware Alteon `radware.radware_alteon`

Certified Radware · version 1.0.2 · updated 2023-09-13 · 85 modules → 9 use cases

Use Case: Publish applications with virtual servers and services

Description	Define virtual servers and services, AppShape++ scripts, content rules, sideband and secure path policies.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)
Key content	**Virtual servers** `alteon_config_virtual_server` **Virtual services** `alteon_config_virtual_service` **AppShape++ scripts** `alteon_config_appshape` **Content rules** `alteon_config_content_rule` **Sideband policies** `alteon_config_sideband_policy` **Secure path policies** `alteon_config_secure_path_policy` **SLB port settings** `alteon_config_slb_port` **SLB proxy IPs** `alteon_config_slb_pip` **SLB IPv6 proxy IPs** `alteon_config_slb_pip6`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Manage real servers, groups and health checks

Description	Add real servers and server groups, check their health over HTTP, TCP or logical expressions, and set operational state.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)
Key content	**Real servers** `alteon_config_server` **Server groups** `alteon_config_server_group` **Group real servers** `alteon_config_group_real_server` **FQDN servers** `alteon_config_fqdn_server` **HTTP health checks** `alteon_config_health_check_http` **TCP health checks** `alteon_config_health_check_tcp` **Logical expression health checks** `alteon_config_health_check_logexp` **Real server operational state** `alteon_oper_server_status`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Route traffic on Layer 7 content

Description	Match requests on cookies, headers, host names, paths, file names and types with Layer 7 content classes.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)
Key content	**Content classes** `alteon_config_l7_content_class` **Cookie classes** `alteon_config_l7_content_class_cookie` **Header classes** `alteon_config_l7_content_class_header` **Host name classes** `alteon_config_l7_content_class_hostname` **URL path classes** `alteon_config_l7_content_class_path` **URL file name classes** `alteon_config_l7_content_class_filename` **URL file type classes** `alteon_config_l7_content_class_filetype`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Distribute traffic across sites with GSLB

Description	Configure GSLB rules and network preferences, DNS responders, global redirection and network classes.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)

Key content	**GSLB rules** `alteon_config_gslb_rule` **GSLB network preferences** `alteon_config_gslb_network` **DNS responders** `alteon_config_dns_responders` **Global traffic redirection** `alteon_config_global_redirection` **IP network classes** `alteon_config_network_class_ip` **Region network classes** `alteon_config_network_class_region` **GEL parameters** `alteon_config_gel`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Terminate and inspect SSL/TLS

Description	Manage SSL certificates, certificate groups, keys, SSL policies, and client and server authentication policies.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)
Key content	**SSL certificates** `alteon_config_ssl_cert` **Certificate groups** `alteon_config_ssl_cert_group` **SSL keys** `alteon_config_ssl_key` **SSL policies** `alteon_config_ssl_policy` **Client authentication policies** `alteon_config_ssl_client_auth_policy` **Server authentication policies** `alteon_config_ssl_server_auth_policy` **Global security settings** `alteon_config_security_global`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Configure Layer 2 and Layer 3 networking

Description	Set up physical ports, VLANs, LACP, spanning tree, LLDP, interfaces, gateways, static routes, BOOTP relay and BGP.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)
Key content	**Physical ports** `alteon_config_physical_port` **VLANs** `alteon_config_l2_vlan` **LACP aggregation** `alteon_config_lacp_aggregation` **Spanning tree** `alteon_config_spanning_tree` **LLDP** `alteon_config_l2_lldp` **Interfaces** `alteon_config_l3_interface` **Gateways** `alteon_config_l3_gateway` **Static routes** `alteon_config_static_routes` **BOOTP relay** `alteon_config_l3_bootp_relay` **BGP global** `alteon_config_bgp_global` **BGP peers** `alteon_config_bgp_peer` **BGP aggregations** `alteon_config_bgp_aggregations`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Configure high availability

Description	Set up HA, floating IPs, HA services, configuration sync, VX peer sync, and force an HA backup state.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)
Key content	**High availability** `alteon_config_high_availability` **Floating IPs** `alteon_config_ha_floating_ip` **HA services** `alteon_config_ha_service` **Configuration sync** `alteon_config_ha_config_sync` **VX peer sync** `alteon_config_system_vx_peer_sync` **HA backup state** `alteon_oper_ha_backup`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Administer access, SNMP and system services

Description	Manage local users, RADIUS and TACACS+, management access, DNS client, time, logging, alerts, and SNMP and SNMPv3.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)

Key content	**Local users** `alteon_config_system_local_user` **Predefined local users** `alteon_config_system_predefined_local_users` **RADIUS authentication** `alteon_config_system_radius_auth` **TACACS+ authentication** `alteon_config_system_tacacs_auth` **Management access** `alteon_config_system_management_access` **DNS client** `alteon_config_system_dns_client` **Time and date** `alteon_config_system_time_date` **Logging** `alteon_config_system_logging` **System alerts** `alteon_config_system_alerts` **SNMP** `alteon_config_system_snmp` **SNMP general** `alteon_config_snmp_general` **SNMPv3 access** `alteon_config_snmpv3_access` **SNMPv3 communities** `alteon_config_snmpv3_community` **SNMPv3 groups** `alteon_config_snmpv3_group` **SNMPv3 notify** `alteon_config_snmpv3_notify` **SNMPv3 target addresses** `alteon_config_snmpv3_target_addr_new_cfg` **SNMPv3 target parameters** `alteon_config_snmpv3_target_params` **SNMPv3 USM users** `alteon_config_snmpv3_usm_user` **SNMPv3 view tree families** `alteon_config_snmpv3_view_tree_family`
Notes	• Module names have the alteon_ prefix omitted.

Use Case: Manage devices, configuration and software

Description	Collect device facts, back up or restore configuration, run CLI commands, reboot, create vADCs and install software images.
Collection	`radware.radware_alteon` (Certified, Radware, v1.0.2, updated 2023-09-13)
Key content	**Device facts** `alteon_device_facts` **Configuration download and upload** `alteon_device_configuration` **Configuration management** `alteon_mng_config` **CLI commands** `alteon_config_alteon_cli_command` **Reboot** `alteon_mng_device_reboot` **vADC instances** `alteon_config_vadc_instance` **Image upload** `alteon_software_image_upload` **Software install** `alteon_software_install` **vADC software install** `alteon_software_install_vadc` **vADC default version** `alteon_software_vadc_default`
Notes	• Module names have the alteon_ prefix omitted.

Seiko SmartCS console server `seiko.smartcs`

Certified Seiko Solutions Inc. · version 1.8.0 · updated 2026-05-25 · 4 modules → 1 use cases

Use Case: Administer SmartCS console servers

Description	Run commands, manage configuration, collect facts and send strings to serial ports on SmartCS devices.
Collection	`seiko.smartcs` (Certified, Seiko Solutions Inc., v1.8.0, updated 2026-05-25)
Key content	**Commands** `smartcs_command` **Configuration** `smartcs_config` **Facts** `smartcs_facts` **Serial port commands** `smartcs_tty_command`

VyOS `vyos.vyos`

Certified VyOS Networks · version 6.0.0 · updated 2025-06-25 · 34 modules → 5 use cases

Use Case: Configure interfaces, VLANs and link aggregation

Description	Manage interfaces, Layer 3 settings, LAGs, VLANs and LLDP.
Collection	`vyos.vyos` (Certified, VyOS Networks, v6.0.0, updated 2025-06-25)
Key content	**Interfaces** `vyos_interfaces` **Layer 3 interfaces** `vyos_l3_interfaces` **LAG interfaces** `vyos_lag_interfaces` **VLANs** `vyos_vlan` **LLDP** `vyos_lldp_global` **LLDP interfaces** `vyos_lldp_interfaces`

Use Case: Configure routing and VRFs

Description	Manage BGP, OSPFv2 and OSPFv3, static routes, prefix lists, route maps and VRFs.
Collection	`vyos.vyos` (Certified, VyOS Networks, v6.0.0, updated 2025-06-25)

Key content	**BGP** `vyos_bgp_global` **BGP address families** `vyos_bgp_address_family` **OSPFv2** `vyos_ospfv2` **OSPFv3** `vyos_ospfv3` **OSPF interfaces** `vyos_ospf_interfaces` **Static routes** `vyos_static_routes` **Prefix lists** `vyos_prefix_lists` **Route maps** `vyos_route_maps` **VRFs** `vyos_vrf`

Use Case: Configure firewall, NAT and VPN

Description	Manage firewall global settings, rules and interface bindings, NAT, IPsec VPN and site-to-site peers.
Collection	`vyos.vyos` (Certified, VyOS Networks, v6.0.0, updated 2025-06-25)
Key content	**Firewall global settings** `vyos_firewall_global` **Firewall rules** `vyos_firewall_rules` **Firewall interfaces** `vyos_firewall_interfaces` **NAT** `vyos_nat` **IPsec VPN** `vyos_vpn_ipsec` **IPsec site-to-site peers** `vyos_vpn_ipsec_s2s` **VRRP and load balancing** `vyos_ha`

Use Case: Administer the device

Description	Manage hostname, banners, system attributes, local users, NTP, logging, SNMP and files.
Collection	`vyos.vyos` (Certified, VyOS Networks, v6.0.0, updated 2025-06-25)
Key content	**System attributes** `vyos_system` **Hostname** `vyos_hostname` **Banners** `vyos_banner` **Local users** `vyos_user` **NTP** `vyos_ntp_global` **Logging** `vyos_logging_global` **SNMP** `vyos_snmp_server` **Files and directories** `vyos_file`

Use Case: Run commands, push configuration and gather facts

Description	Run commands, manage configuration, test reachability and collect device facts.
Collection	`vyos.vyos` (Certified, VyOS Networks, v6.0.0, updated 2025-06-25)
Key content	**Configuration** `vyos_config` **Commands** `vyos_command` **Reachability tests** `vyos_ping` **Device facts** `vyos_facts`

Other certified and validated collections

Module-level content for these collections could not be read from a public source repository. Browse them on Automation Hub.

`ansible.network`	Ansible Network meta Collection to install all network supported content. (Certified, Red Hat)
`arista.dmf`	Ansible Modules to Manage DANZ Monitoring Fabric (Certified, Arista)
`cisco.catalyst_center_iac`	Cisco-validated Ansible playbooks for Infrastructure as Code (IaC) automation of Cisco Catalyst Center configurations. Covers Day 0, Day 1, Day 2, and Day N operations including site hierarchy, device provisioning, SD-Access fabric, SWIM, assurance, and more. (Certified, Cisco)
`citrix.adc`	Citrix ADC modules for Ansible (Certified, Citrix; Superseded by netscaler.adc; the repository now hosts the new collection.)
`juniper.eda`	Ansible Event Driven Automation (Certified, Juniper Networks)
`netscaler.console`	Ansible collection modules for NetScaler Console (Certified, NetScaler)
`network.meraki_ops`	Meraki validated content for Ansible. (Validated, Red Hat)
`nvidia.nvue`	Modules to interact with NVIDIA devices managed by NVUE (Certified, Nvidia)
`opengear.om`	Ansible Network Collection for Opengear OM & CM8100 devices. (Certified, Opengear; The repository now hosts opengear.ng.)
`radware.alteon`	A collection of Ansible Modules for Radware devices (Certified, Radware)
`verticetech.qingteng_host`	A collection containing the qingteng module. (Certified, Vertice Technology)